The Journey of PyTorch: An In-Depth Overview 🚀

Introduction Q

PyTorch is an open-source deep learning library that has revolutionized the field of artificial intelligence (AI) and machine learning (ML). Developed by **Meta AI** (formerly Facebook AI Research), PyTorch combines the intuitive nature of Python programming with the powerful computational capabilities of the original **Torch** framework. Since its inception, PyTorch has become a favorite among researchers and developers due to its flexibility, ease of use, and strong community support.

Official Website:- PyTorch

Background: Python and Torch Uh

Python's Ease of Use (2

- **Simplicity and Readability:** Python is known for its straightforward syntax, making it accessible for beginners and efficient for experts.
- **Extensive Libraries:** It boasts a rich ecosystem of scientific computing libraries like **NumPy, SciPy,** and **Pandas,** which facilitate data manipulation and analysis.

- **Community Support:** A large, active community contributes to a wealth of resources, tutorials, and third-party libraries.

The Torch Framework

- **Origin in Lua:** Torch was originally built using the Lua programming language, focusing on flexibility and speed.
- **High-Performance Tensor Operations:** It excelled in tensor computations, crucial for deep learning tasks, especially when utilizing **Graphics Processing Units (GPUs).**
- **Machine Learning Focus:** Provided a wide range of algorithms and tools specifically designed for ML and neural networks.

Merging Python and Torch: The Birth of PyTorch

- **Bridging the Gap:** PyTorch was created to combine Python's user-friendly nature with Torch's computational prowess.
- **Dynamic Computation Graphs:** Introduced the ability to build neural networks more intuitively and adjust models on-the-fly.
- **Accelerated Development:** Enabled faster prototyping and experimentation, crucial for advancing AI research.

Feature	Benefit	Impact
Flexibility and Ease of Use	Dynamic computation graph and Pythonic design	High accessibility for researchers
Research to Production	TorchScript and model quantization	Seamless transition from experimentation to deployment
Community and Ecosystem	Rich libraries and supportive community	Enhanced capabilities and resources
Performance and Scalability	Optimizations and distributed training	Meets demands of modern AI workloads
Hardware Optimization	Adaptation to various hardware accelerators	Ensures relevance as technology evolves

Dynamic Computation Graph

Overview

- **Define-by-Run:** PyTorch builds the computation graph dynamically during runtime, allowing for more flexible model architectures.

Key Aspects

- **Flexibility:** Supports models with conditional statements, loops, and varying dimensions.
- **Intuitive Coding:** Coding style resembles standard Python, making it easier to implement and understand complex models.
- **Debugging:** Simplifies debugging since the computation graph is built incrementally and can be inspected at each step.

Benefits

- **Experimentation:** Facilitates rapid prototyping and testing of new ideas.
- **Adaptability:** Ideal for models that require variable computation paths, such as recursive neural networks or attention mechanisms.
- **User-Friendly:** Lowers the barrier to entry for beginners and increases productivity for experienced developers.

PyTorch Release Timeline

PyTorch 0.1 (January 2017)

Key Features

- **Dynamic Computation Graphs:**
  - **Flexibility:** Allowed computation graphs to be defined dynamically during runtime, as opposed to static graphs that are defined before execution.
  - **Ease of Debugging:** Simplified the debugging process since the graph's structure could be inspected and modified in real-time.
- **Seamless Integration with Python Libraries:**
  - **NumPy Compatibility:** Enabled effortless conversion between PyTorch tensors and NumPy arrays.
  - **Interoperability:** Users could leverage Python's extensive libraries alongside PyTorch models without friction.

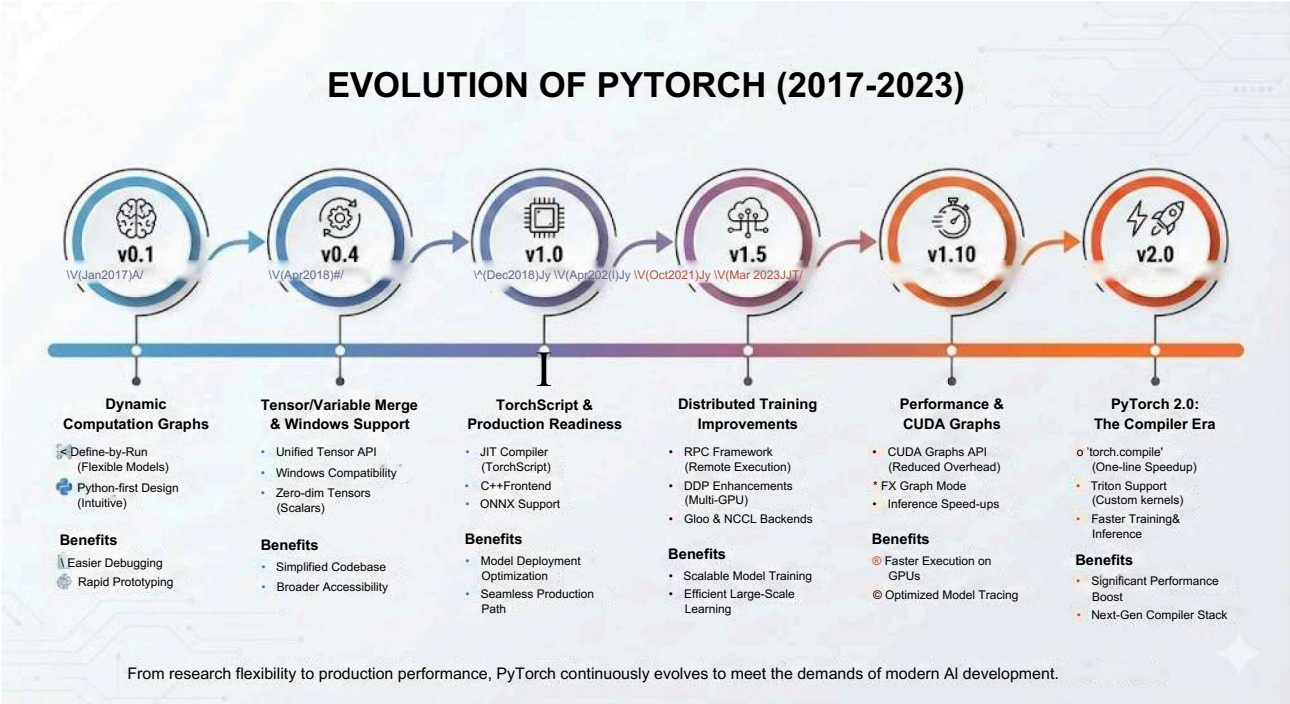
Impact

- **Research Community Adoption:**

- **Intuitive Interface:** The Pythonic design attracted researchers who found other frameworks cumbersome.
- **Rapid Prototyping:** Facilitated quick experimentation with novel neural network architectures.

- **Proliferation in Academic Papers:**
  - **Influence on AI Research:** Became a go-to tool for publishing state-of-the-art results in conferences and journals.

PyTorch 1.0 (December 2018) 0



Key Features

- **Bridging Research and Production:**
  - **Unified Framework:** Merged the ease of use needed for research with the robustness required for production deployment.
- **Introduction of TorchScript:**
  - **Model Serialization:** Allowed models to be saved and loaded independently of Python, enabling deployment in environments where Python isn't available.
  - **Optimization:** Provided just-in-time (JIT) compilation for performance improvements.
- **Caffe2 Integration:**
  - **Enhanced Performance:** Merged with Caffe2 to leverage its production-oriented features.
  - **Cross-Platform Support:** Improved support for mobile and embedded devices.

Impact

- **Smoother Model Deployment:**
  - **From Lab to Production:** Simplified the transition of models from experimental code to scalable, production-ready applications.
- **Industry Adoption:**
  - **Enterprise Use Cases:** Attracted companies looking for a reliable framework that supports both development and deployment.

PyTorch 1.x Series

Key Features

- **Distributed Training:**
  - **Scale-Up Capability:** Enabled training of large models across multiple GPUs and nodes.
  - **Data Parallelism and Model Parallelism:** Provided flexibility in how models are distributed across hardware.
- **ONNX Compatibility:**
  - **Interoperability:** Allowed models to be exported to the Open Neural Network Exchange (ONNX) format for use in other frameworks.
  - **Broader Deployment Options:** Facilitated deployment in various environments, including those optimized for inference.
- **Quantization for Model Compression:**
  - **Efficiency Gains:** Reduced model size and increased inference speed by using lower-precision arithmetic.
  - **Edge Deployment:** Made it feasible to run complex models on resource-constrained devices.
- **Expanded Ecosystem:**
  - **torchvision:** Tools and datasets for computer vision,
  - **torchtext:** Utilities for natural language processing,
  - **torchaudio:** Support for audio data processing.

Impact

- **Increased Research and Industry Adoption:**
  - **Versatility:** Catered to a wide range of applications, from academic research to commercial products.
- **Community Growth:**

- **Third-Party Libraries:** Inspired projects like **PyTorch Lightning,** which streamlines model training, and **Hugging Face Transformers,** offering pre-trained models for NLP.

- **Enhanced Cloud Support:**
  - **Cloud Providers:** AWS, Google Cloud, and Azure provided optimized environments and services for PyTorch.

PyTorch 2.0 (March 2023) 🚀

Key Features

- **Significant Performance Improvements:**
  - **torch.compile Function:** Introduced ahead-of-time compilation to optimize model execution.
  - **Backend Integration:** Worked with compilers like Torchinductor for code generation.
- **Enhanced Deployment and Production-Readiness:**
  - **Stable APIs:** Improved consistency and reliability of APIs for long-term support.
  - **Deployment Tools:** Added features to simplify model serving and integration with production systems.
- **Optimization for Modern Hardware:**
  - **Hardware Accelerators:** Optimized for GPUs, TPUs, and custom AI chips.
  - **Parallelism and Efficiency:** Leveraged advanced hardware capabilities for better performance.

Impact

- **Improved Speed and Scalability:**
  - **Faster Training Times:** Reduced the time required to train complex models.
  - **Efficient Inference:** Enabled real-time applications and services.
- **Better Compatibility with Deployment Environments:**
  - **Versatile Integration:** Supported a range of platforms, from cloud servers to edge devices.
  - **Broader Industry Use:** Attracted sectors requiring high-performance AI solutions, such as autonomous vehicles and healthcare.

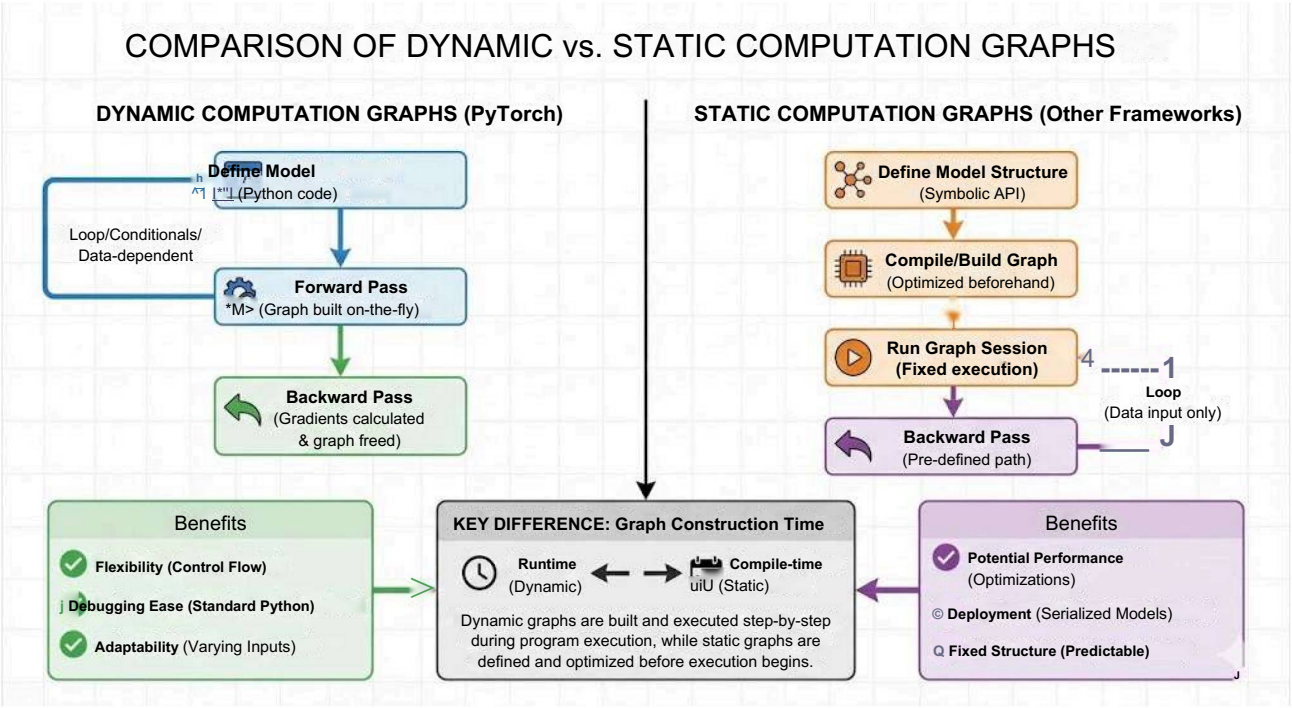
PyTorch Framework Evolution H

Version	Release Date	Key Features	Primary Impact
0.1	January 2017	Dynamic computation graphs, Python integration	Research adoption
1.0	December 2018	TorchScript, Caffe2 integration	Production readiness
1.x Series	2019-2022	Distributed training, ONNX compatibility, Ecosystem expansion	Industry adoption
2.0	March 2023	torch.compile, Hardware optimization	Performance boost

Additional Insights

Dynamic vs. Static Computation Graphs S

- **Dynamic Graphs:**
  - **Runtime Flexibility:** Adjusts to varying input sizes and network architectures during execution.
  - **Conditional Operations:** Easily implements models with control flow, like loops and conditionals.
- **Static Graphs:**
  - **Optimization Potential:** Known ahead of time, allowing for more aggressive compiler optimizations.
  - **Limitations:** Less flexible when experimenting with novel architectures.



TorchScript and Model Deployment

- **Serialization:**
  - o **Portability:** Models can be exported to run in environments without a Python interpreter.
  - o **Language Agnostic:** Facilitates integration with applications written in other programming languages.
- **Just-In-Time Compilation:**
  - **o Performance Gains:** Compiles parts of the model to optimized machine code during execution.
  - o **Reduced Overhead:** Minimizes the performance penalty typically associated with dynamic graphs.

Distributed Training Techniques

- **Data Parallelism:**
  - o **Same Model, Different Data:** Replicates the model across multiple GPUs, each processing a different subset of the data.
- **Model Parallelism:**
  - o **Different Model Parts:** Splits the model across multiple devices, useful for very large models.

Quantization Methods

- **Static Quantization:**

- **Calibration:** Requires a calibration step with a representative dataset to determine scaling factors.

- **Dynamic Quantization:**
  - **On-the-Fly Scaling:** Applies quantization during inference without prior calibration.
- **Quantization-Aware Training:**
  - **Accuracy Preservation:** Simulates quantization effects during training to maintain model performance.

The PyTorch Ecosystem

- **PyTorch Lightning:**
  - **Code Organization:** Separates engineering code from research code, making projects more manageable.
  - **Reusable Components:** Encourages modular design for models, training loops, and data handling.
- **Hugging Face Transformers:**
  - **Pre-Trained Models:** Offers a vast collection of models for tasks like text classification, translation, and question-answering.
  - **Community Contributions:** Hosts models contributed by researchers and organizations worldwide.
- **Fast.ai:**
  - **Accessible Deep Learning:** Simplifies complex tasks with high-level APIs.
  - **Educational Resources:** Provides courses and documentation to help users learn deep learning concepts.

Hardware Acceleration and Optimization

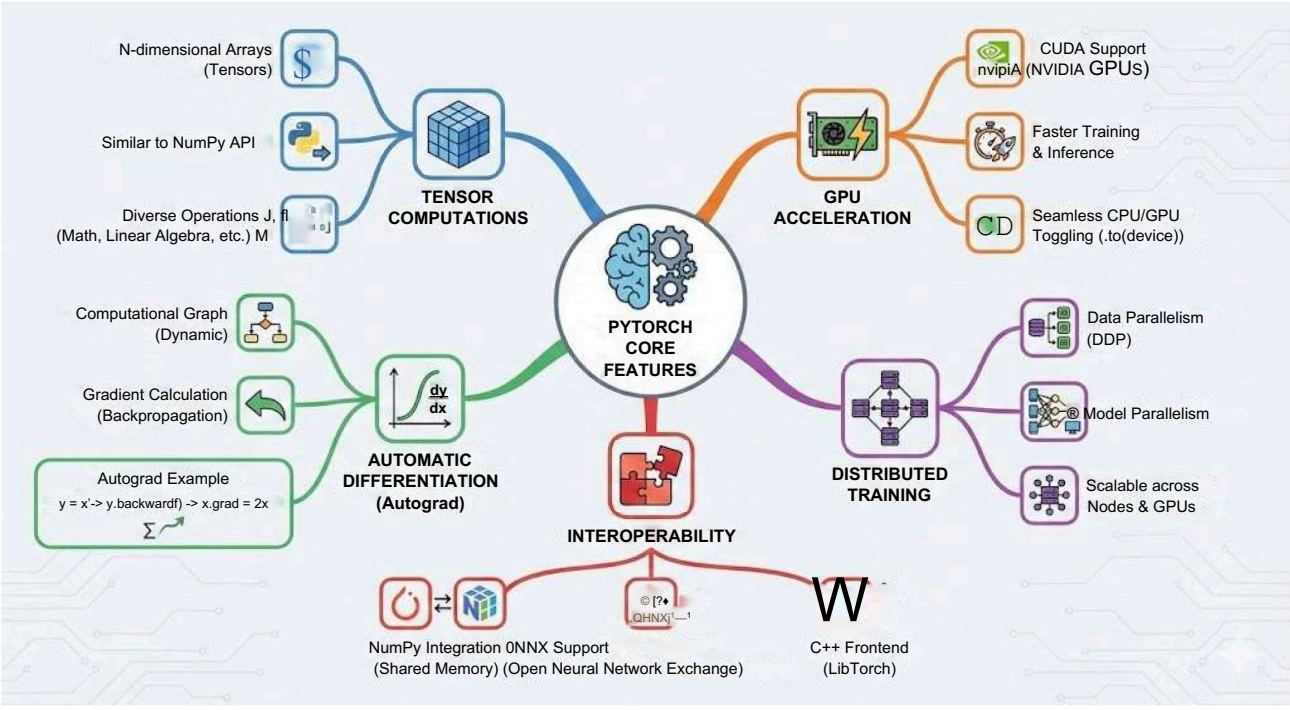
- **GPU Utilization:**
  - **CUDA Support:** Leverages NVIDIA GPUs for accelerated computation.
  - **Automatic Mixed Precision:** Uses both 16-bit and 32-bit floating-point types to speed up training without sacrificing much accuracy.
- **TPU and Custom Chips:**
  - **Broad Hardware Support:** Adapts to different processing units, enabling performance gains on specialized hardware.
- **Future Hardware Trends:**
  - **AI Accelerators:** Ongoing optimizations for emerging technologies like neuromorphic chips and quantum processors.

PyTorch's evolution reflects a deep commitment to advancing AI research and facilitating its practical application. By continually addressing the needs of both researchers and industry professionals, PyTorch has established itself as a cornerstone in the machine learning landscape. Its journey from a flexible research tool to a production-ready framework illustrates the dynamic nature of the AI field and the importance of adaptable, efficient tools in driving innovation.

Core Features of PyTorch

GPU vs CPU Performance Comparison

Task	CPU Time	GPU Time	Speedup
Matrix Multiplication (1000x1000)	500ms	10ms	50x
ResNet-50 Inference (batch=64)	2000ms	50ms	40 x
BERT Training (batch = 32)	15s	0.8s	18.75x
Image Segmentation (512x512)	800ms	30ms	26.7x



4. Automatic Differentiation

Overview

- **Autograd Engine:** PyTorch's automatic differentiation system computes gradients automatically during the backward pass.

Key Aspects

- **Gradient Tracking:** By setting requires_grad=True , PyTorch tracks all operations on a tensor to compute gradients.
- **Backward Propagation:** Calling .backward() computes the gradients needed for optimization.
- **Computational Graph:** Records operations in a graph that represents the dependencies between tensors.

Benefits

- **Simplifies Training:** Eliminates manual calculation of derivatives, reducing potential errors.
- **Custom Operations:** Users can define custom autograd functions by specifying forward and backward computations.
- **Efficiency:** Optimizes memory usage by reusing intermediate computations when possible.

Autograd Example

cP

```
# Define tensors with gradient tracking
x = torch.tensor([2.0], requires_grad=True)
y = torch.tensor([3.0], requires_grad=True)

# Define a computation
z = x**2 + y**3

# Compute gradients
z.backward()

# Access gradients
print(f"dz/dx: {x.grad}") # Output: 4.0
print(f"dz/dy: {y.grad}") # Output: 27.0
```

5. Distributed Training ©

Overview

- **Scaling Across Devices:** PyTorch provides tools to train models across multiple GPUs and machines, enabling handling of large-scale problems.

Key Aspects

- **Data Parallelism:** Utilizes multiple GPUs by splitting input data across them, with each GPU processing a portion of the data.
- **Distributed Data Parallel (DDP):** A module that synchronizes gradients across multiple processes and GPUs for efficient training.
- **Communication Backends:** Supports NCCL, Gloo, and MPI for inter-process communication.

Benefits

- **Speed:** Reduces training time by parallelizing computation.
- **Resource Utilization:** Makes better use of available hardware resources.
- **Scalability:** Allows training of larger models that wouldn't fit on a single GPU.

6. Interoperability with Other Libraries

Overview

- **Ecosystem Integration:** PyTorch works seamlessly with other Python libraries, enhancing its functionality and ease of use.

Key Aspects

- **NumPy Compatibility:** Tensors can be easily converted to and from NumPy arrays using .numpy() and torch.from_numpy() .
- **Integration with Python Libraries:** Works well with libraries like SciPy, Pandas, and scikit-learn.
- **ONNX Support:** Models can be exported to the Open Neural Network Exchange format for use in other frameworks.
- **Third-Party Libraries:** Compatible with libraries like Hugging Face Transformers, PyTorch Lightning, and Fast.ai.

Benefits

- **Versatility:** Enables combining PyTorch with tools for data analysis, visualization, and more.

- **Model Portability:** Facilitates deployment in different environments and platforms.
- **Community Support:** Access to a wide range of pre-trained models and utilities developed by the community.

PyTorch Ecosystem Integration

Library	Purpose	Integration Method
NumPy	Scientific computing	tensor.numpy() , torch.from_numpy()
Pandas	Data manipulation	Convert DataFrames to tensors
Matplotlib	Visualization	Plot tensor data directly
scikit-learn	Machine learning	Use models with PyTorch features
Hugging Face	NLP models	Import pre-trained models
PyTorch Lightning	Training organization	Inherit from LightningModule
ONNX	Model exchange	torch.onnx.export()
TorchServe	Model serving	Export models for production

Additional Insights Q

Working with Tensors

- **Initialization:** Tensors can be initialized with specific values, random numbers, or by loading data from files.
- **Operations:** Supports advanced mathematical functions, including linear algebra operations, Fourier transforms, and statistical functions.
- **Broadcasting Semantics:** Allows operations on tensors of different shapes, following specific broadcasting rules.

GPU Acceleration Best Practices f

- **Device-Agnostic Code:** Write code that can run on both CPU and GPU by checking for GPU availability (torch.cuda.is_available()).
- **Memory Management:** Be mindful of GPU memory usage to avoid out-of-memory errors.
- **Profiling Tools:** Use PyTorch's profiling utilities to optimize performance.

Advantages of Dynamic Computation Graphs S

- **Adaptive Models:** Ideal for models where the computation graph depends on input data, such as in natural language processing tasks.
- **Simplified Control Flow:** Incorporate loops and conditionals directly into the model definition.

Automatic Differentiation in Depth

- **Chain Rule Application:** Autograd uses the chain rule to compute gradients efficiently.
- **Leaf Tensors:** Tensors created by the user are considered leaf nodes; gradients are accumulated in their .grad attribute.
- **No Grad Context:** Use torch.no_grad() to prevent tracking of operations, which is useful during inference to save memory.

Distributed Training Techniques

- **Model Parallelism:** Splitting a model across multiple GPUs, useful when the model is too large for a single GPU.
- **Hybrid Parallelism:** Combining data and model parallelism for maximum efficiency.
- **Synchronization:** Ensuring all processes have the updated model parameters requires careful synchronization.

Interoperability Examples

- **Data Preprocessing:** Use Pandas for data manipulation before converting data to PyTorch tensors.
- **Visualization:** Integrate with Matplotlib or Seaborn for plotting training metrics.
- **Deployment:** Export models using TorchScript or ONNX for deployment in environments like mobile apps or web servers.

Conclusion 2

PyTorch's core features provide a robust foundation for building, training, and deploying deep learning models. Its emphasis on flexibility, performance, and interoperability makes it a preferred choice for both researchers and practitioners in the field of machine learning.

> **Pro Tip:** PyTorch's combination of dynamic computation graphs and intuitive Python-like syntax allows for more natural experimentation while maintaining high performance, making it an excellent choice for both research prototyping and production deployment.

Key Takeaways

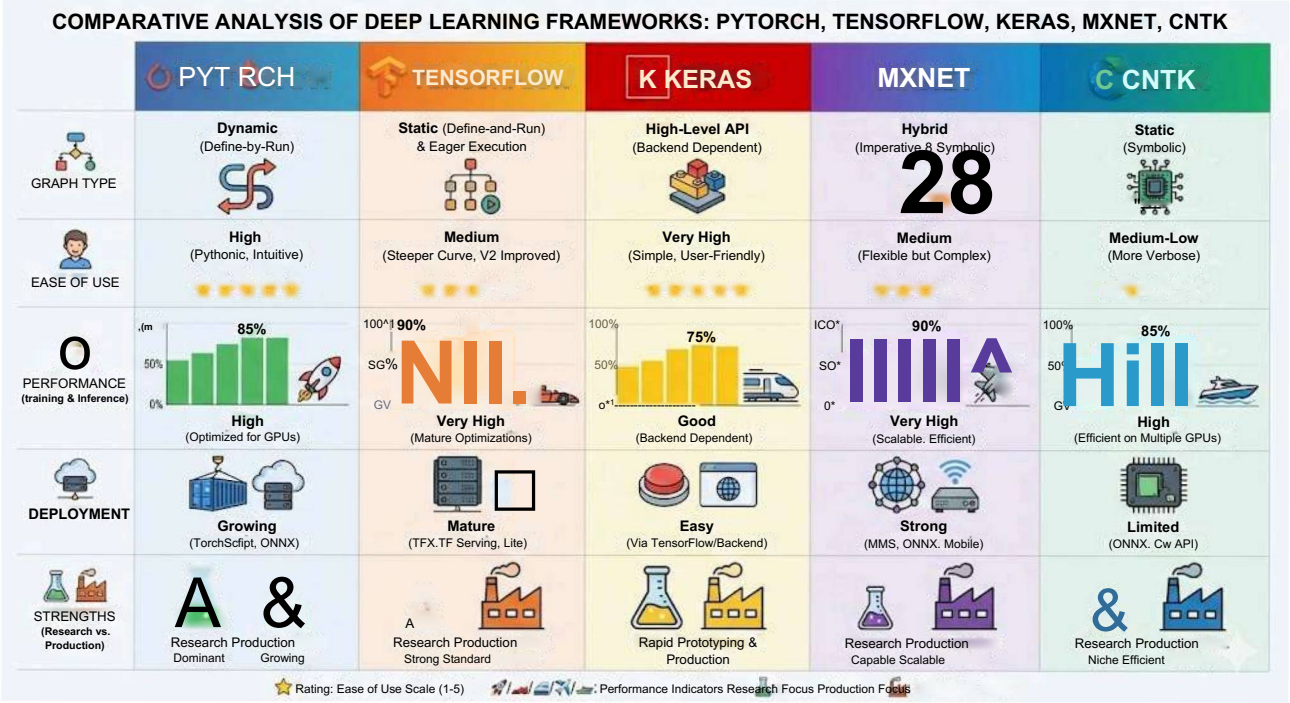
Feature	Description	Primary Benefit
Tensor Computations	Efficient data representation and manipulation	Foundation for all operations
GPU Acceleration	Leverages GPU power for faster computation	Critical speed boost for deep learning
Dynamic Computation Graph	Runtime graph building for flexible architectures	Unparalleled model design flexibility
Automatic Differentiation	Automated gradient computation	Simplified optimization process
Distributed Training	Training across multiple devices	Handles large models and datasets
Interoperability	Integration with other libraries	Enhanced functionality and deployment

Comparison of PyTorch, TensorFlow, and Other Deep Learning Frameworks

Deep learning frameworks are essential tools for developing and deploying machine learning models. PyTorch and TensorFlow are two of the most popular frameworks, each with its own strengths and weaknesses. Below is a detailed comparison of PyTorch, TensorFlow, and other notable frameworks like Keras, MXNet, and CNTK, presented in tabular form for clarity.

Core Features Comparison

Feature	PyTorch	TensorFlow	Keras
Tensor Computations	Yes, with strong NumPy integration	Yes, via tf.Tensor	Yes, as a high-level API over TensorFlow
GPU Acceleration	Native CUDA support, easy to switch between CPU and GPU	Native support, uses tf.device context manager	Inherits GPU support from backend (TensorFlow, etc.)
Dynamic Computation Graph	Yes, dynamic by default	Static by default, dynamic via tf .function and eager execution	Yes, abstracts graph complexities
Automatic Differentiation	Yes, via Autograd module	Yes, via tf.GradientTape	Yes, abstracts gradient computations
Distributed Training	Yes, with torch.distributed module	Yes, with tf.distribute.Strategy	Limited, relies on backend capabilities
Interoperability	High, integrates with NumPy, SciPy, etc.	High, but TensorFlow tensors are not directly compatible with NumPy	High, designed for ease of use and integration



Computation Graph

Aspect	PyTorch	TensorFlow	Keras	MXNet	C^
Graph Type	Dynamic (Define-by-Run)	Static (Define-and-Run), with Eager Execution	Uses backend graph (TensorFlow or Theano)	Supports both static and dynamic graphs	Static
Flexibility	High, allows for easy debugging and model changes at runtime	Less flexible in static mode, more flexible with Eager Execution	High-level API abstracts graph complexities	Moderate, Gluon API provides dynamic graph support	Less fl due tc nature
Ease of Debugging	Easy, uses standard Python debugging tools	More complex in static mode, improved with Eager Execution	Easy, due to high-level API	Improved with Gluon API	More challei due tc graph:

Aspect	PyTorch	TensorFlow	Keras	MXNet	Ch
Performance Optimization	Good, though static graphs can be faster	Excellent, especially with graph optimizations	Good, but depends on backend	Good, with optimizations in static mode	Excelk optim for perfor

Ease of Use

Aspect	PyTorch	TensorFlow	Keras	MXNet
API Design	Pythonic, intuitive, imperative style	Initially complex, improved in TensorFlow 2.x	Very user-friendly, high-level API	Steeper learning curve, but Gluon API simplifies it
Learning Curve	Gentle, especially for Python developers	Steeper, but improved with TensorFlow 2.x	Easy, designed for quick prototyping	Steep without Gluon, moderate with Gluon
Documentation	Comprehensive and clear	Extensive, with official guides and tutorials	Excellent, with many examples	Good, but less extensive than PyTorch/TensorFlov
Community Support	Strong and active community	Very strong, backed by Google	Strong, but relies on backend frameworks	Moderate, growing community

Community and Ecosystem

Aspect	PyTorch	TensorFlow	Keras	MXNet	C
Model Zoo/Hub	Yes, PyTorch Hub with pre-trained models	Yes, TensorFlow Hub and Model Garden	Yes, via Keras Applications	Yes, Model Zoo available	Lim
Third-Party Libraries	Many, including Fast.ai, Hugging Face Transformers, PyTorch Lightning	Many, including Keras, TensorFlow Addons, TFX	Many extensions and plugins	Growing number, but fewer than PyTorch/TensorFlow	Fev thir par libr
Research Adoption	Widely used in academia and research papers	Also widely used, especially in industry	Popular for quick prototyping and education	Used in research, especially with Gluon API	Les con in res<
Industry Adoption	Increasingly used in industry, especially in startups and research labs	Widely adopted across industries	Used in industry through TensorFlow backend	Used by AWS, integrated into Amazon SageMaker	Use witl Mic anc son par

Performance and Optimization

Aspect	PyTorch	TensorFlow	Keras	MXNet	
Speed of Training	Competitive performance, improved	Highly optimized for performance	Depends on backend (usually TensorFlow)	High performance, especially in static mode	High perf optii com

Aspect	PyTorch	TensorFlow	Keras	MXNet	
	with PyTorch 2.0				
Graph Optimization	Limited in dynamic mode, improved with TorchScript and TorchCompile	Advanced graph optimizations in static mode	Abstracted away, relies on backend	Yes, with static graphs	Yes, exte optii
Mixed Precision Training	Yes, supports Automatic Mixed Precision (AMP)	Yes, with AMP and customizable policies	Inherits from backend	Yes	Yes
Hardware Acceleration	Supports CPUs, GPUs, TPUs (limited)	Supports CPUs, GPUs, TPUs	Inherits from backend	Supports CPUs, GPUs	Sup| CPU

Deployment and Production

Aspect	PyTorch	TensorFlow	Keras	MXNet	CNTK
Model Serialization	Yes, via TorchScript and ONNX export	Yes, via SavedModel format and TensorFlow Lite	Uses backend serialization methods	Yes, supports model export	Yes, with model sax capability
Mobile and Embedded Deployment	Yes, via PyTorch Mobile and Lite Interpreter	Yes, via TensorFlow Lite and TensorFlow.js	Limited, relies on backend capabilities	Limited support	Limited support

Aspect	PyTorch	TensorFlow	Keras	MXNet	CNTK
Serving Infrastructure	TorchServe for model serving	TensorFlow Serving, TensorFlow Extended (TFX)	Relies on backend serving solutions	MXNet Model Server	Limited, k focus on serving
Integration with Cloud Platforms	Supported by AWS, Google Cloud, Azure	First-class support across all major cloud providers	Supported via backend frameworks	Fully supported by AWS	Supportei Azure environm

Supported Languages

Aspect	PyTorch	TensorFlow	Keras	MXNet	CNTK
Primary Language	Python	Python	Python	Python	Python
Other Language Support	C + +, Java (experimental), Swift (early stages)	C++, JavaScript, Java, Go, Swift, Rust	N/A (front-end API only)	Scala, Julia, R, Perl, C++	C#, C++
API Consistency Across Languages	Python API is most mature	High consistency across supported languages	N/A	Variable, Python API is most developed	Best with C#, then Python

Hardware and Platform Support

Aspect	PyTorch	TensorFlow	Keras	MXNet	CNTK
CPU Support	Yes	Yes	Yes	Yes	Yes

Aspect	PyTorch	TensorFlow	Keras	MXNet	CNTK
GPU Support	NVIDIA GPUs via CUDA	NVIDIA GPUs via CUDA	Inherits from backend	NVIDIA GPUs via CUDA	NVIDIA GPUs via CUDA
TPU Support	Limited, experimental	Yes, strong support	Inherits from backend	No	No
Custom Hardware Support	Supports custom accelerators via extensions	Yes, via XLA compiler and plugins	Inherits from backend	Limited	Limited
Operating Systems	Linux, Windows, macOS	Linux, Windows, macOS	Same as backend	Linux, Windows, macOS	Linux, Windows

License and Governance

Aspect	PyTorch	TensorFlow	Keras	MXNet	
License	BSD 3-Clause	Apache License 2.0	MIT License	Apache License 2.0	MH
Governance	PyTorch Foundation under the Linux Foundation	Open-source, primarily governed by Google	Open-source, developed with community input	Apache Software Foundation project	Mic ope pro
Contribution Model	Open to community contributions via GitHub	Open to community contributions via GitHub	Open-source, contributions via GitHub	Open to contributions, follows Apache guidelines	Acc cor but acti

Conclusion

PyTorch and TensorFlow are the leading deep learning frameworks, each with their own philosophies and strengths. PyTorch is known for its dynamic computation graph and Pythonic interface, making it a favorite among researchers who require flexibility. TensorFlow, on the other hand, offers robust deployment options and performance optimizations, making it suitable for production environments.

Other frameworks like Keras, MXNet, and CNTK serve specific niches or have features that may appeal to certain users. Keras provides a high-level API for rapid prototyping, MXNet offers a mix of static and dynamic graphs with strong performance, and CNTK, while less popular now, provides efficient computations and was once heavily used within Microsoft.

When choosing a framework, consider factors such as the specific requirements of your project, your team's familiarity with the framework, community support, and deployment targets.

Additional Insights

Recent Developments (as of 2023-09)

- **PyTorch 2.0** introduced significant performance improvements with the torch.compile feature, enhancing its suitability for production environments.
- **TensorFlow** has continued to simplify its API with TensorFlow 2.x, embracing eager execution by default and integrating Keras as its high-level API.
- **Community Trends:** There is a convergence in features between PyTorch and TensorFlow, with both frameworks learning from each other and incorporating similar capabilities.

Considerations for New Projects

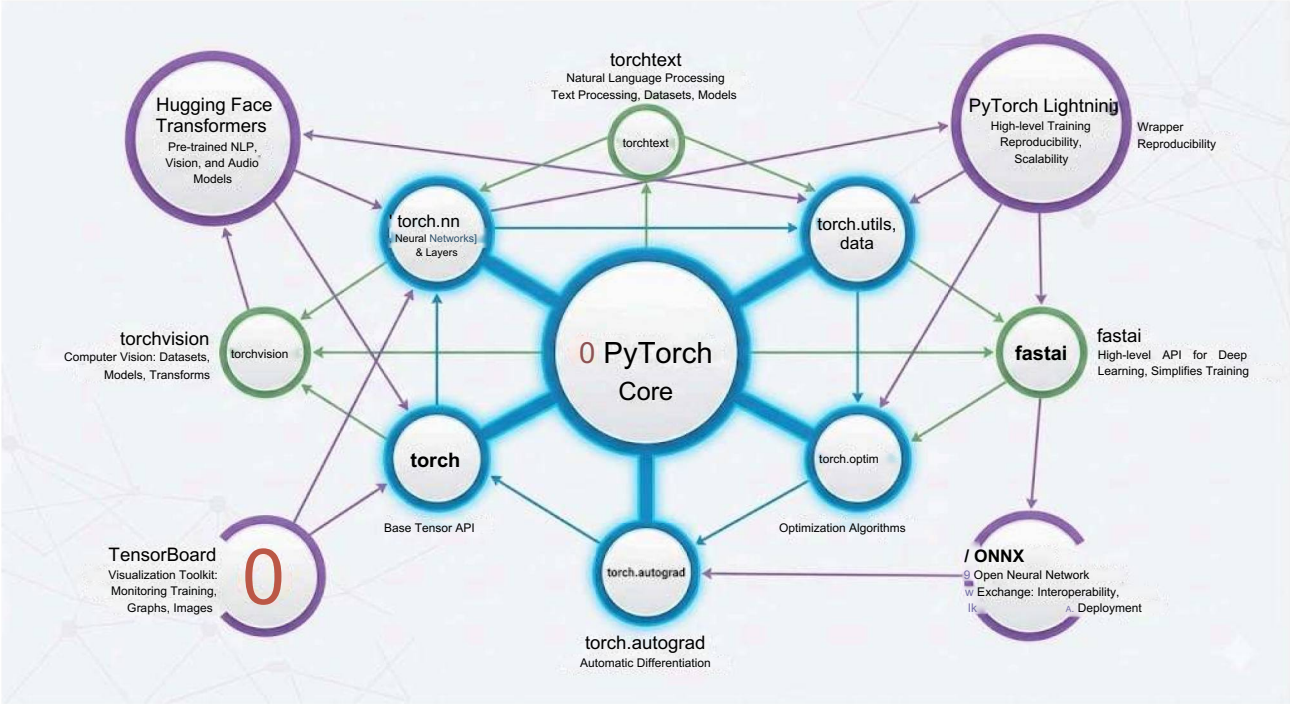
- **Community and Support:** Both PyTorch and TensorFlow have strong communities; choosing between them may come down to specific project needs or team expertise.
- **Performance Needs:** For applications requiring maximum performance and optimizations, TensorFlow might have an edge with its advanced graph optimizations.
- **Flexibility vs. Stability:** PyTorch offers more flexibility during development, while TensorFlow provides more tools for deploying and maintaining models in production.

PyTorch Core Modules and Ecosystem Libraries

PyTorch is a versatile deep learning framework that offers a range of modules and libraries to facilitate building, training, and deploying machine learning models. Understanding its core modules, domain libraries, and popular ecosystem libraries can significantly enhance your productivity and capability in developing AI solutions.

PyTorch Core Modules

PyTorch's core modules provide the fundamental building blocks for creating and training neural networks.



torch

- **Overview:** The base package of PyTorch, containing data structures for multi-dimensional tensors and mathematical operations.
- **Key Features:**
  - **Tensor Operations:** Supports a wide range of operations on tensors, including arithmetic, linear algebra, and random number generation.
  - **Device Management:** Tensors can be moved between CPU and GPU devices using simple commands.

- **Serialization:** Tensors can be saved and loaded easily, facilitating checkpointing and data persistence.

torch.nn

- **Overview:** Provides a suite of classes and modules to build neural networks.
- **Key Features:**
  - **Modules:** Includes layers like nn.Linear, nn.Conv2d , nn.LSTM,etc.
  - **Container Modules:** nn.Sequential , nn.ModuleList , and nn.ModuleDict for organizing layers.
  - **Activation Functions:** Implements common functions like ReLU, Sigmoid, Tanh, etc.
  - **LOSS Functions:** Offers loss functions SUCh as nn.CrossEntropyLoss , nn.MSELoss , and more.

torch.autograd

- **Overview:** The automatic differentiation engine that powers neural network training.
- **Key Features:**
  - **Gradient Calculation:** Computes gradients automatically during the backward pass.
  - **Computational Graph:** Records operations in a graph to compute derivatives via the chain rule.
  - **Customization:** Allows custom autograd functions by defining forward and backward passes.

torch.optim

- **Overview:** Contains optimization algorithms used to adjust model parameters during training.
- **Key Features:**
  - **Optimizers:** Implements algorithms like SGD, Adam, RMSprop, Adagrad, etc.
  - **Parameter Groups:** Allows different hyperparameters for different layers or parameters.
  - **Learning Rate Scheduling:** Supports learning rate schedulers to adjust the learning rate during training.

torch, utils.data

- **Overview:** Provides tools for data loading and preprocessing.

- **Key Features:**
  - **Dataset Class:** An abstract class representing a dataset; can be custom-defined.
  - **DataLoader:** Loads data from a Dataset with support for batching, shuffling, and multiprocess data loading.
  - **Samplers:** Controls the way samples are drawn from the dataset.

torch.jit

- **Overview:** Enables model optimization and deployment through TorchScript.
- **Key Features:**
  - **TorchScript:** A way to create serializable and optimizable models from PyTorch code.
  - **Just-In-Time (JIT) Compilation:** Compiles models for performance improvements.
  - **Deployment:** Allows models to run independently from Python, useful for production environments.

PyTorch Domain Libraries

PyTorch domain libraries are specialized packages that provide datasets, models, and tools tailored to specific application areas.

torchvision

- **Overview:** Focused on computer vision tasks.
- **Key Features:**
  - **Datasets:** Preloaded datasets like ImageNet, CIFAR10, MNIST.
  - **Transforms:** Data augmentation and preprocessing functions.
  - **Models:** Pre-trained models such as ResNet, VGG, AlexNet.
  - **Utilities:** Functions for image reading, writing, and manipulation.

torchtext

- **Overview:** Designed for natural language processing (NLP) tasks.
- **Key Features:**
  - **Datasets:** Includes datasets like IMDB, SQuAD, and WikiText.
  - **Data Processing:** Tools for tokenization, vocabulary management, and numericalization.
  - **Embedding:** Pre-trained word embeddings like GloVe and FastText.

torchaudio

- **Overview:** Tailored for audio and signal processing tasks.
- **Key Features:**
  - I/O: Functions to read and write audio files in various formats.
  - **Transforms:** Audio-specific transforms like MelSpectrogram, MFCC.
  - **Datasets:** Common datasets like LIBRISPEECH, Yesno.
  - **Augmentation:** Tools for manipulating audio data, such as adding noise or time stretching.

torchrec

- **Overview:** A library for building and deploying recommendation systems.
- **Key Features:**
  - **Modules:** Building blocks for creating recommendation models.
  - **Sharding:** Efficiently handles large embedding tables by sharding across multiple devices.
  - **Loss Functions:** Specialized loss functions for ranking and recommendation.
  - **Performance:** Optimized for training on large-scale data.

Popular PyTorch Ecosystem Libraries

These are third-party libraries built on top of PyTorch, enhancing its functionality and simplifying common tasks.

PyTorch Lightning

- **Overview:** A lightweight wrapper for PyTorch that decouples the science code from the engineering code.
- **Key Features:**
  - **Simplified Training Loop:** Abstracts the boilerplate code for training models.
  - **Flexibility:** Allows customization and fine-tuning of the training process.
  - **Scalability:** Supports distributed training across multiple GPUs and nodes.
  - **Logging:** Integrates with logging frameworks like TensorBoard and WandB.

Hugging Face Transformers

- **Overview:** Provides state-of-the-art pre-trained models for natural language processing.

- **Key Features:**
  - **Pre-Trained Models:** Includes models like BERT, GPT-2, RoBERTa, and T5.
  - **Tokenizers:** Efficient tokenization methods for various languages.
  - **Pipelines:** High-level interfaces for tasks like text classification, question answering, and translation.
  - **Integration:** Compatible with both PyTorch and TensorFlow.

Fast.ai

- **Overview:** Offers high-level components that can quickly and easily provide state-of-the-art results in standard deep learning domains.
- **Key Features:**
  - **Learner API:** Simplifies the training process with default settings that work well.
  - **DataBlock API:** Flexible data preprocessing and augmentation pipeline.
  - **Callbacks and Hooks:** Allows customization of the training loop.
  - **Educational Resources:** Accompanied by courses and extensive documentation.

Detectron2

- **Overview:** A next-generation library for object detection and segmentation.
- **Key Features:**
  - **Modular Design:** Easy to configure models and components.
  - **Pre-Trained Models:** Includes models like Faster R-CNN, Mask R-CNN.
  - **Performance:** Optimized for fast training and inference.
  - **Visualization Tools:** Functions for visualizing predictions and annotations.

PyTorch Geometric

- **Overview:** A library for deep learning on irregular structures like graphs and manifolds.
- **Key Features:**
  - **Graph Neural Networks:** Implements models like GCN, GAT, and GraphSAGE.
  - **Data Handling:** Efficiently processes sparse adjacency matrices.
  - **Benchmark Datasets:** Provides datasets like Cora, Citeseer, and PubMed.
  - **Custom Operators:** Optimized CUDA operations for graph computations.

AllenNLP

- **Overview:** An open-source NLP research library built on PyTorch.
- **Key Features:**

- **Modular Components:** Reusable modules for tasks like text classification and sequence tagging.
- **Experiment Management:** Tools for configuring, running, and analyzing experiments.
- **Pre-Trained Models:** Access to models trained on large datasets.
- **Visualization:** Interactive widgets for visualizing model outputs.

Catalyst

- **Overview:** A high-level framework for reproducible and fast experimentation.
- **Key Features:**
  - **Training Utilities:** Simplifies typical training tasks with callbacks and runners.
  - **Configuration Files:** Allows experiments to be defined via configuration files.
  - **Distributed Training:** Easy setup for multi-GPU and TPU training.
  - **Experiment Tracking:** Integrates with tools like TensorBoard and Alchemy.

Skorch

- **Overview:** A scikit-learn compatible neural network library that wraps PyTorch.
- **Key Features:**
  - **scikit-learn Integration:** Allows PyTorch models to be used with scikit-learn utilities like GridSearchCV.
  - **User-Friendly:** Simplifies the process of training neural networks.
  - **Callbacks:** Provides hooks for customizing training behavior.
  - **Compatibility:** Works seamlessly with scikit-learn pipelines.

Pyro

- **Overview:** A deep probabilistic programming language built on PyTorch.
- **Key Features:**
  - **Bayesian Modeling:** Supports stochastic functions and variational inference.
  - **Universal PPL:** Capable of representing any computable probability distribution.
  - **Inference Algorithms:** Implements algorithms like SVI and MCMC.
  - **Extensibility:** Allows for custom inference and modeling strategies.

Additional Insights

Integration and Interoperability

- **Seamless Transition:** Many ecosystem libraries are designed to work together, allowing you to combine functionalities.
- **Customizability:** You can often customize components to suit specific needs, thanks to PyTorch's flexible design.
- **Community Contribution:** The PyTorch ecosystem is continually growing, with contributions from both the community and industry leaders.

Choosing the Right Libraries

- **Project Requirements:** Select libraries that align with your project's domain, such as computer vision or NLP.
- **Ease of Use vs. Control:** High-level libraries like PyTorch Lightning simplify code but may abstract away details; choose based on your comfort level.
- **Performance Needs:** Some libraries offer optimized implementations for speed-critical applications.

Key Takeaways

- **PyTorch Core Modules:** Provide fundamental functionalities for tensor operations, neural network building blocks, automatic differentiation, optimization, data handling, and model deployment.
- **PyTorch Domain Libraries:** Offer specialized tools and datasets for computer vision (torchvision), natural language processing (torchtext), audio processing (torchaudio), and recommendation systems (torchrec).
- **Popular PyTorch Ecosystem Libraries:** Extend PyTorch's capabilities, making it easier to implement complex models, utilize pre-trained models, and manage experiments across various domains.

Detailed Table of PyTorch Users

PyTorch is a widely adopted deep learning framework used by a diverse range of individuals and organizations across academia, industry, and research communities. Below is a detailed table outlining who uses PyTorch and how they utilize it, based on publicly available information up to October 2023.

Academic Institutions

Institution	Use Cases	Notable Projects or Courses
Massachusetts Institute of Technology (MIT)	Teaching, research in AI and ML	**6.S191:** Introduction to Deep Learning with PyTorch
Stanford University	NLP, computer vision research	**CS224N:** Natural Language Processing with Deep Learning (uses PyTorch)
Carnegie Mellon University (CMU)	AI research, robotics	Development of new algorithms using PyTorch
University of Oxford	Deep learning research	Projects on reinforcement learning and generative models with PyTorch
University of California, Berkeley	Machine learning courses, research	**CS285:** Deep Reinforcement Learning (utilizes PyTorch)

Research Organizations

Organization	Use Cases	Notable Contributions
OpenAI	AI research, language models	Some research papers and models implemented in PyTorch
DeepMind	AI and RL research	Publications utilizing PyTorch for experiments
Allen Institute for AI (AI2)	NLP tools and research	**AllenNLP:** An open-source NLP library built on PyTorch
Mila - Quebec AI Institute	Deep learning research	Contributions to advancements in AI using PyTorch
Google Brain (some projects)	Research in machine learning	Select projects and papers employing PyTorch

Technology Companies

Company	Use Cases	Notable Projects
Meta (Facebook)	AI applications across platforms	Creator and primary maintainer of **PyTorch**
Microsoft	Azure services, AI research	**ONNX:** Co-developed for model interoperability; PyTorch support in Azure
Amazon	AWS AI services, research	**Amazon SageMaker:** Provides managed PyTorch environments
IBM	AI solutions, cloud services	Integration of PyTorch in IBM Watson and cloud offerings
NVIDIA	GPU computing, AI frameworks	Collaborates with PyTorch for optimized GPU performance

Startups and SMEs

Company	Use Cases	Notable Applications
Hugging Face	NLP models and tools	**Transformers Library:** PyTorch implementations of transformer models
Lightning AI	Simplifying PyTorch training	**PyTorch Lightning:** High-level interface for PyTorch
fast.ai	Education, simplifying DL	**fastai Library:** High-level API built on PyTorch
Exploding Gradient	AI consulting and solutions	Custom models developed using PyTorch
Databricks	Unified data analytics platform	Supports PyTorch for ML workloads

Industries and Sectors

Healthcare

Organization	Use Cases	Notable Projects
Philips Healthcare	Medical imaging, diagnostics	AI models for image analysis using PyTorch
Siemens Healthineers	Diagnostic imaging, healthcare solutions	Development of AI applications with PyTorch
Johns Hopkins University Applied Physics Laboratory	Medical research	Projects utilizing PyTorch for biomedical data analysis

Automotive

Organization	Use Cases	Notable Projects
Uber Advanced Technologies Group	Autonomous driving research	Perception models using PyTorch in self-driving cars
Lyft Level 5	Self-driving car development	Machine learning models developed with PyTorch
Toyota Research Institute	AI research in robotics and automation	Uses PyTorch for various AI models

Finance

Organization	Use Cases	Notable Projects
JPMorgan Chase	Risk modeling, fraud detection	Machine learning models utilizing PyTorch
Goldman Sachs	Data analytics, trading algorithms	AI research incorporating PyTorch models
FinTech Startups	Customer analytics, predictive modeling	AI solutions built using PyTorch

Retail and E-commerce

Organization	Use Cases	Notable Projects
Alibaba	Recommendation systems, search optimization	Large-scale models developed with PyTorch

Organization	Use Cases	Notable Projects
Shopify	Merchant analytics, demand forecasting	Machine learning models using PyTorch
Zalando	Fashion recommendations, image recognition	Visual search models built with PyTorch

Government and Non-Profit Organizations

Organization	Use Cases	Notable Projects
NASA	Data analysis, image processing	AI models for space exploration data using PyTorch
National Institutes of Health (NIH)	Medical research	Deep learning models for genomics with PyTorch
UNICEF	Humanitarian aid analytics	Projects using PyTorch for data-driven solutions
Open Climate Fix	Climate change research, environmental monitoring	Satellite image analysis using PyTorch

Cloud Service Providers

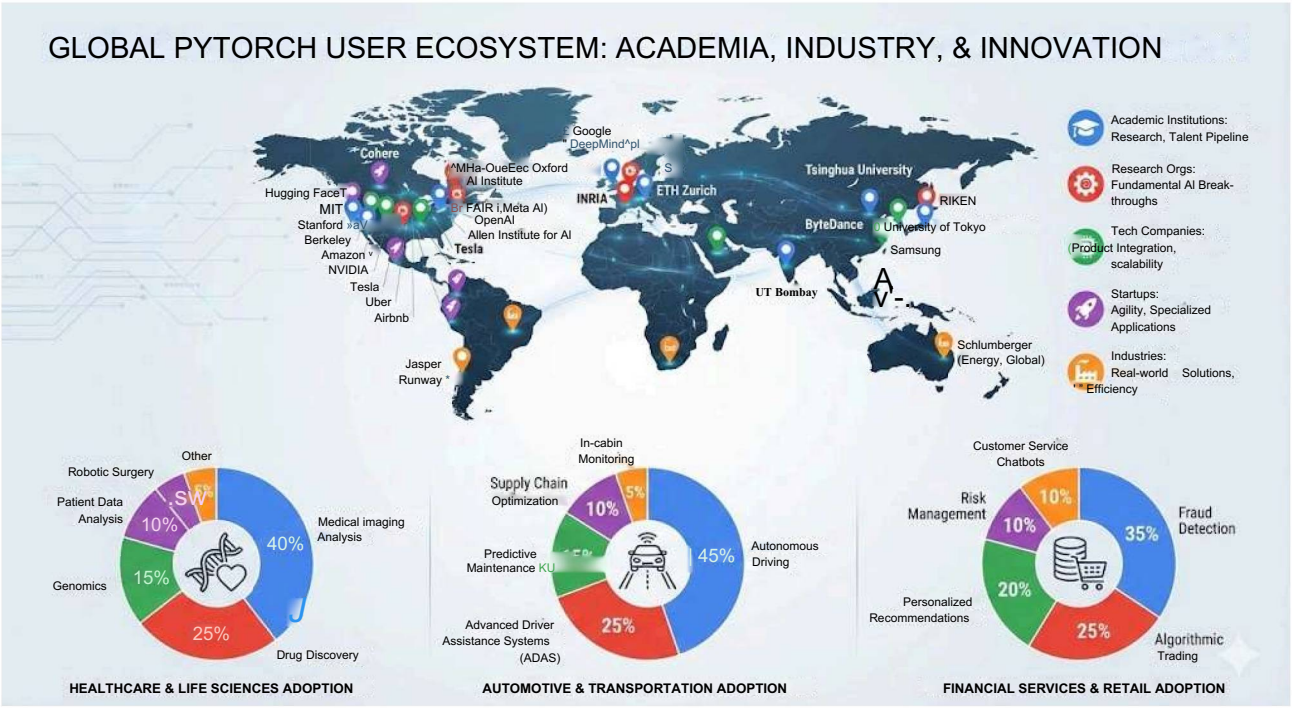
Provider	Use Cases	Notable Services
Amazon Web Services (AWS)	AI/ML services with PyTorch support	**Amazon SageMaker,** Deep Learning AMIs with PyTorch
Microsoft Azure	Machine learning platforms	**Azure Machine Learning** with PyTorch support
Google Cloud Platform (GCP)	AI Platform services	**AI Platform** supports PyTorch models
IBM Cloud	Cloud-based AI solutions	Supports PyTorch in Watson Machine Learning

Educational Platforms and Training Providers

Organization	Use Cases	Notable Courses or Resources
Coursera	Online courses teaching deep learning	**Deep Learning Specialization** includes PyTorch content
Udacity	Nanodegree programs with practical projects	**Deep Learning Nanodegree** utilizing PyTorch
fast.ai	Free online courses in deep learning	**Practical Deep Learning for Coders** using PyTorch
DataCamp	Interactive learning platform	Offers courses on PyTorch and deep learning

Notes

- **Disclaimer:** The information provided is based on publicly available data as of October 2023. Specific use cases may vary, and not all organizations disclose detailed information about their technology stacks.
- **General Adoption:** PyTorch is widely adopted in sectors like healthcare, automotive, finance, retail, and more, fortasks such as computer vision, NLP, recommendation systems, and predictive analytics.
- **Community and Ecosystem:** A strong community contributes to PyTorch's ecosystem, developing libraries and tools that enhance its functionality.



Key Takeaways:

- **Academia:** Used extensively for teaching and research in machine learning and AI.
- **Research:** Preferred by researchers for its ease of experimentation and prototyping.
- **Industry:** Adopted by tech giants, startups, and companies across various sectors.
- **Cloud Providers:** Supported by major cloud platforms for scalable training and deployment.
- **Education Platforms:** Featured in courses and materials for learning deep learning concepts.

By understanding who uses PyTorch and how it is applied across different fields, we gain insight into its versatility and the value it provides to the AI community.

PyTorch: The Journey of Deep Learning Innovation

From Research to Production - A Comprehensive Overview

Introduction

- Open-source deep learning library by Meta AI
- Combines Python's ease with Torch's power
- Dynamic computation graphs
- Research-friendly, production-ready
- Strong community support

Python + Torch = PyTorch

Python
- Simple syntax
- Rich ecosystem
- Large community

Torch
- High-performance
- Tensor operations
- GPU acceleration

Dynamic Computation Graph

/ Define-by-Run
- Graph built during runtime
- Flexible model architectures
- Easy debugging with Python tools
- Supports conditional logic & loops

PyTorch Evolution Timeline

v0.1 (Jan 2017)
- Dynamic graphs
- Python integration
- Research adoption
- NumPy compatible

v1.0 (Dec 2018)
- TorchScript
- Caffe2 merge
- Production ready
- JIT compilation

v1.x (2019-2022)
- Distributed training
- ONNX compatibility
- Quantization
- Ecosystem growth

v2.0 (Mar 2023)
- torch.compile
- Performance boost
- Hardware optimized
- Stable APIs

Future

Core Features

Tensor Computations
- N-dimensional arrays
- Mathematical operations
- Broadcasting support
- NumPy-like interface

GPU Acceleration
- CUDA support
- Easy device switching
- Multi-GPU training
- 40-50x speedup

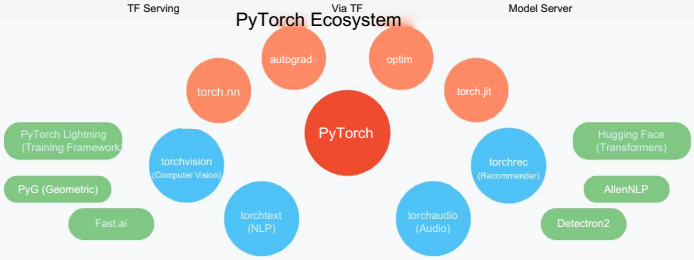
Automatic Differentiation
- Autograd engine
- Gradient tracking
- Custom gradients
- Backward propagation

Distributed Training
- Data parallelism
- Model parallelism
- Multi-node support
- DDP optimization

Tram e woric Comparison

PyTorch	TensorFlow	Keras	MXNet	CNTK
Dynamic	Static/Eager	High-level API	Static/Dynamic	Static
★★★★★	★★★☆☆		★★★☆☆	★★☆☆☆
TorchServe	TF Serving	Via TF	Model Server	Limited

PyTorch Ecosystem



Who Uses PyTorch

Academic
- MIT
- Stanford
- Carnegie Mellon
- Oxford
- UC Berkeley

Research
- OpenAI
- DeepMind
- Allen Institute AI
- Mila
- Google Brain

Tech Giants
- Meta (Creator)
- Microsoft
- Amazon
- IBM
- NVIDIA

Industries
- Healthcare (Philips)
- Automotive (Uber, Tesla)
- Finance (JPMorgan)
- Retail (Alibaba)
- Government (NASA)

Startups & Cloud
- Hugging Face
- Lightning AI
- AWS SageMaker
- Azure ML
- Google Cloud AI

Key Advantages

Flexibility
Dynamic graphs
Pythonic design

Performance
GPU optimization
torchxompHe

Community
Rich ecosystem
Active support

Production
TorchScript
Model serving

GPU PerformanceGalns

PyTorch Development Workflow

Core Modules Summary

torch
Tensor operations
Core data structures

torch.nn
Neural network layers
Loss functions

torch.autograd
Automatic differentiation
Gradient computation

torch.optim
Optimization algorithms
SGD, Adam, etc.

torch.utils.data
DataLoader
Dataset handling

torch.jit
TorchScript
JIT compilation

PyTorch: From Research to Production

Dynamic computation graphs • Pythonic interface • GPU acceleration • Rich ecosystem

Trusted by academia, research labs, tech giants, and industries worldwide

Key Takeaways

- / Leading deep learning framework with dynamic graphs
- •/ Seamless transition from research to production
- >/ Strong community and ecosystem support
- y Continuous evolution: v0.1 (2017) -» v2.0 (2023)
- •J Preferred by researchers for flexibility
- •/ Production-ready with TorchScript and TorchServe
- 7 Extensive domain libraries and third-party tools
- y Official website: pytorch.org

PyTorch 2.0+

CampusX

Tensors in PyTorch

Before Starting

1. **Boring but Useful**
   Some topics may feel tedious, but they form the backbone of deeper understanding.
2. **Knowledge of Deep Learning**
   Basic familiarity with neural networks, tensors, and training loops is assumed.
3. **Very Similar to NumPy**
   Many concepts align closely with NumPy syntax and operations, making the transition smoother.
4. **Lecture Flow**
   See the diagram below for an overview of how the content will progress.
5. **Close to PyTorch Official Documentation**
   The approach is based on the structure and philosophy of the official PyTorch docs.
6. **Watch Like a Lecture**
   Follow along in real-time as if attending a live lecture — pause, rewind, and take notes.

Code

- https://colab.research.google.com/drive/1LgMnRIHSE9NjI2G1YRGSa8wlQqfSf8W8

Detailed Notes: What are Tensors

1. Introduction to Tensors

Definition

- **Tensor:** A specialized multi-dimensional array designed for mathematical and computational efficiency
- Simple explanation: What an array does in 1-dimension, a tensor does in n-dimensions
- It's a generalization of arrays that can span multiple directions

Key Concept: Dimensions

- **Dimension** of a tensor refers to the number of directions in which the tensor spans or extends
- Each dimension represents a different axis along which data is organized
- Understanding dimensions is crucial for working with tensors in deep learning

2. Types of Tensors with Examples

O-Dimensional Tensor (Scalar)

- **Definition:** A single number with no directional extension
- **Examples:**
  - Any single number: 2, 7, 5, 3.14
- **Deep Learning Application:**
  - **Loss Value:** After a forward pass in neural network training, the loss function computes a single scalar value
  - This value indicates the difference between predicted and actual outputs
  - Always results in a single number

1-Dimensional Tensor (Vector)

- **Definition:** Data extending in one direction

- **Programming equivalent:** Arrays or vectors
- **Examples:**
  - Simple array: [1, 2, 3, 4, 5]
- **Deep Learning Application:**
  - **Word Embeddings:** Converting words into numerical representations
  - Example: The word "Hello" -» embedding vector [0.12, -0.84, 0.33,...]
  - Used in NLP with models like Word2Vec and GloVe

2-Dimensional Tensor (Matrix)

- **Definition:** Data extending in two directions (rows and columns)
- **Structure:** Grid-like arrangement
- **Deep Learning Application:**
  - **Grayscale Images:** Each pixel position contains a single intensity value
  - Example from MNIST dataset: Handwritten digits

3-Dimensional Tensor

- **Definition:** Data extending in three directions
- **Deep Learning Application:**
  - **RGB/Colored Images:**
    - Three channels: Red, Green, Blue
    - Each channel is a 2D matrix
    - Shape representation: [Height, Width, Channels]
    - Example: [256, 256, 3] for a 256x256 RGB image

4-Dimensional Tensor

- **Definition:** Data extending in four directions
- **Deep Learning Application:**
  - **Batches of RGB Images:**
    - Used when training neural networks with multiple images simultaneously
    - Never send single images; typically use batches (e.g., 32 images)
    - Shape: [Batch_size, Height, Width, Channels]
    - Example: [32, 128, 128, 3] for 32 RGB images of size 128x128

5-Dimensional Tensor

- **Definition:** Data extending in five directions

- **Deep Learning Application:**
  - **Video Data:**
    - Videos consist of multiple frames
    - Each frame is essentially an RGB image
    - Captures temporal sequences (snapshots at different times)
    - Shape: [Batch_size, Frames, Height, Width, Channels]
    - Example: [10, 16, 64, 64, 3] for 10 video clips with 16 frames each

3. Practical Understanding

Video Data Breakdown

- A video is essentially a combination of multiple images
- When played rapidly, individual frames create the illusion of motion
- Example sequence: Object movement captured frame by frame
  - Frame 1: Object at position A
  - Frame 2: Object moving to position B
  - Frame 3: Object at position B
  - And so on...

Benefits of Using Tensors

- **Flexibility:** Can represent data of any dimensionality
- **Efficiency:** Optimized for mathematical and computational operations
- **Scalability:** From simple scalars to complex multi-dimensional data
- Arrays work in 1D, matrices in 2D, but tensors work in n-dimensions

4. Summary Table

Dimension	Name	Shape Example	Real-World Application
OD	Scalar	Single value	Loss value in neural networks
1D	Vector	[n]	Word embeddings
2D	Matrix	[m, n]	Grayscale images
3D	3D Tensor	[h, w, c]	RGB images
4D	4D Tensor	[b, h, w, c]	Batches of images

Dimension	Name	Shape Example	Real-World Application
5D	5D Tensor	[b, f, h, w, c]	Video data batches

Key Takeaways

1. Tensors are fundamental data structures in deep learning
2. Understanding dimensions is crucial for tensor manipulation
3. Higher dimensions allow representation of more complex data
4. Most deep learning operations involve tensor transformations
5. Batch processing (4D tensors) is standard practice in neural network training

What are Tensors?

O-Dimensional Tensor (Scalar)

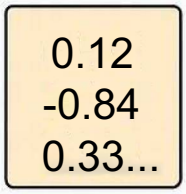
3.14

A single number with no directional extension

Loss value in neural networks

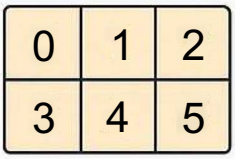
1-Dimensional Tensor (Vector)

Data extending in one direction



Word embeddings

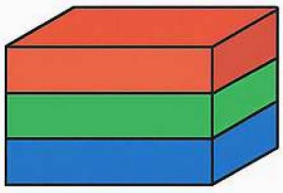
2-Dimensional Tensor (Matrix)



Data extending in two directions (rows and columns)

Grayscale images

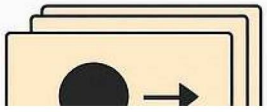
3-Dimensional Tensor



Data extending in three directions

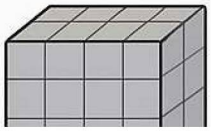
RGB images

5-Dimensional Tensor



Data extending in

£_____1:___ 4.:____



CampusX

Why Are Tensors So Useful in Deep Learning?

Reason 1: Efficient Mathematical Operations

- **Core Benefit:** Tensors enable easy performance of common mathematical operations essential in neural networks
- **Key Operations:**
  - Addition
  - Multiplication
  - Dot products
  - Element-wise operations
- **Significance:** These are the fundamental operations repeatedly performed in neural networks
- **Example:** Matrix addition, matrix multiplication during forward and backward passes

Reason 2: Natural Representation of Real-World Data

Tensors can effectively map all types of real-world data used in deep learning:

Data Modalities

- **Images:**
  - Essentially matrices of numbers
  - 2D for grayscale, 3D for RGB
- **Videos:**
  - Sequences of matrices (frames)
  - 4D or 5D tensors depending on batching
- **Audio:**
  - Can be represented as spectrograms (2D tensors)
  - Raw audio as 1D tensors
- **Text:**
  - Represented as vectors (1D tensors)
  - Word embeddings, token sequences

Key Insight: All these data types where deep learning excels can be naturally represented using tensors

Reason 3: Hardware Acceleration & Parallelization

CPU vs GPU Computing

Sequential Processing (CPU - Single Core):

- Example: Adding two 2x2 matrices

Matrix A: [3, 4] Matrix B: [2, 5]
[5, 6] [7, 7]

- CPU performs sequentially:
  i. First: 3+2 = 5
  ii. Then: 4+5 = 9
  iii. Then: 5+7 = 12
  iv. Finally: 6+7 = 13

Parallel Processing (GPU):

- Multiple cores handle operations simultaneously
- Core 1: 3+2
- Core 2: 4+5
- Core 3: 5+7
- Core 4: 6+7
- All operations complete in parallel

Performance Benefits

- **Speed Boost:** Almost 100x improvement for large-scale operations
- **Scale:** Crucial when dealing with:
  - Millions of images in datasets
  - Large video data
  - Complex neural network architectures
- **Hardware Support:**
  - GPUs (Graphics Processing Units)
  - TPUs (Tensor Processing Units)
  - Optimized for tensor operations

Where Are Tensors Used in Deep Learning?

1. Data Storage

- **Purpose:** Store and represent training data
- **Applications:**
    - **Computer Vision:** Images stored as 3D/4D tensors
    - NLP: Text data as embedding tensors
    - **Audio Processing:** Sound waves as tensors
    - **Any Data Type:** All training data is converted to tensor format

2. Model Parameters (Weights and Biases)

- **Weights:**
    - Connection strengths between neurons
    - Stored as 2D tensors (weight matrices)
    - Example: W in the equation y = Wx + b
- **Biases:**
    - Each neuron has its own bias term
    - Stored as 1D tensors (bias vectors)
    - Example: b in the equation y = Wx + b

3. Mathematical Operations

- **Matrix Operations:**
    - Linear transformations: Wx + b
    - Activation functions applied to tensors
    - Batch normalization operations
- **Element-wise Operations:**
    - ReLU, Sigmoid, Tanh activations
    - Dropout operations
    - Pooling operations in CNNs

4. Training Process

Forward Pass

- **Process Flow:**
    i. Input data (tensor) enters the network
    ii. Sequential operations: W-iX + bi -» activation -► $W_2x + b_2$ -*...
    iii. Final output y^(prediction tensor)
- **Continuous Tensor Operations:** Each layer transforms input tensors to output tensors

Backward Pass (Backpropagation)

- **Gradient Calculation:**
  - o Gradients are also tensors
  - o Computed with respect to loss
  - o Flow backward through the network
- **Weight Updates:**
  - o Gradient tensors used to update weight tensors
  - o Learning rate applied to gradient tensors

Complete Training Pipeline

cP

Input Data (Tensor)
1
Forward Pass (Tensor Operations)
1
Output/Predictions (Tensor)
1
Loss Calculation (Scalar Tensor)
1
Backward Pass (Gradient Tensors)
1
Weight Updates (Tensor Operations)

Practical Implications

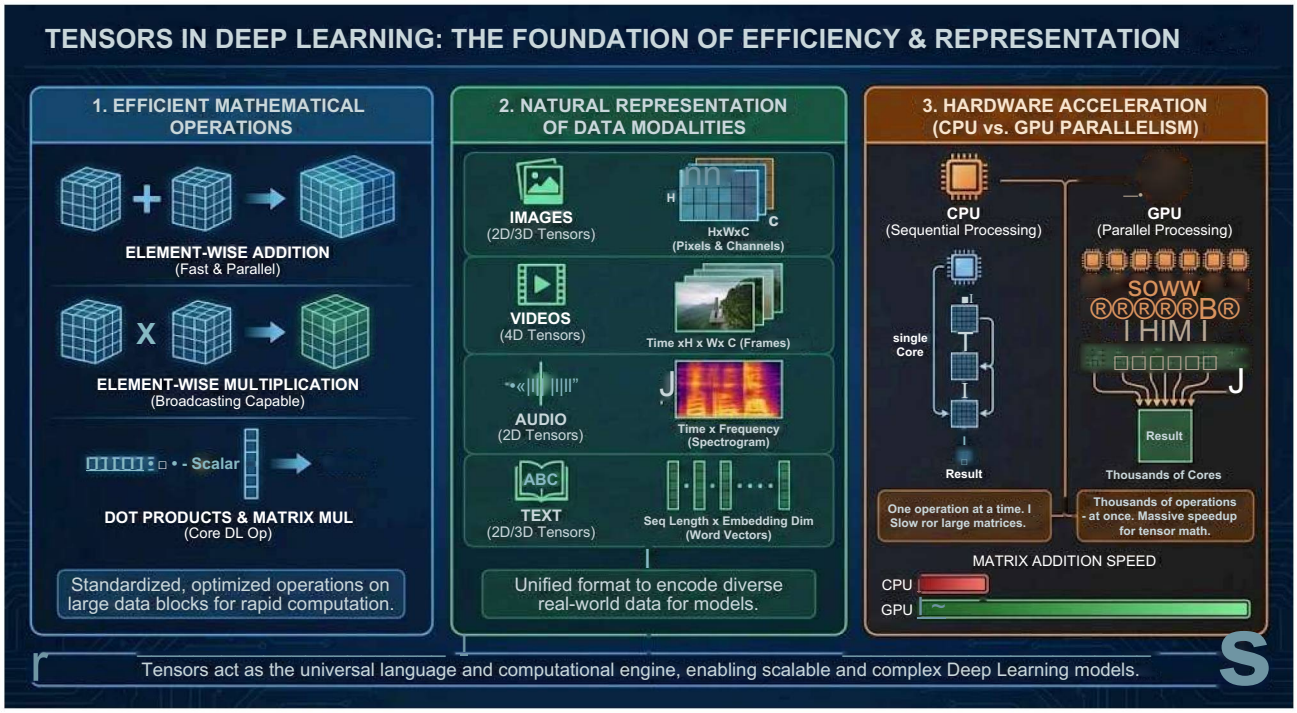
Why TensorFlow is Named After Tensors

- The entire framework is built around efficient tensor operations
- "Flow" represents the flow of tensors through computational graphs
- Highlights the fundamental importance of tensors in deep learning

Key Libraries Supporting Tensor Operations

- **TensorFlow:** Google's deep learning framework
- **PyTorch:** Facebook's dynamic computational graph framework
- **NumPy:** Foundation for numerical computing in Python
- JAX: Google's high-performance ML research framework

Summary Table: Tensor Applications

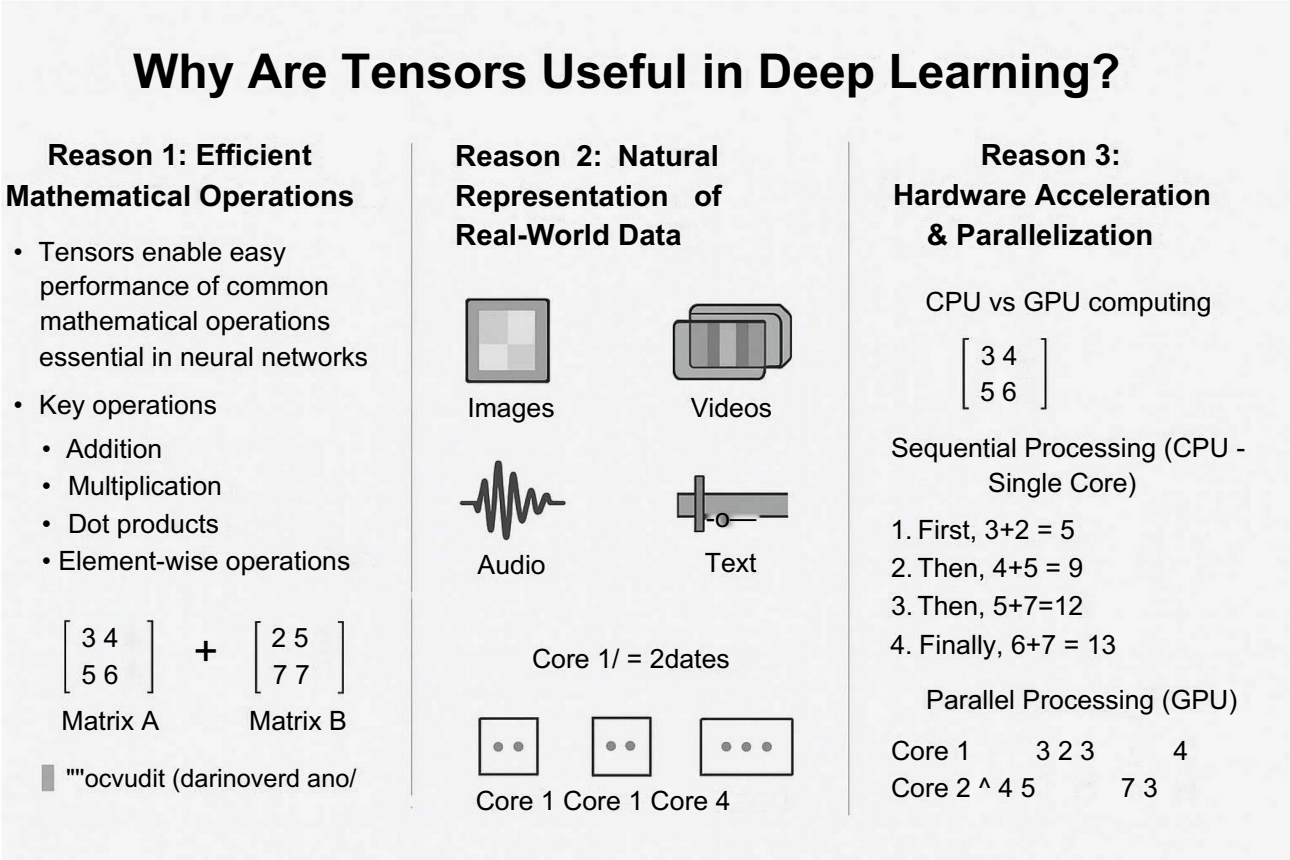


Application Area	Tensor Usage	Example
Data Storage	Input/Output representation	Images as 4D tensors [batch, height, width, channels]
Model Parameters	Weights & Biases	Weight matrix W as 2D tensor
Computations	Mathematical operations	Matrix multiplication, convolutions
Forward Pass	Data transformation	Layer-by-layer tensor transformations
Backward Pass	Gradient computation	Gradient tensors for optimization
Optimization	Parameter updates	Weight tensors updated using gradient tensors

Key Takeaways

1. **Efficiency:** Tensors make deep learning computationally feasible through parallelization
2. **Universality:** All data types in deep learning can be represented as tensors

3. **Hardware Optimization:** Modern hardware (GPUs/TPUs) is specifically designed for tensor operations
4. **Complete Coverage:** Tensors handle the entire deep learning pipeline from data to training
5. **Performance Gains:** 100x speed improvements possible through parallel tensor operations on GPUs



Colab notebook :- https://colab.research.qooqle.eom/drive/1_LqMnRIHSE9Njl2G1YRGSa8wlQqfSf8W8?usp=sharing

Detailed Notes: PyTorch Tensor Operations - Practical Implementation

Date: November 20, 2024

1. Environment Setup & GPU Availability

Version Check

```
import torch
print(torch._____version___ )
# Output: 2.5.1+cul21
```

cP

- Shows PyTorch version with CUDA support (cu121 = CUDA 12.1)

GPU Availability Check

```
if torch.cuda.is_available():
        print("GPU is available!")
        print(f"Using          GPU:          {torch.cuda.get_device_name(0)}")
else:
        print("GPU not available. Using CPU.")
# Output: GPU is available! Using GPU: Tesla T4
```

cP

- Confirms GPU availability for accelerated computation
- Tesla T4 GPU detected in this environment

2. Creating Tensors - Multiple Methods

Method 1: Empty Tensor

```
a = torch.empty(2j3)
type(a) # Output: torch.Tensor
```

cP

- Creates uninitialized tensor with random values
- Memory allocated but not initialized to specific values

Method 2: Zeros Tensor

```
torch.zeros(2j3)
# Output: tensor([[0., 0., 0.],
#                 [0., 0., 0.]])
```

cP

- Creates tensor filled with zeros
- Default dtype is float32

Method 3: Ones Tensor

```
torch.ones(2,3)
# Output: tensor([[l., 1., 1.],
#                 [1., 1., 1.]])
```

[P

- Creates tensor filled with ones

Method 4: Random Tensor

```
torch.rand(2,3)
# Output: tensor([[0.1192, 0.4828, 0.9105],
#                 [0.4266, 0.4203, 0.2719]])
```

tP

- Creates tensor with random values between 0 and 1
- Uniform distribution

Using Seeds for Reproducibility

```
torch.manual_seed(100)
torch.rand(2,3)
# Output: tensor([[0.1117, 0.8158, 0.2626],
#                 [0.4839, 0.6765, 0.7539]])

torch.manual_seed(100) # Same seed
torch.rand(2,3)
# Output: tensor([[0.1117, 0.8158, 0.2626], # Same values
#                 [0.4839, 0.6765, 0.7539]])
```

[P

- Setting seed ensures reproducible random numbers

Method 5: From List/Array

```
torch.tensor([[1,2,3],[4,5,6]])
# Output: tensor([[l, 2, 3],
#                 [4, 5, 6]])
```

[P

- Creates tensor from Python lists

Other Creation Methods

Arange

```
torch.arange(0,10,2)
# Output: tensor([0, 2, 4, 6, 8])
```

[P

- Creates 1D tensor with evenly spaced values
- Parameters: start, end (exclusive), step

Linspace

```
torch.linspace(0,10,10)
# Output: tensor([0.0000, 1.1111, 2.2222, 3.3333, 4.4444,
#                 5.5556, 6.6667, 7.7778, 8.8889, 10.0000])
```

[P

- Creates 1D tensor with linearly spaced values
- Parameters: start, end (inclusive), number of steps

Eye (Identity Matrix)

```
torch.eye(5)
# Output: tensor([[l., 0., 0., 0., 0.],
#                 [0.,  1.,  0.,  0.,  0.],
#                 [0.,  0.,  1.,  0.,  0.],
#                 [0.,  0.,  0.,  1.,  0.],
#                 [0.,  0.,  0.,  0.,  1.]])
```

[P

• Creates identity matrix (diagonal ones, rest zeros)

Full

```
torch.full((3, 3), 5)
# Output: tensor([[5, 5, 5],
#                 [5, 5, 5],
#                 [5, 5, 5]])
```

[P

• Creates tensor filled with specified value

3. Tensor Shapes & Shape-Based Creation

Getting Shape

```
x = torch.tensor([[1,2,3],[4,5,6]])
x.shape # Output: torch.Size([2, 3])
```

[P

- Returns dimensions as torch.Size object

Creating Tensors with Same Shape

Empty Like

```
torch.empty_like(x)
# Creates uninitialized tensor with same shape as x
```

tP

Zeros Like

```
torch.zeros_like(x)
# Output: tensor([[0, 0, 0],
#                 [0, 0, 0]])
```

[P

Ones Like

```
torch.ones_like(x)
# Output: tensor([[l, 1, 1],
#                 [1, 1, 1]])
```

tP

Random Like with Dtype

```
torch.rand_like(x, dtype=torch.float32)
# Output: tensor([[0.4076, 0.6037, 0.9820],
#                 [0.0117, 0.3891, 0.5056]])
```

[P

- Must specify dtype when original is integer

4. Tensor Data Types

Checking Data Type

```
x.dtype # Output: torch.int64
```

tP

Setting Data Type at Creation

tP

```
# Integer type specification
torch.tensor([1.0,2.0, 3.0], dtype=torch.int32)
# Output: tensor([l, 2, 3], dtype=torch.int32)

# Float type specification
torch.tensor([1,2,3], dtype=torch.float64)
# Output: tensor([l., 2., 3.], dtype=torch.float64)
```

Converting Data Type

cP

```
x.to(torch.float32)
# Output: tensor([[l., 2., 3.],
#                 [4.> 5., 6.]])
```

Complete Data Types Reference Table

Data Type	Dtype	Description	Use Case
32-bit Float	torch.float32	Standard floating-point	Most deep learning tasks
64-bit Float	torch.float64	Double-precision	High-precision numerical tasks
16-bit Float	torch.floatl6	Half-precision	Mixed-precision training
BFloat16	torch.bfloatl6	Brain floating-point	TPU optimization
8-bit Float	torch.floats	Ultra-low precision	Experimental/memory-constrained
8-bit Integer	torch.int8	Signed 8-bit	Quantized models
16-bit Integer	torch.intl6	Signed 16-bit	Intermediate precision
32-bit Integer	torch.int32	Standard signed	Indexing, general purpose
64-bit Integer	torch.int64	Long integer	Large indexing arrays

Data Type	Dtype	Description	Use Case
8-bit Unsigned	torch.uint8	Unsigned 8-bit	Image data (0-255)
Boolean	torch.bool	True/False	Masks, logical operations
Complex 64	torch.complex64	32-bit real + imaginary	Signal processing
Complex 128	torch.complexl28	64-bit real + imaginary	High-precision complex
Quantized Int	torch.qint8	Quantized signed	Efficient inference
Quantized Unsigned	torch.quint8	Quantized unsigned	Quantized image tasks

5. Mathematical Operations

5.1 Scalar Operations

Operations between tensor and scalar value:

```
x = torch.rand(2,2)
# Output: tensor([[0.9594, 0.1334],
#                 [0.8591, 0.6639]])

#  Addition: x + 2
#  Subtraction: x - 2
#  Multiplication: x * 3
#  Division: x / 3
#  Integer Division: (x * 100)//3
#  Modulo: ((x * 100)//3)%2
#  Power: x**2
#  Output: tensor([[0.9205, 0.0178],
#                  [0.7380, 0.4408]])
```

cP

5.2 Element-wise Operations

Operations between tensors of same shape:

```
a = torch.rand(2,3)
b = torch.rand(2,3)

#  Addition: a + b
#  Subtraction: a - b
#  Multiplication: a * b
#  Division: a / b
#  Power: a ** b
#  Modulo: a % b
```

Unary Operations

```
c = torch.tensor([l, -2, 3, -4])

# Absolute value
torch.abs(c) # Output: tensor([l, 2, 3, 4])

# Negation
torch.neg(c) # Output: tensor([-l, 2, -3, 4])
```

Rounding Operations

```
d = torch.tensor([1.9, 2.3, 3.7, 4.4])

# Round to nearest
torch.round(d) # Output: tensor([2., 2., 4., 4.])

# Ceiling (round up)
torch.ceil(d) # Output: tensor([2., 3., 4., 5.])

# Floor (round down)
torch.floor(d) # Output: tensor([l., 2., 3., 4.])

# Clamp (limit values to range)
torch.clamp(d, min=2, max=3)
# Output: tensor([2.0000, 2.3000, 3.0000, 3.0000])
```

5.3 Reduction Operations

Operations that reduce tensor dimensions:

```
e = torch.randint(size=(2,3), low=0, high=10, dtype=torch.float32)
# Output: tensor([[l., 2., 9.],
```

```
#                           [7.j 6., 5.]])

#  Sum
torch.sum(e) # Total sum: 30
torch.sum(e, dim=0) # Sum along columns: [8, 8, 14]
torch.sum(e, dim=l) # Sum along rows: [12, 18]

#  Mean
torch.mean(e) # Overall mean: 5.0
torch.mean(e, dim=0) # Column means: [4., 4., 7.]

#  Median
torch.median(e) # Output: tensor(5.)

#  Max and Min
torch.max(e) # Output: tensor(9.)
torch.min(e) # Output: tensor(l.)

#  Product
torch.prod(e) # Output: tensor(3780.)

#  Standard Deviation
torch.std(e) # Output: tensor(3.0332)

#  Variance
torch.var(e) # Output: tensor(9.2000)

#  Argmax (index of maximum)
torch.argmax(e) # Output: tensor(2)

#  Argmin (index of minimum)
torch.argmin(e) # Output: tensor(0)
```

5.4 Matrix Operations

```
f = torch.randint(size=(2,3), low=0, high=10)
g = torch.randint(size=(3,2), low=0, high=10)

#  Matrix Multiplication
torch.matmul(f, g)
# Output: tensor([[63, 73],
#                           [79, 88]])

#  Dot Product (ID vectors only)
vectorl = torch.tensor([1, 2])
vector2 = torch.tensor([3, 4])
torch.dot(vectorl, vector2) # Output: tensor(ll)
```

cP

```
# Transpose
torch.transpose^, 0, 1)
# Swaps dimensions 0 and 1

# Determinant (square matrices only)
h = torch.randint(size=(3,3), low=0, high=10, dtype=torch.float32)
torch.det(h) # Output: tensor(224.)

# Inverse (square matrices only)
torch.inverse(h)
# Output: tensor([[ 0.1429, -0.0357, 0.0000],
#                 [ 0.1071,  0.0982, -0.2500],
#                 [-0.1786,  0.0446,  0.2500]])
```

5.5 Comparison Operations

```
i = torch.randint(size=(2,3), low=0, high=10)
j = torch.randint(size=(2,3), low=0, high=10)

#  Greater than: i > j
#  Less than: i < j
#  Equal to: i == j
#  Not equal to: i != j
#  Greater than or equal: i >= j
#  Less than or equal: i <= j
```

Returns boolean tensors

5.6 Special Functions

```
k = torch.randint(size=(2,3), low=0, high=10, dtype=torch.float32)

# Logarithm
torch.log(k)

# Exponential
torch.exp(k)

# Square Root
torch.sqrt(k)

# Sigmoid (squashes to 0-1)
torch.sigmoid(k)
```

```
# Softmax (probability distribution)
torch.softmax(k, dim=0)

# ReLU (Rectified Linear Unit)
torch.relu(k) # max(0, x)
```

6. Inplace Operations

Inplace operations modify the original tensor and are denoted with underscore suffix:

```
m = torch.rand(2,3)
n = torch.rand(2,3)

#  Inplace addition
m.add_(n) # Modifies m directly
# m is now changed, n remains unchanged

#  Inplace ReLU
m.relu_() # Modifies m directly
```

Key Points:

- Inplace operations save memory
- Denoted by trailing underscore (_)
- Original tensor is modified
- Cannot be used with autograd graph

7. Copying Tensors

Shallow Copy (Reference)

```
a = torch.rand(2,3)
b = a # b is reference to a

a[0][0] = 0
# Both a and b are modified!

id(a) # 134838567488544
id(b) # 134838567488544 (same ID)
```

Deep Copy (Clone)

```
b = a.clone() # Creates independent copy

a[0][0] = 10
#  Only a is modified., b remains unchanged

id(a) # 134838567488544
id(b) # 134838569581552 (different ID)
```

Important Distinctions:

- b = a : Creates reference (shallow copy)
- b = a.cloneQ : Creates independent copy (deep copy)
- Use clone() when you need independent tensors

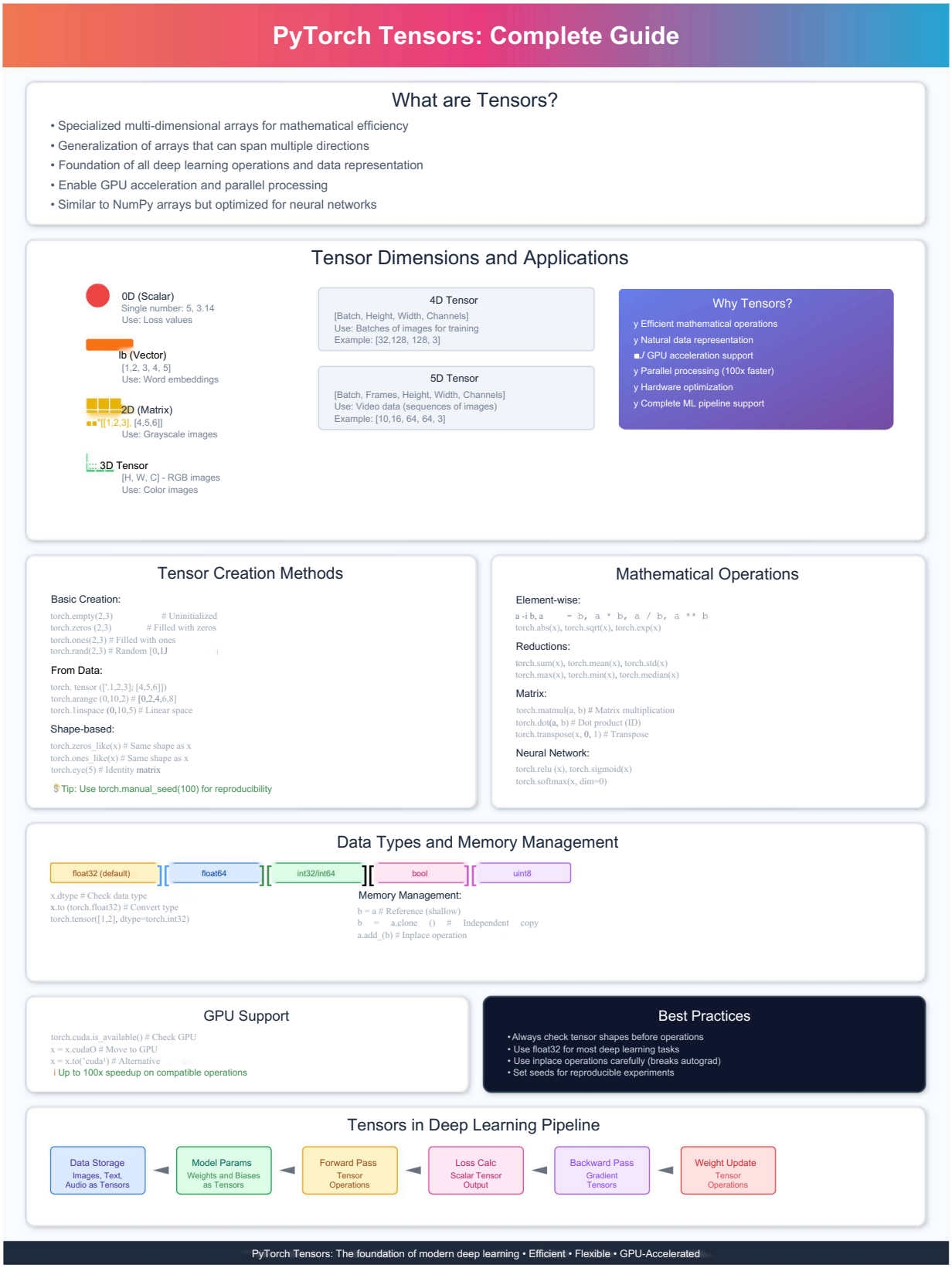
Key Takeaways

1. **Tensor Creation:** Multiple methods available for different use cases
2. **Data Types:** Choose appropriate dtype for memory and precision requirements
3. **Operations:** Rich set of mathematical operations for all deep learning needs
4. **GPU Support:** Seamless GPU acceleration when available
5. **Memory Management:** Understand difference between reference and clone
6. **Inplace Operations:** Use with caution, saves memory but modifies original
7. **Broadcasting:** PyTorch automatically handles compatible shapes
8. **Reproducibility:** Use manual_seed for consistent results

Best Practices

1. Always check tensor shapes before operations
2. Use appropriate data types (float32 for most DL tasks)
3. Leverage GPU when available for large computations
4. Use inplace operations carefully (breaks autograd)
5. Clone tensors when independent copies needed

6. Set seeds for reproducible experiments



PyTorch Autograd Implementation Guide

Code

- htt ps://colab.research. g oog le.com/drive/W22HrY0cW-DNMj 3vG2n6-OjmRXtE8lh2

Understanding the Example Chain Rule Applications ■.

Let's break down the three examples shown in the images to understand PyTorch's autograd implementation requirements.

1. Basic Derivative Computation

Function:

$$y = x^2$$

Derivative:

$$\frac{dy}{dx} = 2x$$

Numeric evaluation:

- When $x = 2$: $\frac{dy}{dx} = 2(2) = 4$

• When $x = 3$: $\frac{dy}{dx} = 2(3) = 6$

Python (example) — quick check:

```
import math
def dy_dx(x): return 2*x
print(dy_dx(2)) # 4
print(dy_dx(3)) # 6
```

[P

2. Two-Layer Chain Rule

Given:

$$y = x^2, \qquad z = \sin(y)$$

Apply the chain rule:

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx} = \cos(y) \cdot 2x = 2x\cos(x^2)$$

Numeric evaluation:

• When x = 2:

$$\frac{dz}{dx}\Big|_{x=2} = 2 \cdot 2 \cdot \cos(2^2) = 4\cos(4) \approx -2.6145744834544478$$

• When $x = 3$:

$$\frac{dz}{dx}\Big|_{x=3} = 2 \cdot 3 \cdot \cos(3^2) = 6\cos(9) \approx -5.4667815713080616$$

Python (example):

```
import math
def dz_dx(x): return 2*x*math.cos(x**2)
print(dz_dx(2)) # approx -2.6145744834544478
print(dz_dx(3)) # approx -5.4667815713080616
```

tP

3. Three-Layer Chain Rule

Given:

$$y = x^2, \qquad z = \sin(y), \qquad u = e^z$$

Use the chain rule across three layers:

$$\frac{du}{dx} = \frac{du}{dz}\frac{dz}{dy}\frac{dy}{dx}$$

$$Tx = Tz'^{\wedge}{}'Tx = eZ'^{\cdot}\cos W'^{2}x$$

Substitute $y = x^2$, $z = \sin(x^2)$:

$$\frac{du}{dx} = 2\%\cos(x^2), e^{\sin x^2}$$

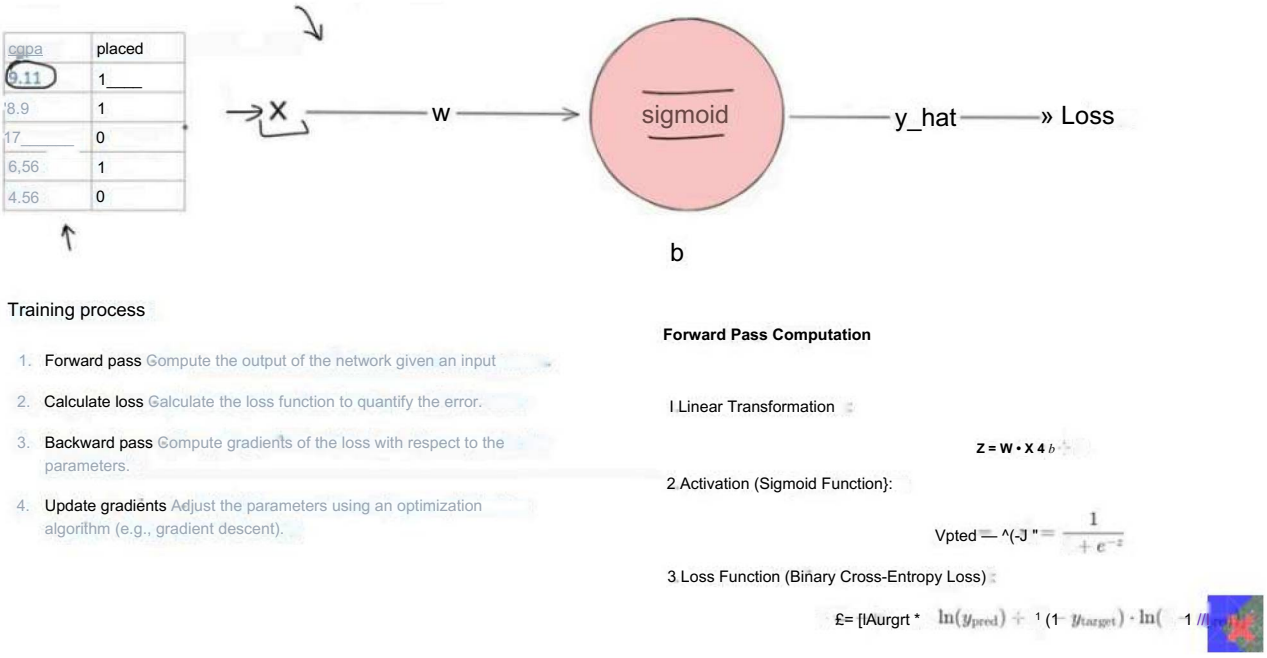
Numeric evaluation (high precision):

- When $x = 2$:
  - $y = 4$
  - $z = \sin(4) \sim -0.7568024953079282$
  - $e^z \approx 46916418587400077$
  - $\cos(y) = \cos(4) \sim -0.6536436208636119$
  - $\left.\frac{du}{dx}\right|_{x=2} \approx -1.2266647089368421$
- When $x = 3$:
  - $y = 9$
  - $z = \sin(9) * 0.4121184852417566$
  - $e^z \sim 4.5100133400254603$
  - $\cos(y) = \cos(9) \sim -0.9111302618846769$
  - $\left.\frac{du}{dx}\right|_{x=3} \approx -8.25491309968052$

Python (example) — compute exactly the same numerics:

```
import math
def du_dx(x):
    y = x**2
    z = math.sin(y)
    return 2*x * math.cos(y) * math.exp(z)

print(du_dx(2)) # approx -1.2266647089368421
print(du_dx(3)) # approx -8.25491309968052
```

cP



Neural Network Training Implementation: Forward & Backward Pass 🧠

Data Overview 📊

The input data shows student performance metrics:

```
X = [CGPA] # Input feature
y = [placed] # Binary target (0 or 1)
```

Network Architecture M

Components

1. **Input Layer** (X)
2. **Linear Transform** (z = w x + b)
3. **Sigmoid Activation** (ŷ = o(z))
4. **Binary Cross-Entropy Loss**

Let's implement this neural network with automatic differentiation:

Implementation Details

1. Forward Pass

- **Linear Transformation: z —** *w * x + b*

```
z = self.w * x + self.b
```

&

- **Sigmoid Activation: <T(Z) = 1 / (1 +** $e^{(}$ — Z))

```
y_pred = 1 / (1 + torch.exp(-z))
```

LP

2. Loss Computation

- **Binary Cross-Entropy:**

```
L = -(ylog(y) + (l-y)-log(l-y))
```

cP

3. Backward Pass 0

- Automatic gradient computation through PyTorch autograd
- Parameter updates using gradient descent:

```
w = w - lr * Vw
b = b - lr * Vb
```

[P

🧠 Neural Network Training (Binary Classification)

📊 Dataset Example

CGPA	Placed (Target)
9.11	1
8.90	1
7.00	0
6.56	1

CGPA	Placed (Target)
4.56	0

Training Process

1. Forward Pass

Compute the output of the network given an input:

$$z = w \cdot x + b$$

$$\text{pred } y = a(z) = \frac{1}{1 + e^{-z}}$$

2. Calculate Loss

We use the **Binary Cross-Entropy Loss:**

$$L = -\left[y_{\text{target}} \cdot \ln(\hat{y}_{\text{pred}}) + (1 - y_{\text{target}}) \cdot \ln(1 - \hat{y}_{\text{pred}}) \right]$$

3. Backward Pass

Compute gradients of the loss with respect to the parameters (w) and (b):

$$\frac{dL}{dw} = (\hat{y}_{\text{pred}} - y_{\text{target}}) \cdot x$$

$$\frac{dL}{db} = (\hat{y}_{\text{pred}} - y_{\text{target}})$$

4. Update Parameters

Using **Gradient Descent:**

$$w := w - \alpha \cdot \frac{dL}{dw}$$

$$b := b - \alpha \cdot \frac{dL}{db}$$

Where (\alpha) is the **learning rate.**

Forward Pass Computation Steps

1. **Linear Transformation**

$$z = w \cdot x + b$$

3. **Activation Function (Sigmoid)**

$$\text{pred} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

5. **Loss Calculation (Binary Cross-Entropy)**

$$L = -\left[\text{target} \cdot \log(\text{pred}) + (1 - \text{target}) \cdot \log(1 - \text{pred}) \right]$$

Key Features

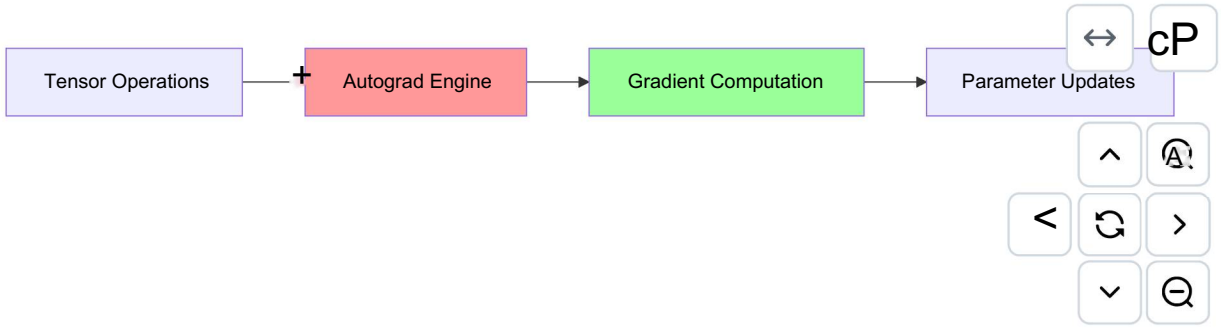
1. **Automatic Differentiation:** Uses PyTorch's autograd
2. **Numerical Stability:** Includes epsilon in log computations
3. **Flexible Learning:** Adjustable learning rate
4. **Progress Monitoring:** Loss tracking during training

Understanding PyTorch Autograd: A Comprehensive Guide

What is Autograd?

Autograd is a foundational component of PyTorch that provides **automatic differentiation** for tensor operations. It's the engine that powers neural network training by automatically computing gradients.

Core Functionality Map



Examples Breakdown

1. Simple Derivative: $y = x^2$

```
# Computing dy/dx
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
y.backward() # dy/dx = 2x = 4 when x = 2
```

2. Chain Rule: $y = x^2$, $z = \sin(y)$

```
# Computing dz/dx using chain rule
x = torch.tensor(2.0, requires_grad=True)
y = x ** 2
z = torch.sin(y)
z.backward() # dz/dx = cos(x²) * 2x
```

3. Neural Network Example

```
class SimpleNN(nn.Module):
  def _____ init____ (self):
        super().____ init____()
        self.linear = nn.Linear(lj 1)
        self.sigmoid = nn.Sigmoid()

  def forward(selfj x):
        x = self.linear(x)          # w*x + b
        x = self.sigmoid(x)         # o(w*x + b)
        return x
```

How Autograd Works 🔧

1. Computational Graph Building

Phase	Action	Description
Forward	Graph Creation	Creates DAG of operations
Operation	Gradient Function	Stores gradient formula
Backward	Chain Rule	Applies chain rule automatically

2. Key Components

- **requires_grad:** Tracks operations for gradient computation
- **grad_fn:** Stores gradient computation function
- **backwardf):** Triggers gradient computation

Common Operations with Gradients x

cP

```
#  Basic operations with gradient tracking
x = torch.tensor([l., 2.]^ requires_grad=True)
y = torch.tensor([3., 4.], requires_grad=True)

#  Operations maintain gradient information
z = x * y                          #  Product
w = torch.sum(z)                   #  Sum
w.backward()                         #  Compute gradients
```

Best Practices 💡

1. **Memory Efficiency**

   o Use with torch.no_grad(): for inference

   o Clear gradients with optimizer.zero_grad()

2. **Debugging Tips**

   - Check grad_fn attributes
   - Use retain_graph=True for multiple backwards

3. **Performance Optimization**
    - Avoid creating unnecessary computation graphs
    - Use detach() to stop gradient tracking when needed

Real-world Applications ©

1. **Training Deep Neural Networks**
2. **Optimization Problems**
3. **Gradient-based Learning**

Neural Network Loss Computation: From Scratch vs PyTorch

Overview: Two Implementation Approaches @

We'll implement the same neural network model using:

1. Pure Python/NumPy (from scratch)
2. PyTorch framework

Let's compare their implementations and results!

Key Components Breakdown H

1. Linear Transformation

Implementation	Code
From Scratch	`z = self.w * x + self.b`
PyTorch	`z = self.w * x + self.b`

2. Sigmoid Activation 0

Implementation	Code
From Scratch	`1 / (1 + np.exp(-z))`

Implementation	Code
PyTorch	torch.sigmoid(z)

3. Binary Cross-Entropy Loss 📉

Implementation	Formula
From Scratch	-np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(l - y_pred))
PyTorch	nn.BCELoss()

Key Differences Q

1. Initialization

tP

```
# Scratch
self.w = np.random.randn()
self.b = np.random.randn()

# PyTorch
self.w = nn.Parameter(torch.randn(l))
self.b = nn.Parameter(torch.randn(l))
```

2. Automatic Differentiation

- **Scratch:** Manual gradient computation required
- **PyTorch:** Automatic through autograd

3. Memory Management

- **Scratch:** Manual NumPy array handling
- **PyTorch:** Automatic tensor management

Advantages & Trade-offs ⚖️

From Scratch Implementation

- Q Complete control over implementation

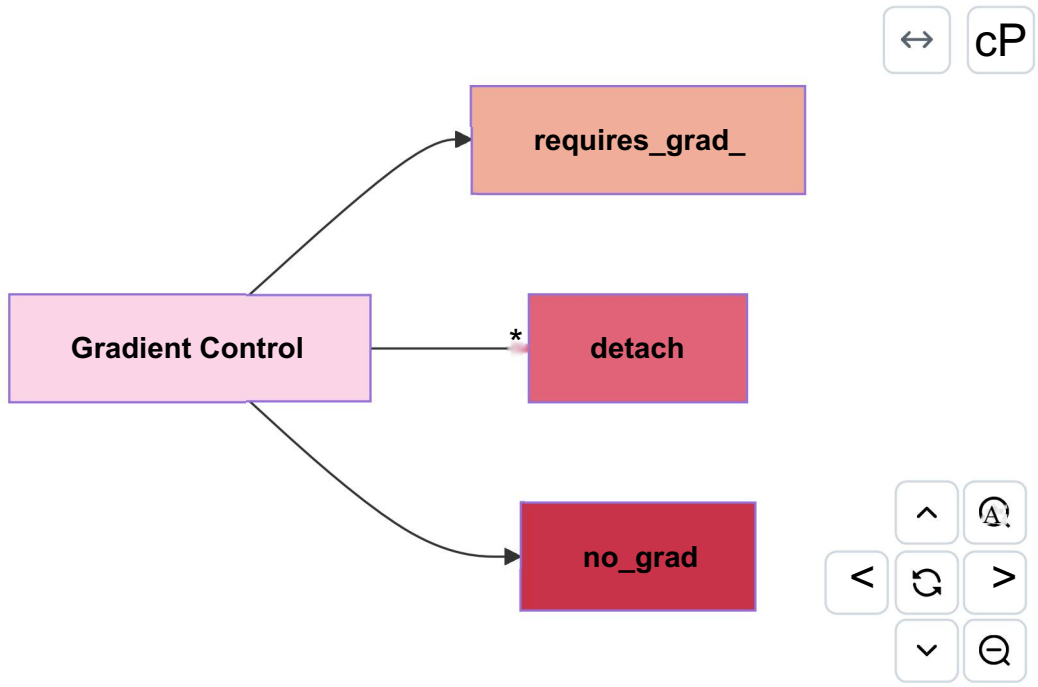
- Q Better understanding of underlying math
- X More complex gradient computation
- X Lower performance

PyTorch Implementation

- Q Automatic differentiation
- Q Optimized performance
- Q Built-in loss functions
- X Less transparent inner workings

PyTorch Gradient Control Methods: In-Depth Comparison hrs SJh

Overview Diagram



1. requires_grad_(False) Method 🎯

Description

Sets the requires_grad flag of a tensor to False , preventing gradient computation.

Implementation Example

```
# Example with requires_grad_(False)
def demonstrate_requires_grad():
    #  Create tensor with gradients
    x = torch.tensor([1.0, 2.0, 3.0], requires_grad=True)
    print(f"Initial requires_grad: {x.requires_grad}") # True

    #  Disable gradient tracking
    x.requires_grad_(False)
    print(f"After requires_grad_(False): {x.requires_grad}") # False

    #  Re-enable if needed
    x.requires_grad_(True)
    print(f"After requires_grad_(True): {x.requires_grad}") # True
```

Key Characteristics

Feature	Description
Persistence	Permanent until changed
Memory Impact	Reduces memory usage
Reversibility	Can be re-enabled
Scope	Affects specific tensor

2. detach() Method

Description

Creates a new tensor that shares the same data but has no gradient history.

Implementation Example

```
# Example with detach()
def demonstrate_detach():
    x = torch.tensor([1.0, 2.0], requires_grad=True)
    y = x * 2

    #  Create detached copy
    z = y.detach()
    print(f"Original requires_grad: {y.requiresgrad}") # True
```

```
print(f"Detached requires_grad: {z.requires_grad}") # False

# Original tensor remains unchanged
print(f"Original tensor still tracks gradients: {y.requires_grad}") # Tru
```

Key Characteristics 📊

Feature	Description
Persistence	Creates new tensor
Memory Impact	Creates new memory
Reversibility	N/A (new tensor)
Scope	New tensor only

3. torch.no_grad() Context 🌟

Description

Context manager that temporarily disables gradient computation.

Implementation Example

tP

```
# Example with torch.no_grad()
def demonstrate_no_grad():
    x = torch.tensor([1.0, 2.0], requires_grad=True)

    print("Outside no_grad:")
    y = x * 2
    print(f"y.requires_grad: {y.requires_grad}") # True

    print("\nInside torch.no_grad():")
    with torch.no_grad():
        z = x * 2
        print(f"z.requires_grad: {z.requires_grad}") # False

    print("\nBack outside:")
    w = x * 2
    print(f"w.requires_grad: {w.requires_grad}") # True
```

Key Characteristics 📊

Feature	Description
Persistence	Temporary (context)
Memory Impact	Minimal
Reversibility	Automatic
Scope	All operations

Comparison Table } 📄

Feature	requires_grad_(False)	detach()	torch.no_grad()
Use Case	Single tensor	Data viewing	Multiple operations
Scope	Permanent	New tensor	Temporary
Memory	Efficient	New allocation	Very efficient
Common Usage	Model freezing	Validation	Inference

Best Practices 💡

When to Use Each Method

1. requires_grad_(False)

```
# Use for freezing model parameters
model.fcl.weight.requires_grad_(False)
```

2. detach()

```
# Use for creating training targets
target = complex_computation.detach()
```

3. torch.no_grad()

```
# Use for inference/evaluation
with torch.no_grad():
    model_predictions = model(testdata)
```

PyTorch Gradient Control Methods: Performance Analysis 🚀

Performance Metrics Comparison || 📊

Let's analyze the performance characteristics of different gradient control methods in PyTorch through comprehensive benchmarking.

Detailed Performance Analysis 📈

1. Time Complexity &

Method	Small Tensors	Large Tensors	Overhead
requires_grad_(False)	Very Fast f	Linear	Minimal
detach()	Fast f	Linear	Memory Copy
torch.no gradQ	Medium 🏃	Linear	Context Switch

2. Memory Usage H

Method	Memory Allocation	Memory Duration	Additional Objects
requires_grad_(False)	None	Permanent	No
detach()	New Tensor	Permanent	Yes
torch.no_grad()	None	Temporary	Context Manager

3. Operation Speed Comparison 🏎️

```
# Relative Speed (normalized to requires_grad_(False))
Speed Ratios = {
    'requires grad (False)': 1.0,          # Baseline
    'detach()': ~1.2,                      # 20% slower
    'torch.no_grad()': ~1.1                # 10% slower
}
```

Practical Implications 🎯

1. Small Operations (< 1000 elements)

- **Best Choice:** requires_grad_(False)
- **Why:** Minimal overhead, no memory allocation

2. Medium Operations (1000-100000 elements)

- **Best Choice:** torch.no_grad()
- **Why:** Good balance of speed and memory usage

3. Large Operations (>100000 elements)

- **Best Choice:** Context-dependent
  - For single tensors: requires_grad_(False)
  - For multiple operations: torch.no_grad()
  - For view operations: detach()

Performance Tips V

1. Memory Optimization

```
# Prefer this (in-place)
tensor.requires_grad_(False)

# Over this (new allocation)
new_tensor = tensor.detach()
```

2. Batch Operations

```
# Efficient
with torch.no_grad():
    results = [model(x) for x in data]

# Less efficient
results = [model(x).detach() for x in data]
```

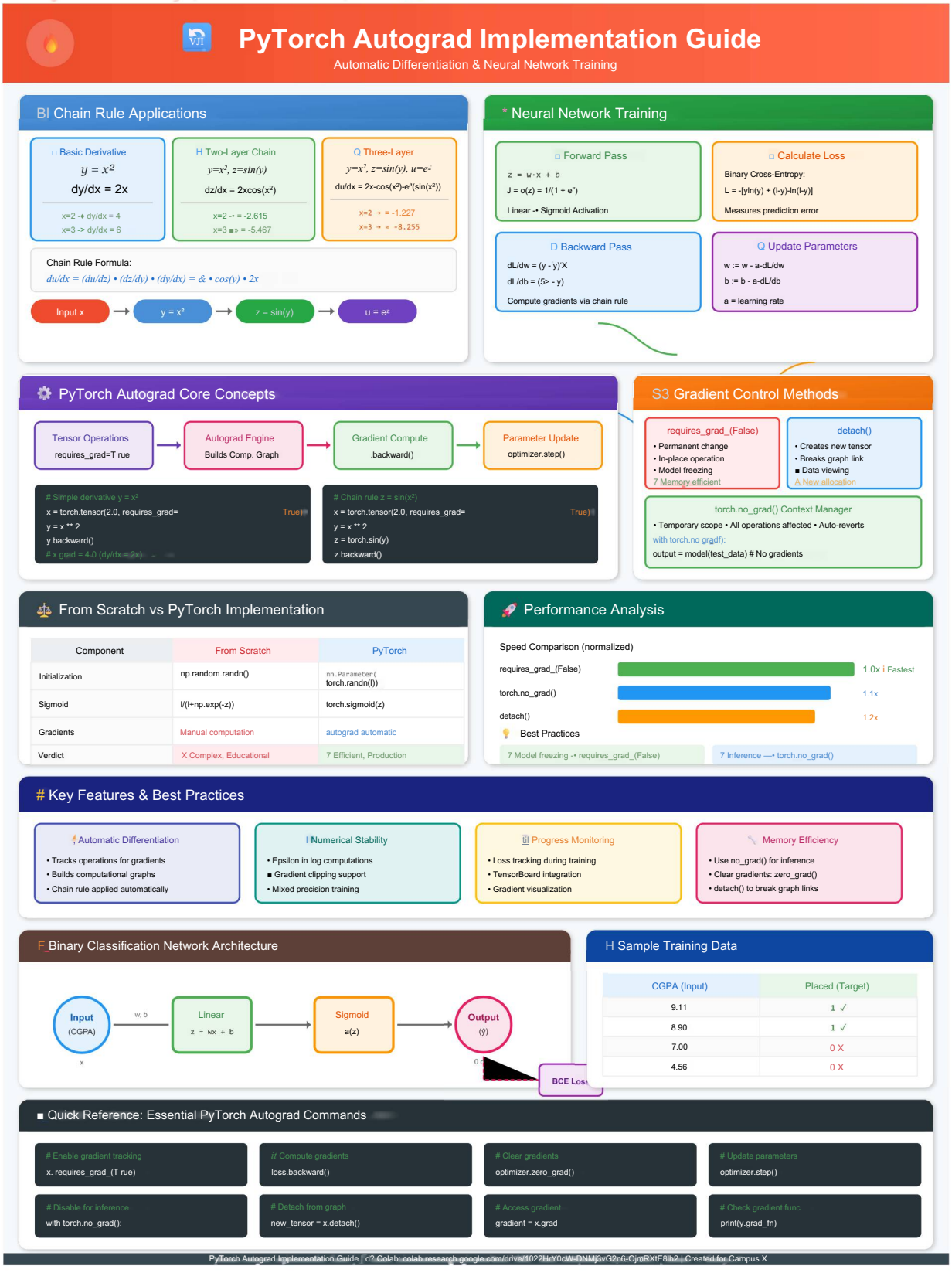
3. Mixed Usage

```
# Optimal combination
with torch.no_grad():
    intermediate = complex_computation()
    final_result = intermediate.detach()
```

Would you like me to:

1. Provide specific optimization techniques for your use case?
2. Show more detailed benchmarks for specific operations?

3. Analyze memory patterns in depth? 🤔



PyTorch Training Pipeline

Code

- htt ps://colab.research.g Qoqle.com/drive/1SzmMiRYPPM1slwu7YpHwHEvNDhU1ylNR

Neural Network Implementation from Scratch \🧠

Project Overview 🎯

We'll build a neural network for breast cancer detection, implementing core deep learning concepts from scratch while following PyTorch-style patterns. Let's break this down into manageable components!

Code Structure Breakdown 0

1. Layer Components \

- **Layer Class:** Implements linear transformation (Wx + b)
    - o Forward pass: Matrix multiplication and bias addition

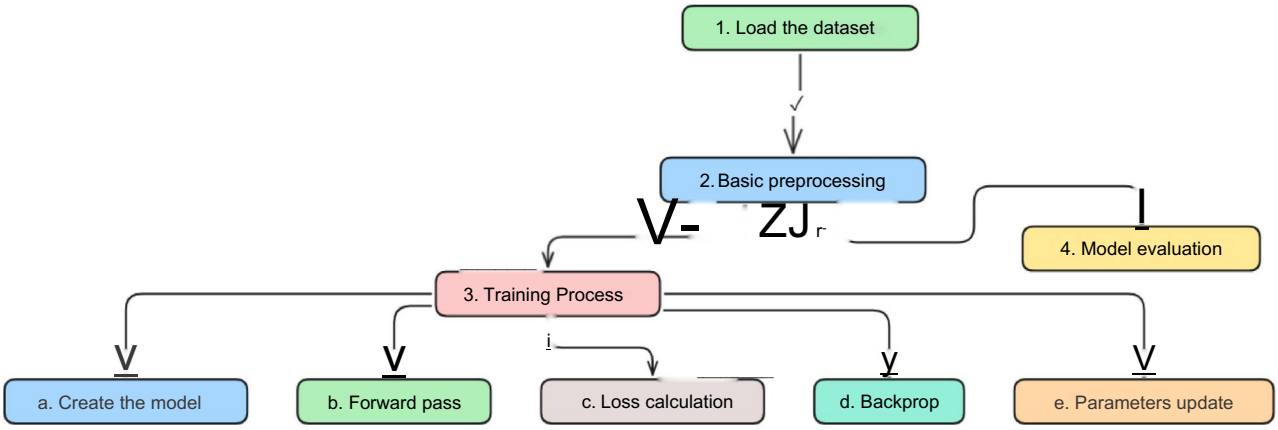
o Backward pass: Gradient computation for weights and biases

- **Activation Functions:**

o ReLU: max (0,x) for hidden layers

o Sigmoid: $\frac{1}{1+e^{-x}}$ for binary classification output

2. Model Architecture *



3. Training Process S

1. **Forward Pass:**
   - o Data flows through layers
   - o Activations computed sequentially
2. **Loss Calculation:**
   - o Binary Cross-Entropy Loss
   - o Handles binary classification task
3. **Backward Pass:**
   - o Gradient computation
   - o Chain rule application
4. **Parameter Updates:**
   - o Simple gradient descent
   - o Learning rate controlled updates

Let's run the model!

Key Features Implemented ♦+

1. **Mini-batch Processing**
   - o Processes data in smaller batches
   - o Improves training stability
2. **Modular Architecture**
   - o Separate classes for layers and activations
   - o Easy to extend and modify
3. **Gradient-based Learning**
   - o Implements backpropagation
   - o Automatic gradient computation
4. **Performance Monitoring**
   - o Tracks loss during training
   - o Evaluates accuracy on test set

Usage Instructions

1. Run the main script
2. Model will:
   - o Load breast cancer dataset
   - o Preprocess features
   - o Train for 100 epochs
   - o Display training progress
   - o Show final accuracies

Dataset

Breast Cancer Wisconsin Dataset

- **Source:** GitHub Repository
- **Features:** 30 numerical features
- **Target:** Binary classification (Malignant/Benign)
- **Samples:** 569 total samples

🛠️ Requirements

```
import numpy as np
import pandas as pd
import torch
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

🚀 Implementation

1. Data Loading and Preprocessing

```
# Load dataset
df = pd.read_csv('https://raw.githubusercontent.com/gscdit/Breast-Cancer-Detec

# Remove unnecessary columns
df.drop(columns=['id', 'Unnamed: 32'], inplace=True)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    df.iloc[:, 1:], df.iloc[:, 0], test_size=0.2
)
```

2. Feature Scaling

```
# Standardize features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

3. Label Encoding

```
# Convert categorical labels to numerical
encoder = LabelEncoder()
y_train = encoder.fit_transform(y_train)
y_test = encoder.transform(y_test)
```

4. Convert to PyTorch Tensors

```
# Convert numpy arrays to PyTorch tensors
X_train_tensor = torch.from_numpy(X_train)
X_test_tensor = torch.from_numpy(X_test)
y_train_tensor = torch.from_numpy(y_train)
y_test_tensor = torch.from_numpy(y_test)
```

5. Neural Network Class Definition

```
class MySimpleNN:
    def ___init____(self, X):
        # Initialize weights and bias
        self.weights = torch.rand(X.shape[l], 1, dtype=torch.float64, requires
        self.bias = torch.zeros(l, dtype=torch.float64, requires_grad=True)

    def forward(self, X):
        # Forward pass: z = X*W + b, y_pred = sigmoid(z)
        z = torch.matmul(X, self.weights) + self.bias
        y_pred = torch.sigmoid(z)
        return y_pred

    def loss_function(self, y_pred, y):
        # Binary Cross-Entropy Loss
        epsilon = le-7 # Prevent log(0)
        y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)
        loss = -(y * torch.log(y_pred) + (1 - y) * torch.log(l - y_pred)).mean
        return loss
```

6. Training Pipeline

```
# Hyperparameters
learning_rate = 0.1
epochs = 25

# Create model
model = MySimpleNN(X_train_tensor)

# Training loop
for epoch in range(epochs):
    # Forward pass
    y_pred = model.forward(X_train_tensor)

    # Calculate loss
    loss = model.loss_function(y_pred, y_train_tensor)
```

```
        # Backward pass
        loss.backward()

        #  Update parameters (gradient descent)
        with torch.no_grad():
            model.weights -= learning_rate * model.weights.grad
            model.bias -= learning_rate * model.bias.grad

        #  Zero gradients for next iteration
        model.weights.grad.zero_()
        model.bias.grad.zero_()

        #  Print progress
        print(f'Epoch: {epoch + 1}, Loss: {loss.item():.4f}')
```

7. Model Evaluation

```
#  Evaluation
with torch.no_grad():
    y_pred = model.forward(X_test_tensor)
    y_pred = (y_pred > 0.5).float() # Apply threshold
    accuracy = (y_pred.squeeze() == y_test_tensor).float().mean()
    print(f'Accuracy: {accuracy.item():.4f}')
```

/Results

- **Initial Loss:** -3.87
- **Final Loss:** Gradually decreases over epochs
- **Accuracy:** Variable (depends on random weight initialization)

> **Note:** The primary goal is learning the PyTorch workflow, not optimizing model performance.

Future Improvements

In upcoming implementations, we will replace manual components with PyTorch built-in modules:

1. **Neural Network Architecture**
    - o Replace manual weight/bias initialization with torch.nn.Linear
    - o Use torch.nn.Module for model definition

2. **Loss Functions**

   o Replace custom loss with torch.nn.BCELoss or torch.nn.BCEWithLogitsLoss

3. **Optimizers**

   o Replace manual gradient descent with torch.optim.SGD , torch.optim.Adam , etc.

4. Training Loop

   - o Implement data loaders for batch processing
   - o Add validation loop and metrics tracking

🎓 Key Learning Points

- Q Understanding PyTorch tensor operations
- Q Manual implementation of forward and backward pass
- Q Gradient computation using autograd
- Q Parameter updates and gradient zeroing
- Q Complete training pipeline structure

PyTorch Training Pipeline

Neural Network Implementation from Scratch | Breast Cancer Detection

0 Project Overview

Breast Cancer Wisconsin Dataset

- Features: 30 numerical measurements
- Target: Binary (Malignant/Benign)
- Samples: 569 total samples
- Split: 80% Train / 20% Test

Goal: Learn PyTorch workflow, not optimize performance

Code Structure

Layer Class
- Linear: Wx + b
- Forward: Matrix mult
- Backward: Gradients

Activations
- ReLU: max(0, x)
- Sigmoid: 1/(1+e⁻ˣ)
- For classification

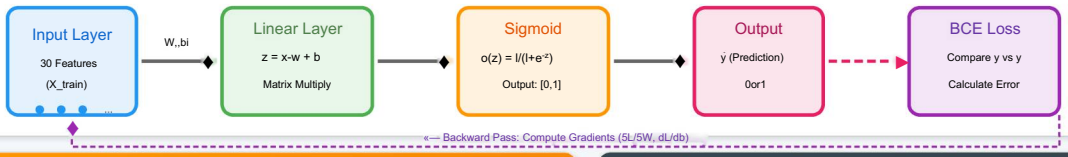
Binary Cross-Entropy Loss

L = -[y·log(ŷ) + (1-y)·log(1-ŷ)]

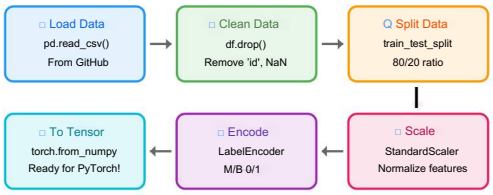
4 Requirements

```
import numpy as np
import pandas as pd
import torch
    from sklearn.model_selection import
        train_test_split
from sklearn.preprocessing import ...
```

Model Architecture - Binary Classification Network



Data Preprocessing Pipeline

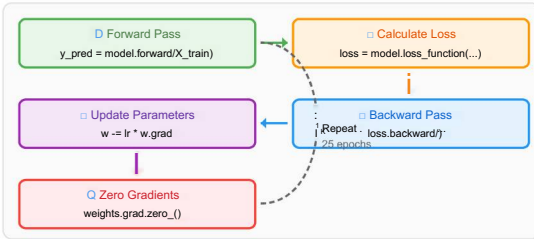


Neural Network Class Definition

```
class MySimpleNN :
  def ___ init___ (self, X):
    self .weights = torch, rand (X.shape[i], 1, requires_grad=          True )
    self .bias = torch, zeros (1, requires_grad=          True )
  def forward (self, X):                    # Forward pass
    z = torch, matmul (X, self.weights) + self.bias
    y_pred = torch, sigmoid (z)
    return y_pred
  def loss_function (self, y_pred, y):                    # BCE Loss
```

Training Pipeline

```
# Hyperparameters
learning_rate =          6.1
epochs =          25
```



Results & Evaluation

```
# Evaluation
with torch. no_grad ():
  y_pred = model.forward(X_test)
  accuracy = (y_pred == y_test).mean()
```

Initial Loss	Final Loss
-3.87	↓
Starting point	Decreases over epochs

Loss Curve:

4 Key Features Implemented

- **Mini-batch Processing** — Smaller batches, better stability
- **Modular Architecture** — Separate classes, easy to extend
- **Gradient-based Learning** — Backprop + auto grad computation
- **Performance Monitoring** — Loss tracking + accuracy eval

5 Future Improvements: Replace with PyTorch Built-ins

Component	Current (Manual)	Future (PyTorch)
Layers	Manual W, b init	nn.Linear
Loss Function	Custom BCE	nn.BCELoss
Optimizer	Manual GD	optim.Adam/SGD

Key Learning Points

- **Tensor Operations** — PyTorch tensor basics
- **Forward/Backward** — Manual implementation
- **Autograd** — Gradient computation
- **Training Loop** — Complete pipeline structure
- **Zero Grad** — Gradient management

Quick Reference: Essential Commands

```
# Create model
model = MySimpleNN(X_train)
# Forward pass
y_pred = model.forward(X)
```

```
# Compute loss
loss = model.loss_function(...)
# Backward pass
loss.backward()
```

```
# Update parameters
w -= lr * w.grad
# Zero gradients
weights.grad.zero_()
```

```
# Evaluation mode
with torch.no_grad():
# Accuracy
acc = (pred == y).mean()
```


CampusX

PyTorch Neural Network (torch.nn) Module:

Code

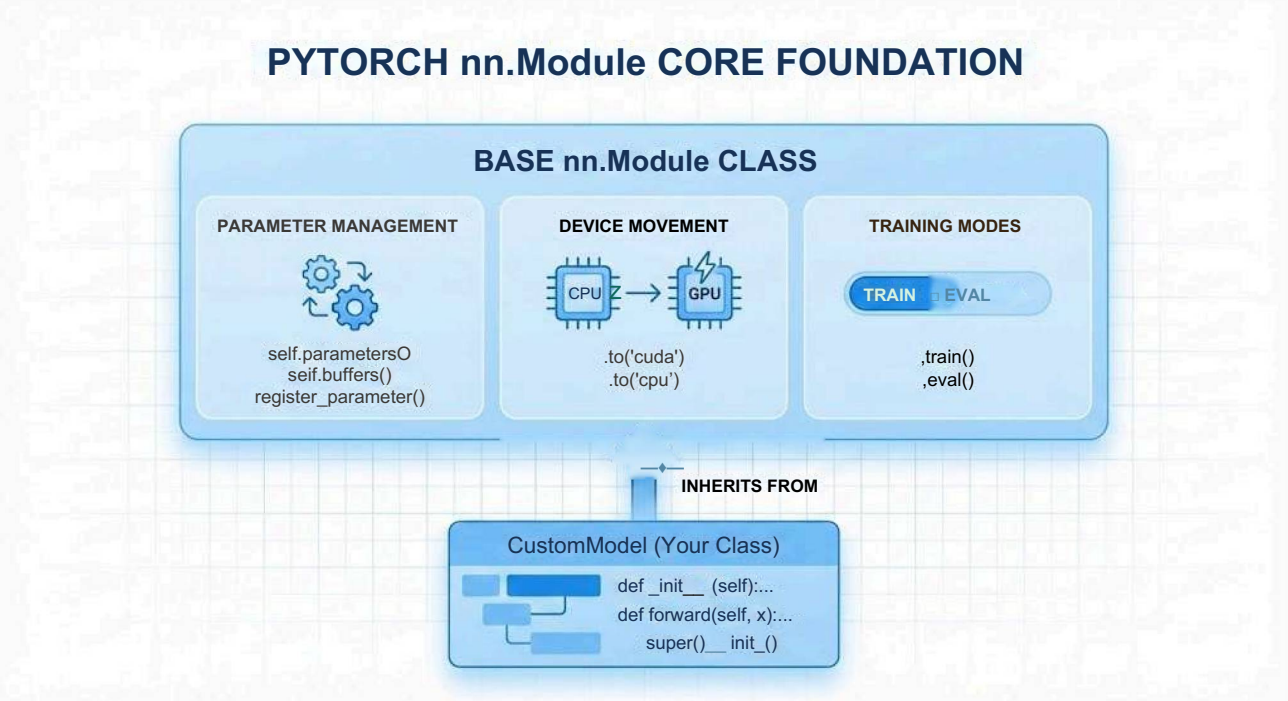
- htt ps://colab.research.g oog le.com/drive/1j9JVaahEDkzwm R0GR3OE7-khODnllg PeL

1. Core Foundation: nn.Module P

Base Class Overview

tP

```
import torch.nn as nn

class CustomModel(nn.Module):
    def _____ init____ (self):
        super(CustomModel, self)._____ init____()

    def forward(self, x):
        return x
```



Key Features of nn.Module

Feature	Description	Example
Parameter Management	Automatically tracks learnable parameters	self.parameters()
Device Movement	Easily move model between CPU/GPU	model.to(device)
Training Modes	Supports train/eval modes	model.train() / model.eval()

2. Essential Neural Network Layers

Linear Layers

```
# Fully Connected Layer
nn.Linear(in_features=784j out_features=256)
```

tP

Convolutional Layers

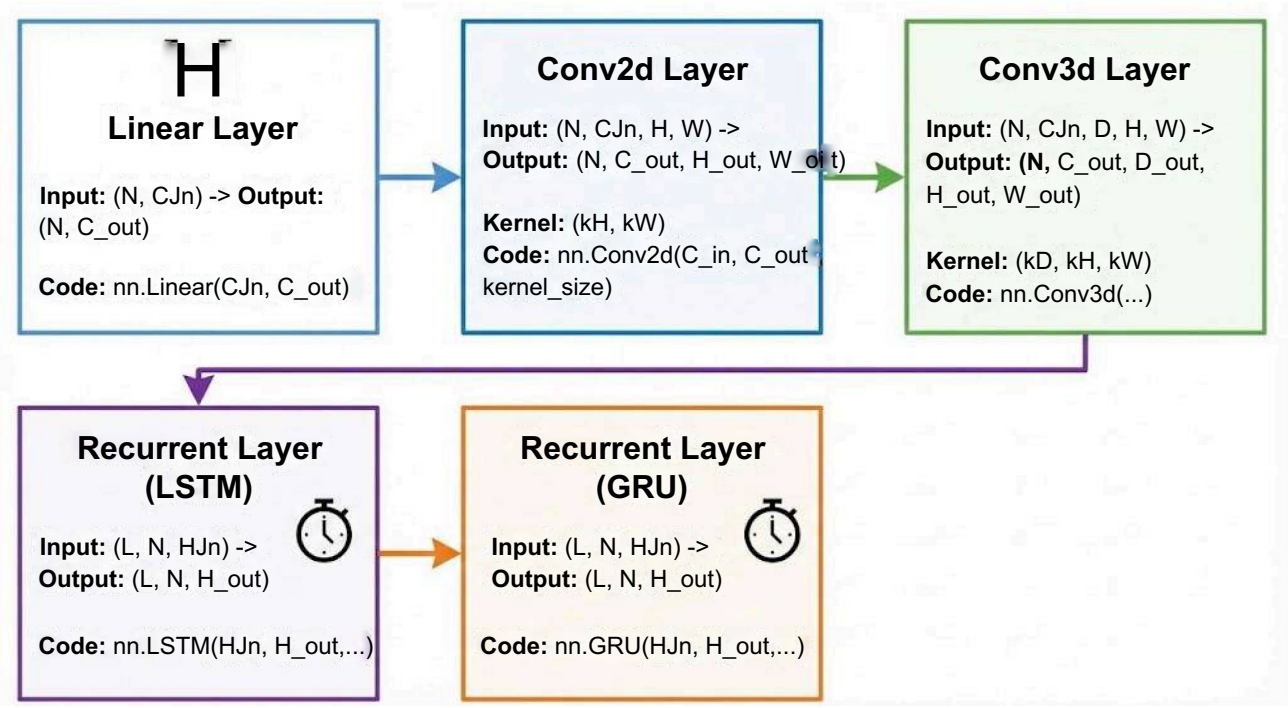
```
# 2D Convolution
nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, stride=l, padding=l)
```

[P

```
# 3D Convolution
nn.Conv3d(in_channels=3j out_channels=64, kernel_size=3)
```

Recurrent Layers

```
#  LSTM Layer
nn.LSTM(input_size=100j hidden_size=256, num_layers=2j batch_first=True)

#  GRU Layer
nn.GRU(input_size=100, hidden_size=256, num_layers=2)
```



3. Activation Functions f

Common Activations

Function	Mathematical Form	Usage
Re LU	max(0,x)	nn.Rel_U()
Sigmoid	$1/(1+e^{A}(-x))$	nn.Sigmoid()
Tanh	$(e^{A}x - e^{A}(-x))/(e^{A}x + e^{A}(-x))$	nn.Tanh()

```
# Example of using activations
model = nn.Sequential(
        nn.Linear(784, 256),
        nn.ReLU(),
        nn.Linear(256, 10),
        nn.Sigmoid()
)
```

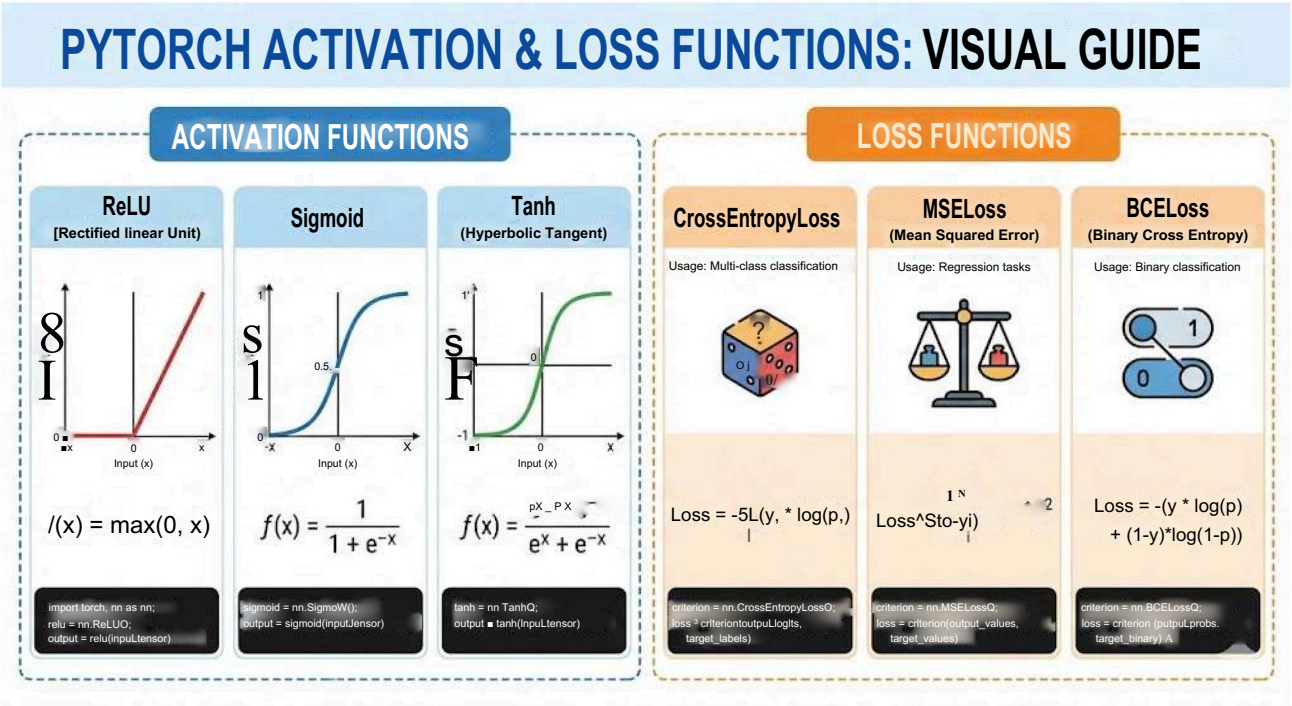
4. Loss Functions

Popular Loss Functions

```
# Classification
criterion = nn.CrossEntropyLoss()

# Regression
criterion = nn.MSELoss()

# Binary Classification
criterion = nn.BCELoss()
```

Usage Example

```
loss_fn = nn.CrossEntropyl_oss()
output = model(input)
loss = loss_fn(output, target)
loss.backward()
```



5. Container Modules £

Sequential Container

```
model = nn.Sequential(
    nn.Conv2d(3, 64, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool2d(2),
    nn.BatchNorm2d(64)
)
```

ModuleList Container

```
layers = nn.ModuleList([
    nn.Linear(10, 10) for i in range(5)
])
```

6. Regularization Techniques 🎯

Dropout

```
nn.Dropout(p=0.5) # 50% dropout probability
```

```
nn.Dropout2d(p=0.2) # Spatial dropout for CNN
```

Batch Normalization

```
# For CNN
nn.BatehNorm2d(num_features=64)

# For Linear layers
nn.BatehNormld(num_features=256)
```

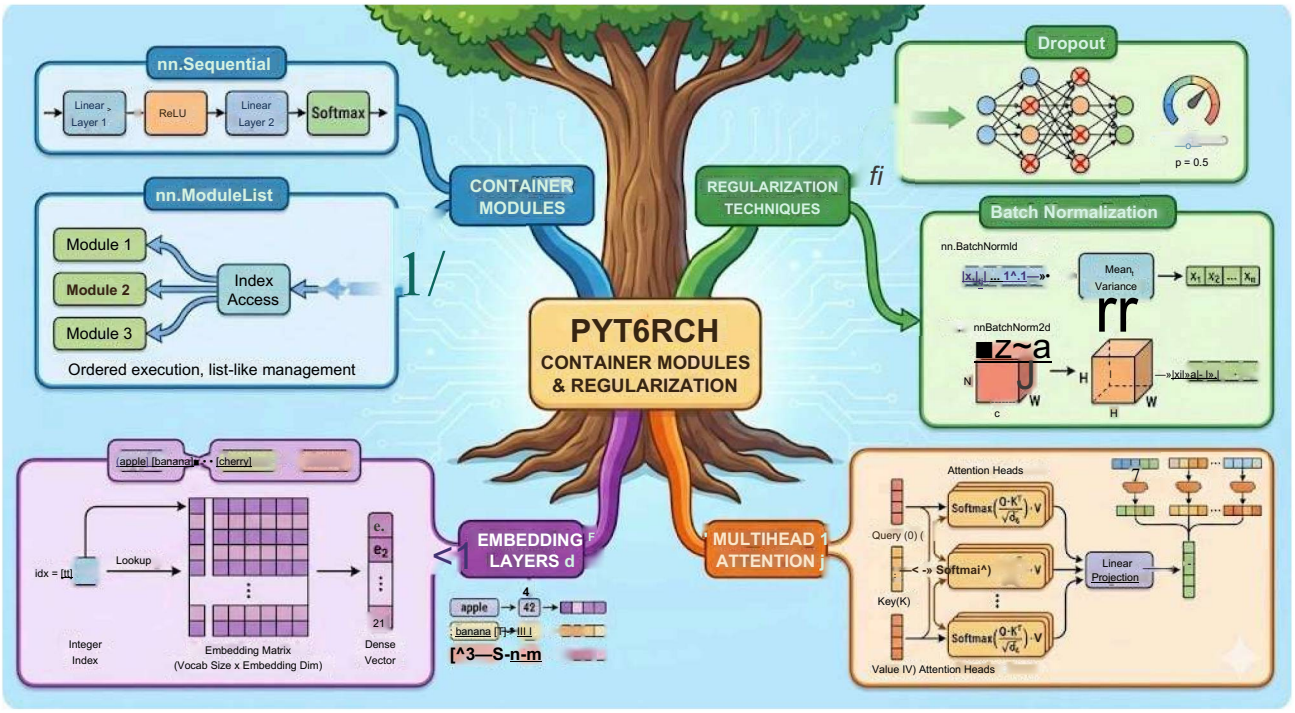
7. Advanced Components

Embedding Layers

```
# Word embeddings
embedding = nn.Embedding(num_embeddings=10000j embedding_dim=300)
```

Attention Mechanisms

```
# Multi-head attention
attention = nn.MultiheadAttention(embed_dim=512, num_heads=8)
```



8. Best Practices & Tips V

1. Module Initialization

```
def weight_init(m):
    if isinstance(m, nn.Linear):
        nn.init.xavier_uniform_(m.weight)
        nn.init.zeros_(m.bias)
```

[P

2. Parameter Management

```
# Count parameters
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

tP

3. Device Management

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
```

tP

1. Core Concepts 🎯

1.1 Basic Optimization Flow

```
import torch.optim as optim

# Initialize optimizer
optimizer = optim.SGD(model.parameters()j lr=0.01)

# Training loop
optimizer.zero_grad() # Clear gradients
loss = criterion(output, target) # Compute loss
loss.backward() # Compute gradients
optimizer.step() # Update parameters
```

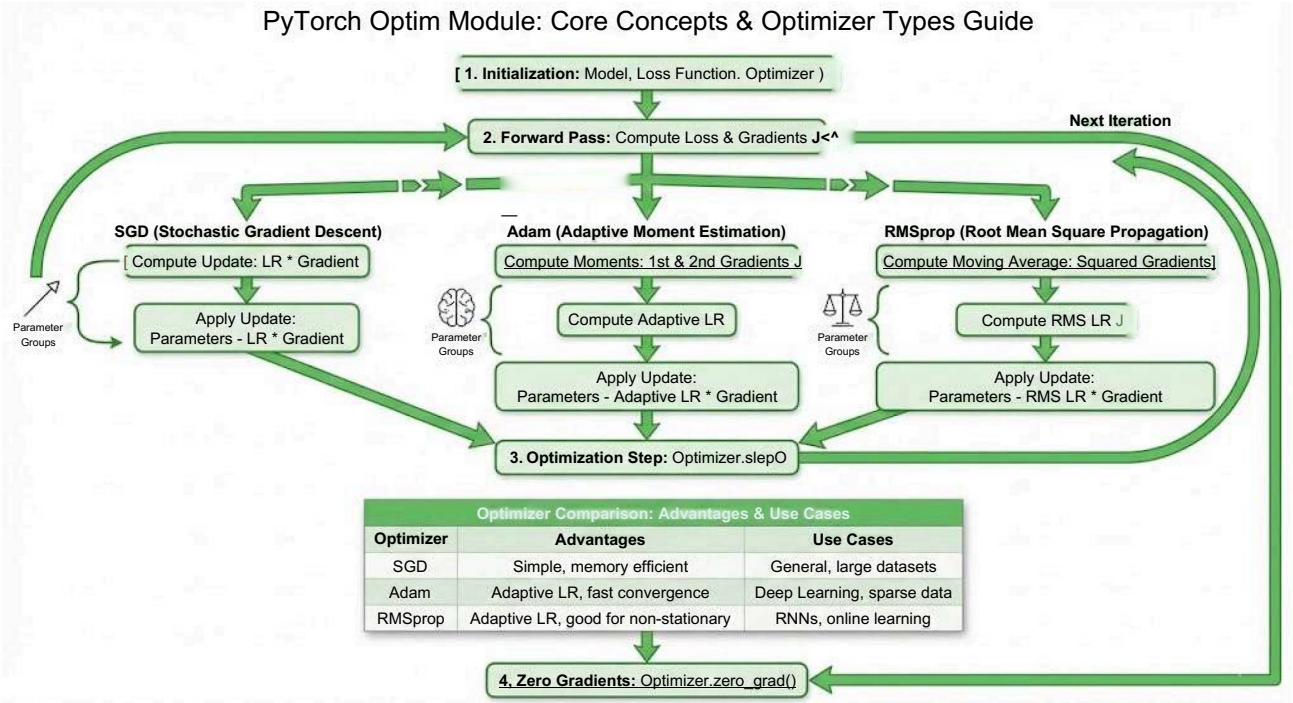
[P

1.2 Parameter Management

Component	Description	Access Method
Parameters	Model weights & biases	model.parameters()

Component	Description	Access Method
Gradients	Parameter derivatives	param.grad
State	Optimizer internal state	optimizer.state_dict()

2. Optimizer Types



2.1 SGD (Stochastic Gradient Descent)

tP

```
# Basic SGD
sgd = optim.SGD(model.parameters()ˌ lr=0.01)

# SGD with momentum
sgd_momentum = optim.SGD(
        model.parameters(),
        lr=0.01,
        momentum=0.9,
        nesterov=Truej
        weight_decay=le-4
)
```

2.2 Adam (Adaptive Moment Estimation)

```
adam = optim.Adam(
    model.parameters(),
    lr=0.001,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=0,
    amsgrad=False
)
```

2.3 RMSprop

```
rmsprop = optim.RMSprop(
    model.parameters(),
    lr=0.01,
    alpha=0.99,
    eps=1e-8,
    weight_decay=0,
    momentum=0
)
```

2.4 Optimizer Comparison Table

Optimizer	Advantages	Best Use Cases
SGD	Simple, reliable	General purpose, CNN
Adam	Adaptive learning rates	Deep networks, NLP
RMSprop	Good for RNNs	Recurrent networks
AdaGrad	Adapts per-parameter	Sparse data

3. Learning Rate Scheduling

3.1 Step Learning Rate

```
scheduler = optim.lr_scheduler.StepLR(
    optimizerj
    step_size=30,
    gamma=0.1
)
```

3.2 Cosine Annealing

```
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer,
    T_max=200,
    eta_min=0
)
```

3.3 Custom Learning Rate Schedule

```
class CustomScheduler(optim.lr_scheduler._LRScheduler):
    def ______ init____ (self, optimizer, total_epochs):
        self.total_epochs = total_epochs
        super(CustomScheduler, self).________ init____(optimizer)

    def get_lr(self):
        epoch = self.last_epoch
        return [base_lr * (1 - epoch/self.total_epochs)
                for base_lr in self.base_lrs]
```

4. Parameter Management

4.1 Gradient Clipping

```
# Clip gradient norm
torch.nn.utils.clip grad norm (model.parameters(), max_norm=l.0)

# Clip gradient value
torch.nn.utils.clip_grad_value_(model.parameters(), clip_value=l.0)
```

4.2 Weight Decay Implementation

```
class OptimWithDecay:
    def _____init_____(self, model, lr=0.01, weight_decay=le-4):
        self.params = model.parameters()
        self.lr = lr
        self.weight_decay = weight_decay

    def step(self):
        with torch.no_grad():
            for param in self.params:
```

```
                if param.grad is not None:
                    param.data -= self.lr * (
                     param.grad + self.weight_decay * param.data
                )
```

5. Best Practices 💡

5.1 Optimizer Selection Guide

```
def get_optimizer(model, optimizer_name='adam'):
    optimizers = {
        ' sgd': optim.SGD(
            model.parameters(),
            lr=0.01,
            momentum=0.9
        ),
        'adam': optim.Adam(
            model.parameters(),
            lr=0.001
        ),
        'rmsprop': optim.RMSprop(
            model.parameters(),
            lr=0.01
        )
    }
    return optimizers[optimizer_name.lower()]
```

5.2 Learning Rate Finding

```
def find_lr(model, train_loader, optimizer, criterion):
    lrs = []
    losses = []
    lr = le-7

    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.param_groups[0]['lr'] = lr

        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()

        lrs.append(lr)
```

```
            losses.append(loss.item())
            lr *= 1.1

            if lr > 10 or loss > 100:
                break

        return lrs, losses
```

6. Advanced Techniques A

6.1 Custom Optimizer

```
class CustomOptimizer(optim.Optimizer):
    def _____ init____ (self, params, lr=le-3, momentum=0.9):
        defaults = dict(lr=lr, momentum=momentum)
        super(Customoptimizer, self)._init____(params, defaults)

    (Storch. no grad()
    def step(self):
        for group in self.paramgroups: _
            for p in group['params']:
                if p.grad is None:
                    continue

                # Custom update rule
                d_p = p.grad
                p.add_(d_p, alpha=-group['lr'])
```

cP

6.2 Multi-Optimizer Training

```
class MultiOptimModel(nn.Module):
    def _____ init____ (self):
        super()._____ init____ ()
        self.feature_extractor = nn.Sequential(...)
        self.classifier = nn.Sequential(...)

    def get_optimizers(self):
        return {
            'features': optim.SGD(
                self.feature_extractor.parameters(),
                lr=0.01
            b
            'classifier': optim.Adam(
                self.classifier.parameters(),
```

cP

```
                lr=0.001
            )
        }
```

6.3 Optimization State Management

```
def save_optimization_state(model, optimizer, path):
    torch.save({
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
    }, path)

def load_optimization_state(model, optimizer, path):
    checkpoint = torch.load(path)
    model.load_state_dict(checkpoint['model_state_dict'])
    optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
```

cP

This comprehensive guide covers all major aspects of PyTorch's optim module, providing both theoretical understanding and practical implementation details. Each section includes code examples and best practices for real-world applications.

CampusX

Dataset & DataLoader Class in PyTorch

Code

- htt ps://colab.research.g oo g le.com/drive/1 P7Nn_nZ2wVKERZxcf4V0JeTD8_BIPC6G
- htt ps://colab.research.g oog le.com/drive/1_RLSpxfu5hvjlcARxihOHelGIGr8xrEuw

Overview

Mini Batch Gradient Descent is an optimization technique that strikes a balance between Stochastic Gradient Descent (SGD) and Batch Gradient Descent by processing small batches of data at a time.

Implementation Components

1. Data Loading Structure

```
from torch.utils.data import DataLoader, Dataset

# Create DataLoader for mini-batching
train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=32, # Mini-batch size
```

```
        shuffle=True    #    Randomize    data
    )
```

2. Training Loop Architecture

```
def train_model(model, train_loader, criterion, optimizer, epochs):
        for epoch in range(epochs):
            for batch_idx, (data, targets) in enumerate(train_loader):
                # Forward pass
                predictions = model(data)
                loss = criterion(predictions, targets)

                # Backward pass
                optimizer.zero_grad() # Clear previous gradients
                loss.backward()              #  Compute gradients
                optimizer.step()             #  Update weights
```

tP

Key Advantages 🎯

Performance Benefits

- **Memory Efficiency:** Processes smaller chunks of data
- **Training Speed:** Faster convergence than batch gradient descent
- **Noise Reduction:** More stable than pure stochastic gradient descent

Implementation Benefits

• **Parallelization:** Efficient GPU utilization

- **Regularization:** Natural noise in mini-batches helps generalization
- **Memory Management:** Better control over memory consumption

Practical Considerations 🤔

1. Batch Size Selection

Batch Size	Impact
Smaller (16-32)	More noise, better generalization
Medium (64-256)	Good balance of speed/stability

Batch Size	Impact
Larger (512+)	More stable, might overfit

2. Code Example with Best Practices

cP

```
import torch
import torch.nn as nn

class NeuralNetwork(nn.Module):
    def _____ init____(self):
        super().___ init___ ()
        self.network = nn.Sequential(
            nn. Linear(784., 128),
            nn.ReLU(),
            nn.Linear(128, 10)
        )

    def f o rwa r d (s e lf, x):
        return self.network(x)

#    Training    configuration
model   =   NeuralNetwork()
criterion    =     nn.CrossEntropyl_oss()
optimizer     =      torch.optim.Adam(model.parameters(),      lr=0.001)
batch_size = 64

#  Training with mini-batches
for epoch in range(num_epochs):
    for batch_x, batch_y in train_loader:
        optimizer.zero_grad()
        output = model(batch_x)
        loss = criterion(output, batch_y)
        loss.backward()
        optimizer.step()
```

Optimization Tips V

1. **Dynamic Batch Sizing**

    o Start with smaller batches

    o Gradually increase based on convergence

2. **Memory Management**

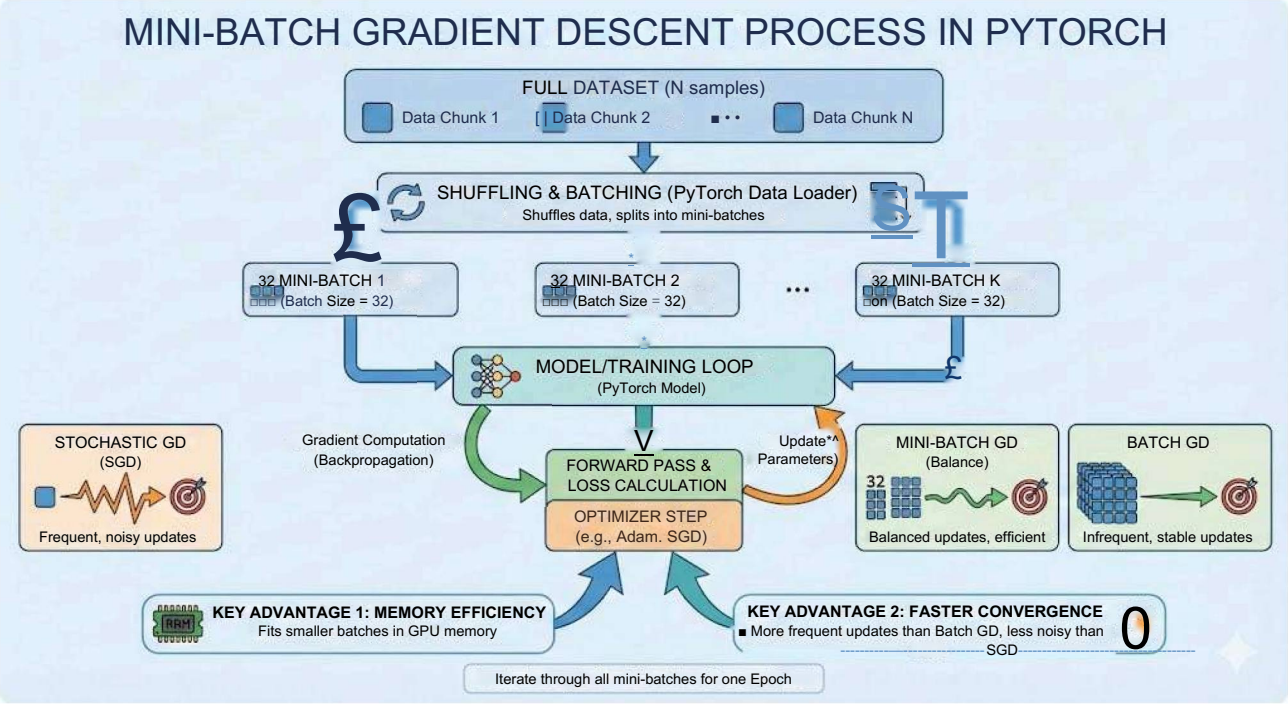
- Use torch.cuda.empty_cache() between epochs
- Monitor GPU memory usage

3. **Performance Monitoring**

- Track loss per mini-batch
- Monitor gradient norms
- Use learning rate scheduling

Common Pitfalls to Avoid ⚠️

- Inconsistent batch sizes
- Not shuffling data between epochs
- Incorrect gradient clearing
- Poor learning rate selection



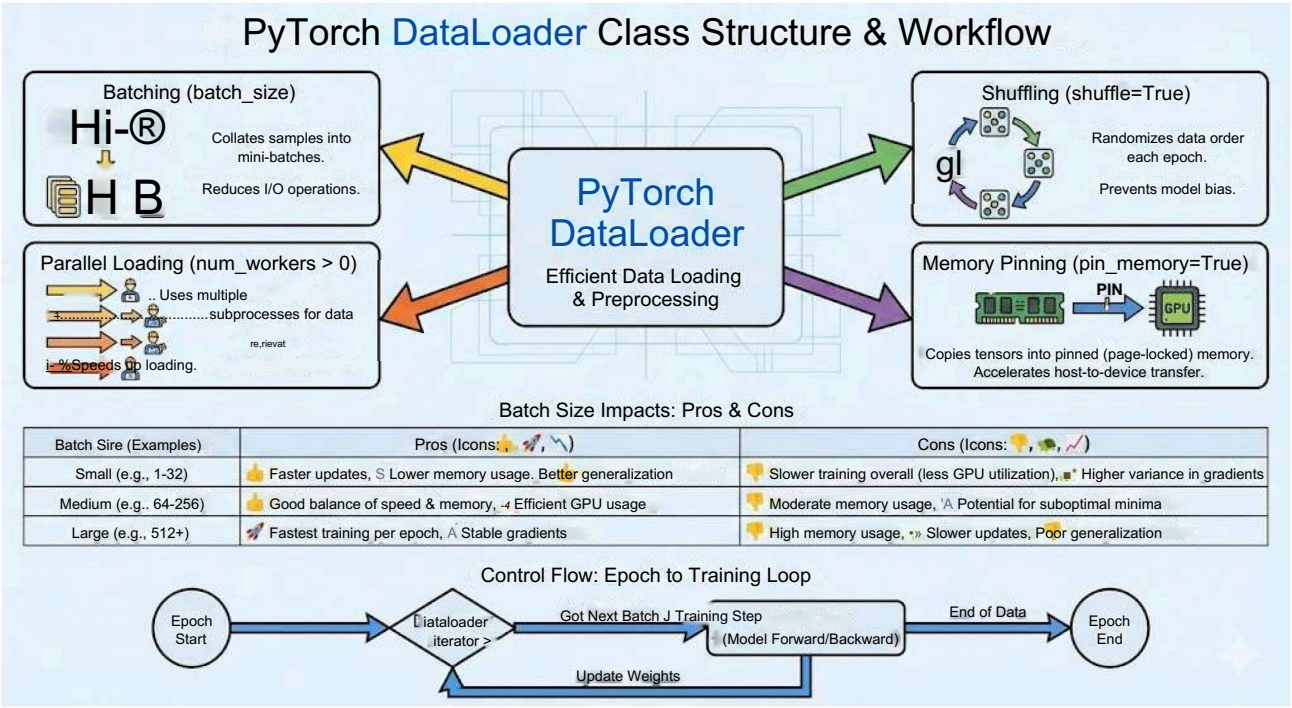
Advanced Techniques 🚀

```
#  Learning rate scheduling
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizerj
    mode='min',
    patience=3,
    factor=0.1
)
```

tP

```
# Gradient clipping
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=l.0)
```

PyTorch Data Challenges & Solutions



1. No Standard Interface for Data ||

The Challenge

```
# Traditional approach (problematic)
data = [
        (imagel, labell),
        (image2, label2)
        # Inconsistent structure
]
```

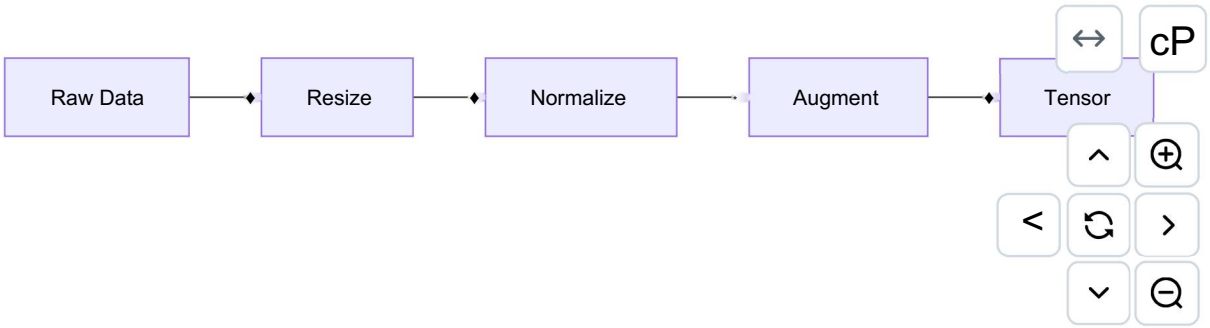
Solution: Custom Dataset Class

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
        def _____ init____(self, data_path):
                self.data = self._load_data(data_path)
```

```
    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx]

    def _load_data(self, path):
        # Standardized data loading logic
        pass
```

2. Data Transformation Complexity

Challenge Visualization



Solution: Composable Transforms

```
from torchvision import transforms

transform_pipeline = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485], std=[0.229]),
    transforms.RandomHorizontalFlip(p=0.5),
])

class EnhancedDataset(Dataset):
    def __init__(self, data_path, transform=None):
        self.transform = transform
        self.data = self._load_data(data_path)

    def __getitem__(self, idx):
        sample = self.data[idx]
        if self.transform:
```

```
            sample = self.transform(sample)
        return sample
```

3. Shuffling and Sampling Management Q

Efficient Implementation

```
from torch.utils.data import DataLoader, Randomsampler

# Create sophisticated sampling strategy
sampler = RandomSampler(
    data_source=dataset,
    replacement=False,
    num_samples=None # Use all samples
)

loader = DataLoader(
    dataset=dataset,
    batch_size=32,
    sampler=sampler,
    drop_last=True # Handle incomplete batches
)
```

4. Batch & Parallelization Optimization f

Enhanced DataLoader Configuration

```
loader = DataLoader(
    dataset=dataset,
    batch_size=32,
    num_workers=4,              # Parallel data loading
    pin_memory=True,            # Faster data transfer to GPU
    prefetch_factor=2,          # Pre-load future batches
    persistent_workers=True     # Keep workers alive
)
```

Memory Management Strategies

```
    class MemoryEfficientDataset(Dataset):
    def _____ init____(self, data_path):
            self.data_path = data_path
```

```
        self.indices = self._create_index()

    def __getitem__(self, idx):
        # Load data on-demand
        return self._load_single_item(self.indices[idx])

    def _create_index(self):
        # Create lightweight index structure
        pass
```

Advanced Optimization Techniques 🚀

1. Custom Collate Functions

```
def custom_collate(batch):
    """Handle variable-sized data....."""
    data = [item[0] for item in batch]
    target = [itemfl] for item in batch]
    return torch.stack(data), torch.tensor(target)

loader = DataLoader(
    dataset=dataset,
    collate_fn=custom_collate
)
```

2. Dynamic Batch Sizing

```
class DynamicBatchSampler(Sampler):
    def__init__(self, dataset, min_batch, max_batch):
        self.dataset = dataset
        self.min_batch = min_batch
        self.max_batch = max_batch

    def __iter__(self):
        # Implement dynamic batch size logic
        pass
```

Performance Monitoring Dashboard 📊

Metric	Tool	Purpose
Memory Usage	torch.cuda.memory_stats()	Track GPU memory

Metric	Tool	Purpose
Load Time	time.perf_counter()	Measure data loading
Worker Efficiency	torch.utils.data.DataLoader logs	Monitor parallelization

This comprehensive guide addresses the main challenges in PyTorch data handling with practical, optimized solutions and best practices for implementation.

PyTorch Dataset and DataLoader: Core Data Handling Classes

Dataset Class Blueprint L

Core Components

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def_____init____(self, data_source):
        """Initialize data loading strategy.....
        self.data = self._load_data(data_source)

    def __ len__ (self):
        """Return total number of samples.....
        return len(self.data)

    def___ getitem___(self, index):
        """Return single sample (and label)"""
        return self.data[index]
```

Implementation Example \

```
class ImageDataset(Dataset):
    def __ init___ (self, image_dir, transform=None):
        self.image_paths = glob.glob(f"{image_dir}/*.jpg")
        self.transform = transform

    def __ len__ (self):
        return len(self.image_paths)

    def __ getitem___(self, idx):
```

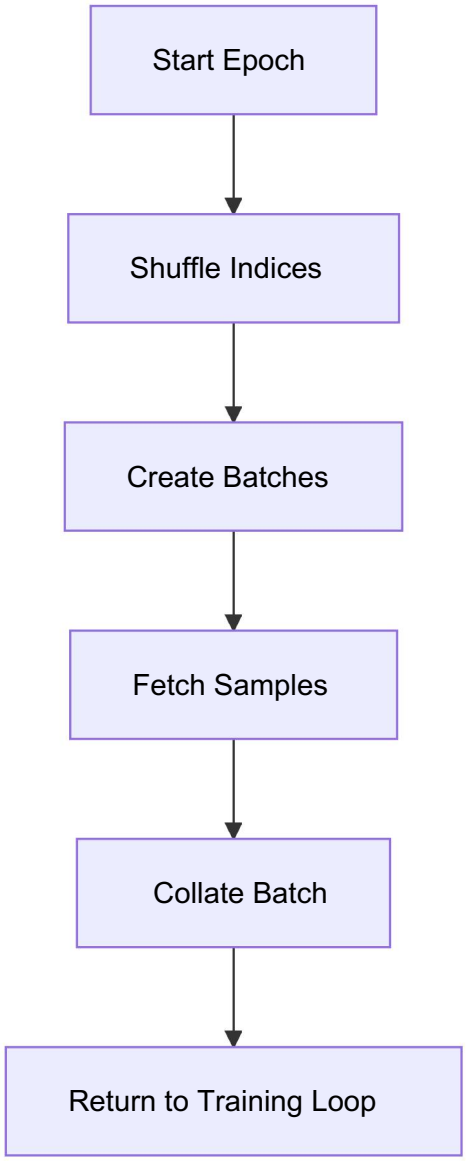
```
        image = Image.open(self.image_paths[idx])
        if self.transform:
            image = self.transform(image)
        return image
```

DataLoader Architecture . 🚀

Core Functionality

Component	Purpose	Configuration
Batching	Combines samples	batch_size=32
Shuffling	Randomizes order	shuffle=True
Parallel Loading	Multi-process data loading	num_workers=4
Memory Pinning	Optimizes GPU transfer	pin_memory=T rue

Control Flow Diagram



Integration Example

```
# Dataset Definition
class TimeSeriesDataset(Dataset):
    def __init__(self, data, sequence_length):
        self.data = data
        self.sequence_length = sequence_length

    def __len__(self):
        return len(self.data) - self.sequence_length

    def __getitem__(self, idx):
        sequence = self.data[idx:idx + self.sequence_length]
```

```
            target = self.data[idx + self.sequence_length]
            return sequence, target

# DataLoader Configuration
loader = DataLoader(
    dataset=TimeSeriesDataset(data, sequence_length=10),
    batch_size=32,
    shuffle=True,
    num_workers=4,
    pin_memory=True,
    prefetch_factor=2
)

# Training Loop Integration
for epoch in range(num_epochs):
    for batch_data, batch_targets in loader:
        # Training step implementation
        pass
```

Advanced Features & Best Practices &

1. Custom Collate Function

```
def custom_collate(batch):
    """Customize batch formation"""
    data = torch.stack([item[0] for item in batch])
    targets = torch.tensor([item[l] for item in batch])
    return data, targets

loader = DataLoader(
    dataset,
    collate_fn=custom_collate
)
```

cP

2. Sampler Configuration

```
from torch.utils.data import WeightedRandomSampler

sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(dataset),
    replacement=True
)
```

tP

```
loader = DataLoader(
        dataset,
        sampler=sampler
)
```

3. Memory Optimization Techniques

```
# Optimized DataLoader configuration
loader = DataLoader(
        dataset,
        batch_size=32,
        num_workers=4,
        pin_memory=True,
        persistent_workers=True,
        prefetch_factor=2
)
```

Performance Tips @

1. **Optimal Worker Count**

```
num_workers = min(cpu_count(), 4 * torch.cuda.device_count())
```

2. **Memory Pinning**
    - Enable for GPU training
    - Disable for CPU-only training
3. **Batch Size Selection**
    - Start with power of 2
    - Adjust based on memory constraints

This comprehensive overview covers the essential aspects of PyTorch's data handling classes, providing both theoretical understanding and practical implementation guidelines.

Dataset & DataLoader

PyTorch Data Pipeline & Mini-Batch Training

Dataset Class Blueprint

```
class CustomDataset(Dataset):
    def __init__(self, data):
        self.data = data # Initialize
    def __len__(self):
        return len(self.data) # Total samples
    def __getitem__(self, idx):
        return self.data[idx] # Single sample
```

♦ These 3 methods are REQUIRED for any custom Dataset

DataLoader Architecture

```
loader = DataLoader(
    dataset      =train_dataset,
    batch_size   =32,
    shuffle      =True,
    num_workers  =4,
    pin_memory   =True
)
```

- batch_size: Samples per batch
- num_workers: Parallel loading threads

Mini-Batch Overview

Batch GD	Mini-Batch	SGD
All data	32-256 samples	1 sample
Slow, stable	Best balance!	Fast, noisy

Advantages:

- Memory efficient processing
- Faster convergence than Batch GD
- More stable than pure SGD
- Efficient GPU parallelization
- Natural regularization effect

Data Pipeline Flow: Dataset —» DataLoader —> Training Loop

Raw Data (files, arrays, images, text) → Dataset (__getitem__(idx), __len__(), + transforms) → Sampler (shuffle/order) → BatchSampler (batch_size=32, drop_last) → Collate Fn (stack tensors) → DataLoader (num_workers=4, pin_memory, prefetch) → Training Loop (for batch in loader: forward(), backward())

Data Transforms

```
transform = transforms.Composed
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
    transforms.RandomHorizontalFlip()
])
```

Batch Size Guide

Size	Trade-offs
16-32	More noise -► Better generalization Slower training, lower GPU usage
64-256	Recommended balance Good speed & stability
512+	Stable gradients, fast training Risk of overfitting, high memory

Training Loop

```
for epoch in range (epochs):
    for data, target in train_loader:
        » Forward pass
        output = model(data)
        loss = criterion(output, target)
        # Backward pass
        optimizer.zero_grad()          loss.backward()
        optimizer.step( )
```

Performance Optimization Config

```
loader = DataLoader(
    dataset^dataset,
    batch_size=32,
    num_workers=4,
    pin_memory=True,
    prefetch_factor=2,
    persistent_workers=True
)
```

- num_workers
  Parallel data loading processes
  Tip: min(cpu_count(), 4*gpu_count)
- pin_memory
  Faster GPU transfer (CUDA only)
- prefetch_factor
  Pre-load batches per worker

Samplers & Custom Collate

RandomSampler

```
RandomSampler(
dataset,
replacement=False)
```

WeightedRandomSampler

```
WeightedRandomSampler(
weights=sample_weights,
num_samples=len(ds))
```

Custom Collate Function:

```
def custom_collate (batch): # Handle variable-size data
return torch.stack([x[0] for x in batch]), torch.tensor([x[l] for x in batch])
```

Common Pitfalls

- X Inconsistent batch sizes at epoch end
  + Use drop_last=True
- X Not shuffling data between epochs
  + Set shuffle=True
- X Forgetting optimizer.zero_grad()
  + Gradients accumulate incorrectly
- X Loading all data into memory
  + Load on-demand in ____ getitem__

Best Practices

- ✓ Use pin_memory=True for GPU training
- ✓ Start with batch_size = power of 2
- ✓ Monitor memory with cuda.memory_stats()
- ✓ Use persistent_workers for long training
- ✓ Apply LR scheduling after epochs
  scheduler.step()

Memory Efficient Dataset

```
class LazyDataset (Dataset):
    def ___ init___ (self, paths):
        # Only store file paths
        self.paths = paths
    def ____ getitem__ (self, idx):
        # Load on-demand
        return self._load(self.paths[idx])
```

Quick Reference: DataLoader Parameters

Parameter	Type	Default	Description	Recommended
batch_size	int	1	Samples per batch	32-256 (power of 2)
shuffle	bool	False	Randomize data order each epoch	True for training
num_workers	int	0	Subprocesses for data loading	4-8 (based on CPU cores)
pin_memory	bool	False	Copy tensors to CUDA pinned memory	True for GPU training
drop_last	bool	False	Drop incomplete last batch	True for BatchNorm layers

Campus X

torch.utils.data Module

pytorch.org/docs/stable/data.html

from torch.utils.data import Dataset, DataLoader, Sampler

2 Colab Notebooks Included

Data Pipeline Mastery Guide

CampusX

Building an Artificial Neural Network (ANN) with PyTorch

Code

- htt ps://colab.research.g oog le.com/drive/1 REudoW3X8RNA53vf8j_pir540JyzB7uR

1. Introduction and Prerequisites

This guide covers building an **Artificial Neural Network (ANN) —** also known as **Multi-Layer Perceptron (MLP) —** using PyTorch from scratch. The neural network performs multi-class classification on the Fashion MNIST dataset.

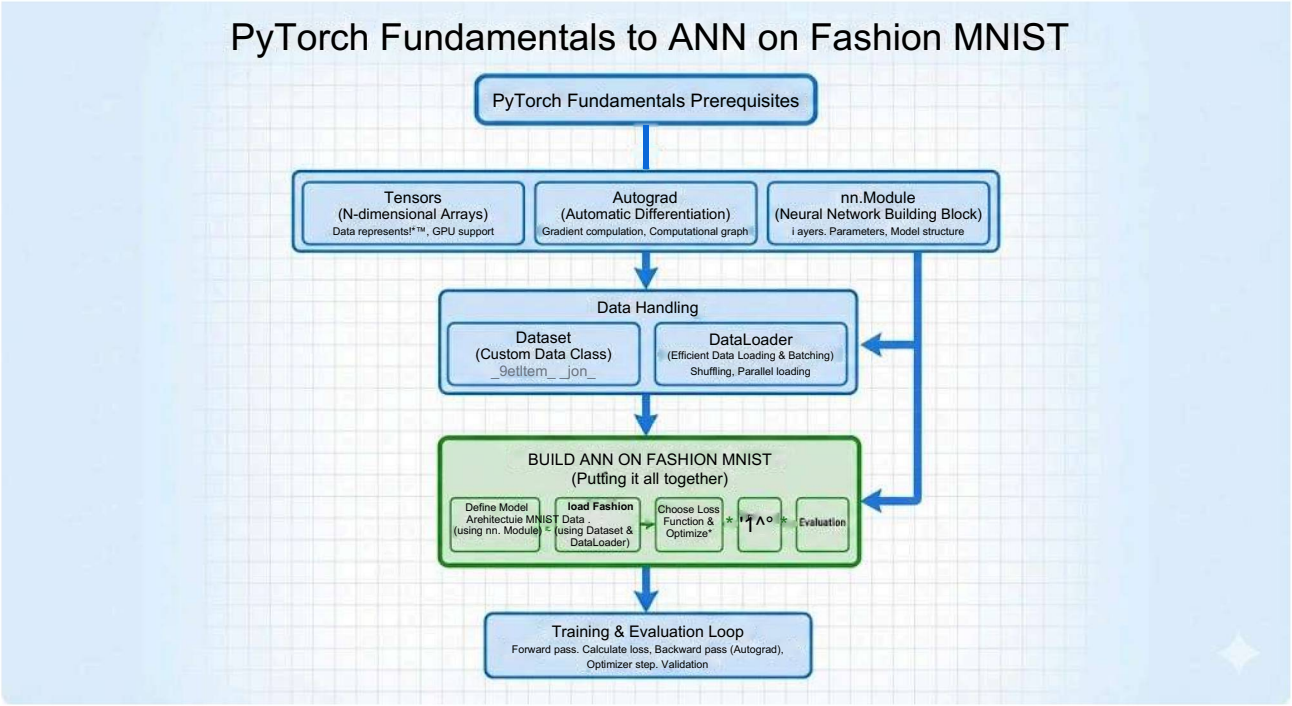
What You'll Learn

Concept	Description
Custom Dataset Class	Creating PyTorch-compatible dataset objects
DataLoader Objects	Batch processing fortraining and testing
Neural Network Architecture	Designing feed-forward networks with nn.Module

Concept	Description
Training Pipeline	Forward pass, loss calculation, backpropagation, optimization
Model Evaluation	Accuracy calculation on test data

2. PyTorch Fundamentals Recap

Before building the ANN, it's essential to understand the foundational PyTorch concepts covered in prerequisite learning:



Prerequisite Topics Summary

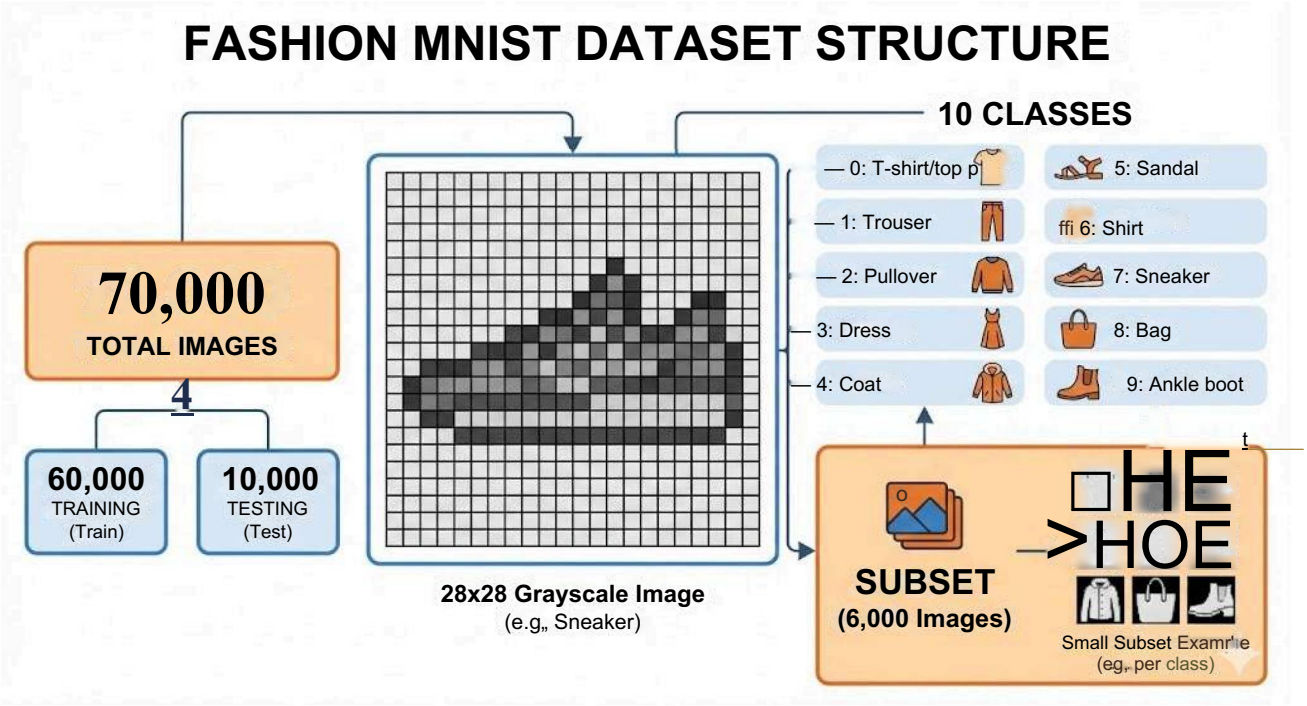
Video	Topic	Key Concepts
1	Introduction to PyTorch	Framework overview, installation, basic operations
2	Tensors	Multi-dimensional arrays, tensor operations, GPU support
3	Autograd	Automatic gradient computation, computational graphs
4	Training Pipeline	Manual implementation of training loop

Video	Topic	Key Concepts
5	nn.Module	Abstraction for neural network layers and models
6	Dataset & DataLoader	Batch creation, shuffling, data iteration

3. Dataset Overview: Fashion MNIST

About the Dataset

Fashion MNIST is a dataset of Zalando's article images, serving as a drop-in replacement for the classic MNIST dataset. It's commonly used for benchmarking machine learning algorithms.



Dataset Specifications

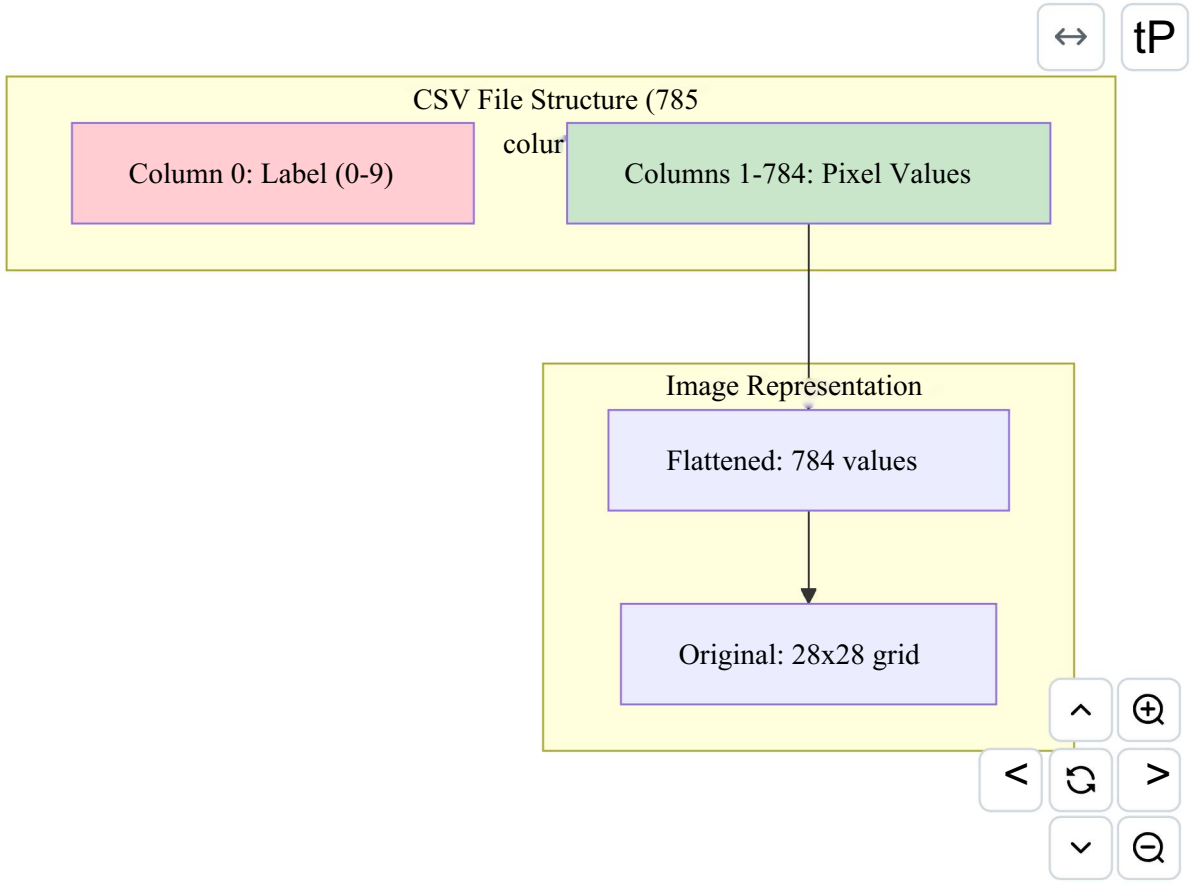
Property	Value
Image Dimensions	28 x 28 pixels
Total Images	70,000 (60,000 train + 10,000 test)
Number of Classes	10
Pixel Value Range	0 - 255 (grayscale)

Property	Value
Flattened Features	784 (28 x 28)
Data Format	CSV with 785 columns (1 label + 784 pixels)

Class Labels

Label	Class Name	Label	Class Name
0	T-shirt/Top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle Boot

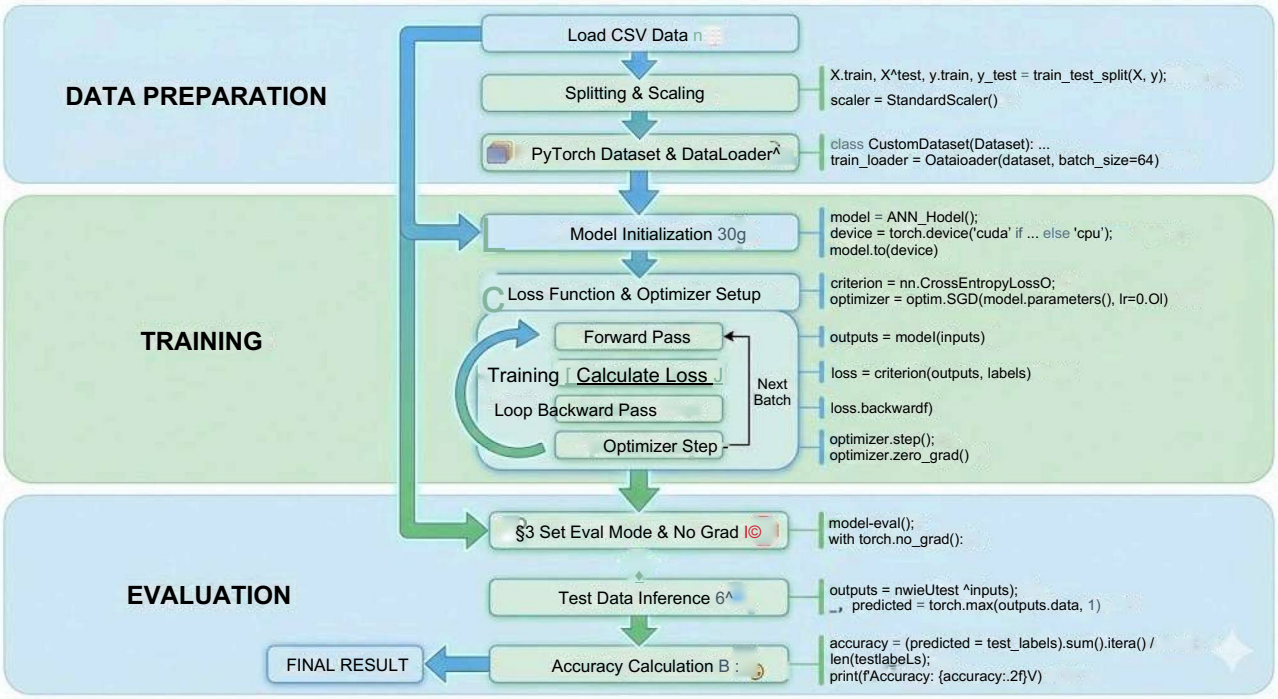
Data Structure Visualization



4. Project Architecture and Workflow

Three-Step Workflow

The entire project follows a systematic three-step approach:



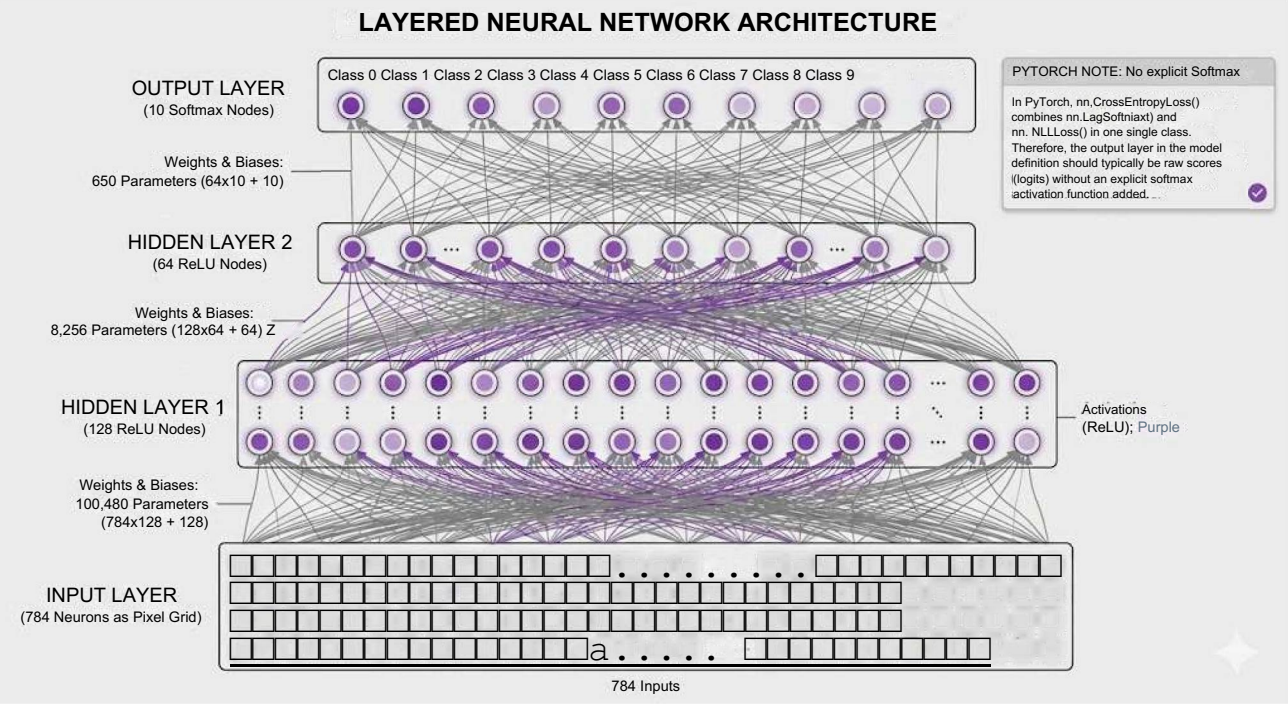
Detailed Workflow Table

Step	Sub-Step	Description	PyTorch Component
1. Data Preparation	Load Data	Read CSV file into DataFrame	pandas
	Extract Features	Separate X (pixels) and y (labels)	numpy slicing
	Split Data	80% train, 20% test	train_test_split
	Scale Features	Normalize to [0, 1] range	Division by 255
	Create Dataset	Custom Dataset class	torch.utils.data.Dataset
	Create DataLoader	Batch and shuffle data	torch.utils.data.DataLoader

Step	Sub-Step	Description	PyTorch Component
2. Training	Initialize Model	Create NN instance	nn.Module subclass
	Loss Function	Cross-entropy for multi-class	nn.CrossEntropyLoss
	Optimizer	Stochastic Gradient Descent	optim.SGD
	Training Loop	Forward, loss, backward, update	Custom loop
3. Evaluation	Eval Mode	Disable training behaviors	model. eval()
	No Gradients	Disable gradient computation	torch.no_grad()
	Accuracy	Compare predictions vs labels	Manual calculation

5. Neural Network Architecture Design

Architecture Diagram



Architecture Specifications

Layer	Type	Input Size	Output Size	Activation Function	Parameters
Input	Linear	784	128	Re LU	784X128 + 128 = 100,480
Hidden 1	Linear	128	64	Re LU	128x64 + 64 = 8,256
Output	Linear	64	10	Softmax (implicit)	64X10 + 10 = 650
Total	-	-	-	-	**109,386**

Parameter Calculation

For a fully connected (Linear) layer, the number of parameters is calculated as:

$$\text{Parameters} = (\text{Input Size} \times \text{Output Size}) + \text{Output Size (bias)}$$

Layer 1 (Input -> Hidden 1):

$$P_r = (784\text{x } 128) + 128 = 100{,}352 + 128 = 100{,}480$$

Layer 2 (Hidden 1 -» Hidden 2):

$$P_2 = (128 \text{ x } 64) + 64 = 8{,}192 + 64 = 8{,}256$$

Layer 3 (Hidden 2 -> Output):

$$P_3 = (64 \text{ x } 10) + 10 = 640 + 10 = 650$$

Total Parameters:

$$Ptotal = Pi + P_2 + P3 = 100{,}480 + 8,\ 256 + 650 = 109{,}386$$

Why No Explicit Softmax?

In PyTorch, when using nn.CrossEntropyLoss , the softmax activation is **implicitly applied** internally. This is different from frameworks like Keras where you explicitly add a softmax layer.

CrossEntropyLoss combines:

1. **LogSoftmax:** $\log\left(\frac{e^{x_i}}{\sum_j e^{x_j}}\right)$
2. **Negative Log Likelihood Loss (NLLLoss)**

6. Implementation: Step-by-Step

6.1 Data Loading and Preprocessing

Import Dependencies

```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as pit

# Set random seeds for reproducibility
torch.manual_seed(42)
```

Load and Explore Data

```
# Load the dataset
df = pd.read_csv('fmnist_small.csv')
df.head()
```

Extract Features and Labels

```
#  X: All columns except the first (pixel values)
#  y: First column only (labels)
X = df.ilocf:, 1:].values                    # Shape: (6000, 784)
y = df.iloc[:, 0].values                     # Shape: (6000,)
```

Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,                  # 20% for testing
    random_state=42                 # For reproducibility
```

```
)
# X_train: (4800, 784), X_test: (1200, 784)
```

Feature Scaling (Normalization)

```
# Scale pixel values from [0, 255] to [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0
```

Why Scale?

Aspect	Before Scaling	After Scaling
Value Range	0 - 255	0 - 1
Training Stability	Less stable	More stable
Gradient Flow	Can be problematic	Smoother
Convergence	Slower	Faster

The mathematical transformation:

$$\blacksquare A\ scaled = \frac{\hat{}original}{255}$$

Where 255 is the maximum pixel value in grayscale images.

6.2 Custom Dataset Class

The custom Dataset class inherits from torch.utils.data.Dataset and must implement three methods:

Implementation

```
class CustomDataset(Dataset):
    def____ init____ (self, features, labels):
        # Convert to PyTorch tensors with appropriate dtypes
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def ____len___ (self):
        return len(self.features)
```

```
    def ____getitem____ (self, index):
        return self.features[index], self.labelsfindex]
```

Data Type Requirements

Data	Required dtype	Reason
Features (X)	torch.float32	Neural network computations require floating-point
Labels (y)	torch.long	CrossEntropyLoss expects integer class indices

Create Dataset Objects

```
# Training dataset
train_dataset = CustomDataset(X_train, y_train)
print(len(train_dataset)) # Output: 4800

# Testing dataset
test_dataset = CustomDataset(X_test, y_test)
print(len(test_dataset))                    # Output: 1200
```

6.3 DataLoader Objects

DataLoader wraps a Dataset and provides batching, shuffling, and parallel data loading.

Create DataLoader Objects

```
# Training DataLoader with shuffling
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True # Shuffle for training
)

# Test DataLoader without shuffling
test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False          # No shuffle for evaluation
)
```

DataLoader Parameters Explained

Parameter	Training	Testing	Reason
batch_size	32	32	Process 32 samples at a time
shuffle	True	False	Randomize training; keep order for evaluation

Number of Batches:

$$\text{Number of Batches} = \left\lceil \frac{\text{Total Samples}}{\text{Batch Size}} \right\rceil$$

- Training: [4800 / 32] = 150 batches
- Testing: [1200 / 32] = 38 batches

6.4 Neural Network Class Definition

The neural network class inherits from nn.Module and uses nn.sequential for layer organization.

Implementation

```
class MyNN(nn.Module):
    def _____init_____(self, num_features):
        #  IMPORTANT: Always call parent constructor first
        super()._____init____ ()

        #  Define architecture using Sequential container
        self.model = nn .Sequential
            nn.Linear(num_features, 128), # Input -» Hidden 1
            nn.ReLU(),                    # Activation 1
            nn.Linear(128, 64),           # Hidden 1 -» Hidden 2
            nn.ReLU(),                    # Activation 2
            nn. Linear (64, 10)           # Hidden 2 → Output
            #   Note: No Softmax here (included in CrossEntropyLoss)
        )

    def f o rwa r d (s e lf, x):
        return self.model(x)
```

Why super()._____ init___ () is Critical

Without super().init()	With super().init()
nn.Module not initialized	nn.Module properly set up

Without super().init()	With super().init()
Parameter tracking fails	Parameters tracked correctly
.parameters() won't work	.parameters() works
Optimizer cannot access weights	Optimizer can access weights

6.5 Training Configuration

Before the training loop, set up hyperparameters and instantiate components:

tP

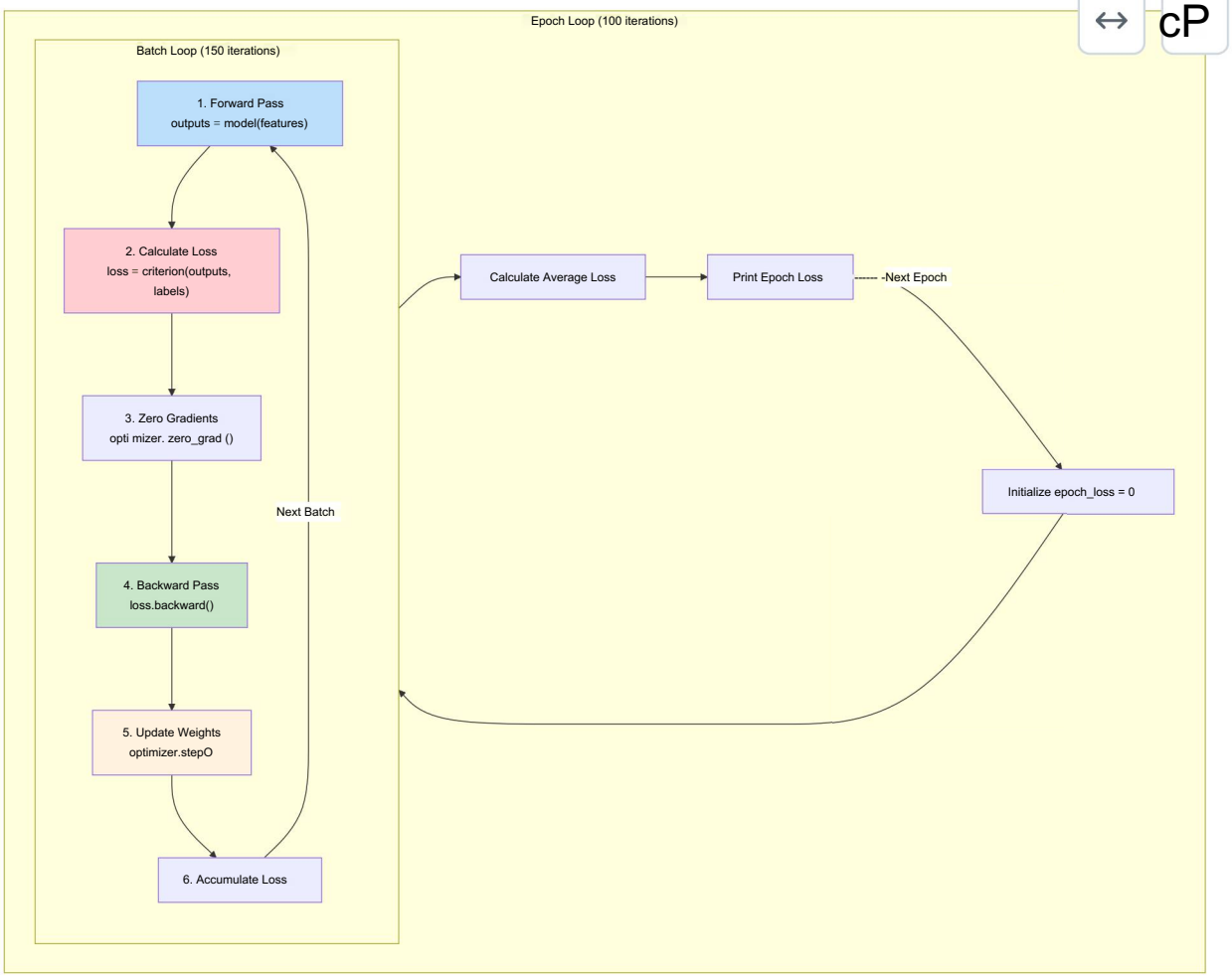
```
# Hyperparameters
epochs = 100
learning_rate = 0.1

#  Instantiate the model
model = MyNN(X_train.shapefl]) # 784 features

#  Loss function (includes Softmax internally)
criterion = nn.CrossEntropyLoss()

#  Optimizer
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
```

Training Components Summary

Component	Type	Purpose
model	MyNN	Neural network instance
criterion	CrossEntropyLoss	Measures prediction error
optimizer	SGD	Updates weights based on gradients
epochs	int (100)	Number of full dataset passes
learning_rate	float (0.1)	Step size for weight updates

6.6 Training Loop

The training loop is the core of the learning process, consisting of nested iterations.



Implementation

```
# Training loop
for epoch in range(epochs):
    total_epoch_loss = 0
```

```
for batch_features, batch_labels in train_loader:
        # Step 1: Forward pass
        outputs = model(batch_features)

        #   Step 2: Calculate loss
        loss = criterion(outputs, batch_labels)

        #   Step 3: Zero gradients (clear previous gradients)
        optimizer.zero_grad()

        #   Step 4: Backward pass (compute gradients)
        loss.backward()

        #   Step 5: Update weights
        optimizer.step()

        #   Accumulate loss for this epoch
        total_epoch_loss = total_epoch_loss + loss.item()

# Calculate average loss per batch
avg_loss = total_epoch_loss / len(train_loader)
print(fEpoch: {epoch + l}, Loss: {avg_loss}')
```

Training Steps Explained

Step	Method	Mathematical Operation
Forward Pass	model(x)	$\hat{y}$ = f^x- 0,
Loss Calculation	criterion(outputSj labels)	L = - 2$_z$yjog(y$_f$) ^
Zero Gradients	optimizer.zero_grad()	V$_e$^0
Backward Pass	loss.backward()	$\nabla_\theta L = \frac{\partial L}{\partial \theta}$
Update Weights	optimizer.step()	*9 <- 9 — rfi7$_g$L*

Why Zero Gradients?

PyTorch **accumulates gradients** by default. Without zeroing:

$$V_\theta® = \nabla_\theta^{(t-1)} + \nabla_\theta^{(current)}$$

This causes incorrect gradient values. We want:

$$\frac{_(t)}{^V0} _ _ (current)$$

6.7 Model Evaluation

Evaluation assesses model performance on unseen test data.

Implementation

```
# Set model to evaluation mode
model.eval()

# Initialize counters
total = 0
correct = 0

# Disable gradient computation
with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        # Forward pass
        outputs = model(batch_features)

        # Get predictions (class with highest probability)
        _,  predicted = torch.max(outputSj 1)

        # Update counters
        total = total + batch_labels.shape[0]
        correct = correct + (predicted == batch_labels).sum().item()

# Calculate and print accuracy
accuracy = correct / total
print(f'Accuracy: {accuracy * 100:.2f}%')
```

Why model.eval()?

Some layers behave differently during training vs. inference:

Layer Type	Training Behavior	Inference Behavior
Dropout	Randomly drops neurons	Uses all neurons
BatchNorm	Computes batch statistics	Uses stored statistics

`model.eval()` tells these layers to switch to inference behavior.

Why torch.no_grad()?

With Gradients	Without Gradients
Tracks computation graph	No graph tracking
Higher memory usage	Lower memory usage
Slower execution	Faster execution
Needed for training	Sufficient for inference

7. Understanding Model Outputs

Output Tensor Structure

When you pass a batch through the model, you get probability-like scores (logits) for each class:

Output Interpretation

For a batch of 32 images with 10 classes, the output is a 32x10 tensor:

$$\text{outputs} \rightarrow \begin{bmatrix} s_{1,0} & s_{1,1} & \cdots & s_{1,9} \\ s_{2,0} & s_{2,1} & \cdots & s_{2,9} \\ \vdots & \vdots & \ddots & \vdots \\ s_{32,0} & s_{32,1} & \cdots & s_{32,9} \end{bmatrix}$$

Where *s^j* is the score for image *i* belonging to class *j.*

Extracting Predictions

```
# torch.max returns (values, indices)
_, predicted = torch.max(outputs, 1)
```

For dimension = 1 (along columns), this finds the maximum value in each row:

$$\text{predicted}_i = \arg\max_j s_{i,j}$$

Visual Example

Image	Class 0	Class 1	Class 2	...	Class 9	Prediction
1	0.1	0.2	**0.8**	...	0.05	**2**

Image	Class 0	Class 1	Class 2	...	Class 9	Prediction
2	0.05	**0.9**	0.02	...	0.01	**1**
3	**0.7**	0.1	0.1	...	0.02	**0**

8. Performance Improvement Strategies

Improvement Options Table

Category	Technique	Current	Potential Change
Data	Dataset Size	6,000	70,000 (full)
Optimizer	Type	SGD	Adam, RMSprop
Hyperparameter	Learning Rate	0.1	Experiment (0.001 -0.01)
Training	Epochs	100	200+
Regularization	Dropout	None	0.2 - 0.5
Regularization	BatchNorm	None	After each linear layer
Architecture	Hidden Layers	2	3-4
Hardware	Compute	CPU	GPU (CUDA)

Expected Impact

Change	Expected Accuracy Improvement
Full Dataset + GPU	+ 5-10%
Adam Optimizer	+ 1-3%
Dropout (0.3)	+ 1-2%
Batch Normalization	+2-3%
Learning Rate Tuning	+ 1-2%

Goal: Cross 90% accuracy through combination of techniques.

9. Complete Code Reference

Full Implementation

cP

```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim

# ============== CONFIGURATION ==============
torch.manual_seed(42)
BATCH_SIZE = 32
EPOCHS = 100
LEARNING_RATE =  0.1

# ============== DATA LOADING ==============
df = pd.read_csv('fmnist_small.csv')
X = df.ilocf:, 1:].values
y = df.iloc[:, 0].values

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Feature scaling
X_train = Xtrain / 255.0
X_test = X_test / 255.0

# ============== CUSTOM DATASET ==============
class CustomDataset(Dataset):
    def ____init____ (self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def ____len___ (self):
        return len(self.features)

    def ____getitem____ (self, index):
        return self.featuresfindex], self.labels[index]

# Create datasets
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)
```

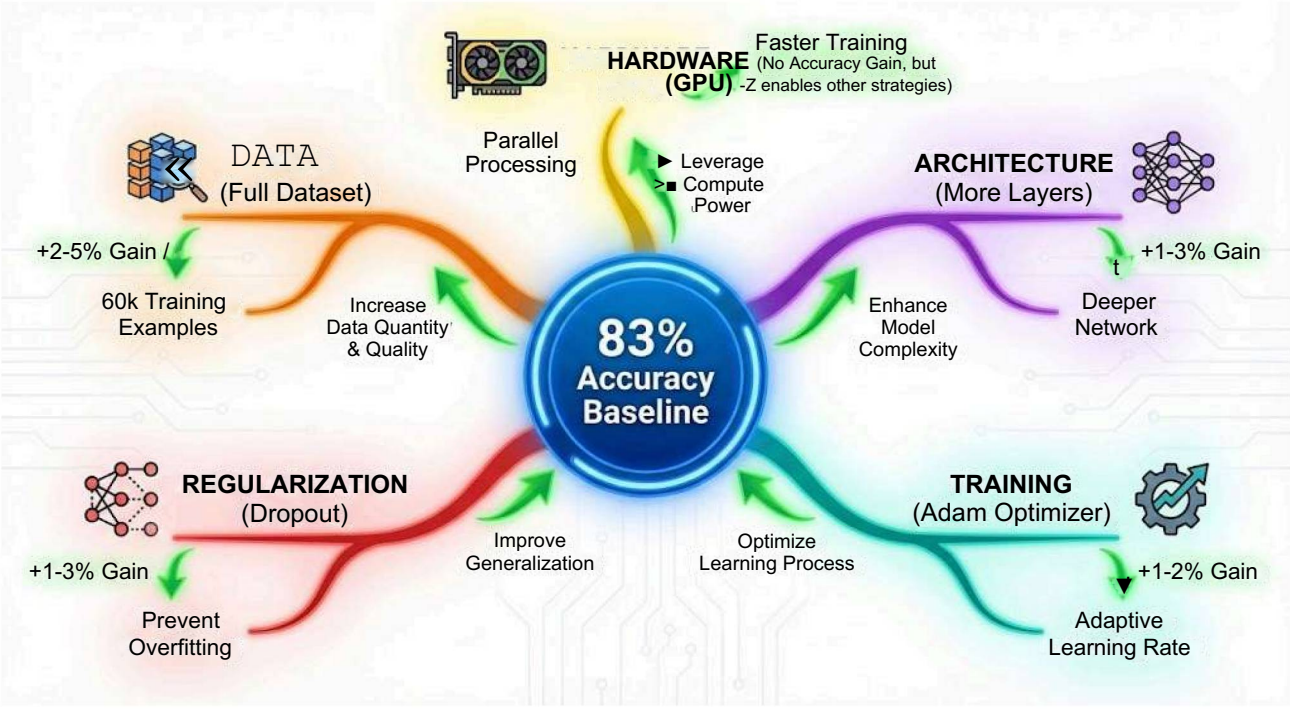
```
# Create dataloaders
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

#  ==============  NEURAL NETWORK ==============
class MyNN(nn.Module):
    def init____(self, num_features):
        super()._______init_____()
        self.model = nn .Sequential
            nn.Linear(num_features, 128),
            nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)

#  ==============  TRAINING SETUP    ==============
model = MyNN(X_train.shapefl])
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=LEARNING_RATE)

#  ==============  TRAINING LOOP    ==============
for epoch in range(EPOCHS):
    total_epoch_loss = 0
    for batch_features, batch_labels in train_loader:
        outputs = model(batch_features)
        loss = criterion(outputs, batch_labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_epoch_loss += loss.item()

    avg_loss = total_epoch_loss / len(train_loader)
    if (epoch + 1) % 10 == 0: # Print every 10 epochs
        print(f'Epoch: {epoch + 1}, Loss: {avg_loss:.4f}')

# ==========  ====  EVALUATION  ==============
model.eval()
total = 0
correct = 0

with torch.no gradQ_:
    for batch_features, batch_labels in test_loader:
        outputs = model(batch_features)
        _, predicted = torch.max(outputs, 1)
        total += batch_labels.shape[0]
```

```
            correct += (predicted == batch_labels).sum().item()

print(f'\nTest Accuracy: {100 * correct / total:.2f}%')
```

10. Key Takeaways

Summary Checklist

Concept	Key Point
Custom Dataset	Inherit from Dataset , implement init __ __ , __ len __ , _getitem____ .
Data Types	Features -» fioat32 , Labels -» long
DataLoader	Shuffle training data, don't shuffle test data
nn.Module	Always call super()._init_() first
nn.Sequential	Container for layer stacking
CrossEntropyLoss	Includes Softmax internally
Training Loop	Forward -» Loss -> Zero Grad -► Backward -► Step
Evaluation	Use model.eval() and torch.no_grad()



Building ANN with PyTorch

Fashion MNIST Multi-Class Classification

3-Step Workflow

1 Data Preparation

- Load CSV (6000 images)
- Extract X (784 pixels) & y (labels)
- Split: 80% train, 20% test
- Scale: pixel/255 [0,1]
- Create Custom Dataset
- Create DataLoader (batch=32)

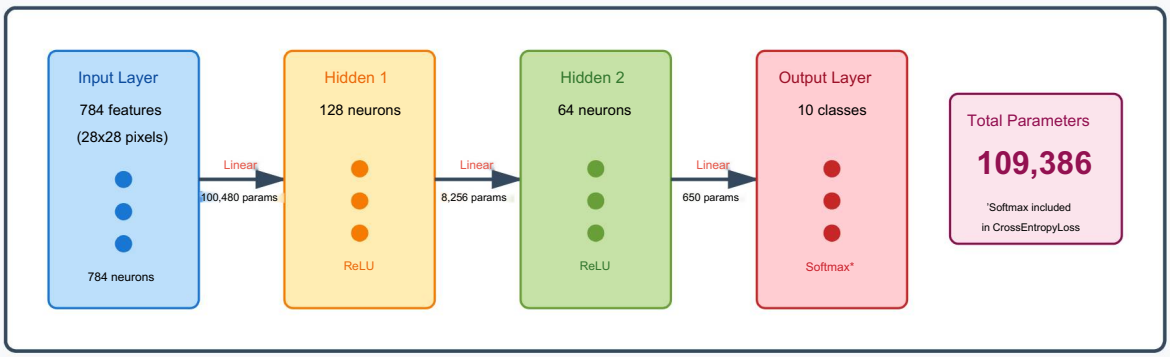
0 Training

- Initialize Model (MyNN)
- Loss: CrossEntropyLoss
- Optimizer: SGD (lr=0.1)
- Epochs: 100
- Forward —+ Loss -»Zero Grad
- Backward —>Optimizer Step

0 Evaluation

- model. eval()
- torch.no_grad()
- Forward pass
- torch.max(outputs, 1)
- Calculate accuracy
- Compare predictions

Neural Network Architecture



Fashion MNIST Dataset

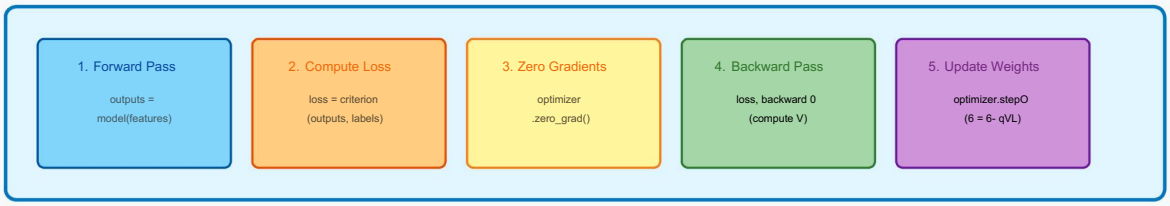
Specifications:

- Image Size: 28x28 pixels (grayscale)
- Total Images: 70,000 (60k train + 10k test)
- Number of Classes: 10
- Pixel Range: 0-255 —> normalized to 0-1
- Features: 784 (flattened 28x28)
- CSV Format: 1 label + 784 pixel columns

10 Fashion Classes

0: T-shirt/Top	5: Sandal
1: Trouser	6: Shirt
2: Pullover	7: Sneaker
3: Dress	8: Bag
4: Coat	9: Ankle Boot

Training Loop (Per Batch)



Key Takeaways

/ Always call super()._____ init___() in nn.Module

7 Use model.eval() + torch.no_grad() for testing

CampusX

🚀 GPU Training in PyTorch - Complete Guide

Code

- htt ps://colab.research. g oog le.com/drive/17hjS23Cg Fg IZjuB2XKt2u7 pVu1 M pD2Qr

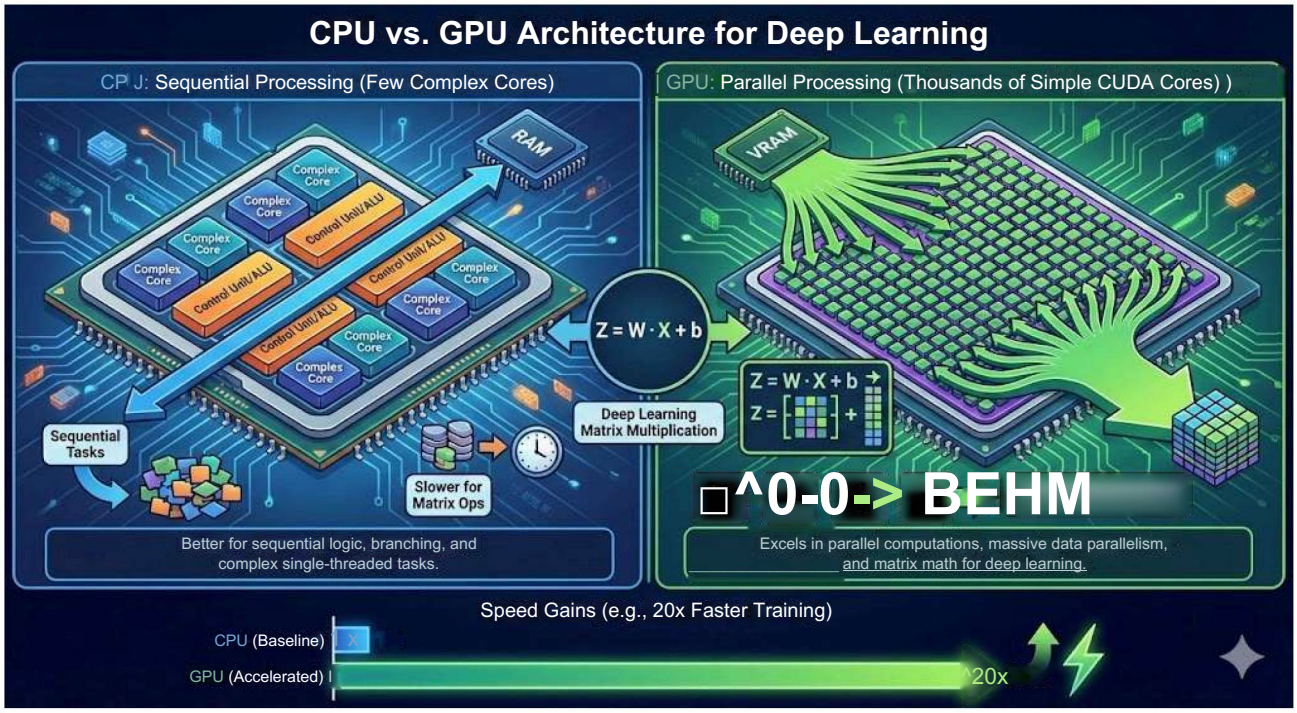
1. Introduction & Motivation

Context: Fashion MNIST Classification Problem

This guide continues from a Fashion MNIST classification project where:

- **Previous Setup:** Training on CPU with a subset of 6,000 images
- **Previous Accuracy:** -83%
- **Goal:** Use full dataset (60,000 training images) to improve accuracy
- **Solution:** GPU-accelerated training

Why Move to GPU?



Key Benefits of GPU Training

Aspect	CPU Training	GPU Training
Data Size	Limited (subset required)	Full dataset support
Speed	Slower	Significantly faster
Parallelization	Limited cores	Thousands of CUDA cores
Memory	System RAM	Dedicated VRAM
Accuracy Potential	Lower (less data)	Higher (more data)

2. CPU vs GPU: Why GPU for Deep Learning?

Why Deep Learning Favors GPUs

Deep learning operations are fundamentally **matrix operations:**

Forward Pass (Matrix Multiplication):

$$Z = W \blacksquare X + b$$

Where:

- W = Weight matrix of shape *(output features, input features)*

- X = Input tensor of shape *(batch$_s$ize, input feature s')*
- b = Bias vector of shape *{output features)*
- Z = Output tensor

Parallelization Opportunity:

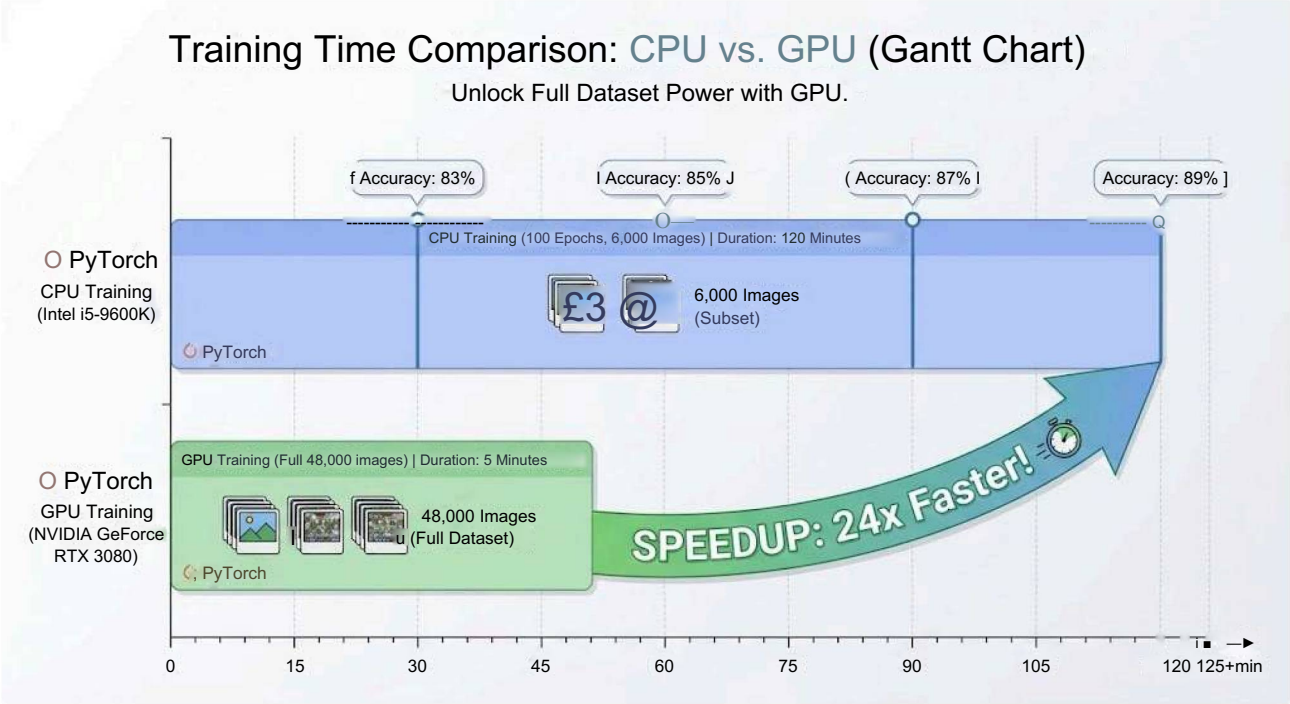
For a batch of 32 samples with 784 features each:

Operations per layer = *batch$_s$ize* x *inputfeatures* x *output features*

= 32 x 784 x 128 = 3,211,264 operations

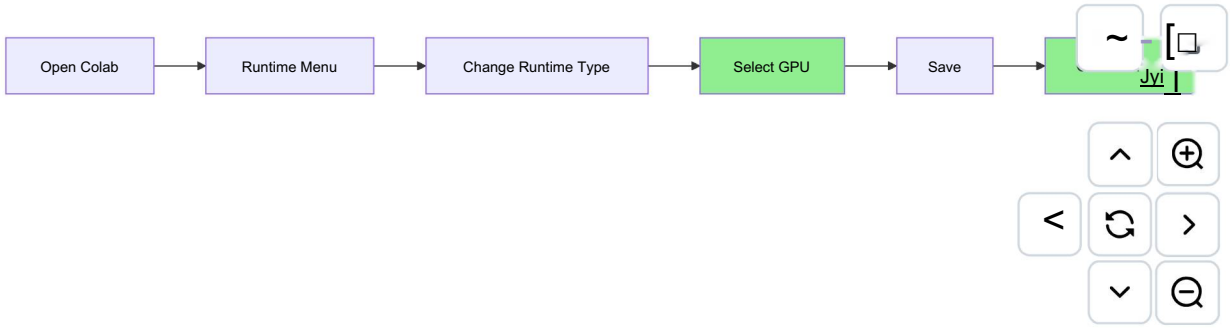
GPUs can perform these operations in parallel, while CPUs process them sequentially.

Training Time Comparison



3. Prerequisites & Setup

Step 1: Environment Setup (Google Colab)



Step 2: Dataset Preparation

Dataset Statistics:

Split	Images	Purpose
Training	60,000	Model training
Test	10,000	Model evaluation
After 80-20 Split		
Training (80%)	48,000	Model training
Validation (20%)	12,000	Model validation

Data Properties:

- Image Size: 28 x 28 pixels
- Channels: 1 (Grayscale)
- Features per image: 784 (28 x 28 flattened)
- Classes: 10 (Fashion categories)
- File Size:-126 MB

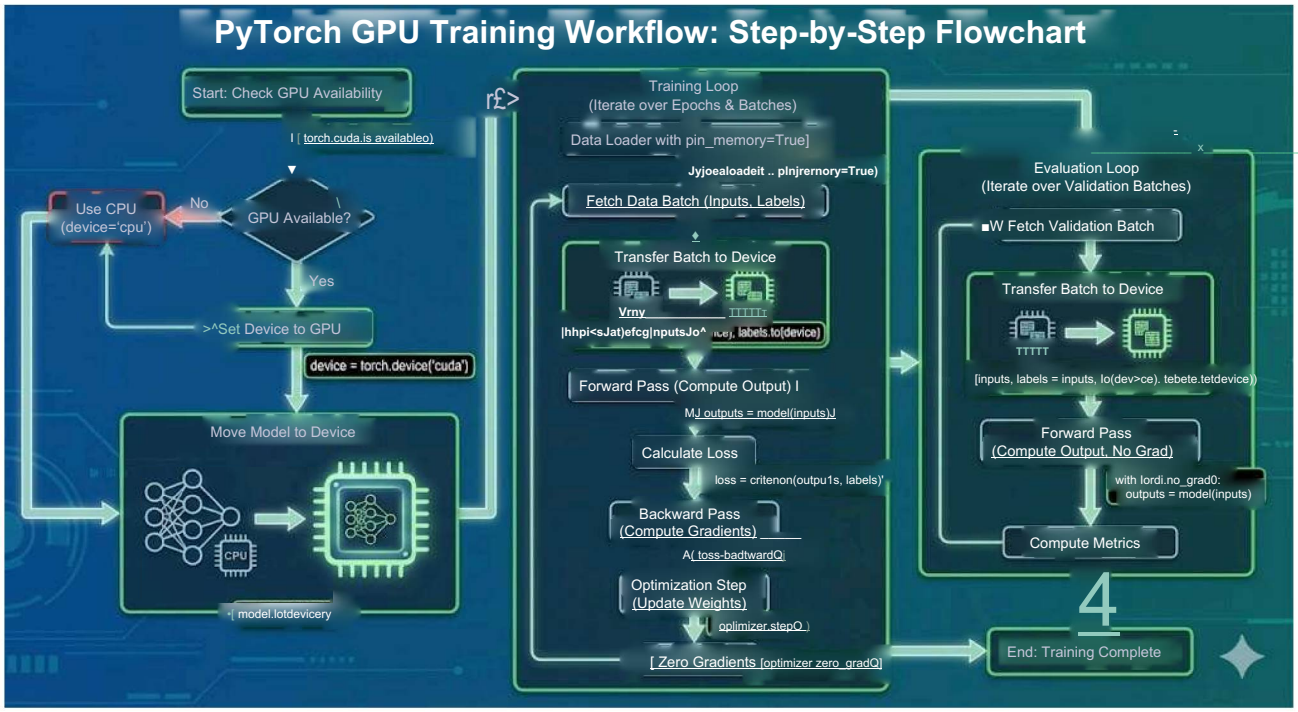
Fashion MNIST Class Labels

Label	Category
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress

Label	Category
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

4. Step-by-Step GPU Training Workflow

Complete Workflow Diagram



Detailed Step Breakdown

Step 1: Check GPU Availability

```
import torch

# Check GPU availability and set device
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

Output:

```
Using device: cuda
```

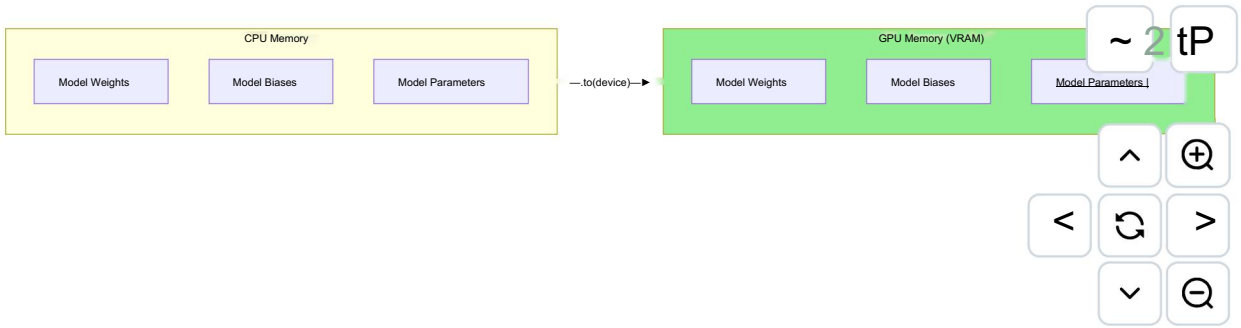
Explanation:

- torch.cuda.is_available() returns True if CUDA-capable GPU is available
- We store the device in a variable for consistent use throughout the code
- This pattern ensures code portability between GPU and CPU systems

Step 2: Move Model to GPU

```
# Create model instance
model = MyNN(num_features=784)

# Move model to GPU
model = model.to(device)
```

What Happens Internally:



Step 3: Move Training Data to GPU

```
for epoch in range(epochs):
    for batch_features, batch_labels in train_loader:
        # Move data to GPU - CRITICAL STEP
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        # Now perform computations on GPU
        outputs = model(batch_features)
```

```
            loss = criterion(outputs, batch_labels)
            # . .. rest of training loop
```

Step 4: Move Evaluation Data to GPU

```
model.eval()
with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        # Move test data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        outputs = model(batch_features)
        # ... evaluation logic
```

5. Complete Code Implementation

Full Training Pipeline with GPU Support

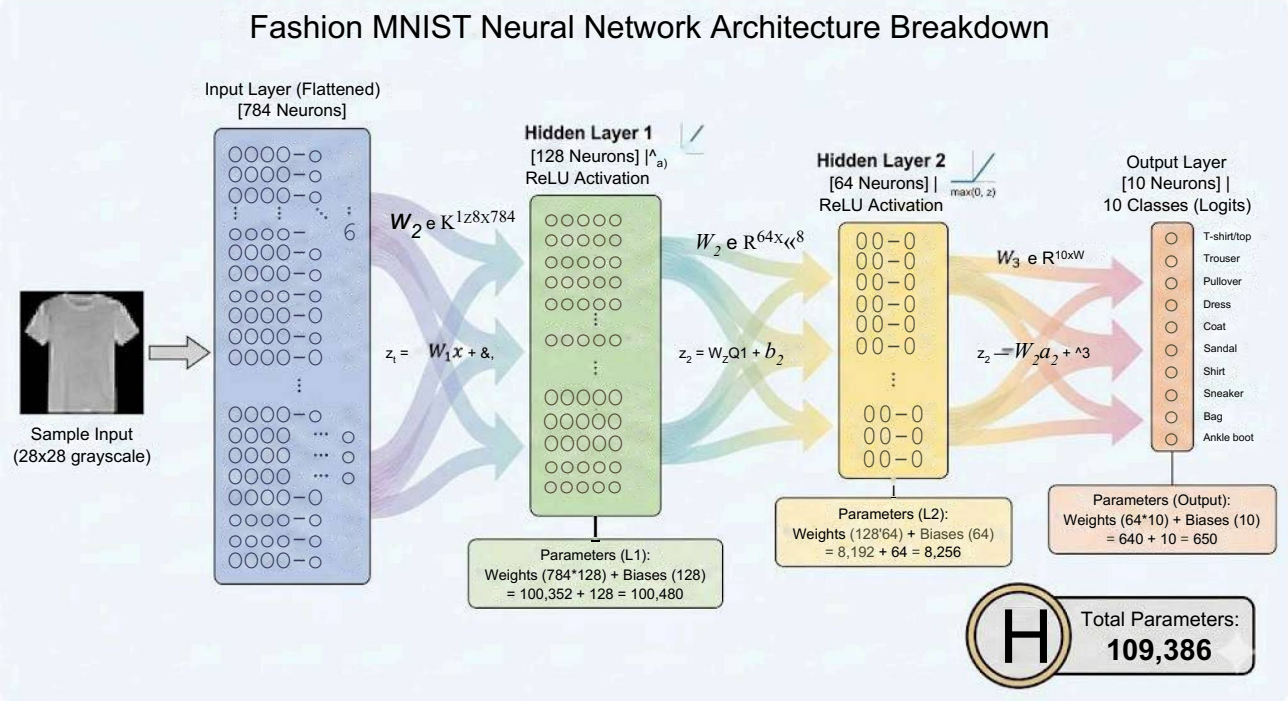
```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as pit

# ====================================
# STEP 1: Set Random Seeds & Check GPU
# ====================================
torch.manual_seed(42)

# Check for GPU availability
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# ==========================================
# STEP 2: Load and Explore Data
# ==========================================
df = pd.read_csv('/content/fashion-mnist_train.csv')
print(f"Dataset shape: {df.shape}")
print(f"Total images: {df.shape[0]}")

# =========================
# STEP 3: Data Preprocessing
```

```
# ==========================================
# Separate features and labels
X = df.iloc[:, 1:].values # All columns except first (pixels)
y = df.iloc[:, 0].values # First column (labels)

# Train-test split (80-20)
X_train, X_test, y_train, y_test = train_test_split(
    X , y, test_size=0.2, random_state=42
)

# Normalize pixel values to [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0

print(f"Training samples: {X_train.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")

# ==========================================
# STEP 4: Custom Dataset Class
# ==========================================
class CustomDataset(Dataset):
    def____init____ (self, features, labels):
        # Convert numpy arrays to PyTorch tensors
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def ___ len___(self):
        return len(self.features)

    def ___ getitem____(self, index):
        return self.featuresfindex], self.labelsfindex]

# Create dataset objects
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)

# ==========================================
# STEP 5: Create DataLoaders with pin_memory
# ==========================================
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=True # Optimization for GPU training
)
test_loader = DataLoader(
    t e s t _d a t a s e t,
    batch_size=32,
    shuffle=False,
```

```
        pin_memory=True # Optimization for GPU training
)

print(f"Number of training batches: {len(train_loader)}")

# ========================================= =====
# STEP 6: Define Neural Network Model
# ========================================= =====
class MyNN(nn.Module):
    def ___ init____ (self, num_features):
        super().__init____ ()
        self.model = nn.Sequential(
            nn.Linear(num_features, 128),          # Input -> Hidden 1
            nn.ReLU(),
            nn.Linear(128, 64),                    # Hidden 1 -> Hidden 2
            nn.ReLU(),
            nn.Linear(64, 10)                      # Hidden 2 -> Output (10 classes)
        )

    def forward(self, x):
        return self.model(x)

# ========================================= ===
# STEP 7: Initialize Model, Loss, Optimizer
# ========================================= ===
learning_rate = 0.1
epochs = 100

# Create model and MOVE TO GPU
model = MyNN(X_train.shape[l])
model = model.to(device) # Critical: Move model to GPU

# Loss function
criterion = nn.CrossEntropyLoss()

# Optimizer
optimizer = optim.SGD(model.parameters(), lr=learning_rate)

# ============================== ==============
# STEP 8: Training Loop with GPU
# ============================== ==============
print("\nStarting Training...")
for epoch in range(epochs):
    total_epoch_loss = 0

    for batch_features, batch_labels in train_loader:
        # CRITICAL: Move data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)
```

```
            #  Forward pass
            outputs = model(batch_features)

            #  Calculate loss
            loss = criterion(outputs, batch_labels)

            #  Backward pass
            optimizer.zero_grad()
            loss.backward()

            #  Update weights
            optimizer.step()

            total_epoch_loss += loss.item()

        avg_loss = total_epoch_loss / len(train_loader)

        if (epoch + 1) % 10 == 0: # Print every 10 epochs
            print(fEpoch: {epoch + 1:3d}, Loss: {avg_loss:.4f}')

print("\nTraining Complete!")

# ========================================    ==
# STEP 9: Evaluation with GPU
# ========================================    ==
model.eval() # Set model to evaluation mode

total = 0
correct = 0

with torch.no_grad(): # Disable gradient computation
    for batch_features, batch_labels in test_loader:
        # CRITICAL: Move test data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        outputs = model(batch_features)
        _,  predicted = torch.max(outputs, 1)

        total += batch_labels.shape[0]
        correct += (predicted == batch_labels).sum().item()

accuracy = correct / total
print(f"\nTest Accuracy: {accuracy * 100:.2f}%")
```

Model Architecture Visualization



Layer-by-Layer Parameter Count

Layer	Input Size	Output Size	Parameters
Linear 1	784	128	784X128 + 128 = 100,480
Linear 2	128	64	128x64 + 64 = 8,256
Linear 3	64	10	64x10 + 10 = 650
Total			**109,386**

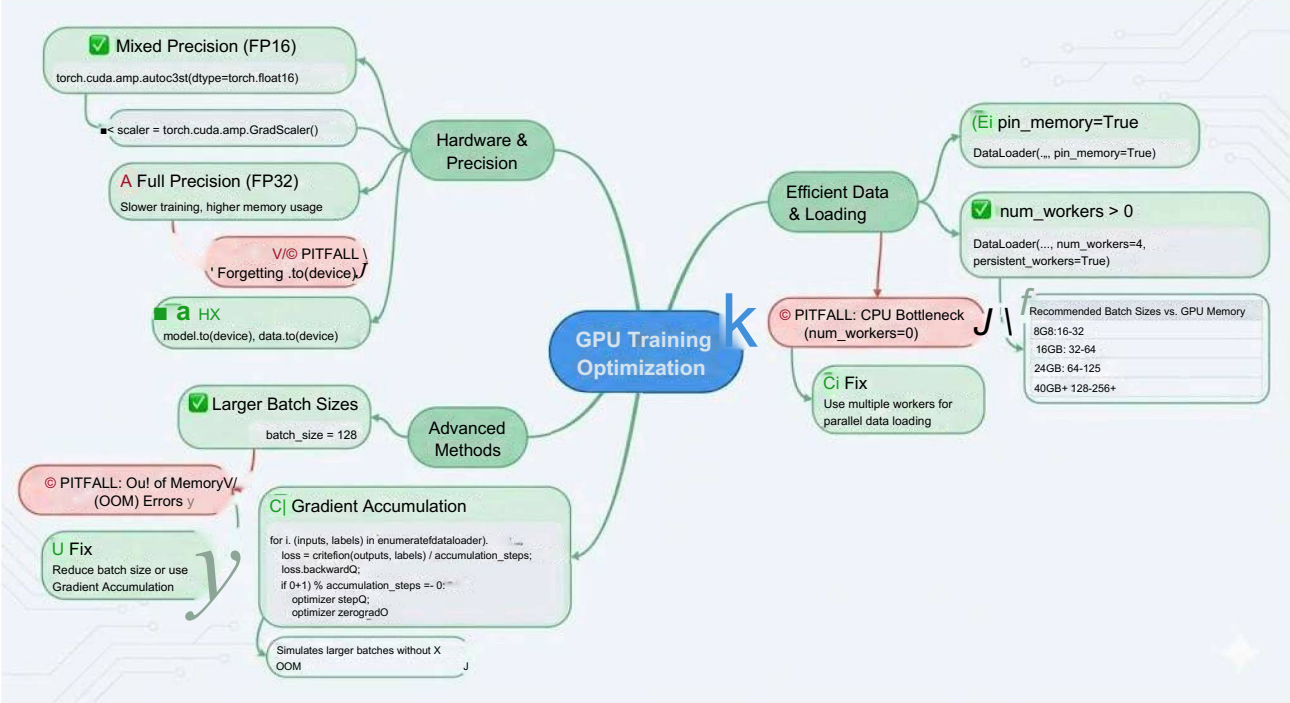
Parameter Calculation Formula:

Parameters = *(input features* x *output features)* + *output features*

= Weights + Biases

6. Memory Architecture: Paged vs Pinned Memory

Understanding Memory Types



Data Flow Comparison

When to Use pin_memory=True

Scenario	Use pin_memory?	Reason
Small dataset fits in GPU memory	Optional	Minimal benefit
Large dataset (streaming batches)	**Yes**	Significant speedup
Training with many epochs	**Yes**	Cumulative time savings
Memory-constrained system	Careful	Uses more RAM
Multi-GPU training	**Yes**	Critical for performance

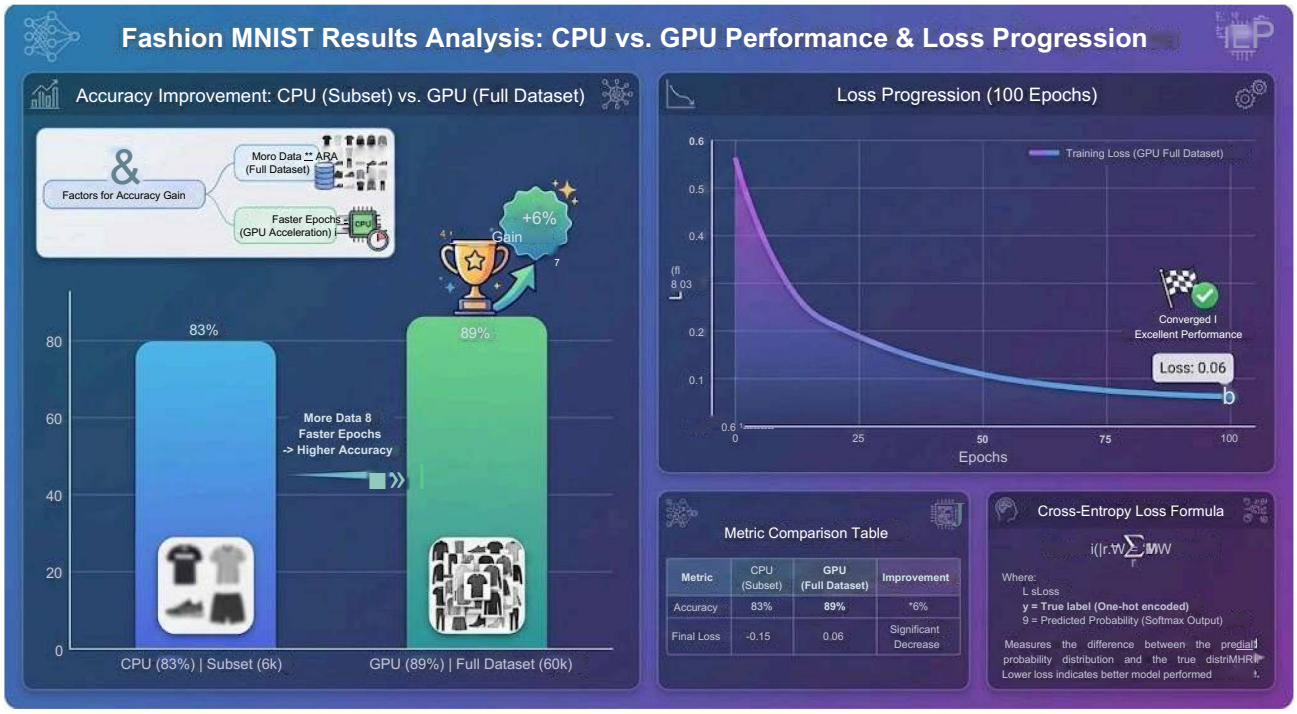
Code Implementation

cP

```
# DataLoader with pin_memory optimization
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=True,          # Pre-allocate pinned memory
    num_workers=2             # Optional: parallel data loading
)
```

7. Performance Optimization Techniques

Optimization Hierarchy



Batch Size Recommendations

GPU VRAM	Recommended Batch Size	Notes
4 GB	32 - 64	Entry-level GPUs
8 GB	64 - 128	Mid-range GPUs
12 GB	128 - 256	High-end consumer
16+ GB	256 - 512	Professional/Server

Batch Size Impact on Training:

'_' allowed only in math mode

$$
\text{Gradient Variance} \propto \frac{l}{\text{batch_size}}
$$

CampusX

Larger batches = More stable gradients = Potentially faster convergence

Optimization Techniques Summary

Technique	Implementation	Benefit
pin_memory	DataLoader(..., pin_memory=True)	Faster CPU GPU transfer
Larger batch	batch_size=64 or 128	Better GPU utilization
num_workers	DataLoader(..., num_workers=4)	Parallel data loading
Mixed Precision	torch.cuda.amp	2x speedup, less memory
Gradient Accumulation	Accumulate over mini-batches	Simulate larger batches

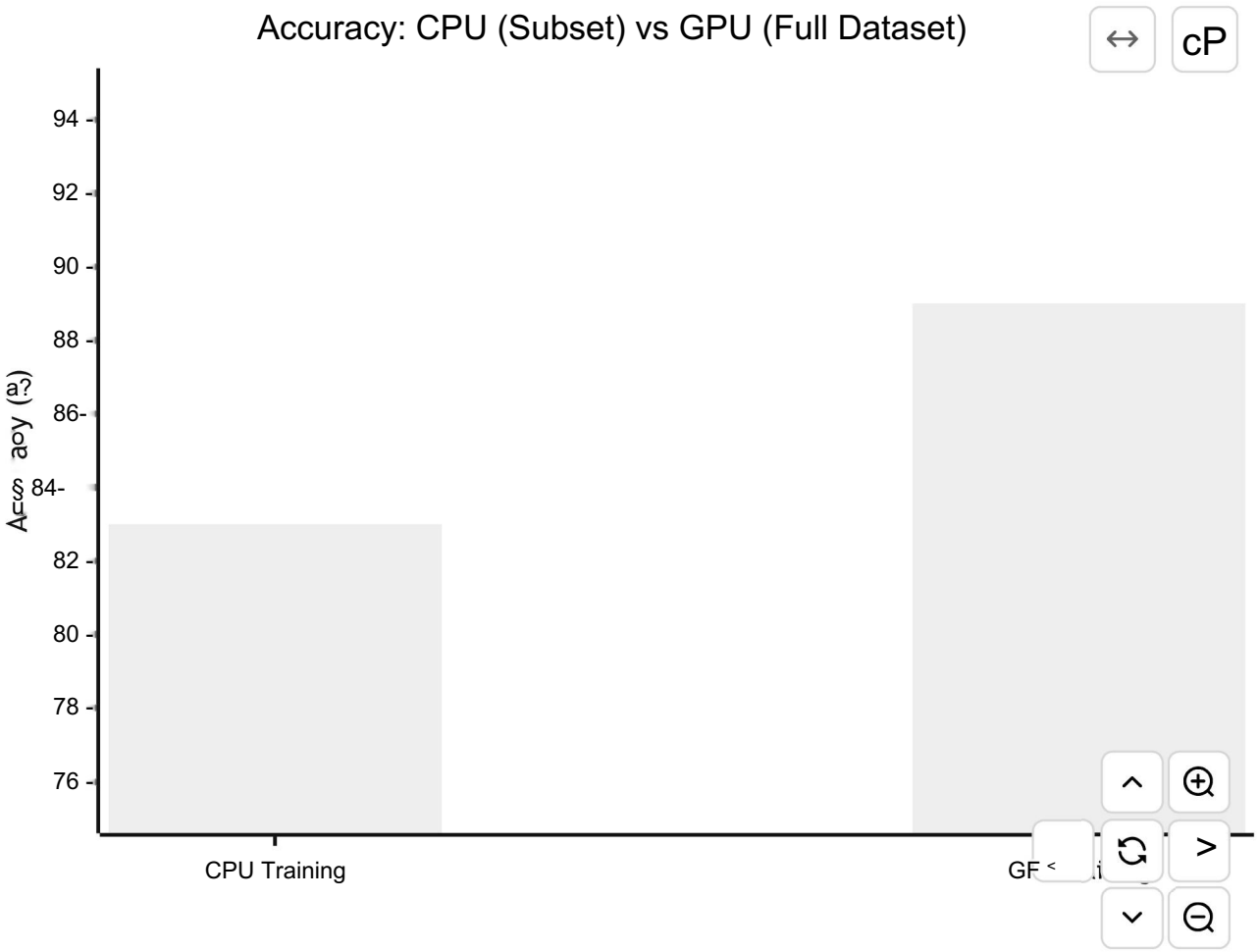
Optimized DataLoader Configuration

tP

```
# Fully optimized DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=128,              # Larger batch for GPU efficiency
    shuffle=True,
    pin_memory=True,             # Faster memory transfer
    num_workers=4,               # Parallel data loading
    prefetch_factor=2j           # Prefetch batches
    persistent_workers=True      # Keep workers alive between epochs
)
```

8. Results Analysis & Comparison

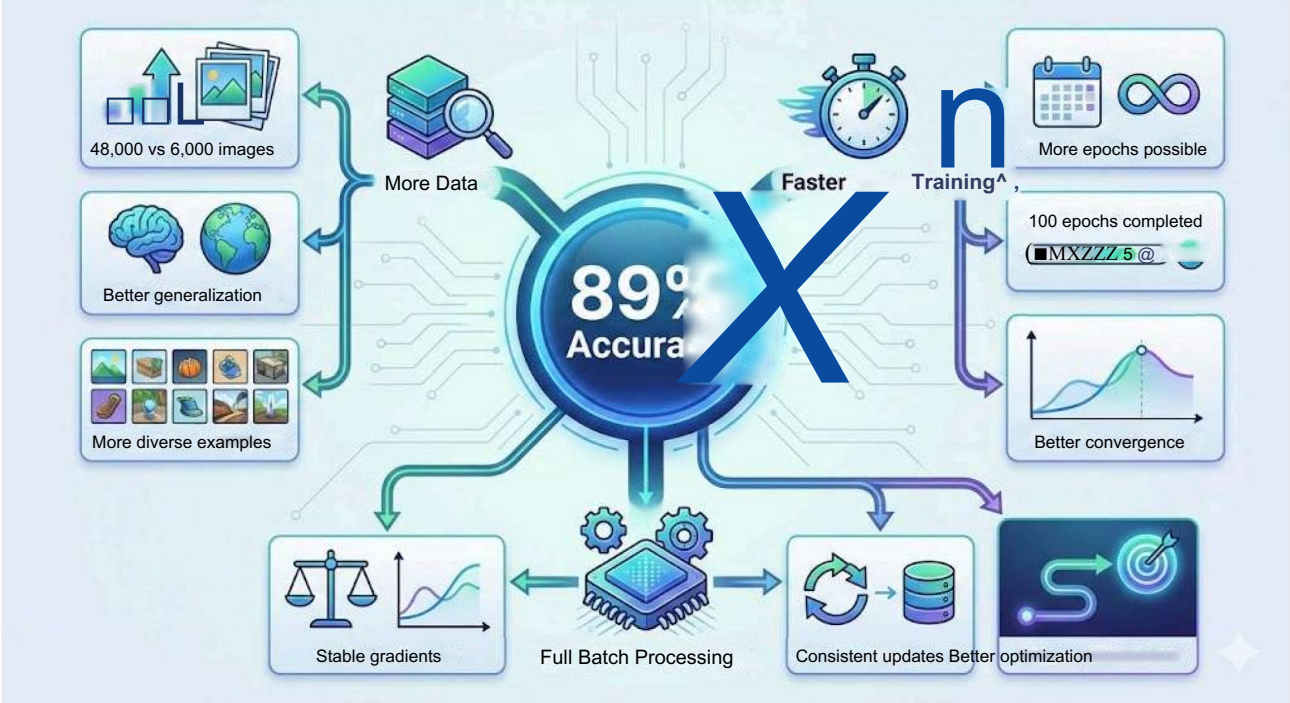
Accuracy Improvement



Detailed Comparison

Metric	CPU (Subset)	GPU (Full Dataset)	Improvement
Training Data	6,000 images	48,000 images	8x more data
Test Accuracy	83%	89%	+ 6%
Training Speed	Slow	Fast	~20x faster
Epochs	Limited	100 epochs	More training

Why Did Accuracy Improve?



Training Loss Progression

After 100 epochs:

- **Final Loss:** -0.06
- **Loss Trend:** Consistently decreasing
- **Convergence:** Good convergence observed

Loss Function (Cross-Entropy):

$$L = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{i,c}\log(\hat{y}_{i,c})$$

Where:

- N = Number of samples in batch
- C = Number of classes (10)
- $y_{i,c}$ True label (one-hot encoded)
- $\hat{y}_{i,c}$ Predicted probability for class c

9. Best Practices & Common Pitfalls

X Common Pitfalls

Pitfall	Problem	Solution
Forgot •to(device)	RuntimeError: tensors on different devices	Always move both model AND data
Wrong order	Optimizer has CPU parameters	Move model to GPU before creating optimizer
Memory leak	OOM during evaluation	Use torch.no_grad()
Inconsistent devices	Mixed CPU/GPU tensors	Check all tensors before operations
Not using eval mode	Dropout/BatchNorm active during test	Call model.eval()

Code Patterns to Avoid vs Preferred

X Wrong: Creating optimizer before moving model

```
model = MyNN(784)
optimizer = optim.SGD(model.parameters(), lr=0.1) # Optimizer has CPU params
model = model.to(device) # Model moved, but optimizer still references CPU
```

[P

Q Correct: Move model first

```
model = MyNN(784)
model = model.to(device) # Move model first
optimizer = optim.SGD(model.parameters(), lr=0.1) # Now optimizer has GPU par
```

tP

X Wrong: Missing data transfer

```
for batch_features, batch_labels in train_loader:
    outputs = model(batch_features) # Error! Data on CPU., model on GPU
```

[P

Q Correct: Transfer data to device

```
for batch_featuresbatch_labels in train_loader:
    batch_features = batch_features.to(device)
```

[P

```
        batch_labels = batch_labels.to(device)
        outputs = model(batch_features) # Works!
```

10. Quick Reference Cheat Sheet

Essential GPU Training Commands

tP

```
# ===== DEVICE SETUP =====
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# ===== MODEL TO GPU =====
model = model.to(device)

# ===== DATA TO GPU (in loops) =====
batch_features = batch_features.to(device)
batch_labels = batch_labels.to(device)

# ===== DATALOADER OPTIMIZATION =====
DataLoader(..., pin_memory=True, num_workers=4)

# ===== CHECK GPU MEMORY=====
print(torch.cuda.memory_summary())
print(torch.cuda.get_device_name(0))
```

Complete Training Template

cP

```
# 1. Setup
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = YourModel().to(device)
optimizer = optim.Adam(model.parameters())
criterion = nn.CrossEntropyLoss()

# 2. Training Loop
for epoch in range(epochs):
    model.train()
    for data, labels in train_loader:
        data, labels = data.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(data)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

# 3. Evaluation
```

```
model.eval()
with torch.no gradQ :
    for data, labels in test_loader:
        data, labels = data.to(device), labels.to(device)
        outputs = model(data)
        # Calculate metrics
```

GPU TRAINING IN PYTORCH: FROM SETUP TO SUCCESS! (89% ACCURACY)

SETUP: Check for GPU
(torch.cuda Js_available()')

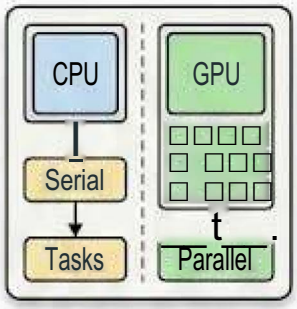
MIGRATE TO DEVICE (4 STEPS}:
1. Model (**.to(device)')**
2. Data **('.to(device)')**
3. Loss Function
4. Optimizer

TRAINING LOOP:
Forward Pass -> Loss Calculation -> Backward Pass -> Optimizer Step

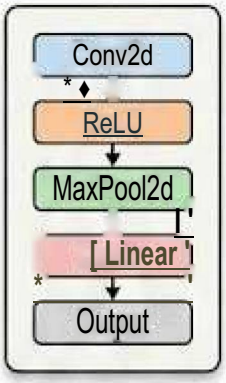
EVALUATION:
ACHIEVE 89% ACCURACY!
(Success!)

KEY TAKEAWAYS:
4 Migration Steps,
20x Speedup!
(Compared to CPU)

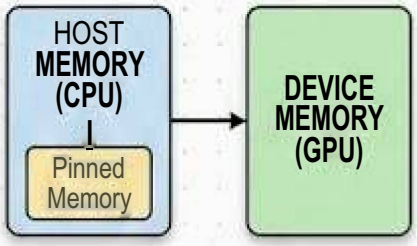
Q. **OPTIMIZATIONS:**
'pin_memory=True' for faster data transfer.
'num_workers > O' r



CPU VS, GPU DIFF



MODELARCH (SIMPLE CNN)



MEMORY TYPES

CHEAT SHEET COMMANDS
- **torch.cuda.is_available():** Check GPU
- **device = torch.devicefcuda' if torch.cuda.is available() else 'cpu'):** Set Device
- **model.to(device), data.to(device):** Move to GPU
- **pin_memory=True (DataLoader):** Faster Transfer
- **nvidia-smi:** Monitor GPU Usage

CampusX

FUTURE IMPRUVEMENTS: Muiti-UPU, Mixed Precision (AMP), Distributed training
REFERENCES: PyTorch Docs, NVIDIA CUDA Guide, Official Tutorials

Key Takeaways

1. **GPU Training is Essential:** For large datasets like Fashion MNIST (60,000 images), GPU training enables using the full dataset instead of subsets.
2. **Simple Migration:** Moving from CPU to GPU requires only 4 key changes:
    - Check GPU availability
    - Move model to GPU
    - Move training data to GPU in loop
    - Move evaluation data to GPU in loop
3. **Performance Gains:**
    - Accuracy: 83% -» 89% (+6%)
    - Speed: ~20x faster training
    - Capability: Full dataset utilization
4. **Optimization Tips:**
    - Use pin_memory=True for faster data transfer
    - Increase batch size for better GPU utilization
    - Use multiple workers for parallel data loading
5. **Future Improvements:** The current 89% accuracy can be further improved with:
    - Weight initialization techniques
    - Regularization (Dropout, L2)
    - Different optimizers (Adam, RMSprop)
    - Hyperparameter tuning
    - Learning rate scheduling

References & Resources

- PyTorch CUDA Documentation
- Fashion MNIST Dataset
- PyTorch Best Practices
- Google Colab GPU Guide

GPU Training in PyTorch

Complete Guide • Fashion MNIST • 83% -> 89% Accuracy

f Why Move to GPU?

CPU Training
- Limited to subset (6,000 imgs)
- Slower training speed
- Sequential processing

83% Accuracy

GPU Training
- Full dataset (60,000 imgs)
- ~20x faster training
- Parallel CUDA cores

89% Accuracy

% CPU vs GPU Architecture

CPU (4-8 Cores) — Sequential Processing

VS

GPU (1000s of Cores) — Massive Parallel Processing

Gf 4-Step GPU Training Workflow

1 Check GPU

```
import torch
device = torch.device(
    'cuda' if
    torch.cuda.is_available()
    else 'cpu'          )
```

2 Modei GPU

```
model = MyNN(784)

model = model, to(device)
```

3 Data — GPU (Train)

```
for X, y in train_loader:
  X  =  X.  to(device)
  y = y. to(device)
  # train loop...
```

4 Data -> GPU (Eval)

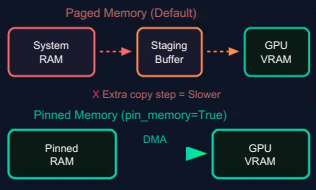
```
model. eval()
with torch. no_grad() :
  for X, y in test_loader:
      x = X. to(device)
      y = y. to(device)
```

• Model Architecture

Total Parameters: 109,386

Layer 1: 784x128 + 128 = 100,480 Layer 2: 128x64 + 64 = 8,256 Layer 3: 64x10 + 10 = 650

£ Memory: Paged vs Pinned




```
# Optimized DataLoader
DataLoader(
  dataset,
  pin_memory =True,
  num_workers = 4
)
```

When to use pin_memory:
- 7 Large datasets
- ■./ Many epochs
- / Multi-GPU training
- X Memory-constrained

ffl Fashion MNIST Dataset

Dataset Split		Image Properties	
Training:	48,000	Size:	28 x 28 px
Validation:	12,000	Channels:	1 (Gray)
Test:	10,000	Features:	784
Total:	70,000	Classes:	10

$ Batch Size Guide

GPU VRAM	Batch Size	GPU Type
4 GB	32 - 64	Entry-level
8 GB	64 - 128	Mid-range
12 GB	128 - 256	High-end
16+ GB	256 - 512	Professional

f Optimization Tips

f pin_memory=True	Faster transfer
♦ num_workers=4	Parallel loading
f Larger batch_size	GPU utilization
f torch.cuda.amp	Mixed precision
♦ prefetch_factor=2	Prefetch batches

A Common Pitfalls

X Forgot .to(device)

```
# Data on CPU, model on GPU
outputs ■ model(batch_features)
# RuntimeError!
```

X Wrong Order

```
# Optimizer before .to(device)
optimizer - SGD(model.params())
model = model.to(device)
```

X Memory Leak in Eval

```
# Missing torch.no_grad()
for X, y in test_loader:
outputs = model(X) # OOM!
```

X Missing model.eval()

```
# Dropout/BatchNorm still active
for X, y in test_loader:
# Inconsistent results!
```

jin Results Summary

BEFORE — CPU Training — 83% — 6,000 images — Slow training

→

AFTER — GPU Training — 89% — 48,000 images — -20x faster

GAIN +6% Accuracy Boost

@ Essential Commands Cheat Sheet

```
# Device Setup
device = torch.device(
  'cuda' if torch.cuda.
  is_available() else 'cpu' )
```

```
# Model to GPU
model = model. to(device)
```

```
# Data to GPU
X = X. to(device)
y = y. to(device)
```

```
# Evaluation Mode
model. eval()
with torch. no_grad()  :
```

```
# Check GPU
torch.cuda.
memory_summary()
get_device_name(0)
```

++ Key Takeaways

1. GPU training is essential for large datasets - enables full dataset utilization
2. Simple migration: 4 key changes - check GPU, move model, move train data, move eval data
3. Performance gains: 83% -»89% accuracy (+6%), ~20x faster training

PyTorch GPU Training Guide • Fashion MNIST • Created with V by CampusX

CampusX

CampusX

Neural Network Optimization: Techniques to Reduce Overfitting

Code

- htt ps://colab.research. g oog le.com/drive/1YDVmsVD8zkdDh5lq umA_Htlh_Wq H10FC

1. Introduction

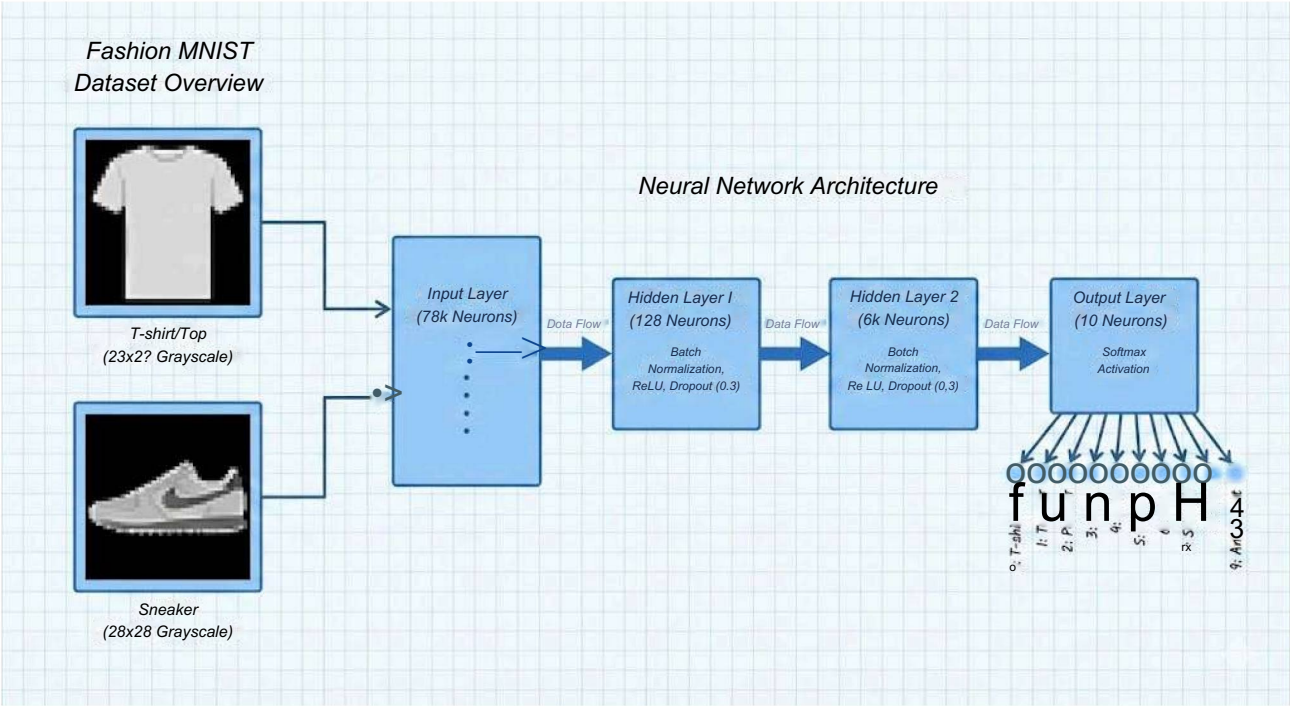
This guide covers optimization techniques for neural networks, specifically focusing on reducing overfitting in deep learning models. We use the Fashion MNIST dataset as our case study—a multi-class classification problem with 10 classes (T-shirts, shoes, jackets, etc.) containing 60,000 images of handwritten clothing items.

Dataset Overview

Property	Value
Dataset	Fashion MNIST
Total Images	60,000 (training)
Image Size	28 x 28 pixels
Input Features	784 (flattened)

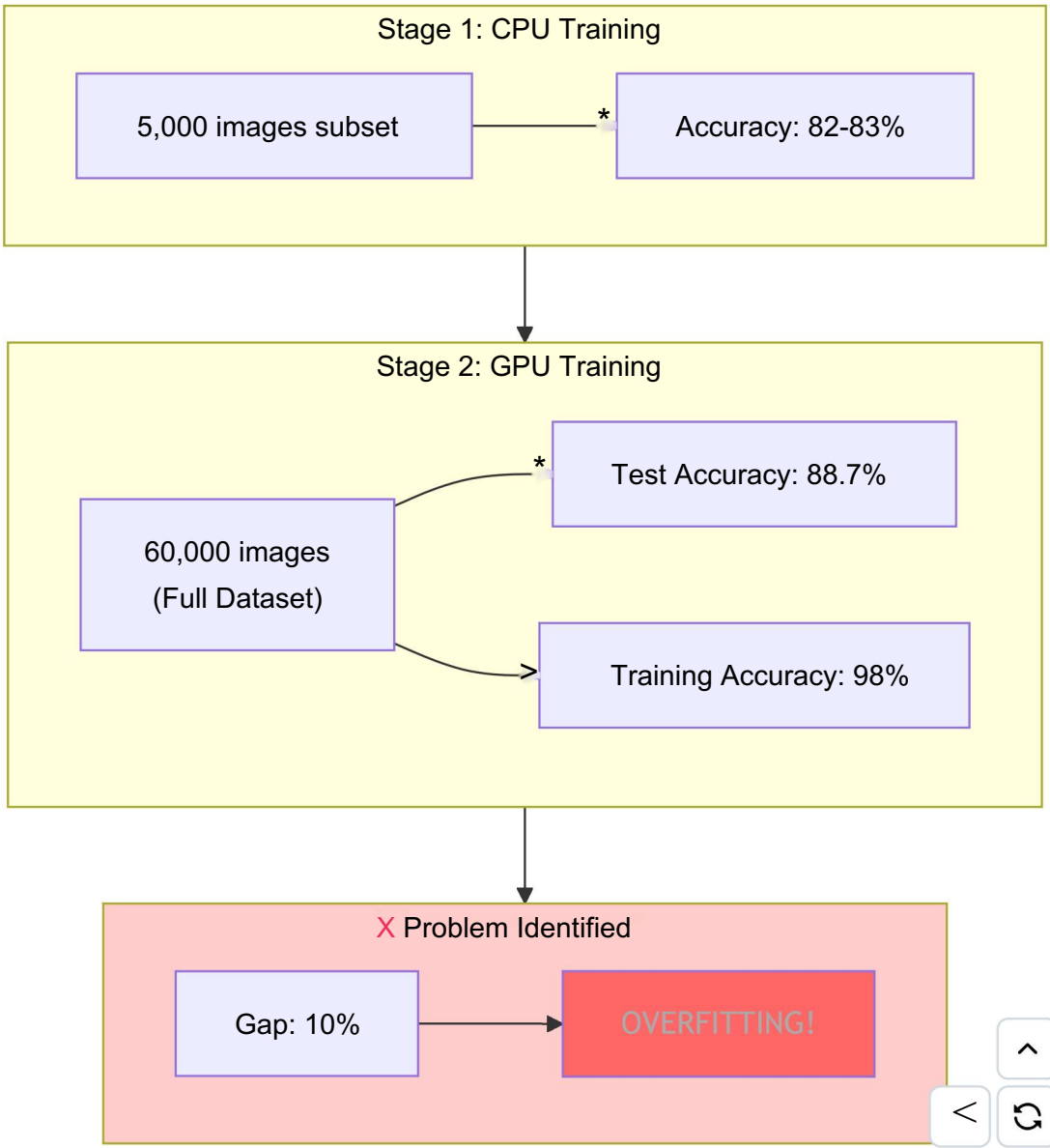
Property	Value
Number of Classes	10
Task Type	Multi-class Classification

Neural Network Architecture



2. Problem Statement: Overfitting

Training Progress Overview



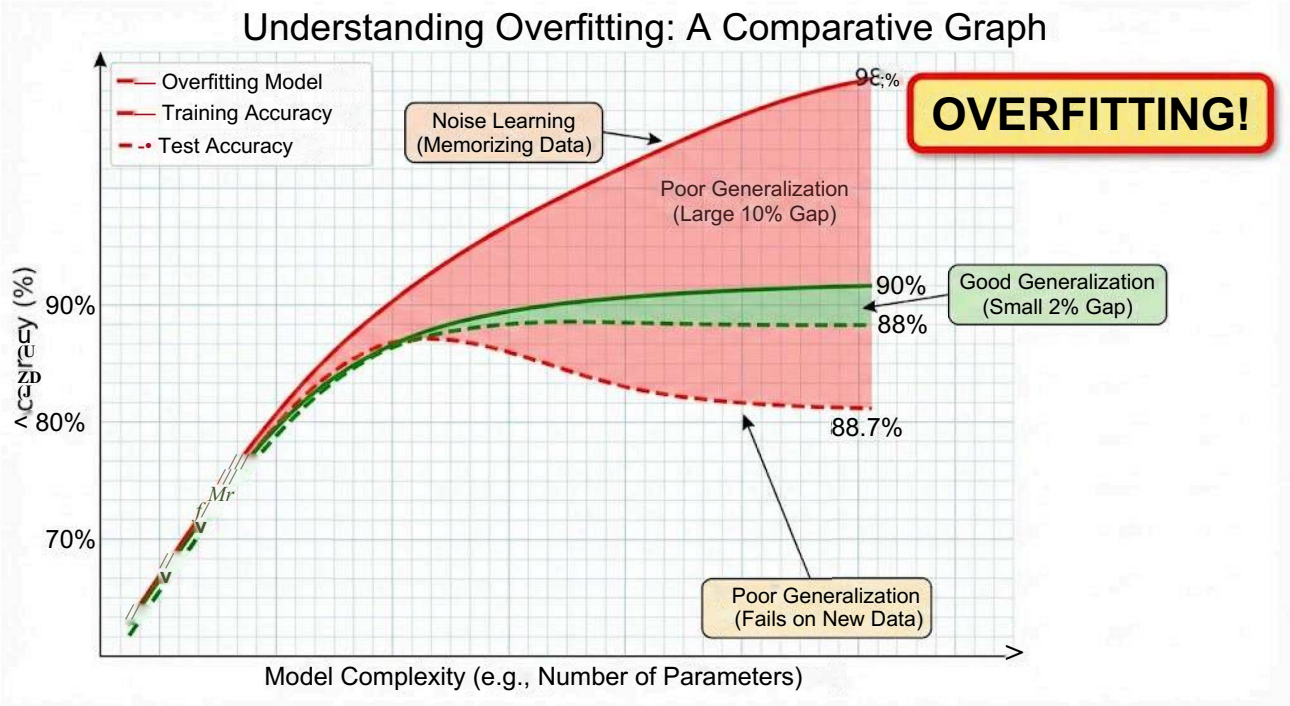
Understanding the Overfitting Problem

Metric	Value	Status
Training Accuracy	98%	⚠ Too High
Test Accuracy	88.7%	Q Good
Accuracy Gap	~10%	X Overfitting

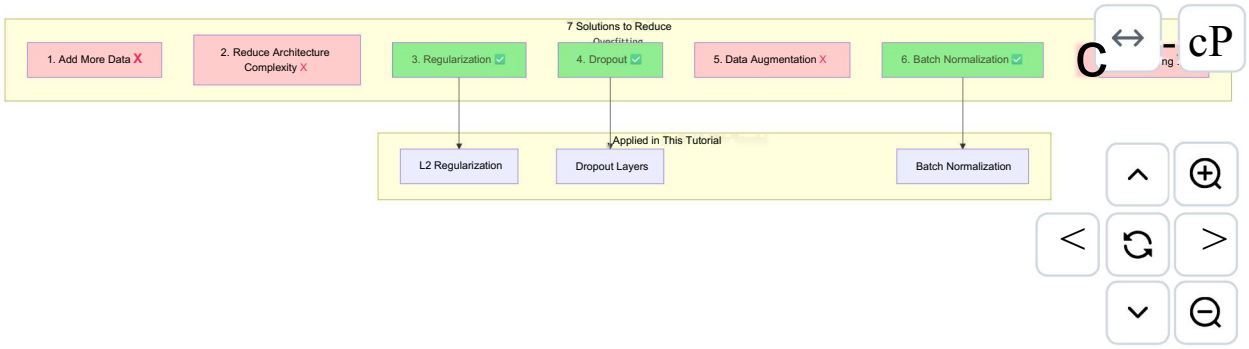
What is Overfitting?

Overfitting occurs when a model learns the training data too well, including its noise and random fluctuations, rather than learning the underlying pattern. This results in:

- Excellent performance on training data
- Poor generalization to unseen (test) data
- Large gap between training and test accuracy



3. Seven Solutions for Overfitting



Detailed Solution Analysis

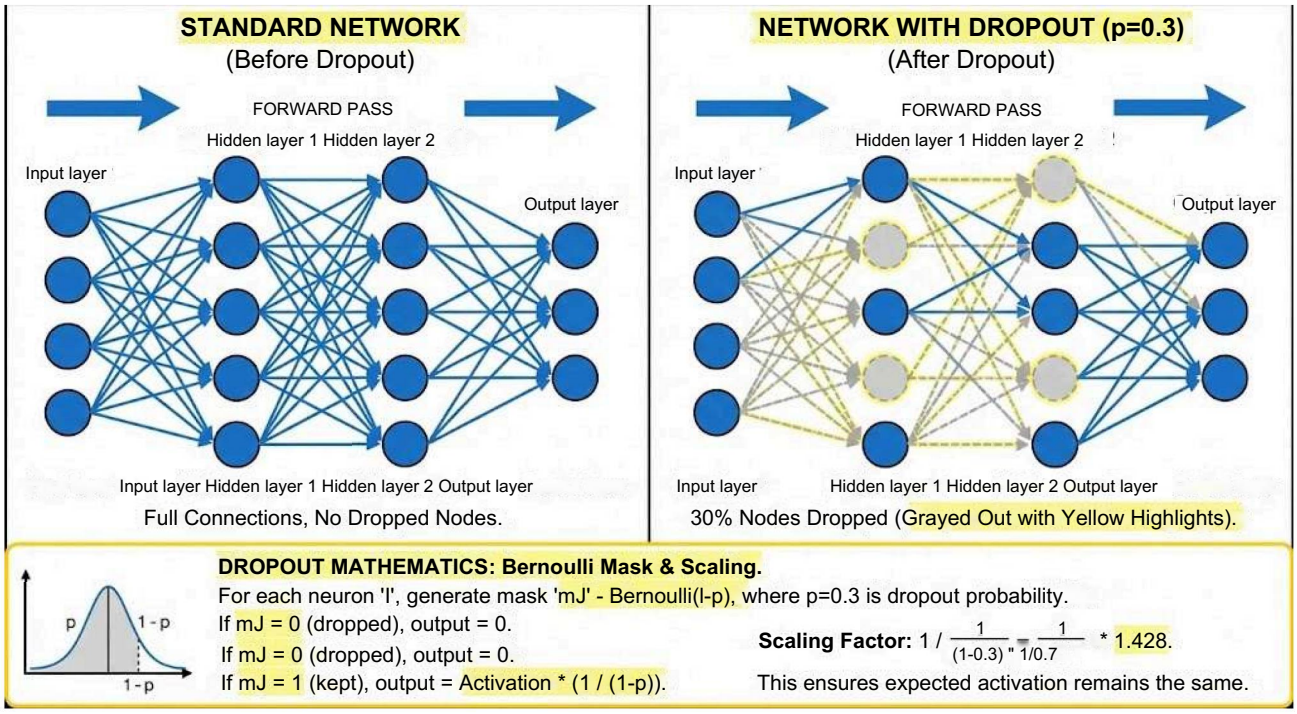
#	Solution	Applicable?	Reason
1	Add More Data	X No	Already using full 60,000 images

#	Solution	Applicable?	Reason
2	Reduce Architecture Complexity	X No	Architecture already simple (2 hidden layers)
3	Regularization (L2)	✅ Yes	Add penalty term to loss function
4	Dropout	Q Yes	Randomly turn off neurons during training
5	Data Augmentation	X No	More effective with CNNs, not ANNs
6	Batch Normalization	✅ Yes	Stabilize training + regularization effect
7	Early Stopping	X No	Not covered in this tutorial

4. Dropout

4.1 Theory and Intuition

Dropout is a regularization technique that randomly "turns off" (sets to zero) a fraction of neurons during each forward pass in training. This prevents the network from becoming too dependent on any particular neuron.



Key Properties of Dropout

Property	Description
Applied To	Hidden layers only (not input or output)
When Applied	After activation function (e.g., ReLU)
Parameter p	Probability of dropping a neuron (e.g., p=0.3 means 30% dropped)
During Training	Randomly drops neurons each forward pass
During Evaluation	Dropout is disabled (all neurons active)
Effect	Regularization, prevents co-adaptation

4.2 Mathematical Foundation

Forward Pass with Dropout

During training, for each neuron in a layer with dropout rate p:

Step 1: Generate Bernoulli Mask

$$m_t \sim \text{Bernoulli}(1 — p)$$

Where m, is a binary mask (0 or 1) for neuron i.

Step 2: Apply Mask to Activations

$$\widetilde{Cl}; \not{m}i \text{ ‘ } Clj$$

Where:

- a_t = original activation of neuron i
- $\widetilde{a}t$ = activation after dropout
- TTp = mask value (0 if dropped, 1 if kept)

Step 3: Scale During Training (Inverted Dropout)

To maintain expected output, scale by !$_{-p}$:$\frac{1}{}$

$$\sim \quad \frac{m_i \bullet a_{\underline{L}}}{1 \quad 1\text{-p}}$$

Why Scaling is Necessary

Without scaling, the expected value changes:

$$E[\tilde{a}_i] = (1 - p) \cdot a_t \quad \neq a_i$$

With scaling:

$$E\left[\frac{m; \cdot at}{1 — p}\right] = \frac{(1 - p) \cdot fl; _}{1 - P} \; 1$$

This ensures consistent expected values between training and evaluation.

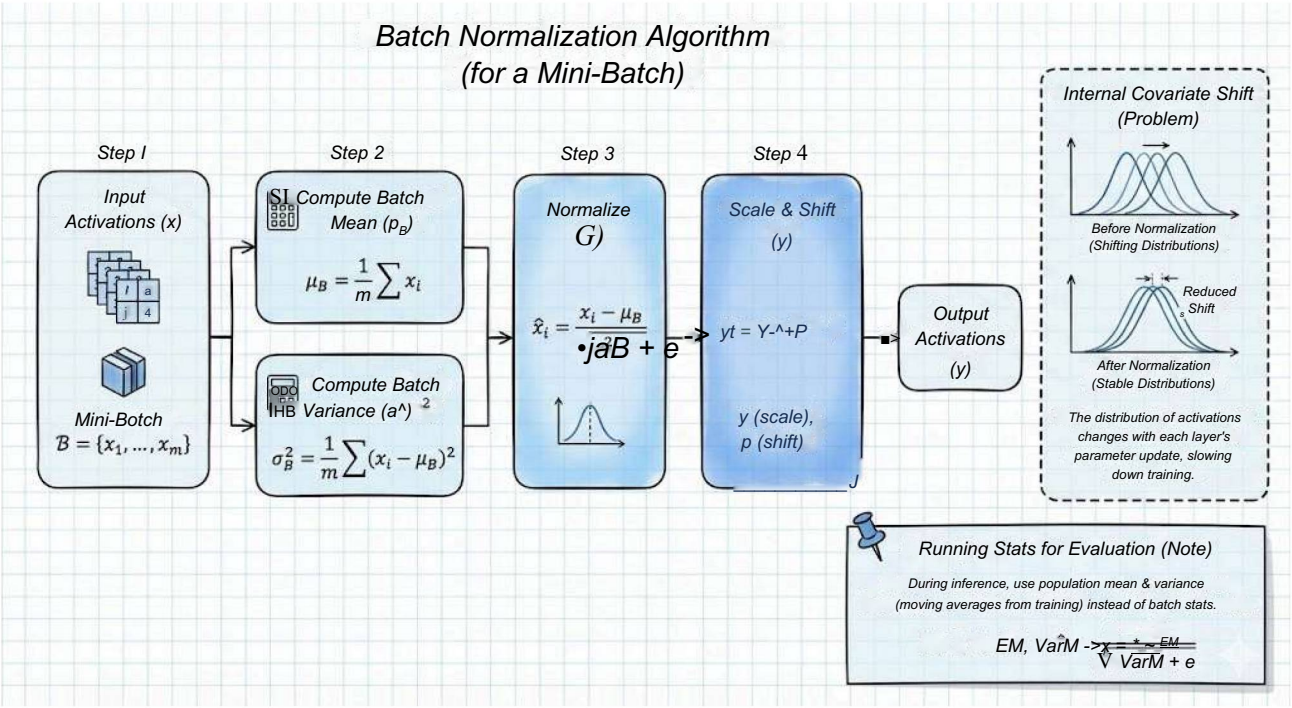
4.3 PyTorch Implementation

```
import torch.nn as nn                                                    tP

class MyNN(nn.Module):
    def _____ init____ (self, num_features):
        super().____ init____ ()
        self.model = nn.Sequential(
            # First Hidden Layer
            nn.Linear(num_features, 128),
            nn.ReLU(),
            nn.Dropout(p=0.3), # 30% neurons dropped

            #  Second Hidden Layer
            nn. Linear(128., 64),
            nn.ReLU(),
            nn.Dropout(p=0.3), # 30% neurons dropped

            #   Output Layer
            nn.Linear(64, 10)
        )

    def f o rwa r d (s e lf, x):
        return self.model(x)
```

5. Batch Normalization

5.1 Internal Covariate Shift

Internal Covariate Shift is the phenomenon where the distribution of each layer's inputs changes during training as the parameters of previous layers change.



Why is this a problem?

- Each layer must continuously adapt to new input distributions
- Slows down training
- Requires lower learning rates
- Can cause training instability

5.2 Mathematical Formulation

Batch Normalization normalizes the activations of each layer for each mini-batch.

Step 1: Compute Mini-Batch Statistics

For a mini-batch B = xi,x_2, ■ ■ ->^xm:

Mini-batch Mean:

$$\mu_B = \frac{1}{m}\sum_{I=1}^{m} x_i$$

Mini-batch Variance:

$$\sigma_B^2 = \frac{1}{m}\sum_{I=1}^{m} (x_i - \mu_B)^2$$

Step 2: Normalize

$$\hat{X}i = \frac{Xj \sim \mu_B}{\sqrt{ks^2 + e}}$$

Where e is a small constant (e.g., 10^{-5}) for numerical stability.

Step 3: Scale and Shift (Learnable Parameters)

$$y_i = 7\hat{} + p$$

Where:

- y (gamma) = learnable scale parameter
- P (beta) = learnable shift parameter
- Both are learned during training via backpropagation

Complete Batch Normalization Algorithm

Algorithm: Batch Normalization Transform

I nput: Values of x over a mini-batch: B = {xi, x_{2j} .x_m) . . ,
Parameters to be learned: y> p

Output: {yi = BN_{$y_?$p}(xi)}

1. p_B (1/m) Si Xi // mini-batch mean
2. o^2_B «- (1/m) Si (Xi - p_B)^2 // mini-batch variance
3. Xi «- (Xi - p_B) / V(o^2_B + E) // normalize
4. Yi «- yXi + P = BN_{Y,P}(Xi) // scale and shift

5.3 Learnable Parameters

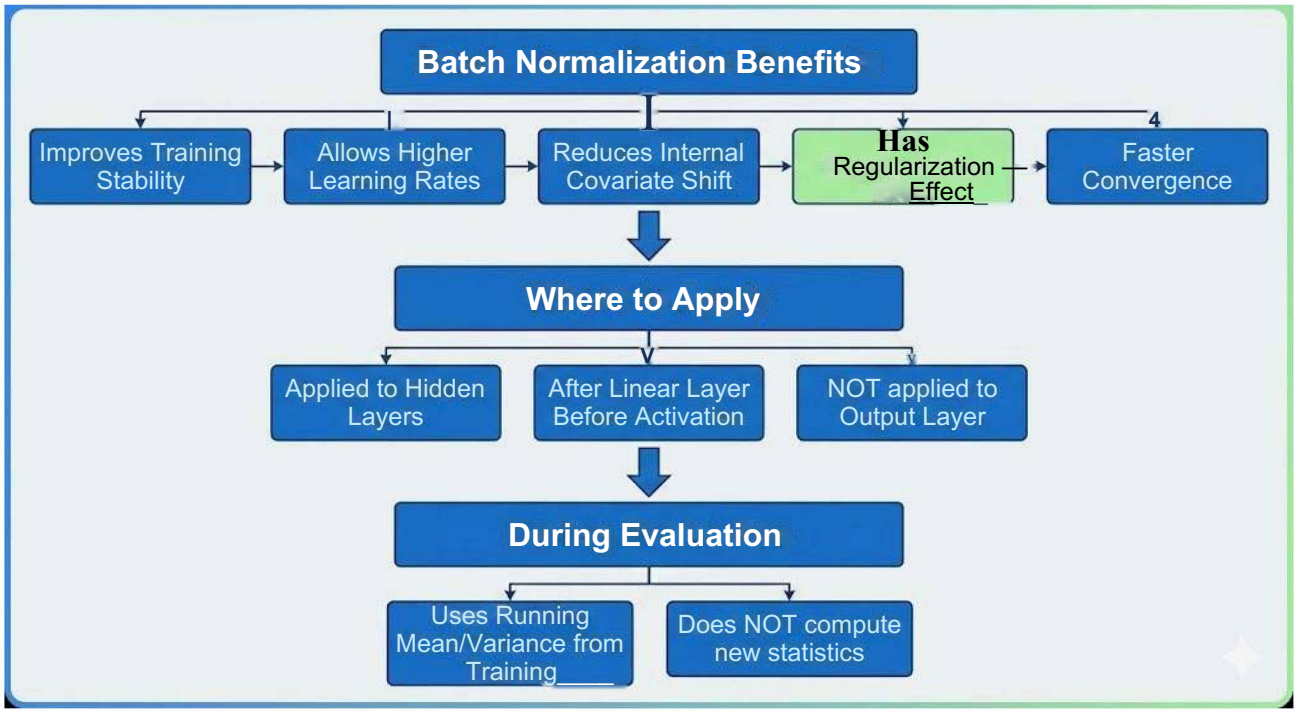
Parameter	Symbol	Purpose	Initial Value
Scale	Y	Controls output variance	1
Shift	P	Controls output mean	0

Why Learnable Parameters?

Without y and $(3,$ the normalized outputs would always have zero mean and unit variance. The learnable parameters allow the network to recover the original representation if needed:

If y σ_B and ft = n_B, then $y \bar{i}$ x_i

5.4 Key Properties of Batch Normalization



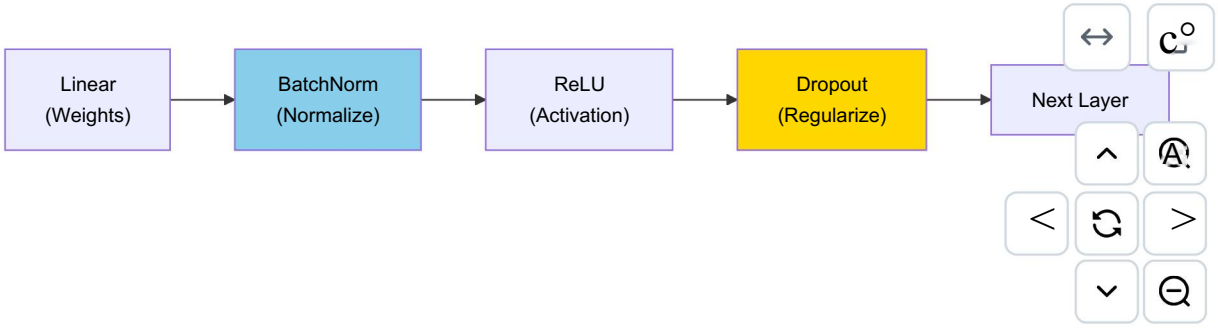
5.5 PyTorch Implementation

```
import torch.nn as nn

class MyNN(nn.Module):
    definit___ (self, num_features):
        super()._____ init____ ()
        self.model = nn.Sequential(
            # First Hidden Layer
            nn.Linear(num_features, 128),
            nn.BatchNormId(128), # BatchNorm BEFORE activation
            nn.ReLU(),
            nn.Dropout(p=0.3),

            #  Second Hidden Layer
            nn.Linear(128, 64),
            nn.BatchNormId(64), # BatchNorm BEFORE activation
            nn.ReLU(),
            nn.Dropout(p=0.3),

            #  Output Layer (NO BatchNorm here)
            nn.Linear(64, 10)
        )
```

cP

```
    def forward(selfj x):
        return self.model(x)
```

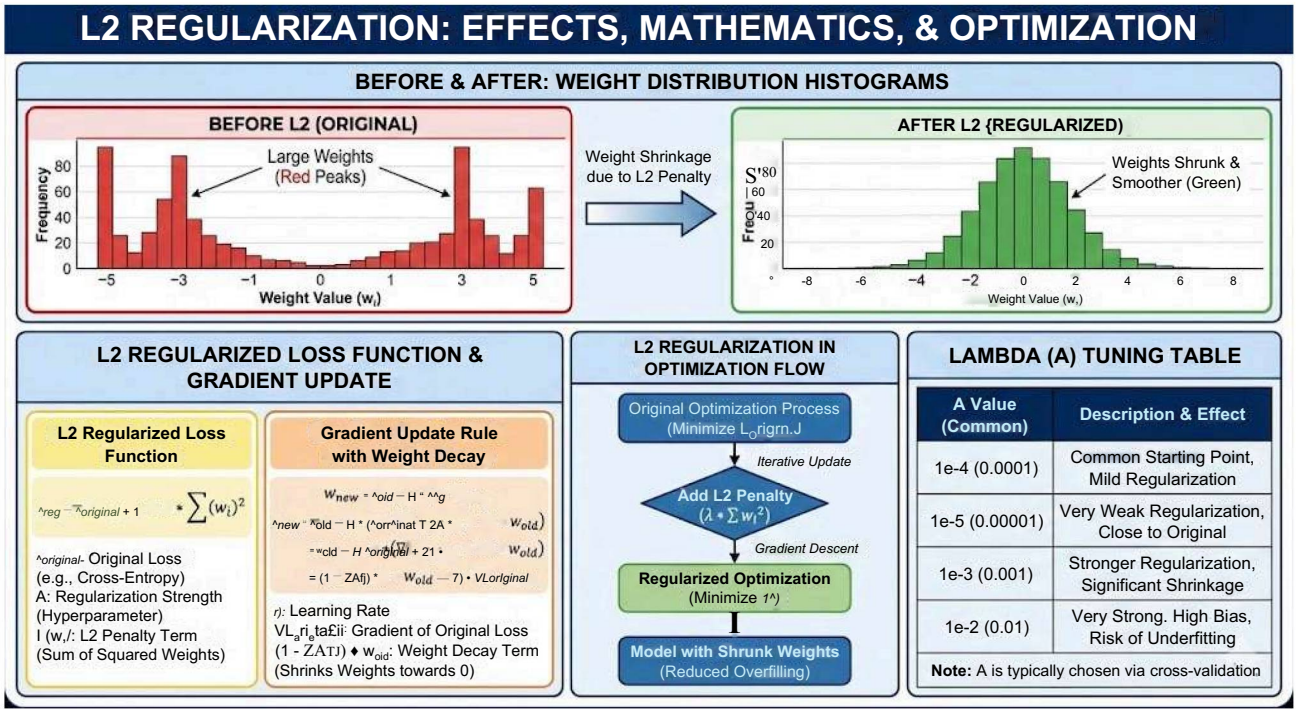
Layer Order in Neural Network



6. L2 Regularization (Weight Decay)

6.1 Regularization Theory

Neural networks are essentially optimization problems where we minimize a loss function by adjusting weights and biases. Regularization adds a penalty term to prevent weights from becoming too large, which helps reduce overfitting.



6.2 Mathematical Derivation

L2 Regularization (Ridge Regression)

The regularized loss function is:

$$\text{^reg ^original "T A W} \sum_i \text{.2}$$

Expanded Form:

$$\text{reg ^original + A(W} + \text{W2} + \text{W3} + \ldots + \text{W}_n) \text{ .2}$$

Where:

- $L_{origina}i$ = original loss (e.g., Cross-Entropy for classification)
- A = regularization coefficient (hyperparameter)
- w_t = weight parameters
- Ej ⇄ w_i^2 um °f squared weights

Gradient with L2 Regularization

The gradient of the regularized loss with respect to weight *w:*

$$\frac{\partial L_{reg}}{dw} = \text{^Loriginai ----- T--------F 2 Aw}$$

Weight Update Rule

Standard gradient descent update:

$$w \text{ «- } w \text{ — 77} \bullet \frac{dL_{reg}}{dw}$$

Substituting the gradient:

$$\text{w <- w — 77} \left(\frac{dLoriginal}{dw} \text{ ---- --------+ 2 Aw} \right)$$

Rearranging:

$$\text{w w - 77 • } VL_{original} \text{ - 2T?AW}$$

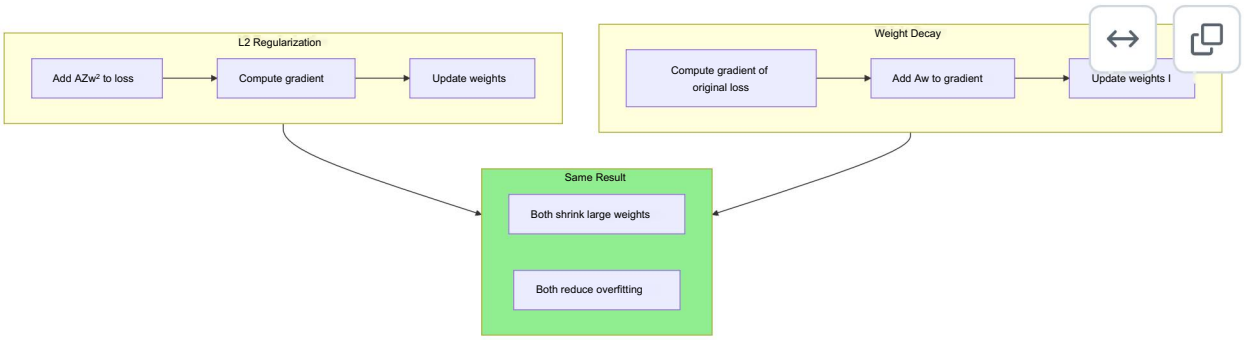
$$\text{w <- w(l - 277A) - 77 • } VL_{origina}i$$

6.3 Weight Decay vs L2 Regularization

Weight Decay directly modifies the gradient update rule:

$$w <- w — Y)(V L + Aw)$$

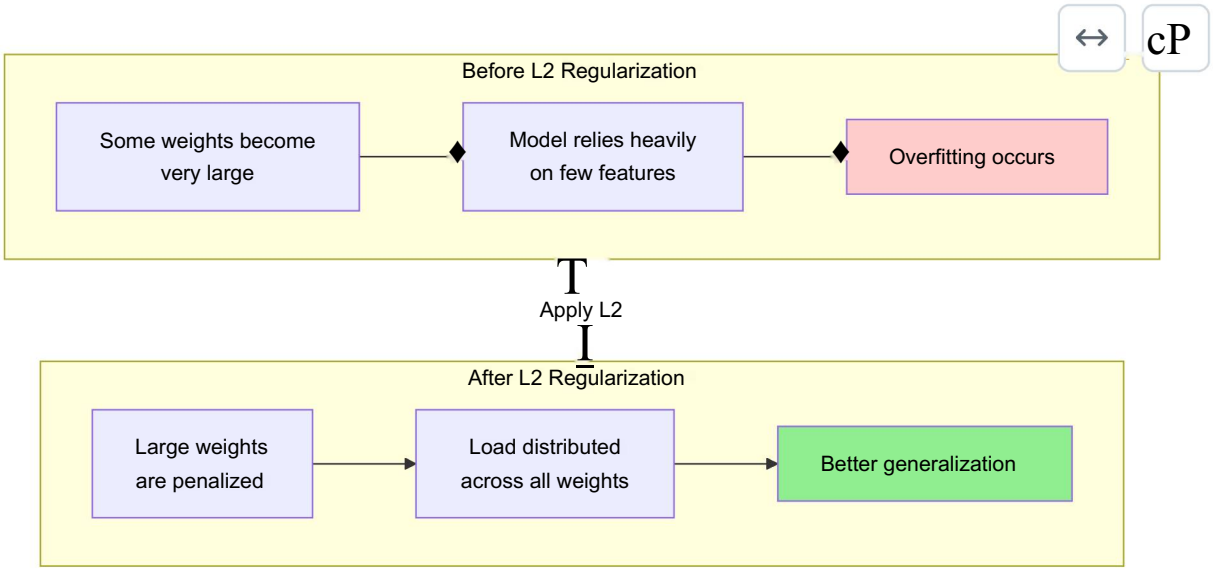
This is mathematically equivalent to L2 regularization for standard SGD!



6.4 Properties of L2 Regularization

Property	Description
Applied To	Weights only (NOT biases)
Effect	Penalizes large weight values
Controlled By	A (lambda) - regularization coefficient
Higher A	Stronger regularization
Lower A	Weaker regularization
When Active	During training only
During Evaluation	No effect

How L2 Regularization Reduces Overfitting



6.5 PyTorch Implementation

```
import torch.optim as optim

# Create model
model = MyNN(num_features=784)

# Define optimizer with weight_decay (L2 regularization)
optimizer = optim.SGD(
    model.parameters(),
    lr=0.1,
    weight_decay=1e-4 # A = 0.0001
)

# That's it! Weight decay automatically applies L2 regularization
```

Choosing A (Regularization Strength)

A Value	Effect
0	No regularization

A Value	Effect
1e-5	Very weak regularization
1e-4	Weak regularization (commonly used)
1e-3	Moderate regularization
1e-2	Strong regularization
1e-1	Very strong regularization

7. Complete Implementation

Full PyTorch Code with All Optimizations

```
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

#  Set random seeds for reproducibility
torch.manual_seed(42)

#  Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")

# ========================================== ===
#  1. DATA PREPARATION
# ========================================== ===
df = pd.read_csv('fashion-mnist_train.csv')

#  Features and labels
X = df.ilocf:, 1:].values
y = df.iloc[:, 0].values

#  Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize pixel values to [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0
```

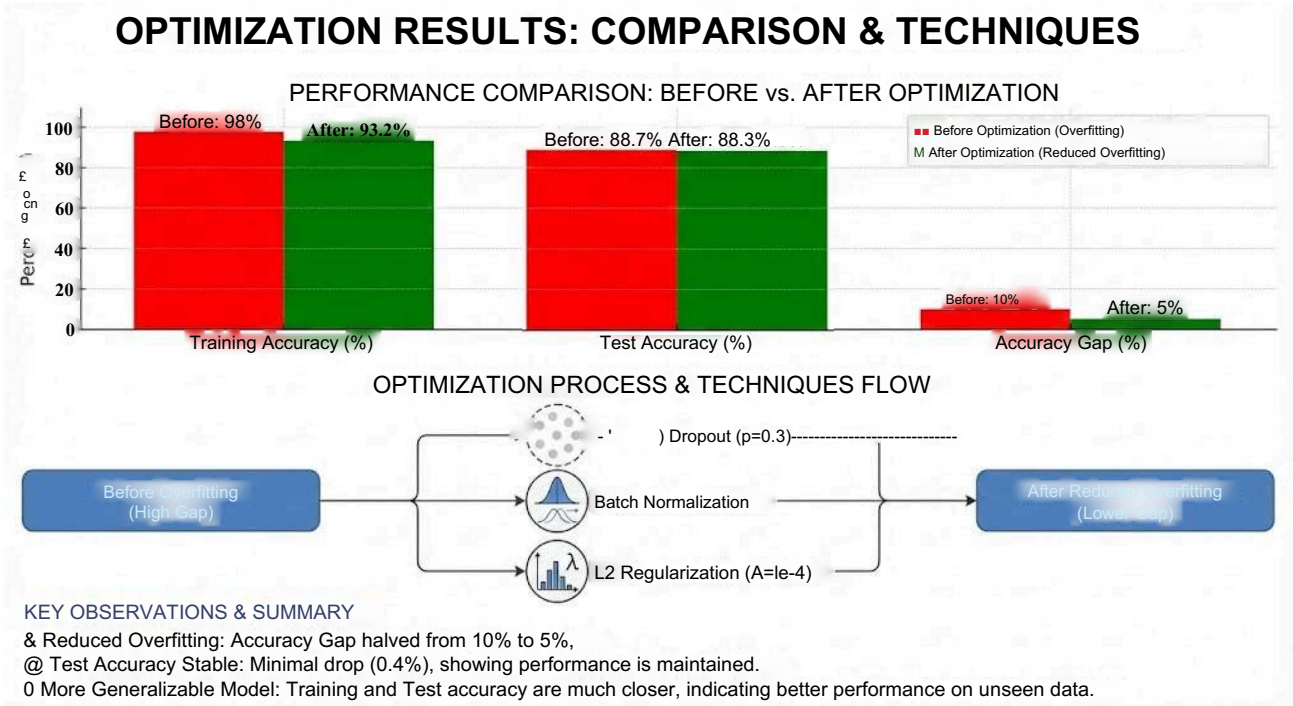
```
# ==========================================
# 2. CUSTOM DATASET CLASS
# ==========================================
class CustomDataset(Dataset):
    def ___init____(self, features, labels):
        self.features = torch.tensor(features, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.long)

    def ___len__  (self):
        return len(self.features)

    def ___getitem___ (self, index):
        return self.featuresfindex], self.labels[index]

# Create datasets and dataloaders
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, pin_memo
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, pin_memor

# =============================================
# 3. NEURAL NETWORK WITH OPTIMIZATIONS
# ========================================= =====
class MyNN(nn.Module):
    def___ init____(self, num_features):
        super().___init_()
        self.model = nn.Sequential(
            # First Hidden Layer
            nn.Linear(num_features, 128),
            nn.BatchNormld(128),              # Q Batch Normalization
            nn.ReLU(),
            nn.Dropout(p=0.3),                 # Q Dropout (30%)

            # Second Hidden Layer
            nn. Linear(128., 64),
            nn.BatchNormld(64),                #Q Batch Normalization
            nn.ReLU(),
            nn.Dropout(p=0.3),                 # Q Dropout (30%)

            #   Output Layer
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.model(x)

# ==========================================
```

```
# 4. MODEL, LOSS, OPTIMIZER
# ============================ ================
model = MyNN(X_train.shape[l])
model.to(device)

criterion = nn.CrossEntropyLoss()

optimizer = optim.SGD(
    model.parameters(),
    lr=0.1,
    weight_decay=le-4 # Q L2 Regularization
)

# ================ ============================
# 5. TRAINING LOOP
# ================ ============================
epochs = 100

for epoch in range(epochs):
    model.train() # Set to training mode
    total_epoch_loss = 0

    for batch_features, batch_labels in train_loader:
        # Move data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        #  Forward pass
        outputs = model(batch_features)

        #  Calculate loss
        loss = criterion(outputs, batch_labels)

        #  Backward pass
        optimizer.zero_grad()
        loss.backward()

        #  Update weights
        optimizer.step()

        total_epoch_loss += loss.item()

    avg_loss = total_epoch_loss / len(train_loader)
    print(f'Epoch: {epoch + l}, Loss: {avg_loss:,4f}')

#  ==========================================
#  6. EVALUATION
#  ==========================================
model.eval() # A IMPORTANT: Disables Dropout and BatchNorm training behavior
```

```
def evaluate(model, data_loader, device):
    total = 0
    correct = 0

    with torch.no_grad():
        for batch_features, batch_labels in data_loader:
            batch_features = batch_features.to(device)
            batch_labels = batch_labels.to(device)

            outputs = model(batch_features)
            _, predicted = torch.max(outputs, 1)

            total += batch_labels.shape[0]
            correct += (predicted == batch_labels).sum().item()

    return correct / total

test_accuracy = evaluate(model, test_loader, device)
train_accuracy = evaluate(model, train_loader, device)

print(f"\nTest Accuracy: {test_accuracy * 100:.2f}%")
print(f"Train Accuracy: {train_accuracy * 100:.2f}%")
print(f"Gap: {(train_accuracy - test_accuracy) * 100:.2f}%")
```

8. Results Comparison

Before vs After Optimization



Detailed Results Table

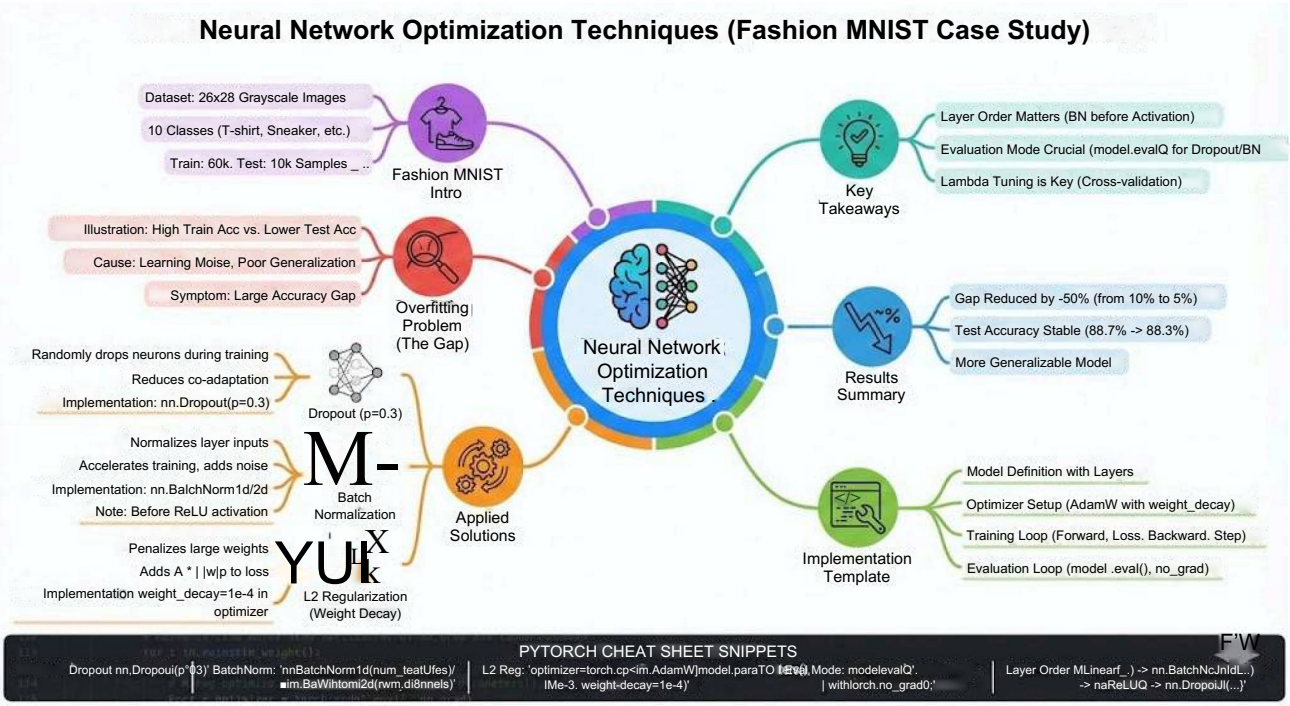
Metric	Before Optimization	After Optimization	Change
Training Accuracy	98%	93.2%	1 4.8%
Test Accuracy	88.7%	88.3%	I 0.4%
Accuracy Gap	**~ 10%**	**~5%**	1 **50%**
Overfitting Status	X Severe	Q Reduced	Improved

Key Observations

1. **Test accuracy remained stable** (-88%) - This is good! We didn't sacrifice generalization.
2. **Training accuracy decreased** (98% -> 93.2%) - This is expected and healthy.
3. **Gap reduced by 50%** (10% -> 5%) - Our goal was achieved!

CampusX

9. Key Takeaways



Summary Points

Technique	Where Applied	When Active	Key Parameter
Dropout	Hidden layers (after activation)	Training only	p (dropout rate)
BatchNorm	Hidden layers (before activation)	Training (computes), Eval (uses running)	num_features
L2 Regularization	Optimizer (weights only)	Training only	weight_decay (A)

Important Reminders

1. **Always call** model.eval() before evaluation to disable Dropout and switch BatchNorm to evaluation mode.
2. **Order matters** in the layer sequence:
   - Linear -» BatchNorm -» Activation -» Dropout
3. **L2 Regularization** only affects weights, not biases.
4. **Dropout rate (p)** is a hyperparameter - common values: 0.2, 0.3, 0.5

5. **BatchNorm** requires batch size > 1 to compute statistics.

References

- Fashion MNIST Dataset: Zalando Research GitHub
- Dropout Paper: Srivastava et al., "Dropout: A Simple Way to Prevent Neural Networks from Overfitting" (2014)
- Batch Normalization Paper: Ioffe & Szegedy, "Batch Normalization: Accelerating Deep Network Training" (2015)
- PyTorch Documentation: pytorch.org/docs

Neural Network Optimization

Techniques to Reduce Overfitting • Fashion MNIST • 10% -» 5% Gap Reduction

4k The Problem: Overfitting

Before Optimization

Training Accuracy:	98%
Test Accuracy:	88.7%
Gap:	-10%

? 7 Solutions (3 Applied)

X Not Applied

1. More Data Full dataset
2. Simpler Modafeady simple
5. Augmenticsetter for CNN
7. Early StoppingNot covered

® Applied in This Tutorial

- 0 L2 Regularization weight_decay
- 0 Dropout n n.Dropout(p=0.3)
- Q Batch Normalization nn.BatchNormid

$ Dropout

Mathematics

Mask: mi ~ Bernoulli(i-p)

Apply: $\tilde{a}_i = m_i \cdot a_i$

Scale: á i = (m i • a j) / (1-p)

Key Properties

/ Applied to: Hidden layers only

J When: After activation (ReLU)

>/ Training: Neurons randomly dropped

X Eval: Disabled (all neurons active)

F Batch Normalization

BatchNorm Algorithm

1. Mean: ^e=(i/m)Sx i
2. Variance: a² B= (1/m) S(x i - p B)²
3. Normalize: x'j =(x i-p B)/V(o² B+C)
4. Scale & Shift: y i = y x" j + 0 (learnable)

Benefits

7 Faster training

V Higher learning rates

7 Regularization effect

7 Stable gradients

! Needs batch_size > 1

-jl L2 Regularization (Weight Decay)

Mathematical Foundation

Regularized Loss:

T reg " L"original + A S W , ²

Gradient Update:

$3L_{reg}$ /dw = $3L_{orig}$ /dw + 2Xw

Weight Decay Effect:
w*— w(i - 2r|X) - qVL

A Values Guide

0	No regularization
1e-5	Very weak
le-4	Common choice -J
1e-3	Moderate
le-2	Strong
1e-1	Very strong

Applied to weights only

Layer Order (Critical!)

Correct Layer Sequence

Linear → BatchNorm → ReLU → Dropout → Next

```
# Complete layer ... with all optimizations
nn.Sequential
    nn. Linear (784, 128),          # Weights
    nn. BatchNormId (128),          # Normalize
    nn. ReLU (),                    # Activate
    nn.  Dropout  (p=  0.3  ),      #  Regularize
    nn. Linear (128, 64),           # Next layer
    . . .
)
```

jji= Results: Before vs After

BEFORE		AFTER		RESULT
Train Acc:	98%	Train Acc:	93.2%	50%
Test Acc:	88.7%	Test Acc:	88.3%	Gap Reduction!
Gap:	~10%	Gap:	-5%	
X Overfitting		Reduced!		

A Critical Reminders

- **model.eval() before testing!** Disables Dropout & changes BatchNorm to eval mode
- **Layer order matters!** Linear ■" BatchNorm -♦ ReLU -» Dropout Wrong order = suboptimal results
- **BatchNorm needs batch_size > 1** Cannot compute statistics with single sample batches
- **9 L2 only affects weights** Biases are NOT regularized optimizer = SGD(..., weight_decay=le-4)

(Technique Summary

Technique	Where Applied	When Active	Key Parameter	PyTorch
0 Dropout	Hidden layers (after activation)	Training only	p (dropout rate)	nn.Dropout(p=0.3)
BatchNorm	Hidden layers (before activation)	Train (compute) / Eval (running)	num_features	nn.BatchNormId(128)
0 L2 Regularization	Optimizer (weights only)	Training only	weight_decay (A)	SGD(..., weight_decay=le-4)

++ Key Takeaways

1. Overfitting = Training acc » Test acc
   Model memorizes noise instead of learning patterns
2. BatchNorm stabilizes training
3. Dropout prevents co-adaptation
   Neurons can't rely on specific other neurons
4. L2 penalizes large weights

Combined Result: 10% Gap —* 5% Gap (50% Reduction)

f Quick Implementation

```
# Add to ... model layers
nn. BatchNormId (128), nn. Dropout (p= 0.3 )
```

```
# Add to ... optimizer
optimizer = optim. SGD (params, weight_decay = le-4 )
```

```
# Before evaluation
model. eval() # Ci  ...itical! Disables dropout
with torch. no_grad() :
```


CampusX

CampusX

Hyperparameter Tuning for Neural Networks using Optuna

Code

- htt ps://colab.research.g oo g le.com/drive/1 uRUPZFQwiBxEQCCiLNQCFtk7b4L_7N_E

1. Introduction & Context

1.1 Project Background

This tutorial builds upon a Fashion-MNIST classification project developed over multiple PyTorch sessions:

Video	Focus Area	Achievement
1-6	PyTorch Fundamentals	Tensors, Autograd, nn Module, DataLoader
7	Basic Neural Network	~83% accuracy (CPU training, partial data)
8	GPU Training	~88% accuracy (Full dataset, CUDA)
9	Regularization	88% test / 93% train (Reduced overfitting)

Video	Focus Area	Achievement
10	**Hyperparameter Tuning**	**~90% accuracy (Optimized architecture)**

1.2 The Challenge

In the previous implementation, we used a neural network architecture with:

- 2 Hidden Layers (128 -> 64 neurons)
- 10 Output neurons (10 classes)
- Manually chosen hyperparameters

The Critical Question: How do we systematically determine optimal values for:

- Number of hidden layers?
- Neurons per layer?
- Learning rate?
- Batch size?
- And many more...

2. The Hyperparameter Problem

2.1 What are Hyperparameters?

Hyperparameters are configuration variables that are set **before** the learning process begins. Unlike model parameters (weights and biases) that are learned during training, hyperparameters must be specified by the practitioner. Create an educational diagram illustrating the difference between model parameters (weights and biases) and hyperparameters (architecture, learning rate, epochs, batch size, regularization), using a flowchart with colored boxes for hyperparameters in light blue and parameters in orange, showing how hyperparameters influence the training process that updates parameters, leading to model performance.

2.2 The Intuition Problem

Hyperparameter	Initial Value	Reasoning
Hidden Layers	2	"Felt right"
Neurons Layer 1	128	Intuition

Hyperparameter	Initial Value	Reasoning
Neurons Layer 2	64	Intuition
Epochs	100	Arbitrary choice
Batch Size	32	Common default
Learning Rate	0.01	Standard starting point
Dropout Rate	0.3	Typical value

> ⚠️ **Problem:** In production environments, you cannot rely solely on intuition. You need a **systematic approach** to find optimal values.

2.3 The Solution: Experimentation

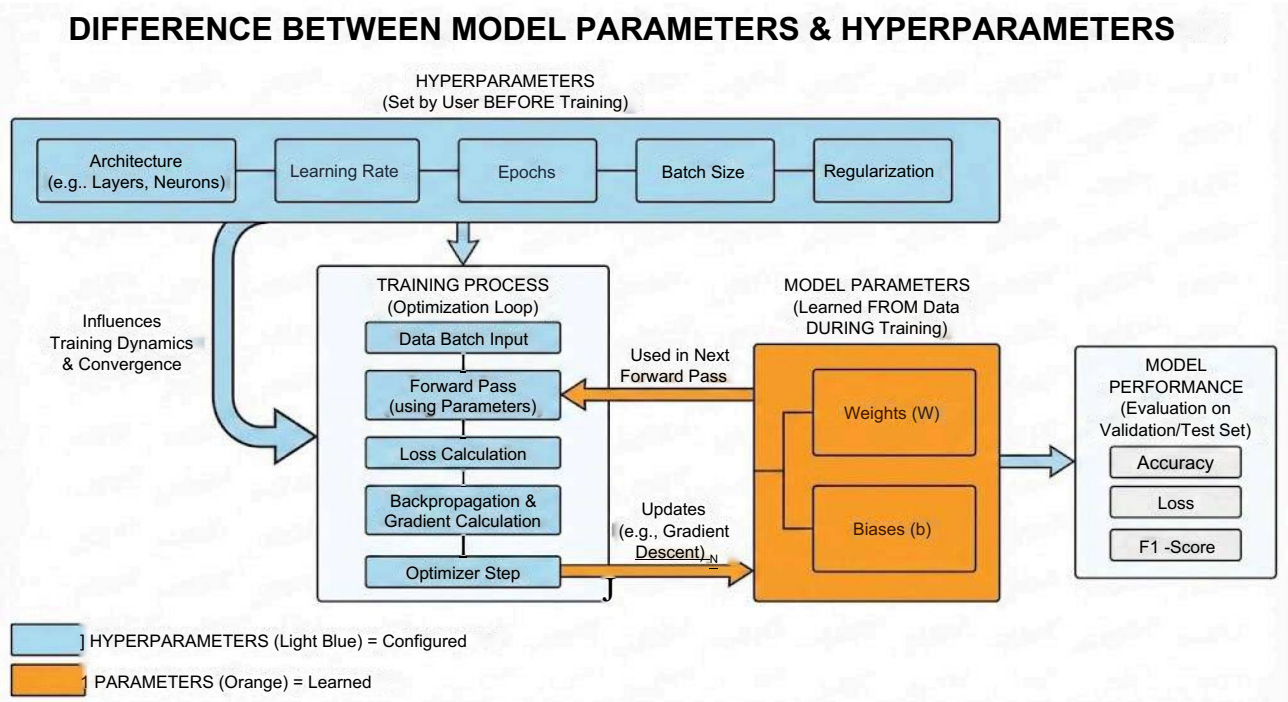
The answer is **Hyperparameter Tuning** through systematic experimentation:

1. Define a search space for each hyperparameter
2. Run multiple experiments with different combinations
3. Evaluate performance for each combination
4. Select the best-performing configuration

3. Hyperparameters to Tune

3.1 Complete List of Hyperparameters

Based on our neural network architecture, we need to tune **8 hyperparameters:**



3.2 Detailed Hyperparameter Specifications

#	Hyperparameter	Type	Description	Impact
1	**Number of Hidden Layers**	Integer	Depth of the network	Model complexity & capacity
2	**Neurons per Layer**	Integer	Width of each hidden layer	Feature representation power
3	**Number of Epochs**	Integer	Training iterations over full dataset	Convergence & overfitting
4	**Optimizer**	Categorical	Optimization algorithm	Convergence speed & quality
5	**Learning Rate**	Float	Step size for weight updates	Training stability & speed
6	**Batch Size**	Categorical	Samples per gradient update	Memory usage & gradient noise
7	**Dropout Rate**	Float	Probability of neuron dropout	Regularization strength
8	**Weight Decay (A)**	Float	L2 regularization strength	Overfitting prevention

3.3 Mathematical Formulation

Learning Rate Impact on Weight Updates

The weight update rule in gradient descent is:

$$w_{t+1} = w_t - p \cdot VL(w_t)$$

Where:

- w_t = weights at step t
- p = learning rate (hyperparameter)
- VL(w_t) = gradient of loss with respect to weights

Weight Decay (L2 Regularization)

The regularized loss function becomes:

$$\hat{}_{reg} - \hat{}_{original} \text{ T" A } \sum_i W_i^2$$

Where:

- $\hat{}_{original}$ = original cross-entropy loss
- A = weight decay coefficient (hyperparameter)
- *Wi* = individual weights

This modifies the weight update to:

$$w_{t+} ! = w_t - p \bullet VL(w_t) - p \bullet A \bullet w_t$$

Dropout Probability

During training, each neuron output is:

$$\tilde{y}_i = \begin{cases} 0 \text{ with probability } p \\ \frac{y_i}{1-p} \text{ with probability } 1 - p \end{cases}$$

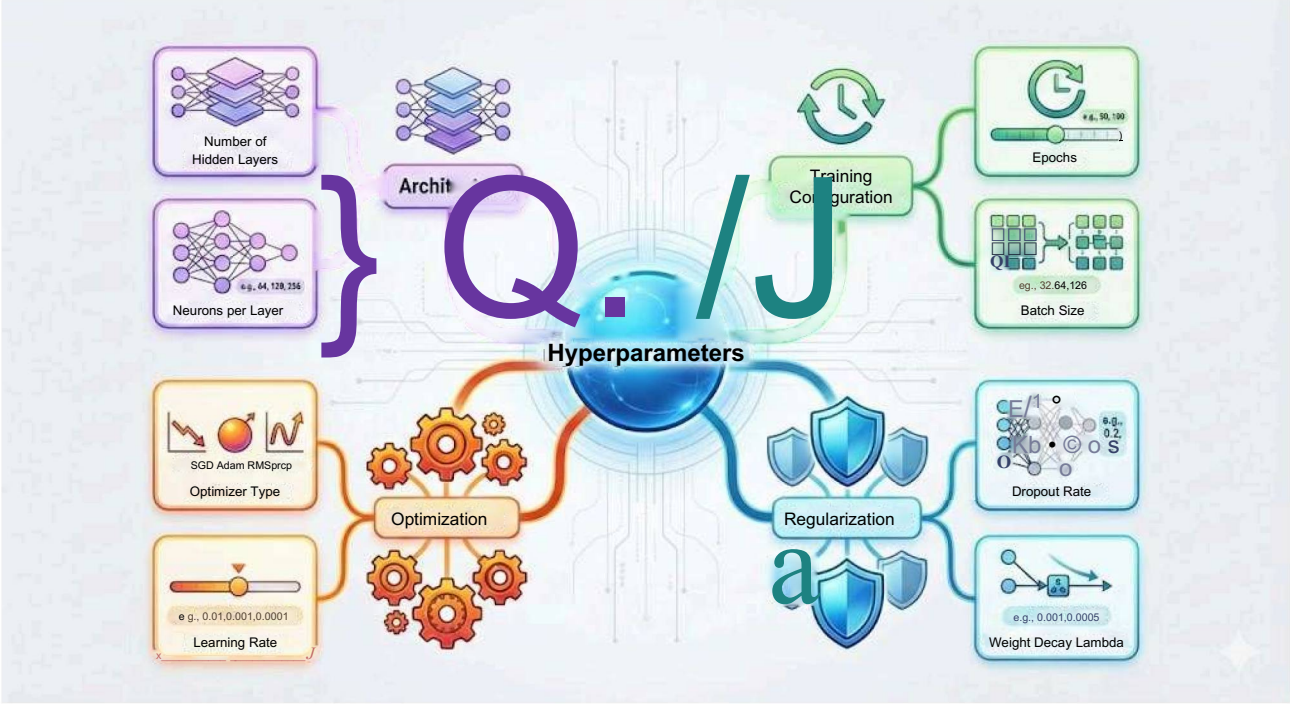
Where p is the dropout rate (hyperparameter).

4. Hyperparameter Tuning Approaches

4.1 Comparison of Methods

Method	Strategy	Efficiency	Best For
Grid Search	Exhaustive search of all combinations	Low	Small search spaces
Random Search	Random sampling from search space	Medium	Large search spaces
Bayesian Search	Probabilistic model-guided search	High	Complex optimization

4.2 Visual Comparison



4.3 Why Bayesian Search (Optuna)?

Bayesian optimization builds a probabilistic model of the objective function:

$$P(y \mid x) = \text{Surrogate Model}$$

It uses an **acquisition function** to decide which hyperparameters to try next:

$$x_{next} = \arg\max_{X} \text{Acquisition}(x)$$

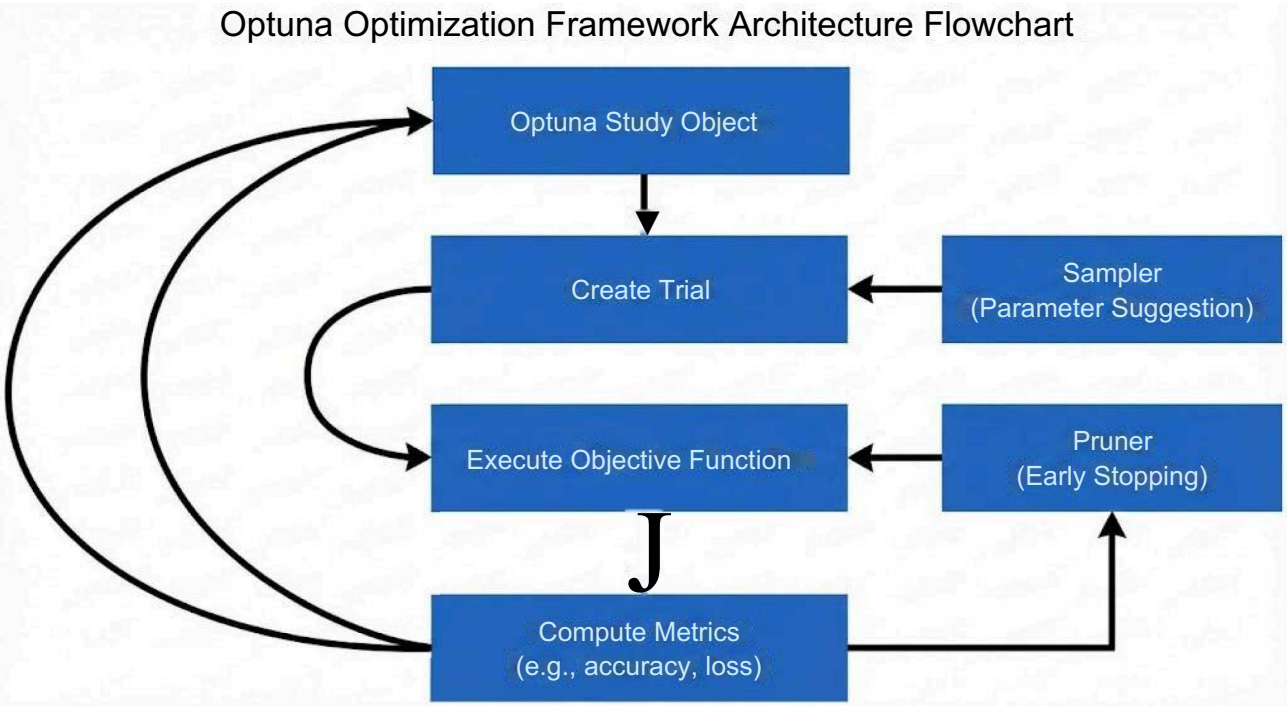
Key Advantages:

1. **Learns from previous trials -** Each trial informs future exploration
2. **Efficient exploration -** Focuses on promising regions
3. **Handles complex spaces -** Works well with mixed parameter types
4. **Early stopping -** Can prune unpromising trials

5. Introduction to Optuna

5.1 What is Optuna?

Optuna is an open-source hyperparameter optimization framework that implements state-of-the-art Bayesian optimization algorithms.



5.2 Core Concepts

Concept	Description
Study	An optimization session consisting of multiple trials
Trial	A single execution of the objective function
Objective Function	The function that trains the model and returns the metric
Sampler	Algorithm that suggests hyperparameter values

Concept	Description
Pruner	Mechanism to early-stop unpromising trials

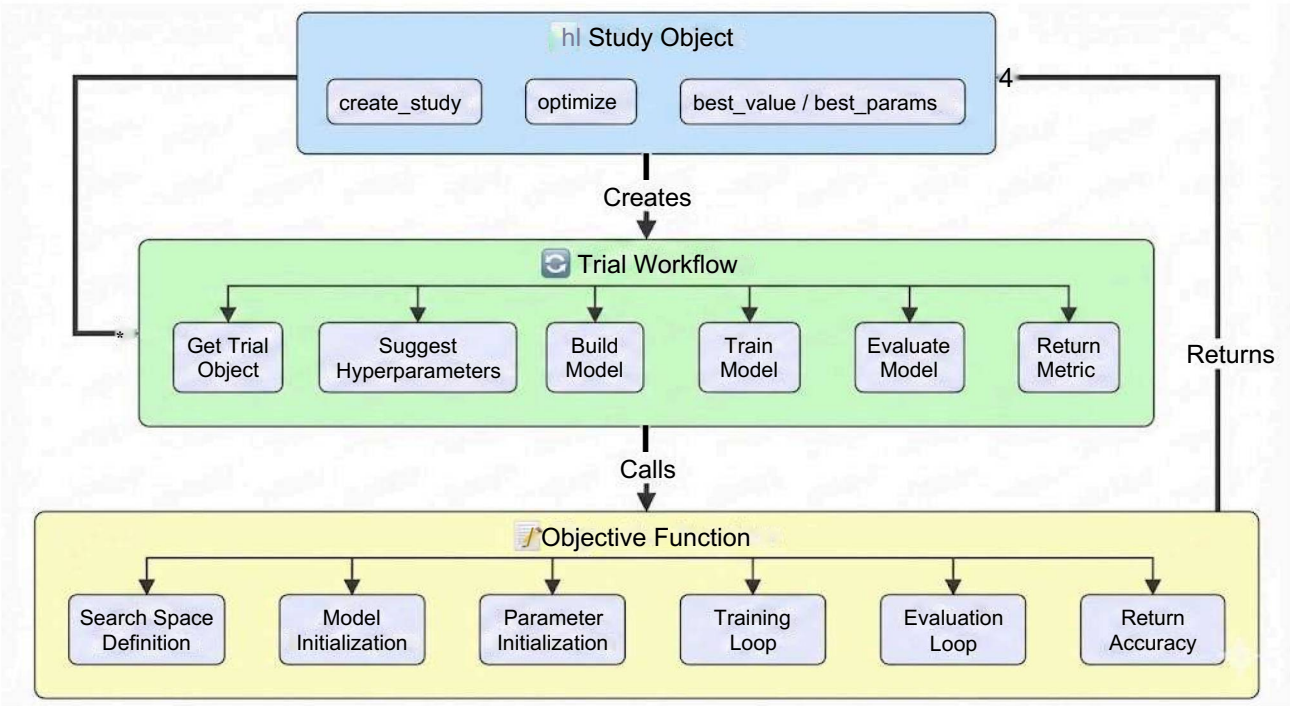
5.3 Installation

```
pip install optuna
```

cP

6. Optuna Architecture & Workflow

6.1 High-Level Architecture



6.2 Objective Function Structure

The objective function is the core of Optuna optimization. It must:

1. **Accept a Trial object** as input
2. **Define search space** using trial.suggest_* methods
3. **Build and train the model**
4. **Return the metric** to optimize

6.3 Trial Object Methods

Method	Purpose	Example
suggest_int(name, low, high, step)	Integer hyperparameter	Layers, neurons, epochs
suggest_-Float (name, low, high, log)	Float hyperparameter	Learning rate, dropout
suggest_categorical(name, choices)	Categorical selection	Optimizer type, batch size

7. Implementation Guide

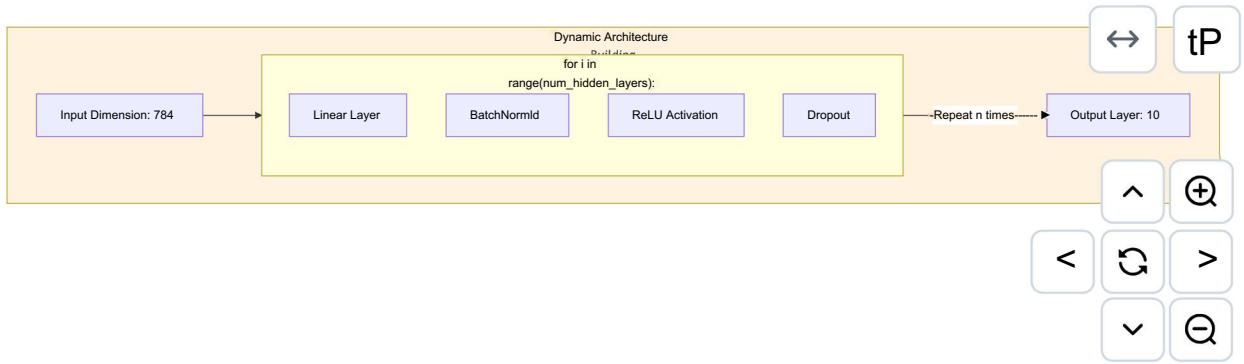
7.1 Two-Round Strategy

We implement hyperparameter tuning in **two rounds:**

Round	Parameters Tuned	Trials	Purpose
Round 1	Hidden Layers, Neurons per Layer	10	Learn the workflow
Round 2	**All** 8 parameters	50+	Production optimization

7.2 Dynamic Neural Network Architecture

The key challenge is creating a network that adapts to different hyperparameter values:



7.3 Architecture Construction Logic

When num_hidden_layers = 2 and neurons_per_layer = 128 :

Step	Layer Created	Dimensions
1	Linear	784 → 128
2	BatchNormId	128
3	Re LU	-
4	Dropout	-
5	Linear	128 → 128
6	BatchNormId	128
7	Re LU	-
8	Dropout	-
9	Linear (Output)	128 → 10

8. Complete Code Implementation

8.1 Imports and Setup

cP

```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
import optuna

#  Set random seeds for reproducibility
torch.manual_seed(42)

#  Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

8.2 Data Loading and Preprocessing

cP

```
#  Load Fashion-MNIST dataset
df = pd.read_csv('fashion-mnist_train.csv')

#  Separate features and labels
```

```
X = df.iloc[:, 1:].values # Pixel values (784 features)
y = df.iloc[:, 0].values                    # Labels (0-9)

#  Train-test split
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
)

# Normalize pixel values to [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0
```

8.3 Custom Dataset Class

```
class CustomDataset(Dataset):
        """PyTorch Dataset wrapper for Fashion-MNIST data."""

        def ___init____(self, features, labels):
                self.features = torch.tensor(features, dtype=torch.float32)
                self.labels = torch.tensor(labels, dtype=torch.long)

        def ___len__  (self):
                return len(self.features)

        def___ getitem___ (self, index):
                return self.features[index], self.labels[index]

# Create dataset objects
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)
```

8.4 Dynamic Neural Network Class

```
class MyNN(nn.Module):
        """
        Dynamic Neural Network with configurable:
        - Number of hidden layers
        - Neurons per layer
        - Dropout rate
        """

        def ___init____(self, input_dim, output_dim, num_hidden_layers,
                                neurons_per_layer, dropout_rate):
                super().____init____()
```

```
        layers = []

        #  Build hidden layers dynamically
        for i in range(num_hidden_layers):
            #  Add Linear layer
            layers.append(nn.Linear(input_dim, neurons_per_layer))
            # Add Batch Normalization
            layers.append(nn.BatchNorm1d(neurons_per_layer))
            #  Add Activation
            layers.append(nn.ReLU())
            #  Add Dropout
            layers.append(nn.Dropout(dropout_rate))
            # Update input dimension for next layer
            input_dim = neurons_per_layer

        #  Add output layer
        layers.append(nn.Linear(neurons_per_layer, output_dim))

        #  Create sequential model
        self.model = nn.Sequential(*layers)

    def forward(self, x):
        return self.model(x)
```

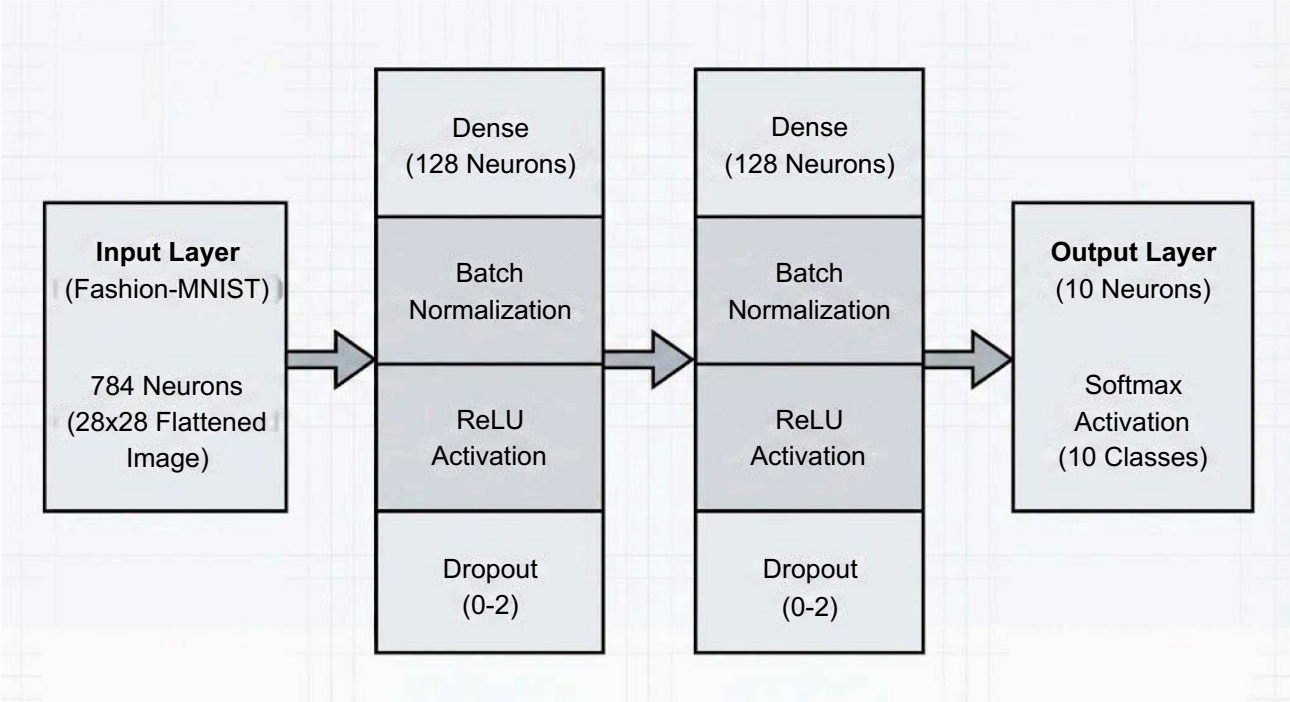
8.5 Objective Function (Complete)

```
def objective(trial):
    """
    Optuna objective function for hyperparameter optimization.

    Args:
        trial: Optuna trial object

    Returns:
        float: Test accuracy for this trial
    """

    # ====================== ======================
    #  1. DEFINE SEARCH SPACE
    #  ============================================

    #  Architecture parameters
    num_hidden_layers = trial.suggest_int("num_hidden_layers", 1, 5)
    neurons_per_layer = trial.suggest_int("neurons_per_layer", 8, 128, step=8)

    #  Training parameters
    epochs = trial.suggest_int("epochs"> 10, 50, step=10)
```

```
    batch_size = trial.suggest_categorical("batch_size", [16, 32, 64, 128])

    #  Optimization parameters
    learning_rate = trial.suggest_float("learning_rate", 1e-5, 1e-1, log=True)
    optimizer_name = trial.suggest_categorical("optimizer",
                                                ['Adam', 'SGD', 'RMSprop'])

    #  Regularization parameters
    dropout_rate = trial.suggest_float("dropout_rate", 0.1, 0.5, step=0.1)
    weight_decay = trial.suggest_float("weight_decay", 1e-5, 1e-3, log=True)

    #  ======================================================
    # 2. CREATE DATA LOADERS
    # ======================================================

    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=T rue
    )
    test_loader = DataLoader(
        test_dataset,
        batch_size=batch_size,
        shuffle=False,
        pin_memory=T rue
    )

    # ======================================================
    # 3. INITIALIZE MODEL
    # ======================================================

    input_dim = 784
    output_dim = 10

    model = MyNN(
        input_dim,
        output_dim,
        num_hidden_layers,
        neurons_per_layer,
        dropout_rate
    )
    model.to(device)

    # ======================================================
    # 4. SETUP TRAINING COMPONENTS
    # ======================================================

    criterion = nn.CrossEntropyl_oss()
```

```
# Select optimizer based on trial suggestion
if optimizer_name == 'Adam':
    optimizer = optim.Adam(
        model.parameters(),
        lr=learning_ratej
        weight_decay=weight_decay
    )
elif optimizer_name == 'SGD':
    optimizer = optim.SGD(
        model.parameters(),
        lr=learning_rate,
        weight_decay=weight_decay
    )
else: # RMSprop
    optimizer = optim.RMSprop(
        model.parameters(),
        lr=learning_rate;
        weight_decay=weight_decay
    )

# ================ ============================
# 5. TRAINING LOOP
# ================ ============================

model.train()
for epoch in range(epochs):
    for batch_features, batch_labels in train_loader:
        # Move data to GPU
        batch_features = batch_features.to(device)
        batch_labels = batch_labels.to(device)

        #  Forward pass
        outputs = model(batch_features)
        loss = criterion(outputs, batch_labels)

        #  Backward pass
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

# ============ ================================
# 6. EVALUATION
# ============: ================================

model.eval()
total = 0
correct = 0
```

```
    with torch.no_grad():
        for batch_features, batch_labels in test_loader:
            batch_features = batch_features.to(device)
            batch_labels = batch_labels.to(device)

            outputs = model(batch_features)
            _,  predicted = torch.max(outputSj 1)

            total += batch_labels.shape[0]
            correct += (predicted == batch_labels).sum().item()

    accuracy = correct / total

    return accuracy
```

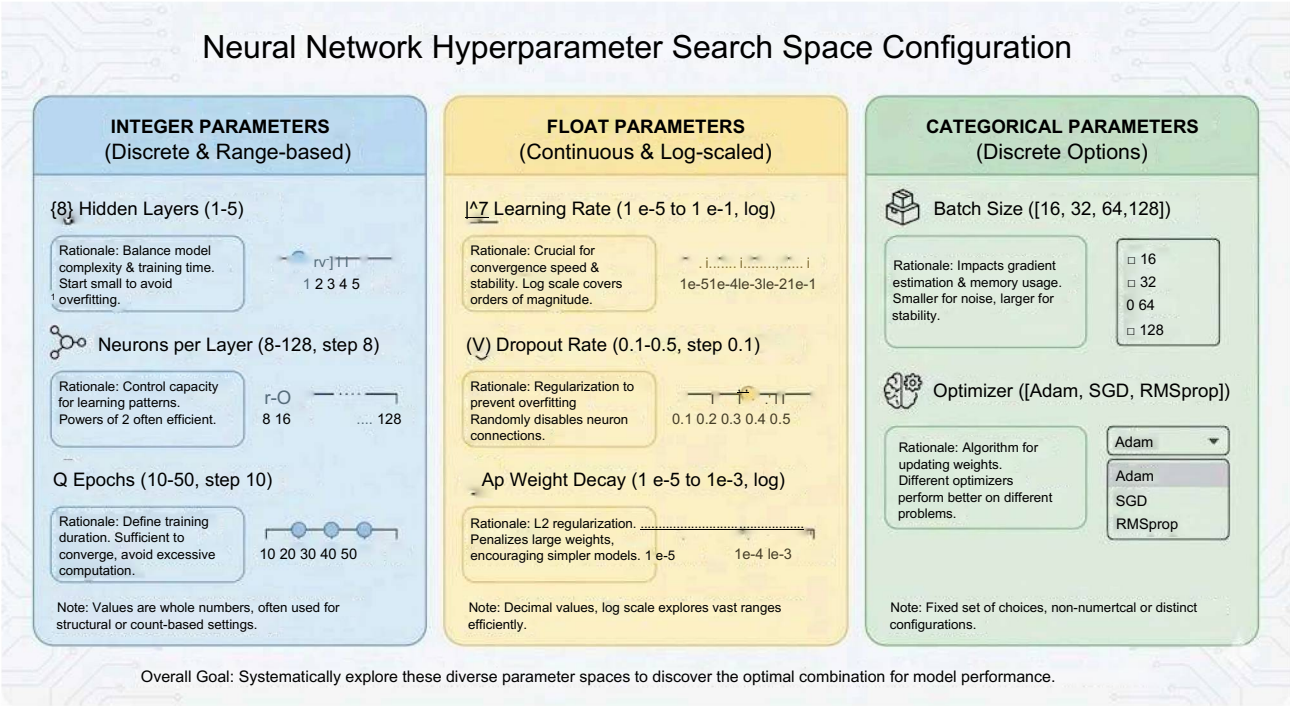


8.6 Running the Optimization

```
#  Create study (maximize accuracy)
study = optuna.create_study(direction^'maximize')

#  Run optimization
study.optimize(objectivej n_trials=50)

#  Get results
print(f"Best Accuracy: {study.best_value:.4f}")
print(f"Best Parameters: {study.best_params}")
```

9. Search Space Configuration

9.1 Search Space Summary



9.2 Search Space Details

Parameter	Type	Range/Values	Scale	Rationale
num_hidden_layers	int	1-5	Linear	Balance complexity and training time
neurons_per_layer	int	8-128 (step=8)	Linear	Powers of 2 work well
epochs	int	10-50 (step=10)	Linear	Prevent over/under-training
batch_size	categorical	[16, 32, 64, 128]	-	Common batch sizes
learning_rate	float	1e-5 to 1 e-1	**Logarithmic**	Spans multiple orders of magnitude

Parameter	Type	Range/Values	Scale	Rationale
optimizer	categorical	[Adam, SGD, RMSprop]	-	Popular optimizers
dropout_rate	float	0.1-0.5 (step=0.1)	Linear	Standard dropout range
weight_decay	float	1e-5 to 1e-3	**Logarithmic**	Regularization strength

9.3 Logarithmic vs Linear Scale

For learning rate, we use logarithmic scale because optimal values can vary across orders of magnitude:

$$\text{log scale: } [10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-4}]$$

This ensures equal probability of sampling from each order of magnitude.

10. Experiment Tracking with MLflow

10.1Why Track Experiments?

When running 50+ trials, you need to:

- **Log** all hyperparameter values
- **Record** all metrics (accuracy, loss)
- **Visualize** relationships between parameters and performance
- **Compare** trials systematically

10.2 Integration Pattern

```
import mlflow

def objective(trial):
    # . . . hyperparameter suggestions ...

    with mlflow.start_run():
        #  Log all hyperparameters
        mlflow.log_params({
            "num_hidden_layers": num_hidden_layers,
            "neurons_per_layer": neurons_per_layer,
```

cP

```
            "epochs": epochs,
            "learning_rate": learning_rate,
            "optimizer": optimizer_name,
            "batch_size": batch_size,
            "dropout_rate": dropout_rate,
            "weight_decay": weight_decay
        })

        #  ... training code ...

        #  Log final metric
        mlflow.log_metric("accuracy", accuracy)

    return accuracy
```

11. Results Analysis & Insights

11.1 Sample Results (50 Trials)

After running 50 trials with full hyperparameter tuning:

Metric	Value
Best Accuracy	~90%
Best num_hidden_layers	2-3
Best neurons_per_layer	~ 100-120
Best epochs	~40
Best optimizer	SGD
Best learning_rate	0.002-0.004
Best dropout_rate	0.1-0.2

11.3 Parameter-Performance Relationships

Parameter	Observation	Recommendation
Hidden Layers	2-3 layers most tried, best results	Use 2-3 layers

Parameter	Observation	Recommendation
Neurons	100-120 dense cluster of good results	Use -100 neurons
Optimizer	SGD has highest accuracy and most trials	Prefer SGD
Dropout	0.1 gives best results	Use lower dropout with BatchNorm
Learning Rate	0.002-0.004 optimal range	Start in this range

12. Best Practices & Tips

Common Mistakes to Avoid

X Mistake	Q Better Approach
Too few trials (< 10)	Run 50+ trials for reliable results
Fixed search space	Adjust based on initial results
Ignoring boundaries	If best value at edge, expand range
No reproducibility	Set random seeds
No logging	Use MLflow or similar

When to Stop

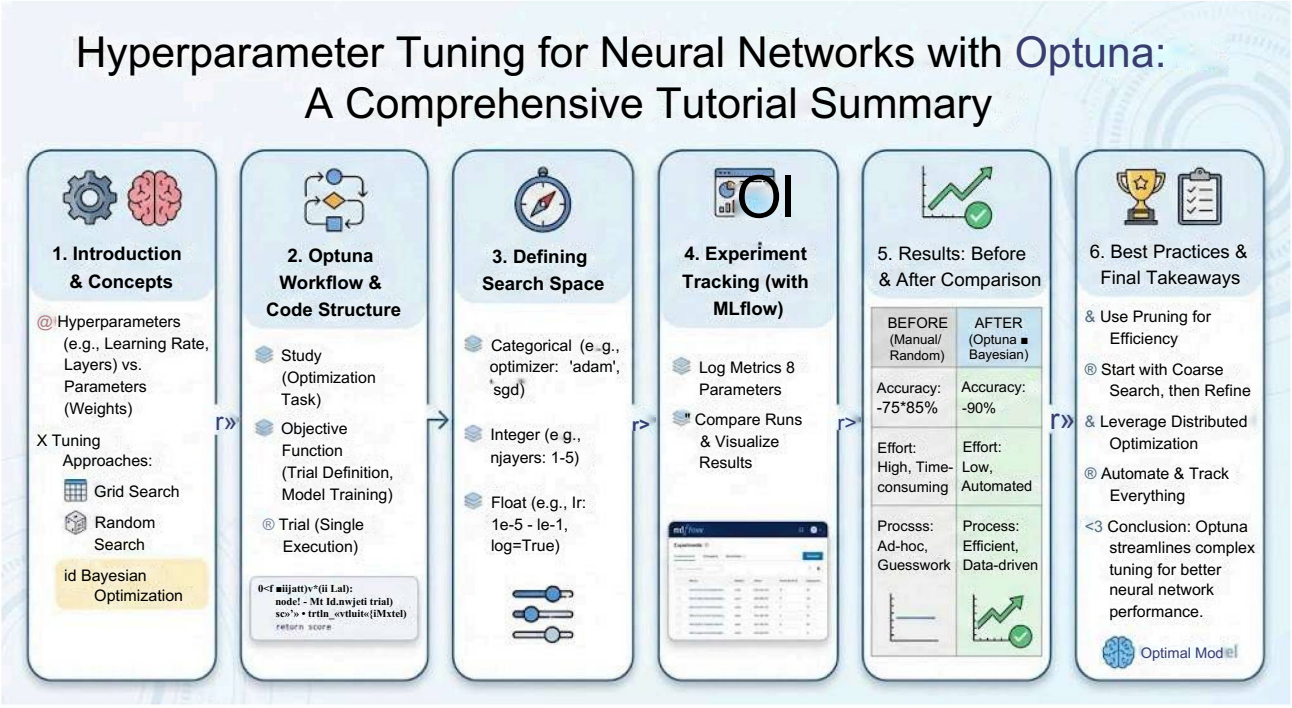
Consider stopping hyperparameter tuning when:

1. **Convergence:** Best value hasn't improved for **N** trials
2. **Budget exhausted:** Time/compute limit reached
3. **Diminishing returns:** Improvements are marginal
4. **Goal achieved:** Target accuracy reached

13.3 Key Takeaways

1. **Don't rely on intuition -** Use systematic hyperparameter tuning
2. **Optuna is efficient** - Bayesian search learns from trials

3. **Dynamic architecture -** Build flexible code for different configs
4. **Track experiments -** Use MLflow for analysis
5. **Iterate and refine** - Adjust search space based on results
6. **Same approach for all DL -** Works for ANN, CNN, RNN, etc.



Hyperparameter Tuning

Neural Networks with Optuna

FI PARAMETERS vs HYPERPARAMETERS

MODEL PARAMETERS
- Weights & Biases
- Learned during training
- Updated by optimizer
- Internal to model

HYPERPARAMETERS
- Architecture, LR, Epochs
- Set before training
- Requires tuning/search
- External configuration

S3 8 HYPERPARAMETERS TO TUNE

1. Hidden Layers: 1 - 5 layers
2. Neurons/Layer: 8 - 128 neurons
3. Epochs: 10 - 50
4. Optimizer: Adam/SGD/RMSprop
5. Learning Rate: 1e-5 - 1e-1 (log)
6. Batch Size: [16, 32, 64, 128]
7. Dropout Rate: 0.1 - 0.5
8. Weight Decay: 1e-5 - 1e-3 (log)

TUNING METHODS COMPARISON

Grid Search
- Exhaustive search
- Low efficiency

Random Search
- Random sampling
- Medium efficiency

Bayesian (Optuna)
- Learns from trials
- Probabilistic model
- High efficiency

OPTUNA CORE CONCEPTS

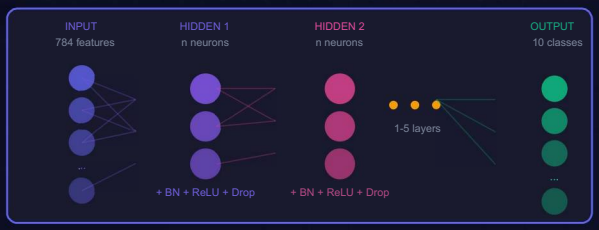
STUDY — Optimization session → TRIAL — Single execution → OBJECTIVE — Function to optimize → METRIC — Accuracy

- trial.suggest_int() → layers, neurons, epochs
- trial.suggest_float() → lr, dropout, decay
- trial.suggest_categorical() → optimizer, batch_size

OPTUNA OPTIMIZATION WORKFLOW

1. **Create Study**
```
optuna.create_study(
direction='maximize')
```
2. **Define Objective**
```
def objective(trial):
params = suggest()
return accuracy
```
3. **Run Optimization**
```
study.optimize(
objective,
n_trials=50)
```
4. **Get Best Results**
```
study.best_value
study.best_params
```
5. **Deploy** — Best Model

DYNAMIC NEURAL NETWORK



KEY FORMULAS

Weight Update Rule

$$w(t+1) = w(t) - \eta \cdot \nabla L(w)$$

η = learning rate (hyperparameter)

L2 Regularization

$$L_{reg} = L + \lambda \sum w_i^2$$

λ = weight decay (hyperparameter)

Dropout

$$\tilde{y} = y \text{ with prob } p$$
$$\tilde{y} = y/(1-p) \text{ with prob } (1-p)$$

p = dropout rate (hyperparameter)

Bayesian Optimization

$$x_{next} = \arg\max_x \text{Acquisition}(x)$$

Learns from previous trials

S OPTIMIZATION RESULTS (50 Trials)

Best Accuracy	Previous Best
~90%	~88%

Optimal Configuration:
- Hidden Layers: 2-3
- Neurons/Layer: 100-120
- Learning Rate: 0.002-0.004
- Optimizer: SGD
- Dropout: 0.1-0.2
- Epochs: 40

A BEST PRACTICES

- ✓ Run 50+ trials for reliable results
- ✓ Use log scale for LR & weight decay
- ✓ Track experiments with MLflow
- ✓ Set random seeds for reproducibility
- ✗ Don't rely on intuition alone
- ✗ Avoid too few trials (<10)

O WHEN TO STOP TUNING

1. Convergence — No improvement for N trials
2. Budget Exhausted — Time/compute limit reached
3. Diminishing Returns — Improvements are marginal
4. Goal Achieved — Target accuracy reached

KEY TAKEAWAYS

1) Systematic tuning > intuition
2) Optuna uses Bayesian search
3) Build dynamic, flexible models
4) Track experiments with MLflow
5) Works for ANN, CNN, RNN

Journey: 83% (Basic) → 88% (GPU+Full) → 88% (Regularized) → ~90% (Optuna)

CampusX • PyTorch Series Video 10 • Hyperparameter Tuning with Optuna

Fashion-MNIST Classification • 8 Hyperparameters • 50 Trials • Bayesian Optimization

CampusX

Convolutional Neural Networks (CNN) with PyTorch - Fashion MNIST Implementation

Code

- htt ps://colab.research. g oog le.com/drive/1uRUPZFQwiBxEQCCiLNQCFtk7b4L_7N_E

1. Introduction and Recap

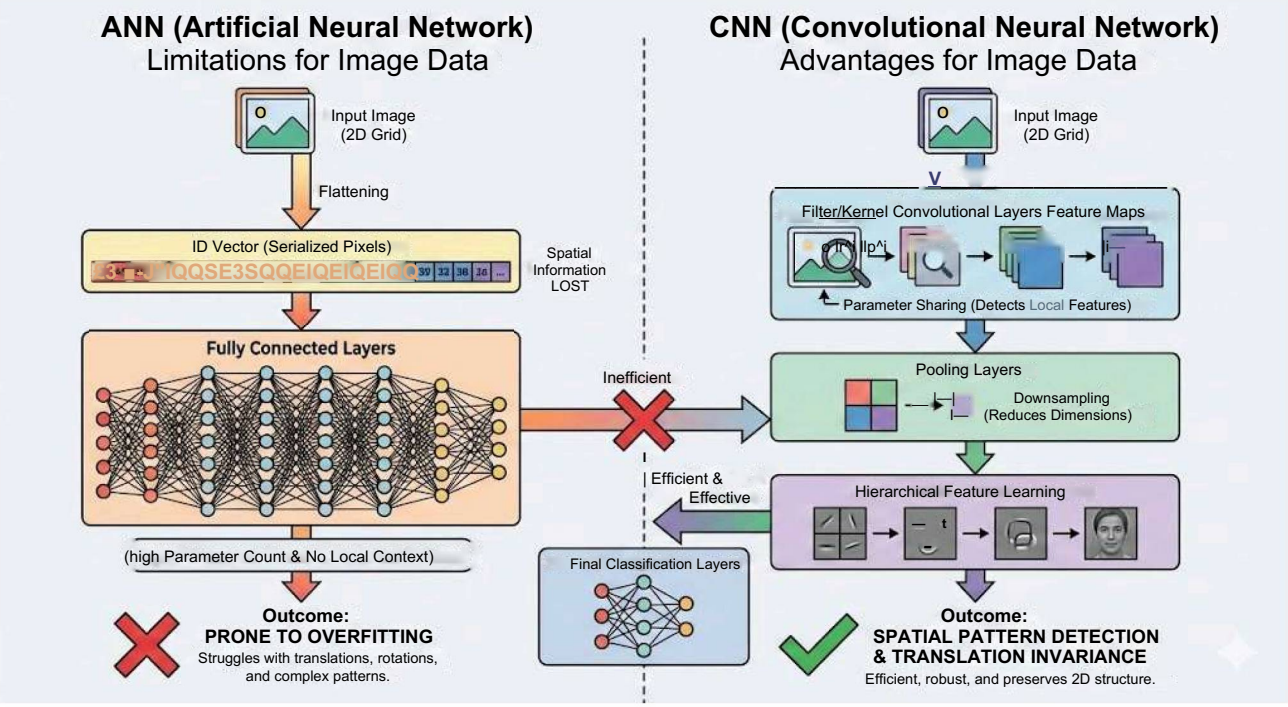
1.1 Journey So Far with ANN

Before diving into CNNs, let's recap the journey with Artificial Neural Networks (ANN) on the Fashion MNIST dataset:

Version	Approach	Training Data	Test Accuracy	Training Accuracy	Key Changes
V1	Basic ANN	Subset (CPU)	~83%	-	Limited by CPU speed, used data subset

Version	Approach	Training Data	Test Accuracy	Training Accuracy	Key Changes
V2	ANN + GPU	Full Dataset	88.9%	~98%	GPU acceleration, full dataset training
V3	Optimized ANN	Full Dataset	88.9%	~93%	Batch Normalization, Dropout, Weight Decay
V4	Hyperparameter Tuning	Full Dataset	~90%	-	Optuna + Bayesian Optimization

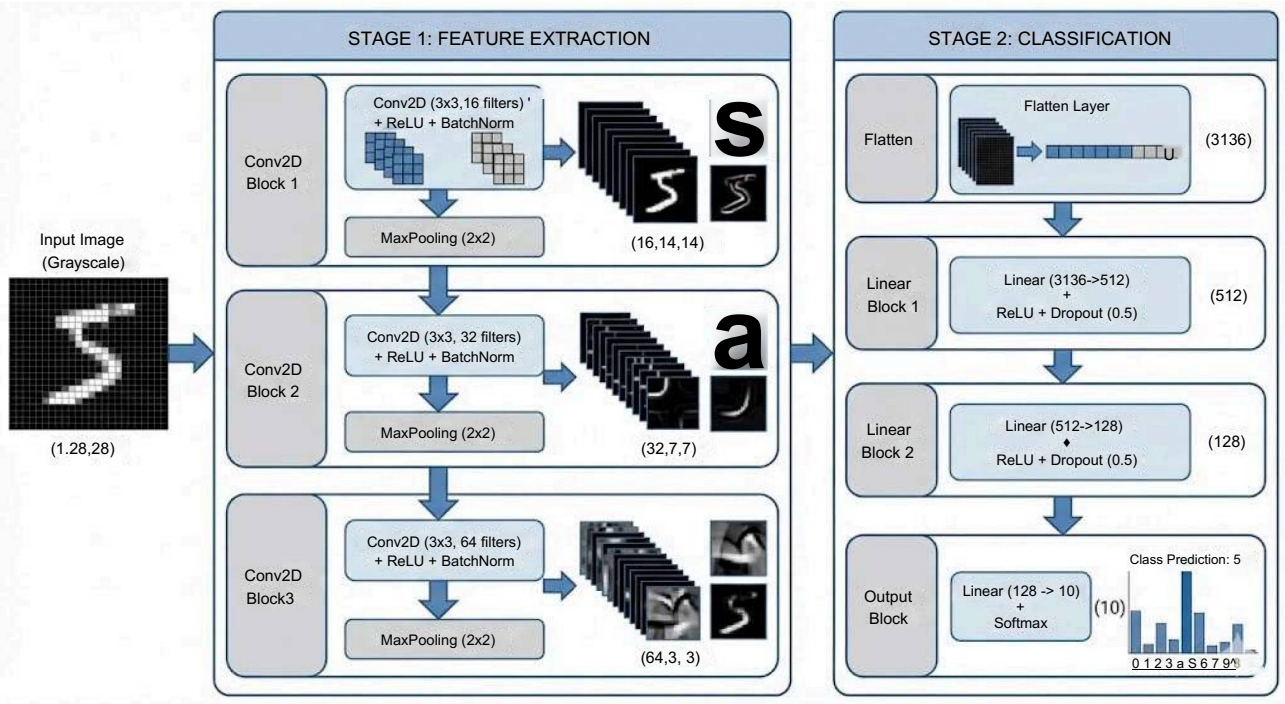
1.2 The Problem with ANNs for Image Data



Key Insight: ANNs have limitations when working with image-based data. Despite extensive optimization efforts (batch normalization, dropout, hyperparameter tuning), the best accuracy achieved was -90%. CNNs are specifically designed for grid-based data like images and typically outperform ANNs significantly.

2. Plan of Action

CNN Implementation Strategy



2.2 Learning Objectives

Stage	Video Focus	Techniques	Goal
1	Current Video	Basic CNN Architecture	Learn PyTorch CNN implementation
2	Next Video	Optimization + Optuna	Reduce overfitting, improve accuracy
3	Future Video	Transfer Learning	Use pre-trained models (ResNet, VGGNet)

3. What is CNN

3.1 Definition

Convolutional Neural Networks (CNNs) are a specialized type of deep learning model designed to process and analyze structured grid-based data, such as images and videos. They excel at tasks like:

- Image Classification
- Image Recognition
- Object Detection
- Video Analysis

3.2 Why CNNs Excel with Grid-Based Data

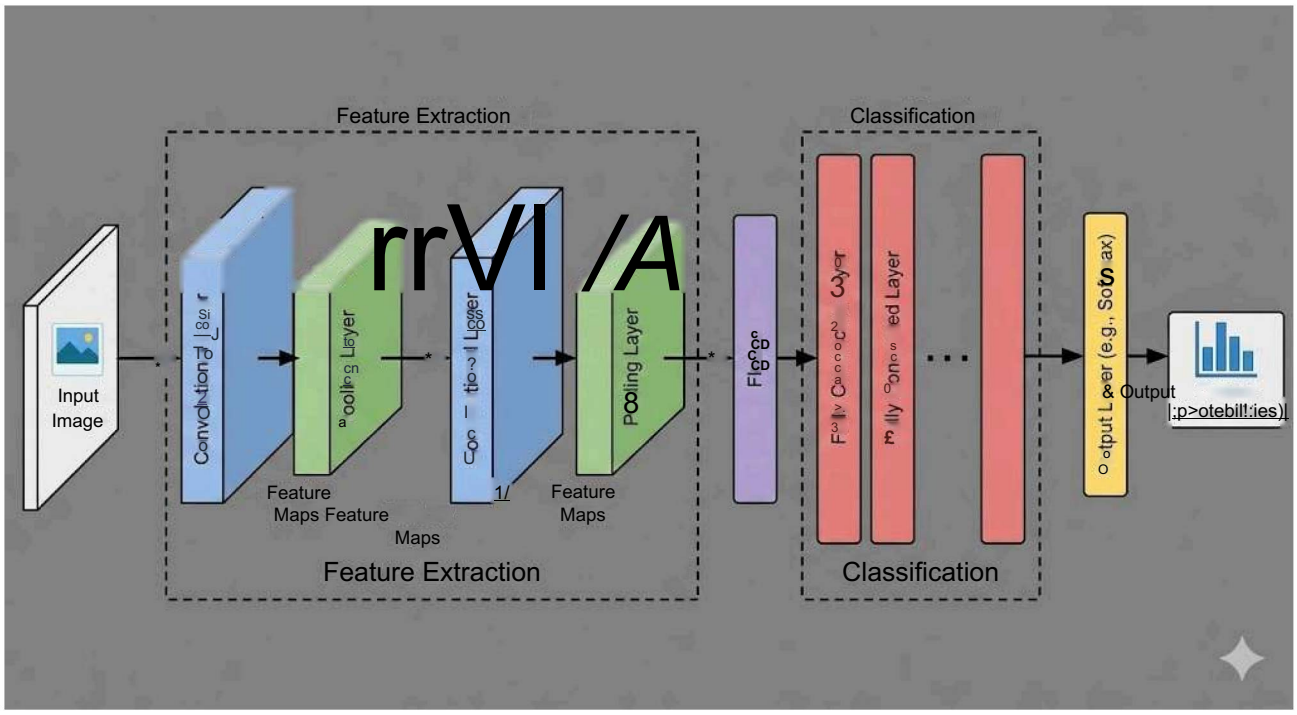
3.3 Image Data Representation

Image Type	Channels	Representation
Grayscale	1	Single 2D matrix of pixel values
RGB Color	3	Three 2D matrices (Red, Green, Blue)
RGBA	4	Four 2D matrices (RGB + Alpha)

Fashion MNIST Specifics:

- Image Size: 28 x 28 pixels
- Channels: 1 (Grayscale)
- Pixel Values: 0-255 (normalized to 0-1)

4. CNN Architecture Overview



Feature Extraction Stage

This stage is responsible for automatically learning and extracting important features from images.

Convolution Layer

Component	Description	Purpose
Filters/Kernels	Small matrices (e.g., 3x3)	Detect specific patterns
Stride	Step size for filter movement	Controls output size
Padding	Adding border pixels	Preserves spatial dimensions
Output	Feature Maps	Detected patterns/features

What Convolution Detects:

- Low-level features: Edges, lines, corners
- Mid-level features: Shapes, textures
- High-level features: Object parts, patterns

Pooling Layer

Type	Operation	Purpose
Max Pooling	Takes maximum value in window	Retains dominant features
Average Pooling	Takes average value in window	Smooths feature maps
Global Pooling	Pools entire feature map	Reduces to single value

Benefits of Pooling:

1. Reduces spatial dimensions (downsampling)
2. Reduces computational complexity
3. Provides translation invariance
4. Helps prevent overfitting

Classification Stage

This stage uses a fully connected neural network (ANN) to classify based on extracted features.

Layer	Function
Flatten	Converts 2D feature maps to 1D vector
Hidden Layers	Learn complex patterns from features
Output Layer	Produces class probabilities

5. Mathematical Foundations

5.1 Convolution Operation

The 2D convolution operation between input *1* and kernel *K* is defined as:

$$(I * K)(i,j) = \sum_{m\ n} \text{zs/(i + } m,j + ri) \bullet K(m,n)$$

Where:

- C^j) is the position in the output feature map
- *(m,n)* are the kernel indices
- *I* is the input image/feature map
- *K* is the kernel/filter

5.2 Output Size Calculation

For a convolution layer, the output dimensions are calculated as:

$$\text{Output Size} = \frac{W - K + 2P}{S} + 1$$

Where:

- 14/ = Input size (width or height)
- *K* = Kernel size
- P = Padding
- 5 = Stride

Example Calculation for Our Architecture:

First Convolution (with 'same' padding):

- Input: 28x28

- Kernel: 3x3
- Padding: 'same' (output = input)
- Stride: 1

$$\text{Output} = 28 \text{ x} 28 \text{x} 32 \text{ (32 filters)}$$

First Max Pooling:

- Input: 28x28
- Kernel: 2x2
- Stride: 2

$$\text{Output Size} = \frac{28\text{-}2 + 0}{2} + 1 = 14$$

5.3 Pooling Output Calculation

For max pooling:

$$\text{Output Size} = \frac{W - K}{S} + 1$$

With 2x2 kernel and stride 2:

$$\text{Output Size} = \frac{147}{2}$$

5.4 Flatten Operation

The flatten operation converts a 3D tensor to 1D:

$$\text{Flattened Size} = C \text{ x } H \text{ x } W$$

Where:

- C = Number of channels
- H = Height
- W = Width

For our architecture:

$$\text{Flattened Size} = 64 \text{ x } 7 \text{ x } 7 = 3136$$

6. Detailed Architecture Design

6.1 Complete Architecture Specification

6.2 Layer-by-Layer Specification Table

Layer	Type	Input Shape	Output Shape	Parameters
1	Conv2d	1 X 28 X 28	32 x 28 x 28	filters=32, kernel = 3x3, padding = 'same'
2	Re LU	32 x 28 x 28	32 x 28 x 28	-
3	BatchNorm2d	32 x 28 x 28	32 x 28 x 28	num_features=32
4	MaxPool2d	32 x 28 x 28	32 x 14 x 14	kernel=2x2, stride=2
5	Conv2d	32 x 14 x 14	64 x 14 x 14	filters=64, kernel = 3x3, padding = 'same'
6	Re LU	64 x 14 x 14	64 x 14 x 14	-
7	BatchNorm2d	64 x 14 x 14	64 x 14 x 14	num_features=64
8	MaxPool2d	64 x 14 x 14	64 x 7 x 7	kernel=2x2, stride=2
9	Flatten	64 x 7 x 7	3136	-
10	Linear	3136	128	in = 3136, out=128
11	Re LU	128	128	-
12	Dropout	128	128	p=0.4
13	Linear	128	64	in = 128, out=64
14	Re LU	64	64	-

Layer	Type	Input Shape	Output Shape	Parameters
15	Dropout	64	64	p=0.4
16	Linear	64	10	in = 64, out=10

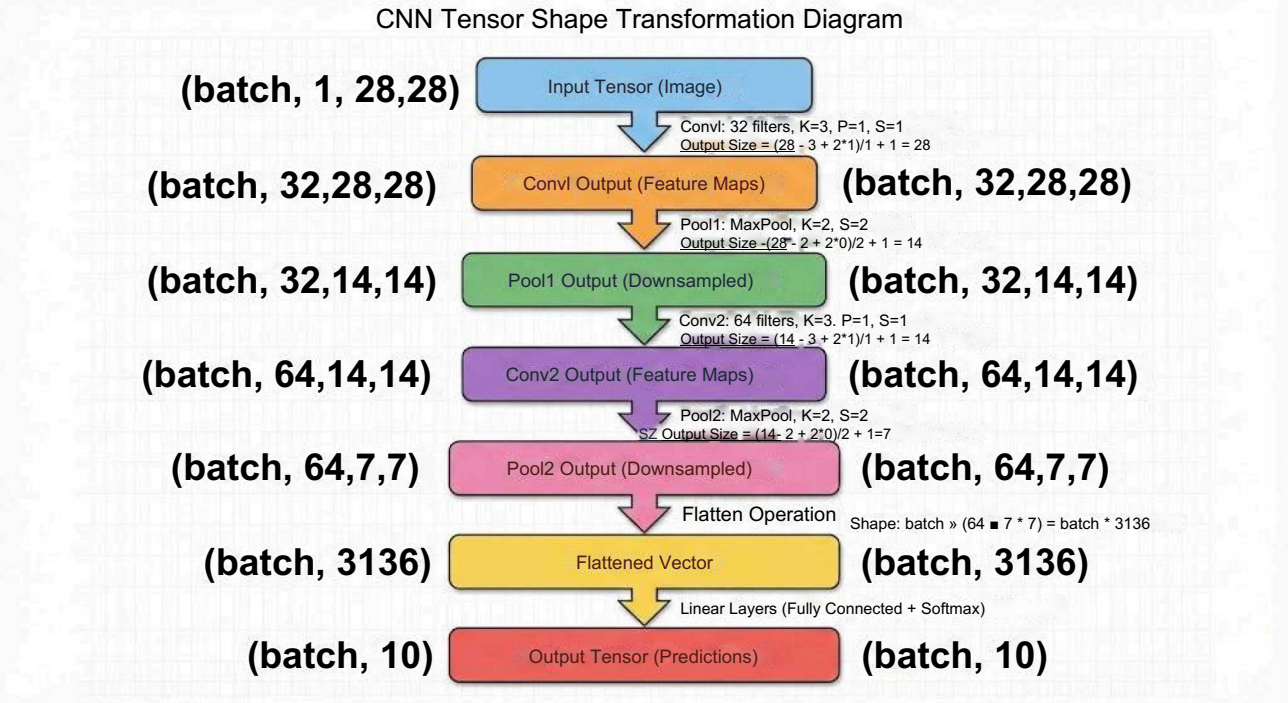
6.3 Architecture Configuration Summary

Component	Configuration
Input Image	1 x 28 x 28 (Grayscale)
Conv Block 1	32 filters, 3x3 kernel, same padding
Conv Block 2	64 filters, 3x3 kernel, same padding
Pooling	Max Pooling 2x2, stride 2
Hidden Layer 1	128 neurons
Hidden Layer 2	64 neurons
Output Layer	10 neurons (classes)
Activation	Re LU
Regularization	BatchNorm2d, Dropout (p=0.4)

7. Tensor Shape Transformations

7.1 Complete Shape Flow

Understanding how tensor shapes change through the network is crucial for CNN implementation:



7.2 Detailed Shape Calculations

Step	Operation	Input Shape	Output Shape	Calculation
1	Input	-	(B, 1, 28, 28)	Given grayscale image
2	Conv2d (32, 3x3, same)	(B, 1, 28, 28)	(B, 32, 28, 28)	Padding preserves size, 32 filters
3	MaxPool2d (2x2, s=2)	(B, 32, 28, 28)	(B, 32, 14, 14)	28 / 2 = 14
4	Conv2d (64, 3x3, same)	(B, 32, 14, 14)	(B, 64, 14, 14)	Padding preserves size, 64 filters
5	MaxPool2d (2x2, s=2)	(B, 64, 14, 14)	(B, 64, 7, 7)	14 / 2 = 7
6	Flatten	(B, 64, 7, 7)	(B, 3136)	64X7X7 = 3136
7	Linear (128)	(B, 3136)	(B, 128)	FC layer
8	Linear (64)	(B, 128)	(B, 64)	FC layer
9	Linear (10)	(B, 64)	(B, 10)	Output layer

Where *B* = Batch Size (32 in our implementation)

7.3 Calculating Flattened Input Size

Step-by-step calculation:

1. **After Convl (with same padding):**
    - Channels: 1 -> 32
    - Spatial: 28 X28 → 28 X28
    - Shape: (32,28,28)
2. **After Pooll:**
    - Channels: 32 (unchanged)
    - Spatial: 28 x28 → 14x14
    - Shape: (32,14,14)
3. **After Conv2 (with same padding):**
    - Channels: 32 -» 64
    - Spatial: 14 x 14 → 14x14
    - Shape: (64,14,14)
4. **After Pool2:**
    - Channels: 64 (unchanged)
    - Spatial: 14x14 → 7x7
    - Shape: (64,7,7)
5. **After Flatten: $$** \text{Flattened Dimension} = 64\times 7\times 7 = 3136 $$

8. PyTorch Implementation

8.1 Import Dependencies

tP

```
import pandas as pd
from sklearn.model_selection import train_test_split
import torch
from torch.utils.data import Dataset, DataLoader
import torch.nn as nn
import torch.optim as optim
import matplotlib.pyplot as pit
```

```
# Set random seeds for reproducibility
torch.manual_seed(42)

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

8.2 Data Loading and Preprocessing

LP

```
# Load dataset
df = pd.read_csv('/content/fashion-mnist_train.csv')

# Display sample images
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
fig.suptitle("First 16 Images", fontsize=16)

for i, ax in enumerate(axes.flat):
    img = df.iloc[i, 1:].values.reshape(28, 28)
    ax.imshow(img)
    ax.axis('off')
    ax.set_title(f"Label: {df.iloc[i, 0]}")

plt.tight_layout(rect=[0, 0, 1, 0.96])
plt.show()
```

8.3 Train-Test Split and Normalization

LP

```
# Separate features and labels
X = df.iloc[:, 1:].values
y = df.iloc[:, 0].values

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalize pixel values to [0, 1]
X_train = X_train / 255.0
X_test = X_test / 255.0
```

8.4 Custom Dataset Class (Key Modification for CNN)

```
class CustomDataset(Dataset):
    def___ init____(self, features, labels):
        # IMPORTANT: Reshape from (samples, 784) to (samples, 1, 28, 28)
        # This is the key change from ANN to CNN
        # Shape: (batch_size, channels, height, width)
        self.features = torch.tensor(
            features,
            dtype=torch,float32
        ).reshape(-1, 1, 28, 28) # -1 for batch, 1 channel, 28x28 image

        self.labels = torch.tensor(labels, dtype=torch.long)

    def ___len__  (self):
        return len(self.features)

    def___ getitem___ (self, index):
        return self.featuresfindex], self.labels[index]
```

Key Point: The reshape from 1D (784 features) to 4D tensor (batch, channel, height, width) is crucial for CNN processing.

8.5 DataLoader Creation

```
# Create datasets
train_dataset = CustomDataset(X_train, y_train)
test_dataset = CustomDataset(X_test, y_test)

# Create data loaders with batch_size=32
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,
    pin_memory=T rue
)
test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False,
    pin_memory=T rue
)
```

8.6 CNN Model Architecture

```
class MyNN(nn.Module):
    def ______ init____ (self, input_features):
        super().______ init____ ()

        # Stage 1: Feature Extraction
        self.features = nn.Sequential(
            # First Conv Block
            nn.Conv2d(input_features, 32, kernel_size=3, padding='same'),
            nn.ReLU(),
            nn.BatchNorm2d(32),
            nn.MaxPool2d(kernel_size=2, stride=2),

            #  Second Conv Block
            nn.Conv2d(32, 64, kernel_size=3, padding='same'),
            nn.ReLU(),
            nn.BatchNorm2d(64),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )

        #  Stage 2: Classification
        self.classifier = nn.Sequential(
            nn.Flatten(),

            #  Hidden Layer 1
            nn.Linear(64 * 7 * 7, 128), # 64 channels x 7 x 7 = 3136
            nn.ReLU(),
            nn.Dropout(p=0.4),

            #  Hidden Layer 2
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Dropout(p=0.4),

            #  Output Layer
            nn.Linear(64, 10) # 10 classes
        )

    def forward(self, x):
        # Feature extraction
        x = self.features(x)
        # Classification
        x = self.classifier(x)
        return x
```

8.7 Model Initialization and Training Setup

```
# Hyperparameters
learning_rate = 0.01
epochs = 100

#  Initialize model
model = MyNN(input_features=l) # 1 for grayscale images
model.to(device)

#  Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
        model.parameters(),
        lr=learning_rate,
        weight_decay=le-4 # L2 regularization
)
```

8.8 Training Loop

```
# Training loop
for epoch in range(epochs):
        total_epoch_loss = 0

        for batch_features, batch_labels in train_loader:
                # Move data to GPU
                batch_features = batch_features.to(device)
                batch_labels = batch_labels.to(device)

                #  Forward pass
                outputs = model(batch_features)

                #  Calculate loss
                loss = criterion(outputs., batch_labels)

                #  Backward pass
                optimizer.zero_grad()
                loss.backward()

                #  Update weights
                optimizer.step()

                total_epoch_loss += loss.item()

        avg_loss = total_epoch_loss / len(train_loader)
        print(fEpoch: {epoch + 1}, Loss: {avg_loss:.4f}')
```

8.9 Evaluation Functions

tP

```
# Set model to evaluation mode
model.eval()

# Evaluation on test data
def evaluate(model, data_loader, device):
    total = 0
    correct = 0

    with torch.no_grad():
        for batch_features, batch_labels in data_loader:
            batch_features = batch_features.to(device)
            batch_labels = batch_labels.to(device)

            outputs = model(batch_features)
            _,  predicted = torch.max(outputs, 1)

            total += batch_labels.shape[0]
            correct += (predicted == batch_labels).sum().item()

    return correct / total

# Calculate accuracies
test_accuracy = evaluate(modelj test_loader, device)
train_accuracy = evaluate(model, train_loader., device)

print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Train Accuracy: {train_accuracy:.4f}")
```

9. Results and Analysis

9.1 Performance Comparison: ANN vs CNN

Model	Test Accuracy	Training Accuracy	Overfitting Gap
ANN V1 (Subset)	~83%	-	-
ANN V2 (GPU)	88.9%	~98%	~9.1%
ANN V3 (Optimized)	88.9%	~93%	~4.1%
ANN V4 (Hypertuned)	~90%	-	-
CNN (Basic)	**92.6%**	**99.9%**	**~7.3%**

9.3 Why CNN Outperforms ANN on Image Data

Factor	ANN	CNN
Spatial Information	Lost (flattened to 1D)	Preserved (2D convolutions)
Parameter Efficiency	Every pixel to every neuron	Weight sharing via filters
Translation Invariance	None	Yes (via convolution + pooling)
Feature Hierarchy	Manual feature engineering	Automatic feature learning
Local Pattern Detection	Cannot detect	Excels at detecting

9.4 Overfitting Analysis

Causes of Overfitting in Current Implementation:

1. **Small Dataset:** Fashion MNIST has limited samples
2. **Low Image Resolution:** 28x28 images lack variation
3. **Similarity in Data:** Images within classes are very similar
4. **Not Real-World Data:** Synthetic/curated dataset

Mitigation Strategies for Next Steps:

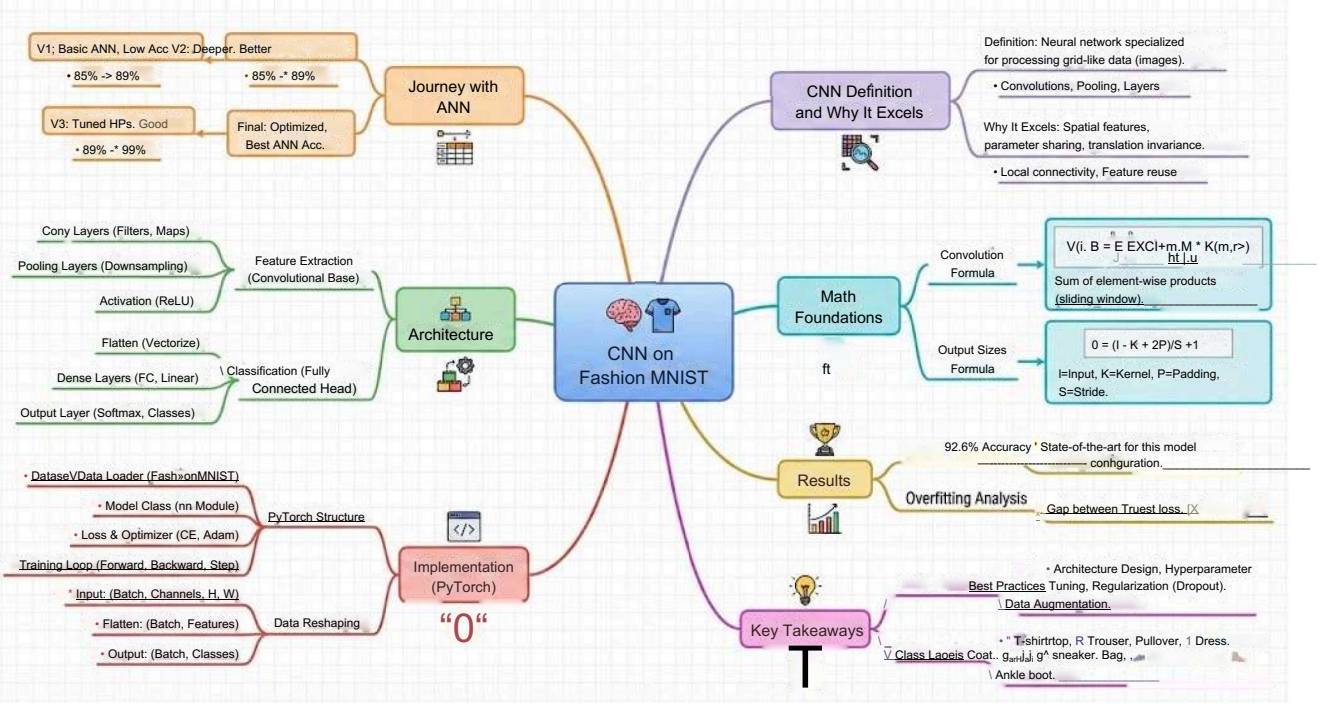
- Data Augmentation
- More aggressive Dropout
- Learning Rate Scheduling
- Early Stopping
- Transfer Learning with pre-trained models

10. Key Takeaways

10.1Technical Summary

Concept	Key Point
CNN Purpose	Neural networks optimized for grid-based data (images, videos)

Concept	Key Point
Feature Extraction	Convolution + Pooling layers detect spatial patterns
Classification	Fully connected layers classify based on extracted features
Data Reshaping	1D to 4D: (samples, 784) (batch, 1, 28, 28)
Tensor Calculation	Track shapes through network to determine layer dimensions
PyTorch Sequential	Use nn.sequential for clean architecture definition



References

1. PyTorch Documentation: pytorch.org/docs
2. Fashion MNIST Dataset: github.com/zalandoresearch/fashion-mnist
3. CNN Fundamentals: 100 Days of Deep Learning Playlist (Videos 1-6 on CNNs)

Convolutional Neural Networks

PyTorch Implementation • Fashion MNIST

ANN — CNN JOURNEY

Model Performance Comparison

ANN V1 (Basic)	83%
ANN V2 (GPU)	88.9%
ANN V4 (Optuna)	90.x%
CNN (Basic)	92.6% +2.6% accuracy boost!

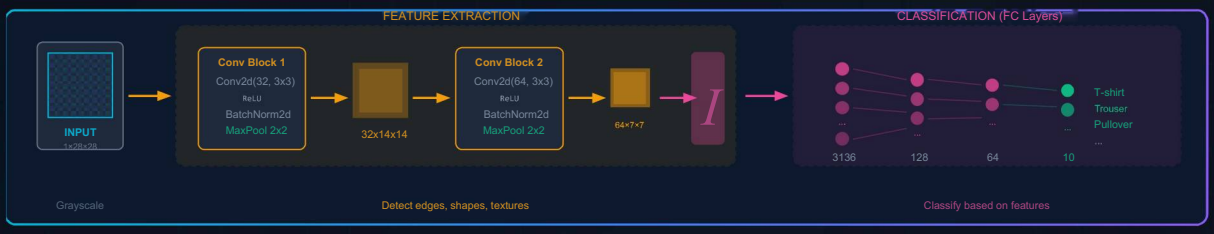
Key Insight

CNNs preserve spatial information that ANNs lose when flattening images.

WHY CNN FOR IMAGES?

Factor	ANN X	CNN /
Spatial Information	Lost (flattened)	Preserved (2D conv)
Parameter Efficiency	All connections	Weight sharing
Translation Invariance	None	Yes (conv + pool)
Feature Learning	Manual engineering	Automatic hierarchy
Local Pattern Detection	Cannot detect	Excels at detecting

CNN ARCHITECTURE: FEATURE EXTRACTION → CLASSIFICATION



CONVOLUTION OPERATION

Convolution Formula

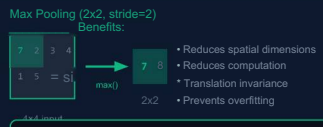
$$(I * K)(ij) = XZI(i+m,j+n) \cdot K(m,n)$$

Element-wise multiplication and sum

POOLING OPERATION

Max Pooling (2x2, stride=2)

Benefits:



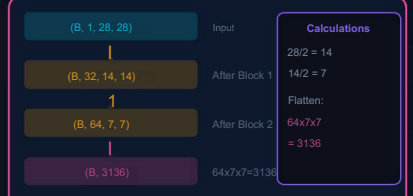
- Reduces spatial dimensions
- Reduces computation
- Translation invariance
- Prevents overfitting

Output Size Formula

$$Output = (W - K + 2P) / S + 1$$

W=input, K=kernel, P=padding, S=stride

TENSOR SHAPE FLOW



PYTORCH IMPLEMENTATION HIGHLIGHTS

Key Change: Data Reshape

```
# ANN: (samples, 784)
# CNN: (samples, 1, 28, 28)
,reshape(-l, 1, 28, 28)
```

Convolution Layer

```
nn.Conv2d(
  in_channels, out_channels,
  kernel_size=3, padding='same'
)
```

Model Structure

```
self-features = nn.Sequential(
  Conv2d, ReLU, BatchNorm2d, MaxPool2d
)
self.classifier = nn.Sequential(...)
```

Pooling & Flatten

```
nn.MaxPool2d(
  kernel_size=2, stride=2
)
nn.Flatten()
```

ARCHITECTURE CONFIG

Component	Configuration
Input Image	1 x 28 x 28 (Grayscale)
Conv Block 1	32 filters, 3x3 kernel, same padding
Conv Block 2	64 filters, 3x3 kernel, same padding
Pooling	Max Pooling 2x2, stride 2
Hidden Layers	128 -» 64 neurons
Output Layer	16 neurons (classes)
Regularization	BatchNorm2d, Dropout (p=0.4)

RESULTS: CNN vs ANN

CNN Test Accuracy	Best ANN Accuracy
92.6%	~90%

Improvement Analysis

CNN outperforms best ANN by +2.6%

With only basic architecture (no optimization yet!)

WHAT CONVOLUTION DETECTS

Low-Level	Mid-Level	High
Edges Lines Corners	Shapes Textures Patterns	Parts Objects

Fashion MNIST Classes:

T-shirt | Trouser | Pullover | Dress | Coat | +5

NEXT STEPS

1. **Optimization + Optuna**
   Reduce overfitting, hyperparameter tuning
2. **Data Augmentation**
   Increase dataset diversity
2. **Transfer Learning**
   Use pre-trained models (ResNet, VGGNet)

Current: 99.9% train vs 92.6% test = Overfitting gap -7.3%

KEY TAKEAWAYS

1. CNNs preserve spatial info
- Conv + Pool = Feature extraction
0. Reshape: (N,784) — (N,1,28,28)
- Track tensor shapes
- nn.Sequential for clean code Q 92.6% accuracy!

CNN Architecture: INPUT (1×28×28) → CONV1 (32) → POOL → CONV2 (64) → POOL → MHs"- FC <128-64-10) - OUTPUT

CampusX

Transfer Learning Using PyTorch: A Comprehensive Guide

Colab Notebook

htt ps://colab.research. g oog le.com/drive/196F8ZuAdbeS9_96v3dLpf9oCpMLEuRwB

Introduction

This guide covers Transfer Learning implementation using PyTorch, specifically demonstrating how to use the **VGG-16** pre-trained model on the **Fashion MNIST** dataset. The goal is to leverage knowledge from a model trained on ImageNet (1.4 million images) to achieve better accuracy on a smaller dataset.

Project Context

Metric	ANN (Baseline)	Basic CNN	CNN + Hyperparameter Tuning	Transfer Learning (VGG-16)
Accuracy	~90%	92.6%	93.1%	~94%+
Training Complexity	Low	Medium	High	Medium

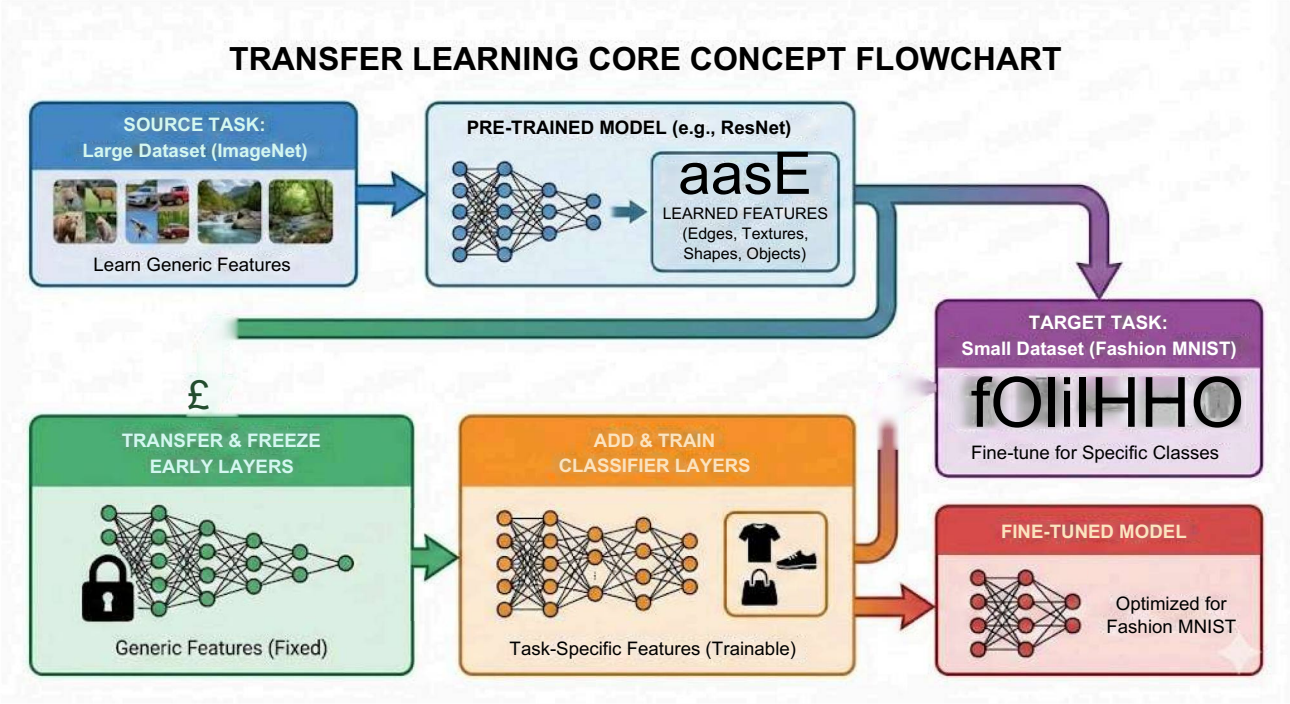
Metric	ANN (Baseline)	Basic CNN	CNN + Hyperparameter Tuning	Transfer Learning (VGG-16)
Data Requirements	Standard	Standard	Standard	Leverages pre-trained knowledge

What is Transfer Learning?

Definition

> **Transfer Learning** is a machine learning technique where a model trained on one task is reused (partially or fully) for a different but related task. Instead of training a model from scratch, which can be computationally expensive and require large datasets, transfer learning leverages knowledge from a pre-trained model to improve learning efficiency and performance.

Visual Representation



Real-World Analogy

Person A	Person B
Learned cycling as a child	Never learned cycling

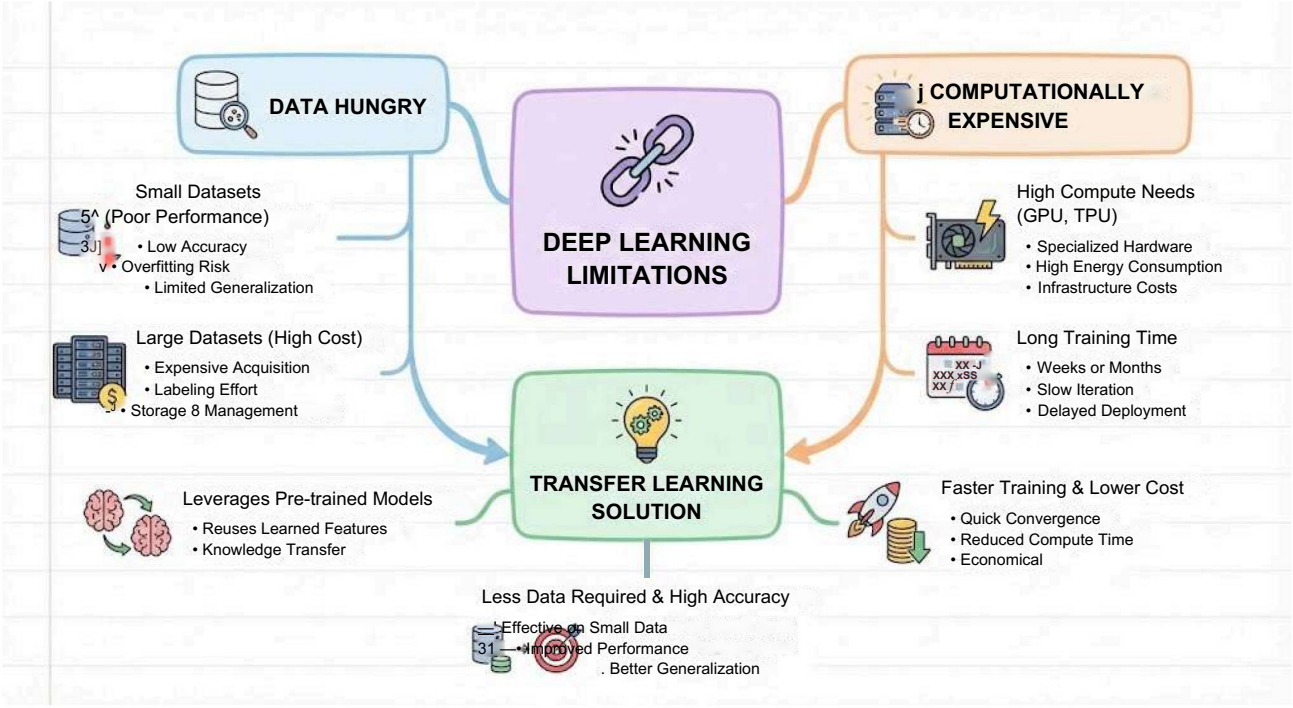
Person A	Person B
Learning motorbike -► **Faster**	Learning motorbike -► **Slower**
Prior knowledge transfers (balance, road focus, multitasking)	Must learn everything from scratch

Key Insight: Just like Person A can transfer cycling skills to learn motorbikes faster, a neural network can transfer learned features from one task to a related task.

Why Do We Need Transfer Learning?

Fundamental Problems with Deep Learning

Deep Learning, while powerful, has two significant limitations:



Comparison Table

Aspect	Training from Scratch	Transfer Learning
Data Required	Very Large (millions)	Small (thousands)
Training Time	Hours to Days	Minutes to Hours
Hardware Cost	High (expensive GPUs)	Lower
Expertise Needed	High	Medium

Aspect	Training from Scratch	Transfer Learning
Result Quality	Depends on data size	Consistently good

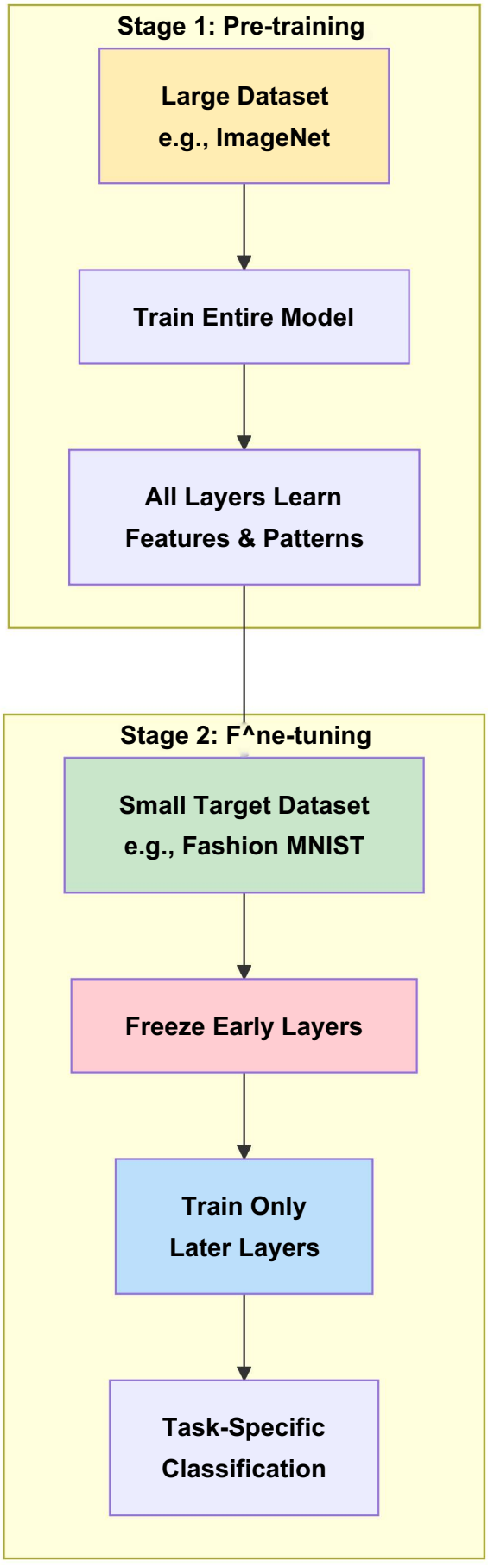
When to Use Transfer Learning

- Q Limited training data available
- Q Computational resources are constrained
- Q Need quick model development
- Q Target task is related to source task
- Q Pre-trained model exists for similar domain

How Transfer Learning Works

The Core Concept

In Transfer Learning, the learning happens in two stages:

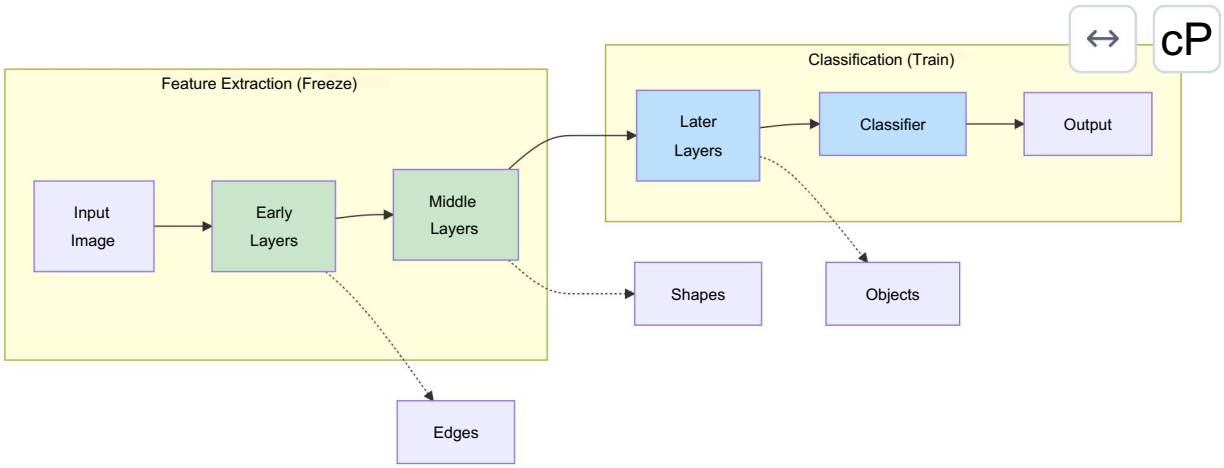

Stage 1: Pre-training
Large Dataset
e.g., ImageNet
Train Entire Model
All Layers Learn
Features & Patterns
Stage 2: F^ne-tuning
Small Target Dataset
e.g., Fashion MNIST
Freeze Early Layers
Train Only
Later Layers
Task-Specific
Classification

CampusX

Layer-wise Learning in CNNs

CNNs learn hierarchically - different layers capture different levels of abstraction:

Layer Position	What It Learns	Reusability
Early Layers (Conv 1-2)	Edges (vertical, horizontal, slanted)	**Highly Reusable** across all images
Middle Layers (Conv 3-4)	Shapes (triangles, circles, squares)	**Reusable** for similar domains
Later Layers (Conv 5+)	Objects (cats, dogs, cars)	**Task-Specific**
Classifier (FC Layers)	Classification decisions	**Must be replaced** for new task



The Two-Stage Process

Stage 1: Pre-training

Objective: Train the model on a large, general dataset to learn universal features.

[P

Pre-training Process:

```
┌─────────────────────────────────────────────────┐
│  Large Dataset (e.g., ImageNet - 1.4 Million Images)  │
│                          ↓                      │
│  Train ALL layers of the model                  │
│                          ↓                      │
│  Model learns: edges shapes -> textures -» objects  │
│                          ↓                      │
│  Save model weights (Pre-trained Model)         │
└─────────────────────────────────────────────────┘
```

Stage 2: Fine-tuning

Objective: Adapt the pre-trained model to the specific target task.

[P

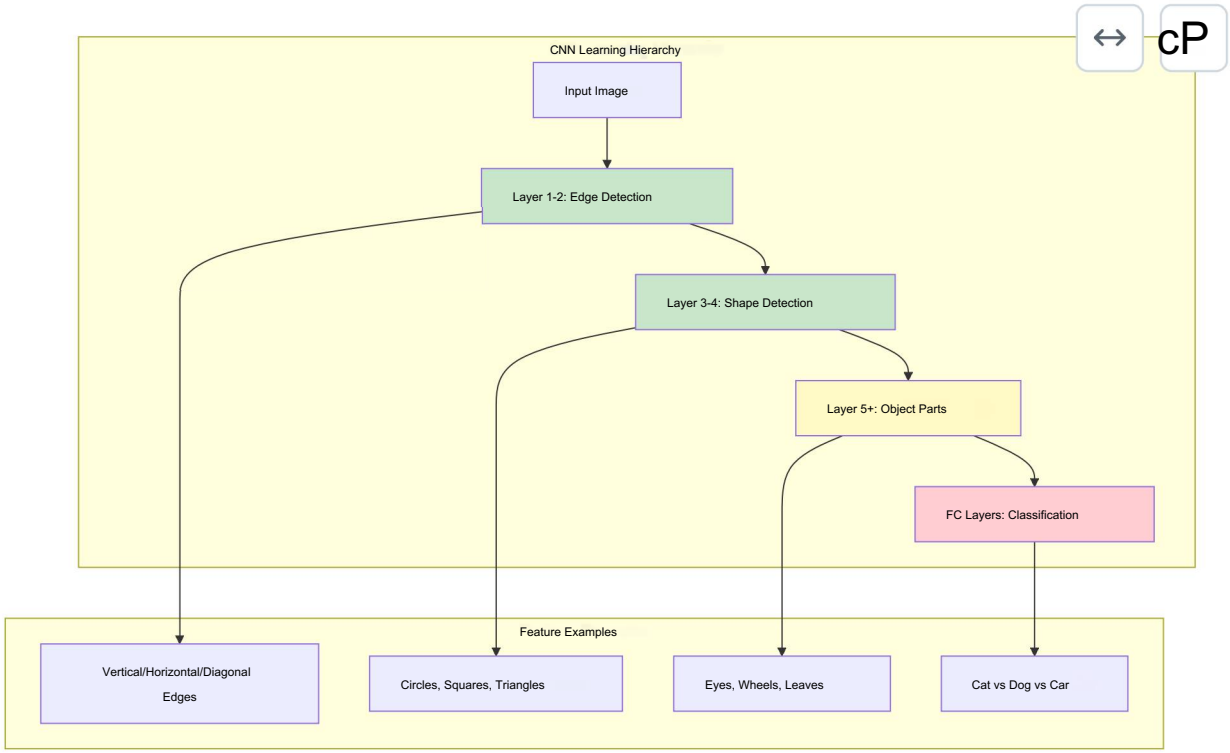
Fine-tuning Process:

```
┌─────────────────────────────────────────────────┐
│  Load Pre-trained Model                         │
│                          ↓                      │
│  Freeze early layers (feature extraction)       │
│  - No gradient updates                          │
│  - Preserve learned features                    │
│                          ↓                      │
│  Replace/Train classifier layers                │
│  - Adapt to new number of classes               │
│  - Learn task-specific patterns                 │
│                          ↓                      │
│  Train on small target dataset                  │
└─────────────────────────────────────────────────┘
```

Why Transfer Learning Works - The Theory

Hierarchical Feature Learning

The reason transfer learning works is rooted in how CNNs learn features hierarchically:



Universal Feature Reusability

Every image in the world can be decomposed into:

1. **Basic edges -** The building blocks of all visual information
2. **Simple shapes -** Combinations of edges form geometric patterns
3. **Complex objects -** Shapes combine to form recognizable objects

Since early layers learn these universal features, they can be **reused across different tasks:**

Feature Type	Example	Transfer Potential
Edges	Lines, corners, gradients	Universal (any image)

Feature Type	Example	Transfer Potential
Textures	Fur, fabric, wood grain	Domain-specific
Parts	Eyes, wheels, handles	Task-specific
Objects	Complete entities	Task-specific

Mathematical Intuition

During fine-tuning, we freeze the feature extraction layers:

For frozen layers:

$$\frac{dL}{3W_{frozen}} = 0$$

For trainable classifier layers:

$$\hat{}_{new} - \hat{}_{old} \hat{}7\ ' - \frac{dL}{{}^{0}W_{classifier}}$$

Where:

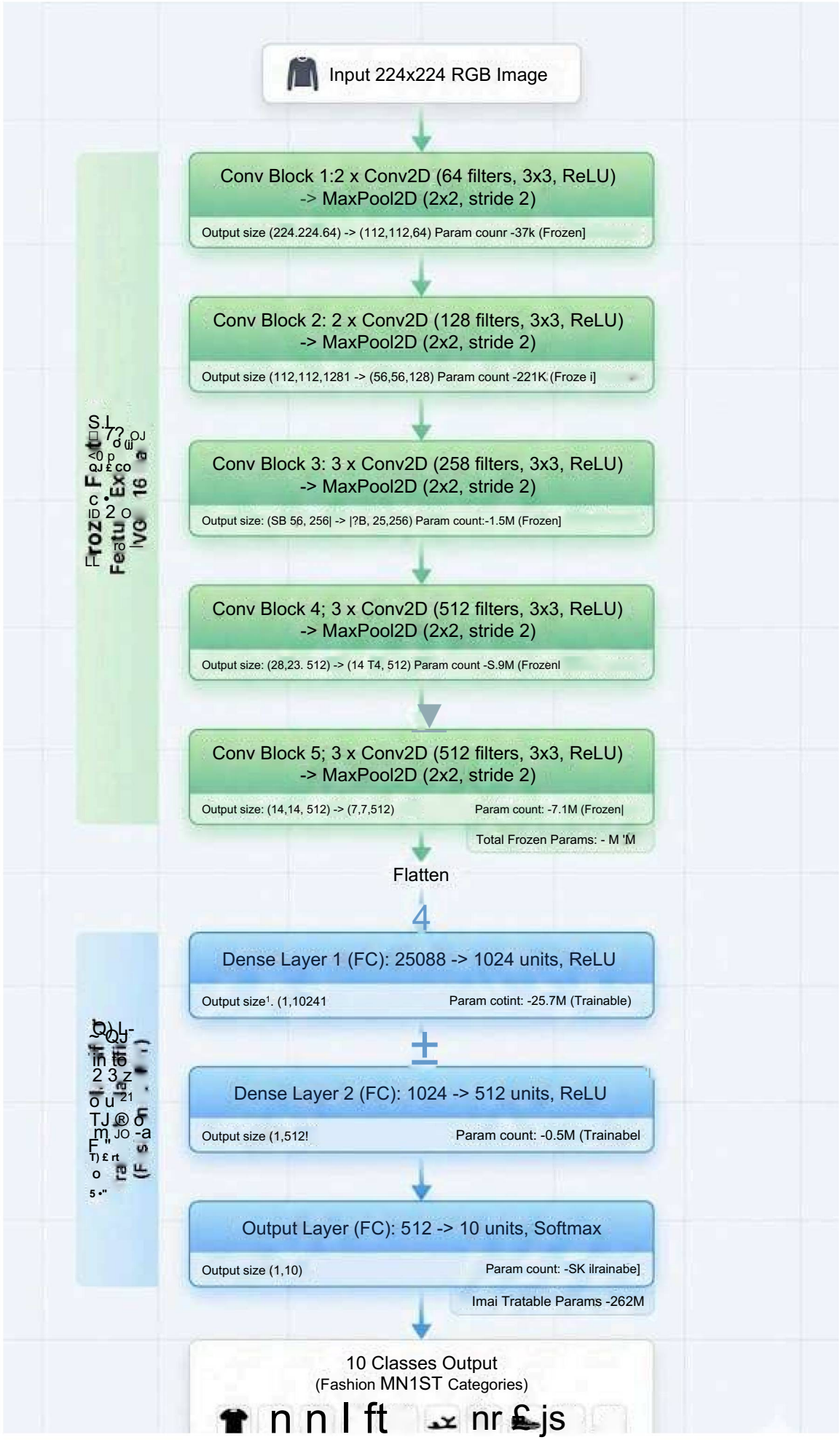
- L = Loss function
- W = Weights
- TJ = Learning rate

VGG-16 Architecture Deep Dive

Overview

VGG-16 is a deep convolutional neural network that won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2014. It's known for its simplicity and depth.

Architecture Diagram


Input 224x224 RGB Image
Conv Block 1:2 x Conv2D (64 filters, 3x3, ReLU)
-> MaxPool2D (2x2, stride 2)
Output size (224.224.64) -> (112,112,64) Param counr -37k (Frozen]
Conv Block 2: 2 x Conv2D (128 filters, 3x3, ReLU)
-> MaxPool2D (2x2, stride 2)
Output size (112,112,1281 -> (56,56,128) Param count -221K (Froze i]
Conv Block 3: 3 x Conv2D (258 filters, 3x3, ReLU)
-> MaxPool2D (2x2, stride 2)
Output size: (SB 56, 256| -> |?B, 25,256) Param count:-1.5M (Frozen]
Conv Block 4; 3 x Conv2D (512 filters, 3x3, ReLU)
-> MaxPool2D (2x2, stride 2)
Output size: (28,23. 512) -> (14 T4, 512) Param count -S.9M (Frozenl
Conv Block 5; 3 x Conv2D (512 filters, 3x3, ReLU)
-> MaxPool2D (2x2, stride 2)
Output size: (14,14, 512) -> (7,7,512)
Param count: -7.1M (Frozen|
Total Frozen Params: - M 'M
Flatten
4
Dense Layer 1 (FC): 25088 -> 1024 units, ReLU
Output size1. (1,10241
Param cotint: -25.7M (Trainable)
Dense Layer 2 (FC): 1024 -> 512 units, ReLU
Output size (1,512!
Param count: -0.5M (Trainabel
Output Layer (FC): 512 -> 10 units, Softmax
Output size (1,10)
Param count: -SK ilrainabe]
Imai Tratable Params -262M
10 Classes Output
(Fashion MN1ST Categories)

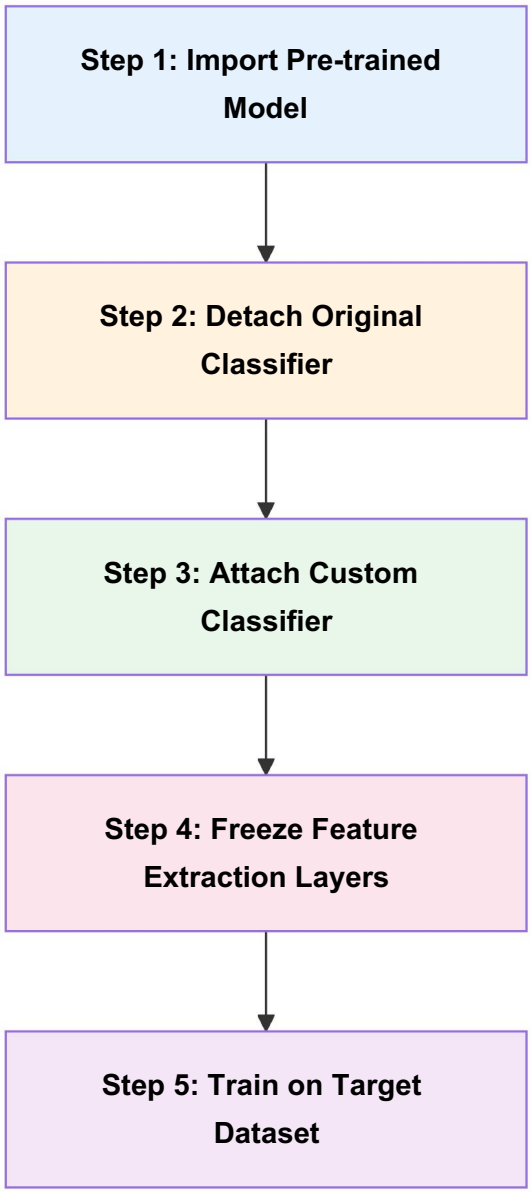
CampusX

Layer-by-Layer Breakdown

Block	Layers	Output Size	Parameters
Input	-	224 x 224 x 3	0
Block 1	2 x Conv3-64, MaxPool	112 x 112 x 64	38,720
Block 2	2 x Conv3-128, MaxPool	56 x 56 x 128	221,440
Block 3	3 x Conv3-256, MaxPool	28 x 28 x 256	1,475,328
Block 4	3 x Conv3-512, MaxPool	14 x 14 x 512	5,899,776
Block 5	3 x Conv3-512, MaxPool	7 x 7 x 512	7,079,424
Flatten	-	25,088	0
FC1	Dense + ReLU	4,096	102,764,544
FC2	Dense + ReLU	4,096	16,781,312
FC3 (Output)	Dense + Softmax	1,000	4,097,000
Total	-	-	**~ 138 Million**

Implementation Workflow

5-Step Transfer Learning Process



Detailed Steps

Step	Action	Code Concept
1	Import VGG-16 from torchvision	models.vggl6(pretrained=True)
2	Remove original classifier	Access vggl6.classifier
3	Add custom classifier for 10 classes	Create new nn.SequentialO
4	Freeze feature extraction layers	Set requires_grad = False
5	Train model on Fashion MNIST	Standard training loop

Why Replace the Classifier?

Original VGG-16	Our Modified VGG-16
Trained on ImageNet	Fine-tuned on Fashion MNIST
1,000 output classes	10 output classes
FC: 25088 -> 4096 4096 → 1000	FC: 25088 → 1024 → 512 → 10

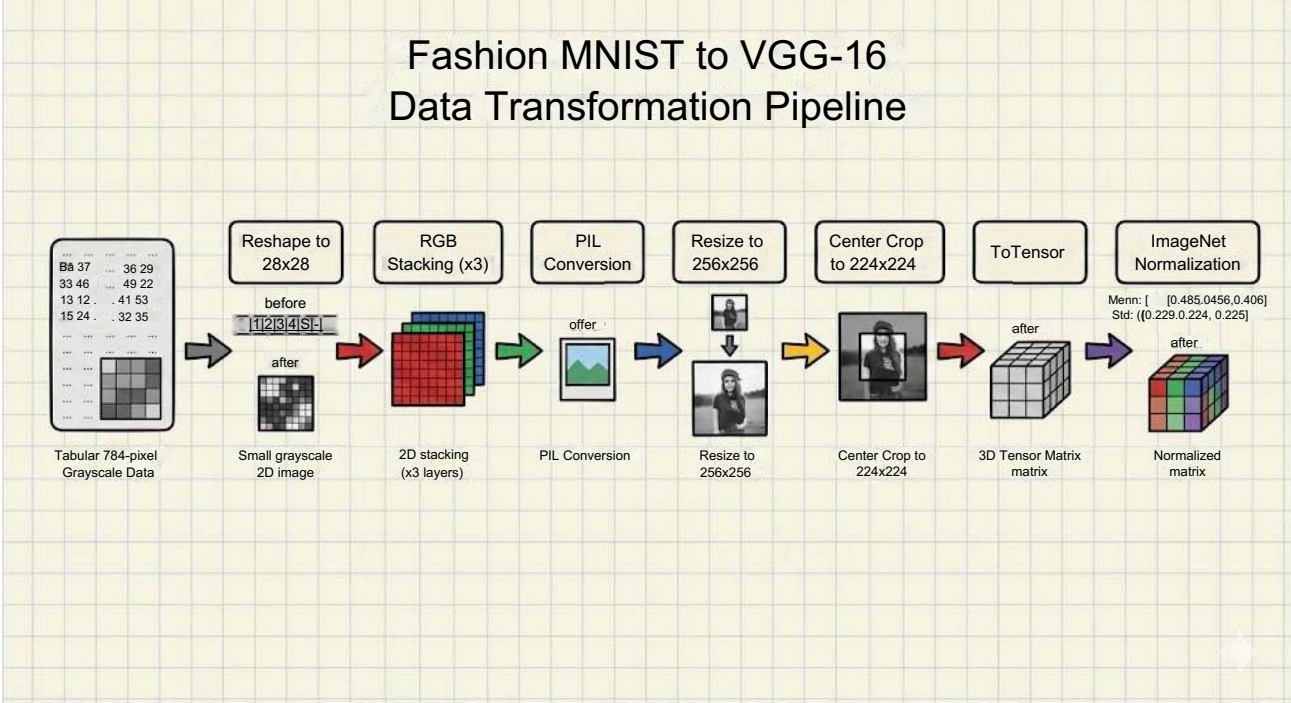
Data Transformation Pipeline

The Challenge

VGG-16 was trained on ImageNet (colored images, 224x224), but Fashion MNIST has:

- **Grayscale images** (single channel)
- **28x28 resolution**
- **Tabular format** (784 features per image)

Required Transformations



Transformation Details

Step	Operation	Input Shape	Output Shape	Purpose
1	Reshape	(784,)	(28, 28)	Convert tabular to 2D image
2	dtype conversion	np.float64	np.uint8	Required for PIL conversion
3	Channel stacking	(28, 28)	(28, 28, 3)	Grayscale -» RGB-like
4	PIL conversion	numpy array	PIL Image	Required for torchvision transforms
5	Resize	(28, 28, 3)	(256, 256, 3)	Match VGG-16 input requirements
6	Center Crop	(256, 256, 3)	(224, 224, 3)	Standard VGG-16 input size
7	ToTensor	PIL Image	(3, 224, 224) tensor	PyTorch format + scale to [0,1]
8	Normalize	[0, 1]	Standardized	ImageNet statistics

ImageNet Normalization Statistics

VGG-16 expects inputs normalized with ImageNet statistics:

$$^{normalized} = \frac{x - \mu}{o'}$$

Channel	Mean (^)	Std (a)
Red	0.485	0.229
Green	0.456	0.224
Blue	0.406	0.225

The normalization is applied per-channel:

$$\text{Rnorm} = \frac{R - 0.485}{0.229}$$

$$G_{norm} = \frac{G - 0.456}{0.224}$$

$$B_{norm} = \frac{B - 0.406}{0.225}$$

Complete Code Implementation

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
from PIL import Image
from sklearn.model_selection import train_test_split

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

import torchvision.models as models
from torchvision.transforms import transforms

import matplotlib.pyplot as plt
```

Step 2: Setup Device and Reproducibility

```
# Set random seeds for reproducibility
torch.manual_seed(42)
np.random.seed(42)

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

Step 3: Load and Explore Data

```
# Load Fashion MNIST dataset
df = pd.read_csv('/content/fmnist_small.csv')
print(f"Dataset shape: {df.shape}")

# Visualize sample images
fig, axes = plt.subplots(4, 4, figsize=(10, 10))
```

```
fig.suptitle("First 16 Images", fontsize=16)

for i, ax in enumerate(axes.flat):
    img = df.iloc[i, 1:].values.reshape(28, 28)
    ax.imshow(img, cmap='gray')
    ax.axis('off')
    ax.set_title(f"Label: {df.iloc[i, 0]}")

pit.tight_layout()
pit.show()
```

Step 4: Prepare Train/Test Split

```
#  Separate features and labels
X = df.iloc[:, 1:].values # All pixel values
y = df.iloc[:, 0].values # Labels (0-9)

#  Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Training samples: {X_train.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")
```

Step 5: Define Custom Transforms

```
#  Define transformations for VGG-16 compatibility
custom_transform = transforms.Compose([
    # Resize to 256x256 (bilinear interpolation by default)
    transforms.Resize(256),

    # Center crop to 224x224
    transforms.CenterCrop(224),

    #  Convert PIL Image to Tensor and scale to [0, 1]
    transforms.ToTensor(),

    #  Normalize with ImageNet statistics
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        Std=[0.229, 0.224, 0.225]
    )
])
```

Step 6: Create Custom Dataset Class

tP

```
class CustomDataset(Dataset):
    """
    Custom Dataset for Fashion MNIST with VGG-16 compatibility.

    Transforms tabular data (784 features) to 224x224x3 RGB-like images,
    fl II II

    def___ init____(self, features, labels, transform):
        """
        Args:
            features: Numpy array of shape (n_samples, 784)
            labels: Numpy array of shape (n_samples,)
            transform: torchvision transforms to apply
        """
        self.features = features
        self.labels = labels
        self.transform = transform

    def ___len__  (self):
        return len(self.features)

    def___ getitem___ (self, index):
        # Step 1: Reshape from (784,) to (28, 28)
        image = self.features[index].reshape(28, 28)

        # Step 2: Convert dtype to np.uint8 (required for PIL)
        image = image.astype(np.uint8)

        # Step 3: Convert grayscale to 3-channel (H, W) -> (H, W, C)
        # Stack the same image 3 times to simulate RGB
        image = np.stack([image] * 3, axis=-l)

        # Step 4: Convert numpy array to PIL Image
        image = Image.fromarray(image)

        #  Step 5: Apply all transforms (resize, crop, tensor, normalize)
        image = self.transform(image)

        #  Return image tensor and label tensor
        return image, torch.tensor(self.labels[index], dtype=torch.long)
```

Step 7: Create Data Loaders

```
# Create Dataset objects
train_dataset = CustomDataset(X_train, y_train, transform=custom_transform)
test_dataset = CustomDataset(X_test, y_test, transform=custom_transform)

# Create DataLoader objects
BATCH_SIZE = 32

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    pin_memory=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    pin_memory=True
)

print(f"Number of training batches: {len(train_loader)}")
print(f"Number of test batches: {len(test_loader)}")
```

Step 8: Load and Modify VGG-16

```
# Step 1: Import pre-trained VGG-16
vggl6 = models.vggl6(pretrained=True)

# View original architecture
print("Original VGG-16 Classifier:")
print(vggl6.classifier)
```

Original VGG-16 Classifier Output:

```
Sequential(
  (0): Linear(in_features=25088, out_features=4096)
  (1): ReLU(inplace=True)
  (2): Dropout(p=0.5)
  (3): Linear(in_features=4096, out_features=4096)
  (4): ReLU(inplace=True)
  (5): Dropout(p=0.5)
  (6): Linear(in_features=4096j out_features=1000)
)
```

Step 9: Freeze Feature Extraction Layers

```
# Step 4: Freeze all parameters in feature extraction layers
for param in vggl6.features.parameters():
    param.requires_grad = False

# Verify freezing
total_params = sum(p.numel() for p in vggl6.features.parameters())
trainable_params = sum(p.numel() for p in vggl6.features.parameters() if p.req
print(f"Feature extraction - Total: {total_params:,} Trainable: {trainable_pa
```

Step 10: Replace Classifier

```
# Step 3: Replace classifier with custom one for 10 classes
vggl6.classifier = nn.Sequential(
    # Hidden Layer 1
    nn.Linear(in_features=25088, out_features=1024),
    nn.ReLU(),
    nn.Dropout(0.5),

    # Hidden Layer 2
    nn.Linear(in_features=1024,          out_features=512),
    nn.ReLU(),
    nn.Dropout(0.5),

    # Output Layer (10 classes for Fashion MNIST)
    nn.Linear(in_features=512, out_features=10)
)

print("Modified VGG-16 Classifier:")
print(vggl6.classifier)

# Move model to GPU
vggl6 = vggl6.to(device)
```

Step 11: Define Loss and Optimizer

```
# Hyperparameters
learning_rate = 0.0001 # Lower learning rate for fine-tuning
epochs = 10

# Loss function
criterion = nn.CrossEntropyLoss()
```

```
#  Optimizer - ONLY update classifier parameters
optimizer = optim.Adam(
        vggl6.classifier.parameters()j # Only classifier weights
        lr=learning_rate
)
```

Step 12: Training Loop

```
#  Training loop
train_losses = []

for epoch in range(epochs):
        vggl6.train() # Set model to training mode
        total_epoch_loss = 0

        for batch_idx, (batch_features, batch_labels) in enumerate(train_loader):
                # Move data to GPU
                batch_features = batch_features.to(device)
                batch_labels = batch_labels.to(device)

                # Forward pass
                outputs = vgg!6(batch_features)

                # Calculate loss
                loss = criterion(outputs, batch_labels)

                # Backward pass
                optimizer.zero_grad() # Clear previous gradients
                loss.backward()                 # Compute gradients

                # Update weights
                optimizer.step()

                total_epoch_loss += loss.item()

                # Print progress every 50 batches
                if (batch_idx + 1) % 50 == 0:
                        print(f"Epoch [{epoch+l}/{epochs}], "
                                F'Batch [{batch_idx+l}/{len(train_loader)}]₂ "
                                f'Loss: {loss.item():.4f}")

        # Calculate average loss for epoch
        avg_loss = total_epoch_loss / len(train_loader)
        train_losses.append(avg_loss)

        print(f"Epoch [{epoch+l}/{epochs}] completed. Average Loss: {avg_loss:.4f}
        print("-" * 60)
```

```
#  Plot training loss
plt.figure(figsize=(10j 5))
plt.plot(range(l, epochs + 1), train_losses, marker='o')
plt.xlabel('Epoch')
plt.ylabel('Loss')
pit.title('Training Loss Over Epochs')
plt.grid(True)
plt.show()
```

Step 13: Evaluation

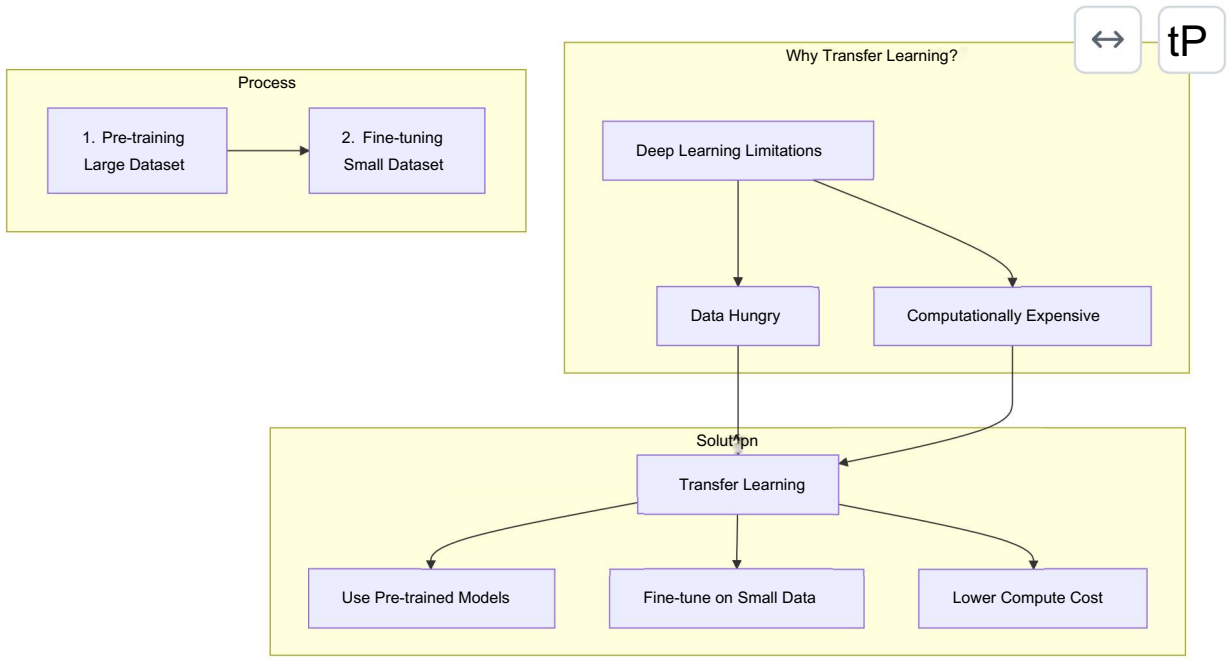
```
def evaluate_model(model, data_loader, device):
    """
    Evaluate model accuracy on given data loader.

    Args:
        model: PyTorch model
        data_loader: DataLoader object
        device: 'cuda' or 'cpu'

    Returns:
        accuracy: float between 0 and 1
    """
    model.eval() # Set to evaluation mode

    total = 0
    correct = 0

    with torch.no grad(): # Disable gradient computation
        for batch_features, batch_labels in data_loader:
            # Move data to device
            batch_features = batch_features.to(device)
            batch_labels = batch_labels.to(device)

            # Forward pass
            outputs = model(batch_features)

            #  Get predictions (index of max logit)
            _,  predicted = torch.max(outputs, 1)

            #  Update counts
            total += batch_labels.size(0)
            correct += (predicted == batch_labels).sum().item()

    accuracy = correct / total
```

tP

```
    return accuracy

# Evaluate on test data
test_accuracy = evaluate_model(vggl6j test_loaderj device)
print(f"Test Accuracy: {test_accuracy * 100:.2f}%")

# Evaluate on training data
train_accuracy = evaluate_model(vggl6, train_loader, device)
print(f"Training Accuracy: {train_accuracy * 100:.2f}%")
```

Key Concepts Summary

Transfer Learning Overview



VGG-16 Modification Summary

Component	Original	Modified
Features (Conv Blocks)	13 Conv layers + 5 MaxPool	Same (Frozen)
Classifier Input	25,088	25,088
Hidden Layer 1	4,096 neurons	1,024 neurons
Hidden Layer 2	4,096 neurons	512 neurons
Output Layer	1,000 classes	10 classes
Trainable Params	~138M	~26M

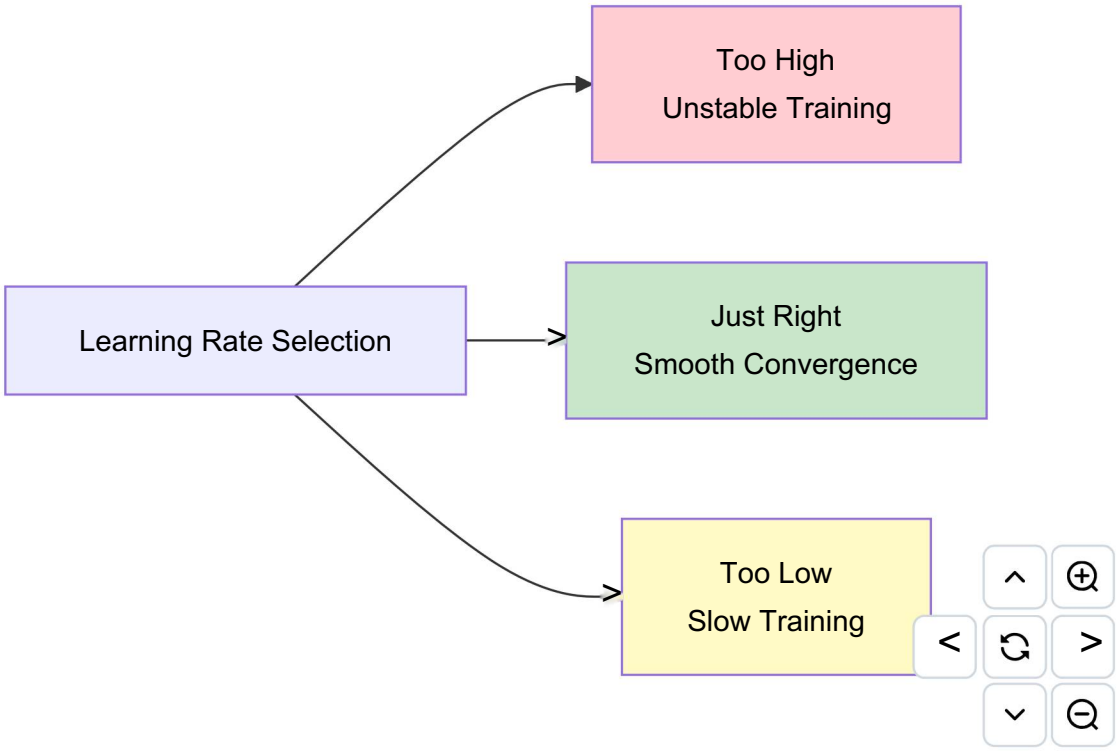
Data Transformation Pipeline Summary

tP

Fashion MNIST (28x28, Grayscale, Tabular)
1
[Reshape + dtype + RGB conversion]
1
PIL Image (28x28x3)
J.
[Resize -> 256x256]
J.
[Center Crop -» 224x224]
↓
[ToTensor -» Scale to [0,1]]
↓
[Normalize with ImageNet stats]
↓
VGG-16 Compatible Input (3x224x224)

Best Practices and Tips

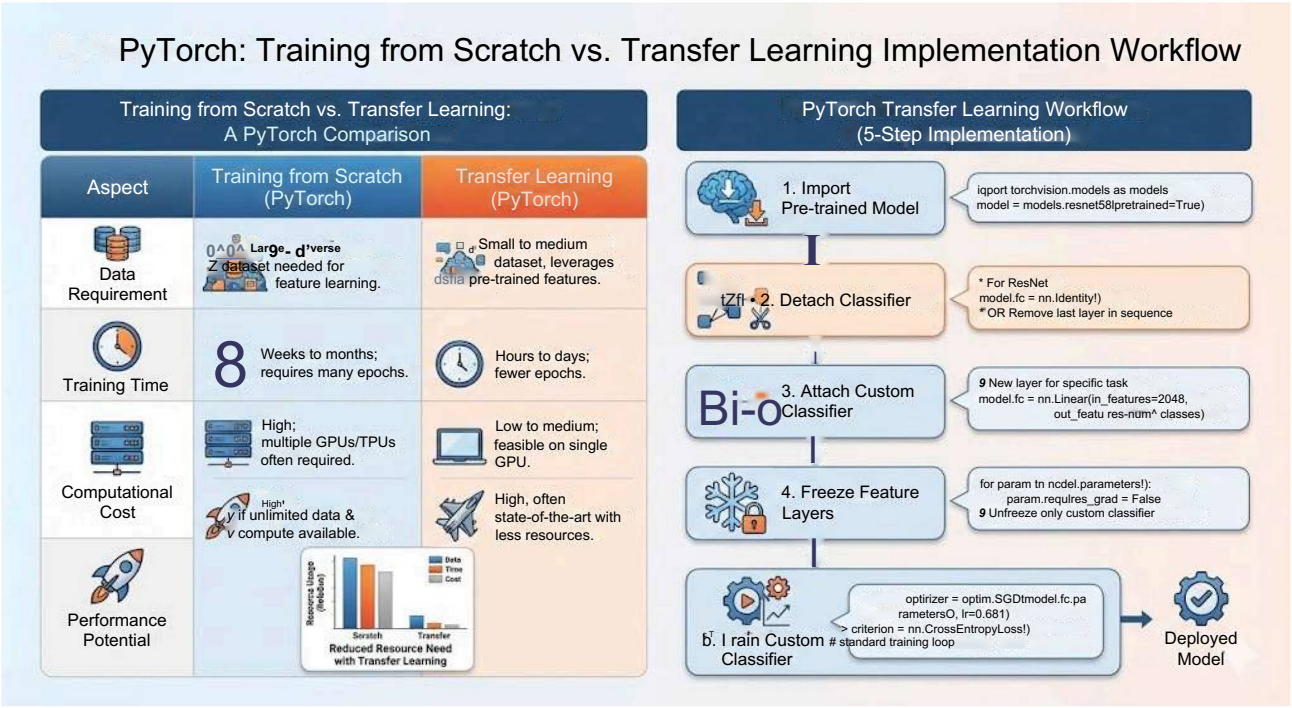
Learning Rate Selection



Scenario	Recommended LR	Reason
Training from scratch	0.01 - 0.001	Need large updates
Fine-tuning classifier	0.001 - 0.0001	Moderate updates
Fine-tuning all layers	0.0001 - 0.00001	Preserve pre-trained knowledge

What to Freeze vs Train

Strategy	Freeze	Train	When to Use
Feature Extraction	All conv layers	Only classifier	Small target dataset
Partial Fine-tuning	Early conv layers	Later conv + classifier	Medium target dataset
Full Fine-tuning	Nothing	Everything	Large target dataset



Common Pitfalls

Pitfail	Solution
Forgetting to freeze layers	Always verify requires_grad status
Wrong input size	Check model documentation for expected input
Missing normalization	Use same normalization as pre-training
High learning rate	Start low, increase if needed
Not using GPU	Always check device and move data appropriately

Code Quality Tips

cP

```
#Q Good: Only update classifier parameters
optimizer = optim.Adam(vggl6.classifier.parameters!), lr=0.0001)

# X Bad: Updates ALL parameters (defeats freezing purpose)
optimizer = optim.Adam(vggl6.parameters()_, lr=0.0001)

#Q Good: Verify freezing worked
for name, param in vgg!6.named_parameters():
    print(f"{name}: requiresgrad = {param.requiresgrad}")
```

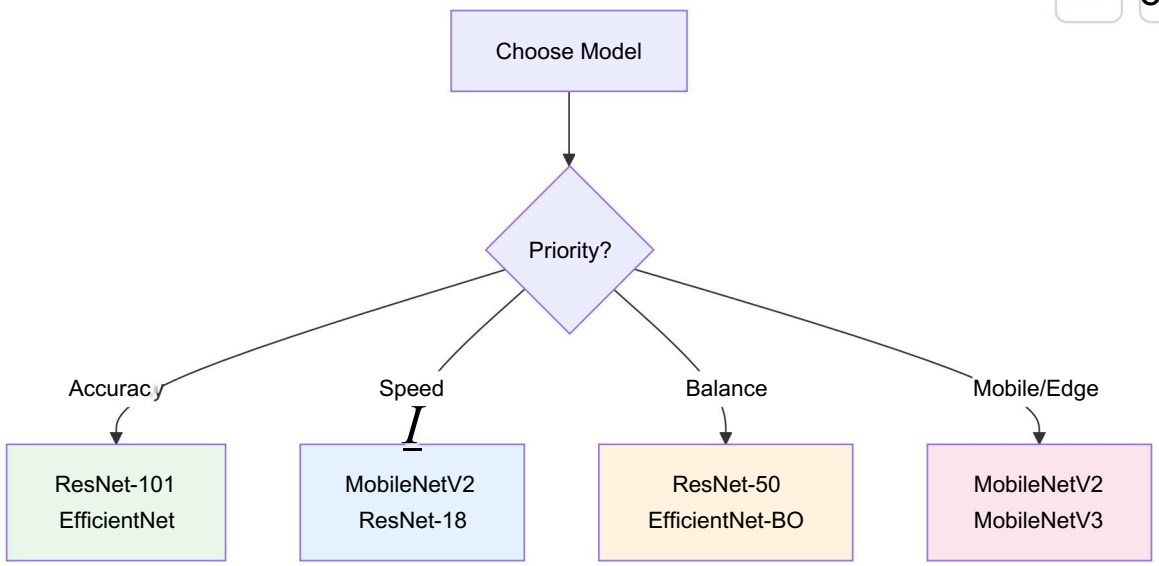
Alternative Pre-trained Models

PyTorch's torchvision.models provides many pre-trained options:

Classification Models Comparison

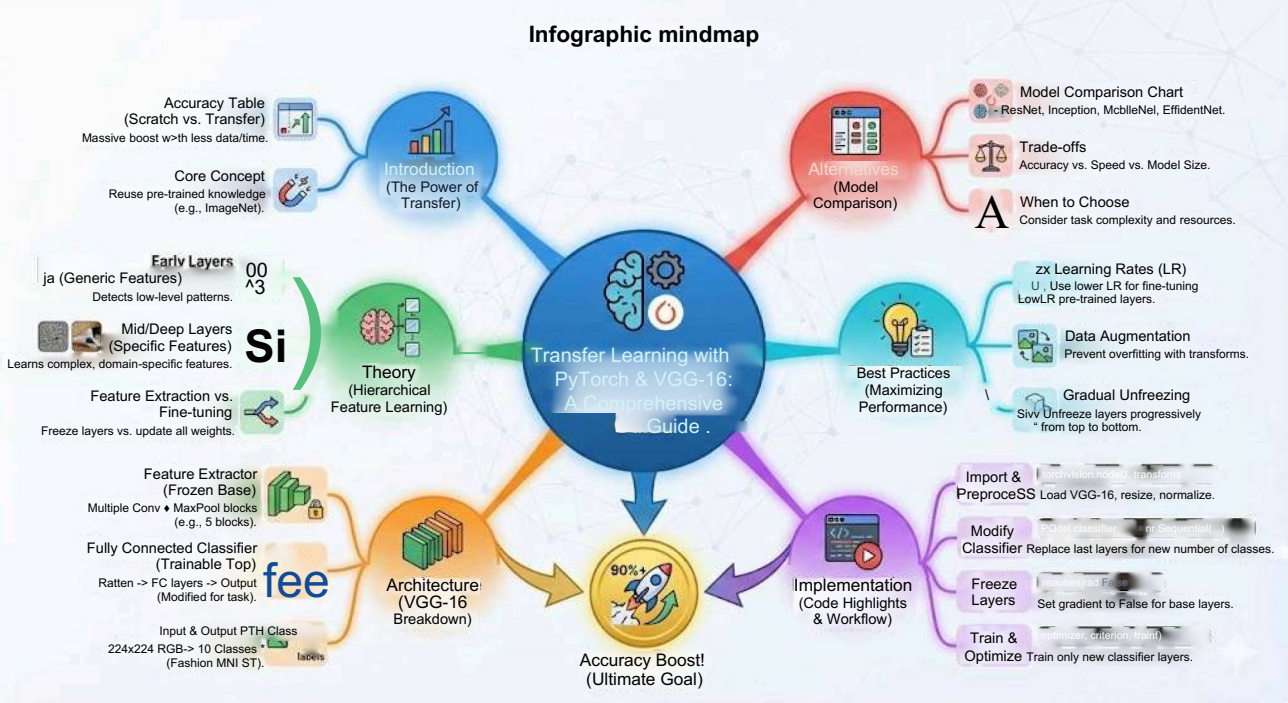
Model	Parameters	Top-1 Accuracy	Speed	Memory
VGG-16	138M	71.6%	Slow	High
VGG-19	144M	72.4%	Slow	High
ResNet-18	11.7M	69.8%	Fast	Low
ResNet-50	25.6M	76.1%	Medium	Medium
ResNet-101	44.5M	77.4%	Slow	High
MobileNetV2	3.4M	72.0%	Very Fast	Very Low
EfficientNet-BO	5.3M	77.1%	Fast	Low
Inception-v3	23.8M	77.3%	Medium	Medium

When to Use Which Model



Example: Using ResNet-50

```
import torchvision.models as models

# Load pre-trained ResNet-50
resnet50 = models.resnet50(pretrained=True)

# Freeze feature extraction
for param in resnet50.parameters():
    param.requires_grad = False

#  Replace final fully connected layer
#  ResNet-50's fc layer has 2048 input features
resnet50.fc = nn.Sequential(
    nn.Linear(2048, 512),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(512, 10)
)

#  Move to device
resnet50 = resnet50.to(device)
```



Resources and References

- **PyTorch Documentation:** https://pytorch.org/docs/stable/index.html
- **TorchVision Models:** https://pytorch.org/vision/stable/models.html
- **VGG Paper:** "Very Deep Convolutional Networks for Large-Scale Image Recognition"
- **ImageNet:** https://www.image-net.org/
- **Colab Notebook:** https://colab.research.google.com/drive/196F8ZuAdbeS9_96v3dLpf9oCpMLEuRwB

Transfer Learning

PyTorch • VGG-16 • Fashion MNIST

Accuracy ~94%+

WHAT IS TRANSFER LEARNING?

Definition

A model trained on one task is reused for a different but related task, leveraging pre-learned knowledge instead of training from scratch.

Real-World Analogy:

- Person A: Knows Cycling — Learning motorbike -»FASTER — Prior knowledge transfers!
- vs
- Person B: No Cycling X — Learning motorbike -> SLOWER — Must learn from scratch

WHY TRANSFER LEARNING?

Deep Learning Problems:

- Data Hungry — Needs millions of images — Data collection expensive
- Computationally Expensive — Hours to days of training — Expensive GPU hardware

Transfer Learning Solution

- Small data OK (thousands)
- Minutes to hours (fast training)
- Lower compute cost (affordable)

TWO-STAGE TRANSFER LEARNING PROCESS

STAGE 1: PRE-TRAINING

ImageNet (1.4M images) → Train ALL layers → Learn Features (edges-.shapes-.obj) → Save Weights

Done by researchers (e.g., VGG team at Oxford)

STAGE 2: FINE-TUNING

Load Pre-trained → Freeze Conv layers → Replace Classifier → Train Smal data → ✓

Done by YOU on your target dataset (Fashion MNIST)

WHY IT WORKS: HIERARCHICAL LEARNING

EARLY LAYERS	MIDDLE LAYERS	LATER + FC
Conv 1-2	Conv 3-4	Conv 5 + Classifier
Edges, Lines	Shapes, Textures	Objects, Classes
$ FREEZE	FREEZE	* TRAIN

Feature Reusability:

Highly Reusable	Domain Specific	Task Specific
Universal (any image)		Must replace for new task

Early layers learn universal features (edges, shapes) that work for ALL images!

VGG-16 ARCHITECTURE

FROZEN FEATURE EXTRACTION — TRAINABLE

INPUT 224x224 ×3 RGB → Block 1 (2×Conv 64 Pool) → Block 2 (2×Conv 128 Pool) → Block 3 (3×Conv 256 Pool) → Block 4 (3×Conv 512 Pool) → Block 5 (3×Conv 512 Pool) → CLASSIFIER 25088-> 1024 1024 — 512 512 → 10 (Fashion MNIST)

VGG-16 Stats:

- Total Parameters: -138 Million
- Feature Extraction: -14.7M (frozen)
- Classifier (ours): -26M (trainable)

Modification:

Original: 1000 classes (ImageNet)
Modified: 10 classes (Fashion MNIST)
FC: 25088-*1024—>512—>10

DATA TRANSFORMATION PIPELINE (Fashion MNIST → VGG-16 Compatible)

Original (784,) Tabular → Reshape (28, 28) 2D Image → Stack x3 (28, 28, 3) RGB-like → Resize (256, 256) Upsample → CenterCrop (224, 224) VGG size → ToTensor (3, 224, 224) [0,1] scale → Normalize ImageNet stats p=[.485, .456, .406] → VGG-16 Ready! (3, 224, 224) Normalized tensor

ImageNet Normalization: $x_norm = (x-n)/a$ where p = [0.485, 0.456, 0.406] and a = [0.229, 0.224, 0.225] (per RGB channel)

5-STEP IMPLEMENTATION PROCESS

1. Import Pre-trained Model
   models.vgg16(pretrained=True)
2. Freeze Feature Layers
   for p in vgg16.features.parameters(): p.requires_grad=False
3. Replace Classifier
   vgg16.classifier = nn.Sequential(Linear(25088,1024),...)
4. Setup Training (Classifier Only!)
   optim.Adam(vgg16.classifier.parameters(), lr-0.9001)
5. Train on Target Dataset
   Standard training loop with Fashion MNIST

RESULTS & BEST PRACTICES

Accuracy Progression:

ANN Baseline	Basic CNN	CNN + Optuna	Transfer VGG-16
~9O%	92.6%	93-1%	~94%+

Best Practices:

- Y Lower learning rate (0.0001)
- Only train classifier params
- Use ImageNet normalization
- X Don't use high learning rate
- X Don't train all parameters
- X Don't skip normalization

KEY TAKEAWAYS & ALTERNATIVE PRE-TRAINED MODELS

1. Pre-train on large, fine-tune on small
2. Freeze feature layers, train classifier
3. Early layers = universal features
4. Transform data to match pre-training
- Use lower LR

Alternative Pre-trained Models (torchvision.models):

VGG-16/19	ResNet-50/101	MobileNetV2	EfficientNet-BO	Inception-v3
138M params • 71.6%	25-44M • 76-77%	3.4M • Fast • 72%	5.3M • 77.1%	23.8M • 77.3%

Choose based on: Accuracy vs Speed vs Memory
Mobile/Edge -> MobileNet | Best accuracy -> EfficientNet/Res

CampusX • PyTorch Series • Transfer Learning with VGG-16

Fashion MNIST • Pre-train —• Fine-tune • Freeze Features —> Train Classifier • ~94%+ Accuracy

CampusX

J

Building a Question-Answering System using RNN in PyTorch

Code

- htt ps://colab.research.g oog le.com/drive/14VN8_INhmfE8fclKx6obLF35rt8 px7JPS

1. Introduction and Overview

Project Goal

Build a **Simple Question-Answering System** using Recurrent Neural Networks (RNN) in PyTorch that can:

- Answer questions based on a training dataset of -90-100 Q&A pairs
- Return "I don't know" for questions outside the training data
- Handle slight variations in question phrasing

Application Demo

Input: "What is the capital of France?"
Output: "Paris"

Input: "What is the capital city of the European country France?"
Output: "Paris" (handles variations!)

Input: "What is CampusX?"
Output: "I don't know" (question not in training data)

Prerequisites

Before diving into this tutorial, ensure familiarity with:

- Basic neural network concepts
- RNN architecture fundamentals
- PyTorch basics (tensors, nn.Module, training loops)
- Python programming

2. What is RNN? - Theoretical Foundation

Definition

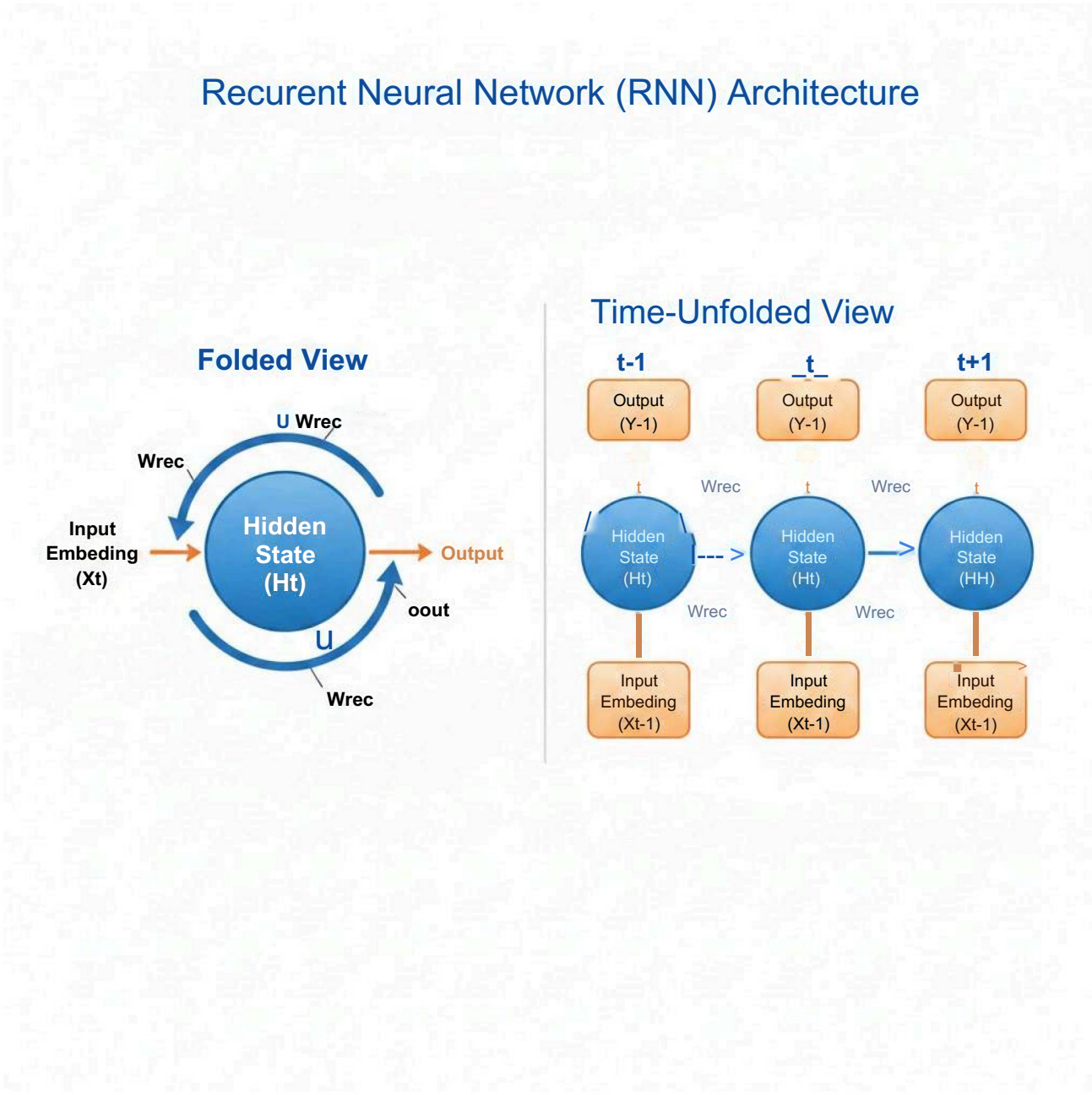
> **A Recurrent Neural Network (RNN)** is a type of neural network designed for processing **sequential data.** Unlike traditional feedforward networks which process inputs independently, RNNs maintain a **memory of previous inputs** by using **loops** in their architecture. This makes them well-suited for tasks where **context and order matter,** such as time series forecasting, speech recognition, and text generation.

Key Characteristics

Feature	Description
Sequential Processing	Processes data one element at a time in sequence
Memory Mechanism	Maintains hidden state carrying information from previous steps
Parameter Sharing	Same weights used across all time steps
Variable Length Input	Can handle sequences of different lengths

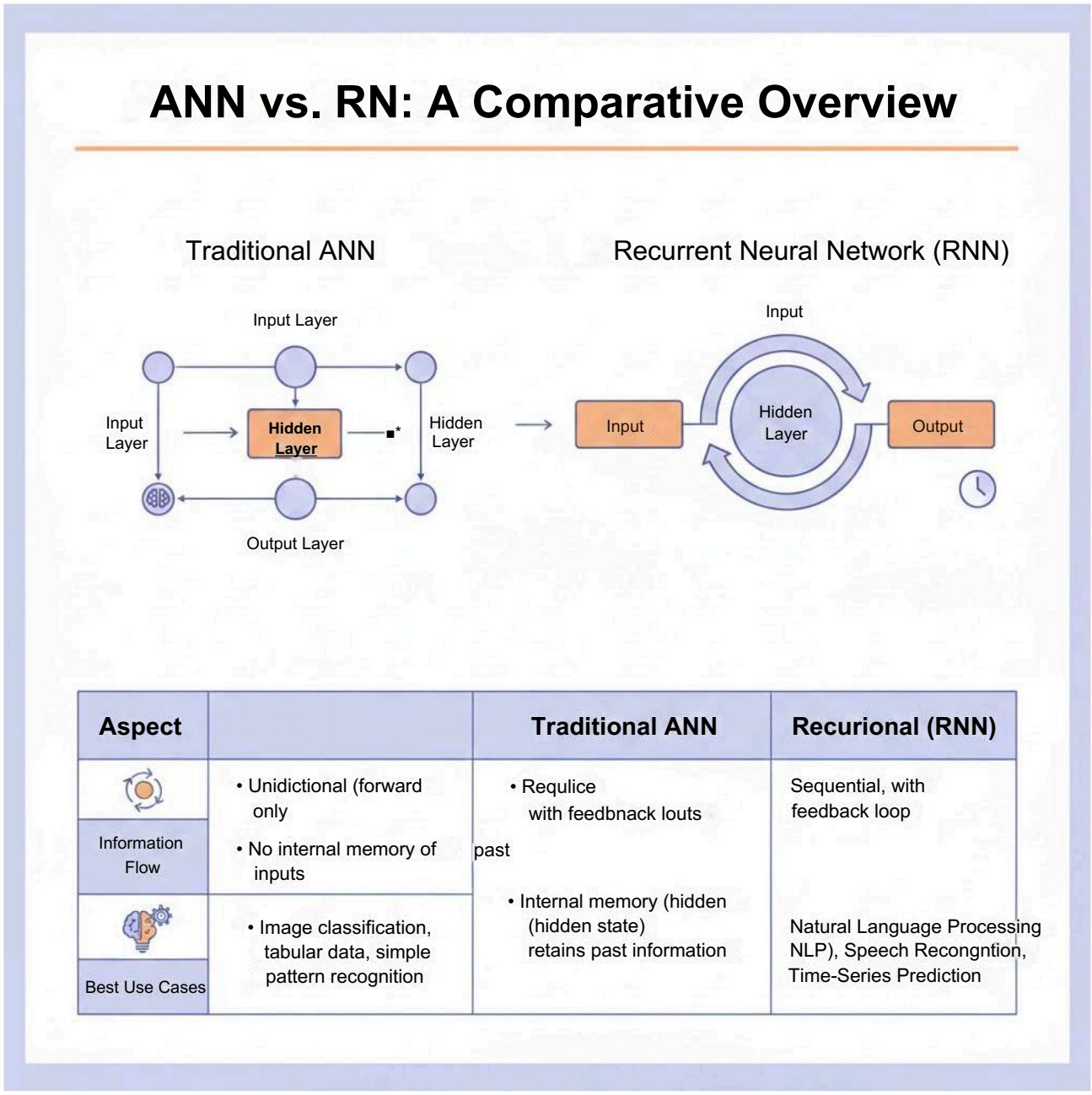
Feature	Description
Context Awareness	Understands order and relationships in data

Visual Representation



3. RNN vs Traditional Neural Networks

Architectural Comparison

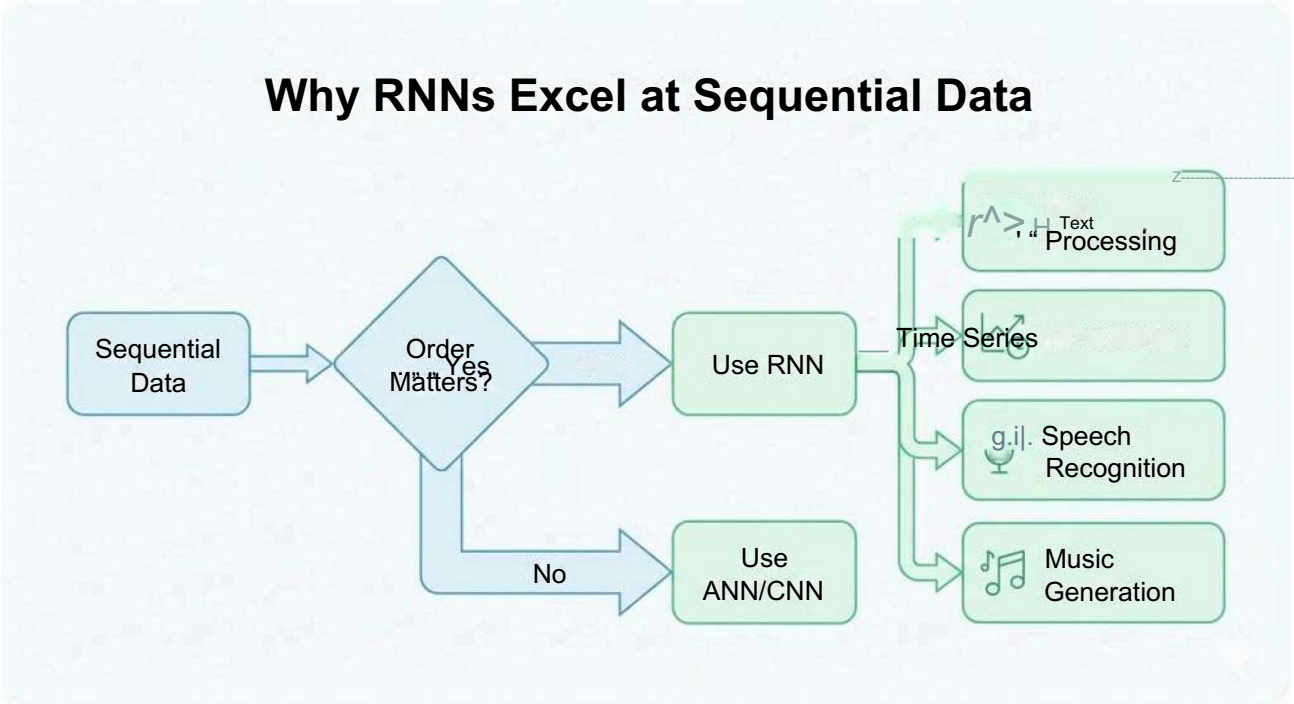


Detailed Comparison Table

Aspect	Traditional ANN	RNN
Information Flow	Unidirectional (forward only)	Loops allow bidirectional flow
Memory	No memory of previous inputs	Maintains memory through hidden states
Input Processing	Processes all inputs at once	Processes inputs sequentially

Aspect	Traditional ANN	RNN
Parameter Count	Separate weights for each layer	Shared weights across time steps
Best For	Tabular data, images	Sequential data (text, time series, speech)
Context Understanding	No inherent context	Understands order and context

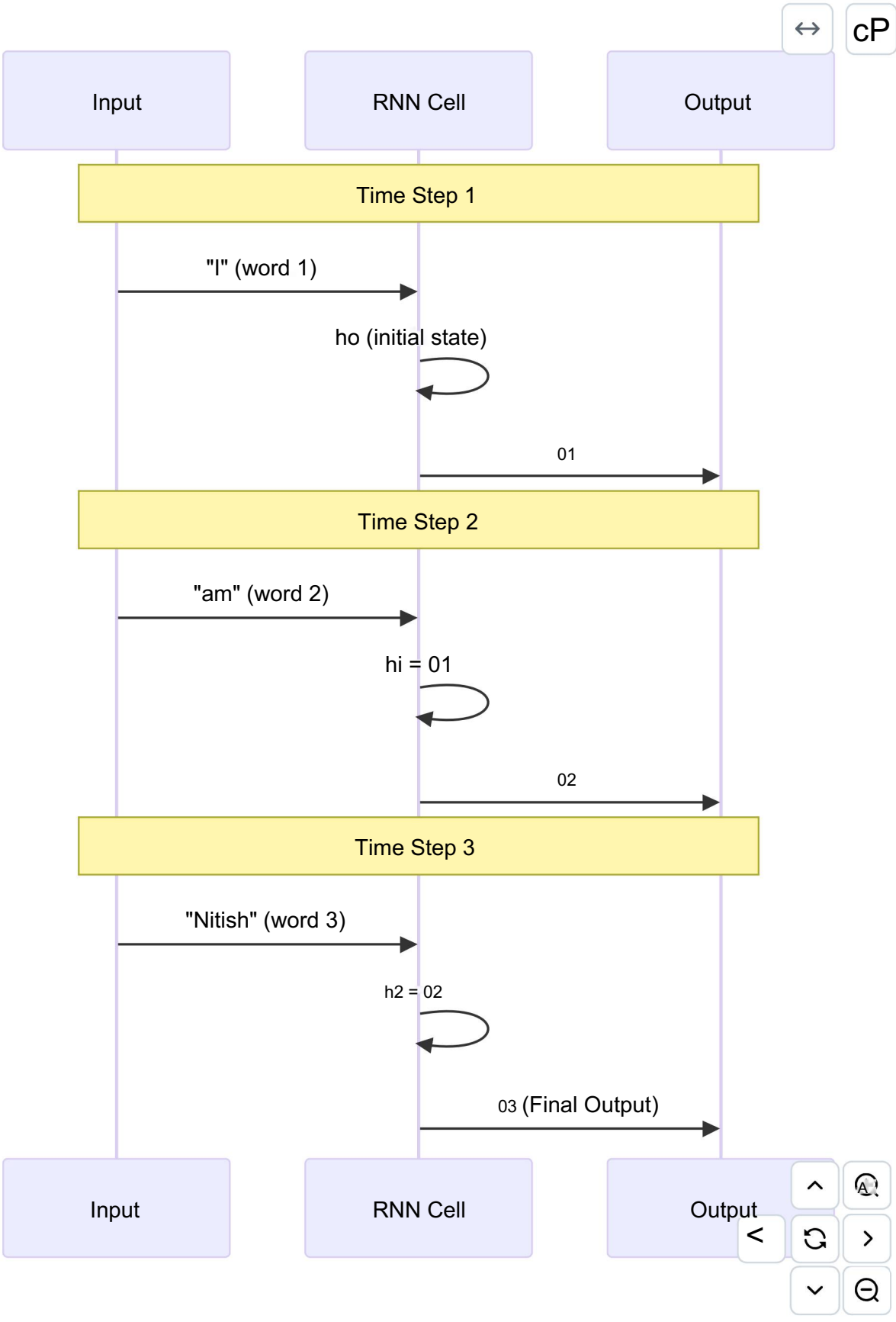
Why RNNs for Sequential Data?



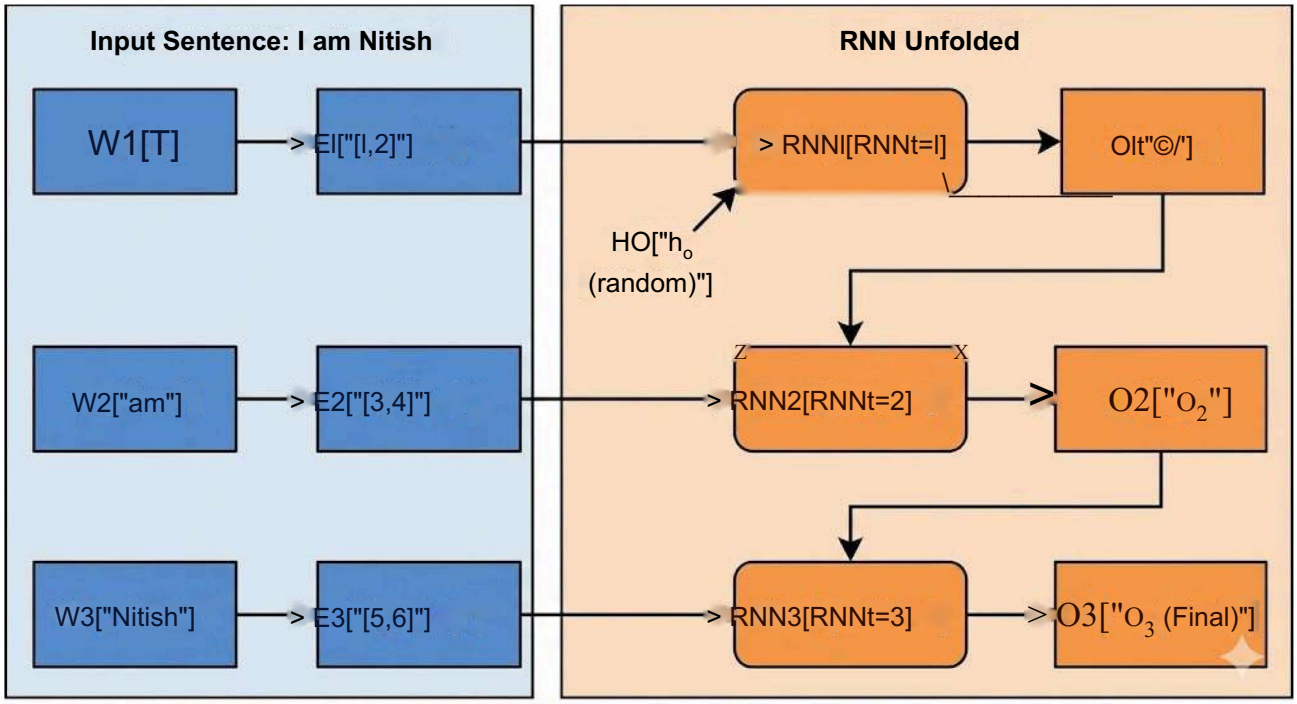
4. How RNNs Process Sequential Data

Unfolding Through Time

When processing a sentence like **"I am Nitish",** the RNN unfolds across time steps:



Step-by-Step Processing



Word Embedding Process

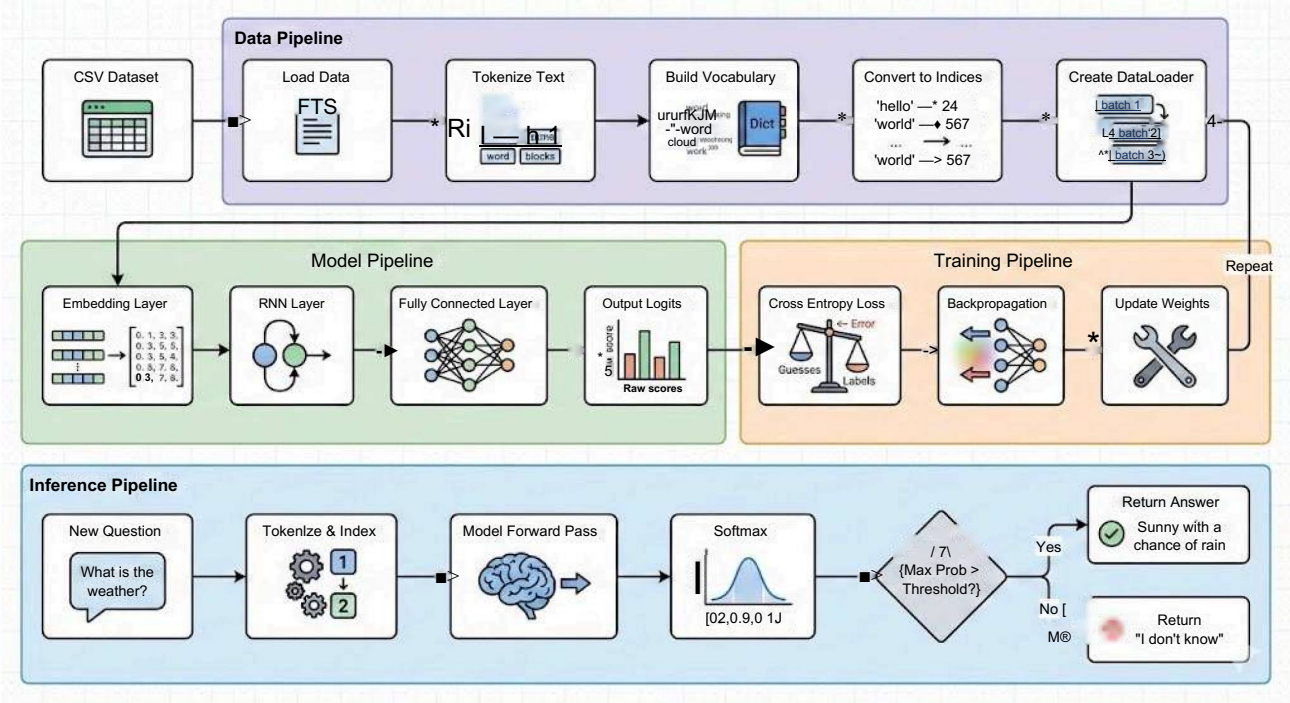
Since RNNs work with numbers, we must convert words to numerical vectors (embeddings):

Word	Index	Embedding Vector (hypothetical)
I	1	[1,2]
am	2	[3, 4]
Nitish	3	[5, 6]

Note: In practice, embedding vectors have much higher dimensions (e.g., 50, 100, 300).

5. Project Strategy and Pipeline

Overall Architecture



Five-Stage Implementation Plan

Stage	Task	Description
1	Data Loading	Load Q&A dataset from CSV
2	Preprocessing	Tokenization, vocabulary building, indexing
3	Architecture	Build RNN model with embedding layer
4	Training	Train model on dataset
5	Prediction	Implement inference with confidence threshold

6. Implementation Guide

Step 1: Data Loading

```
import pandas as pd                                                    tP

# Load the Q&A dataset
df = pd.read_csv('/content/100_Unique_QA_Dataset.csv')
df.head()
```

Dataset Structure:

Column	Description
question	The question text in English
answer	The corresponding answer

Example entries:

question	answer
What is the capital of France?	Paris
What is the largest planet in our solar system?	Jupiter
Who developed the theory of relativity?	Albert Einstein

Step 2: Tokenization

Purpose: Split text into individual words (tokens) for processing.

```
def tokenize(text):
    """
    Tokenize text by:
    1. Converting to lowercase
    2. Removing punctuation (? and ')
    3. Splitting by whitespace

    Args:
        text: Input string
    Returns:
        List of tokens (words)
    """
    text = text.lower()
    text = text.replace('?', '')
    text = text.replace("'", '')
    return text.split()
```

Example:

```
>>> tokenize('What is the capital of France?')
['what', 'is', 'the', 'capital', 'of', 'france']
```

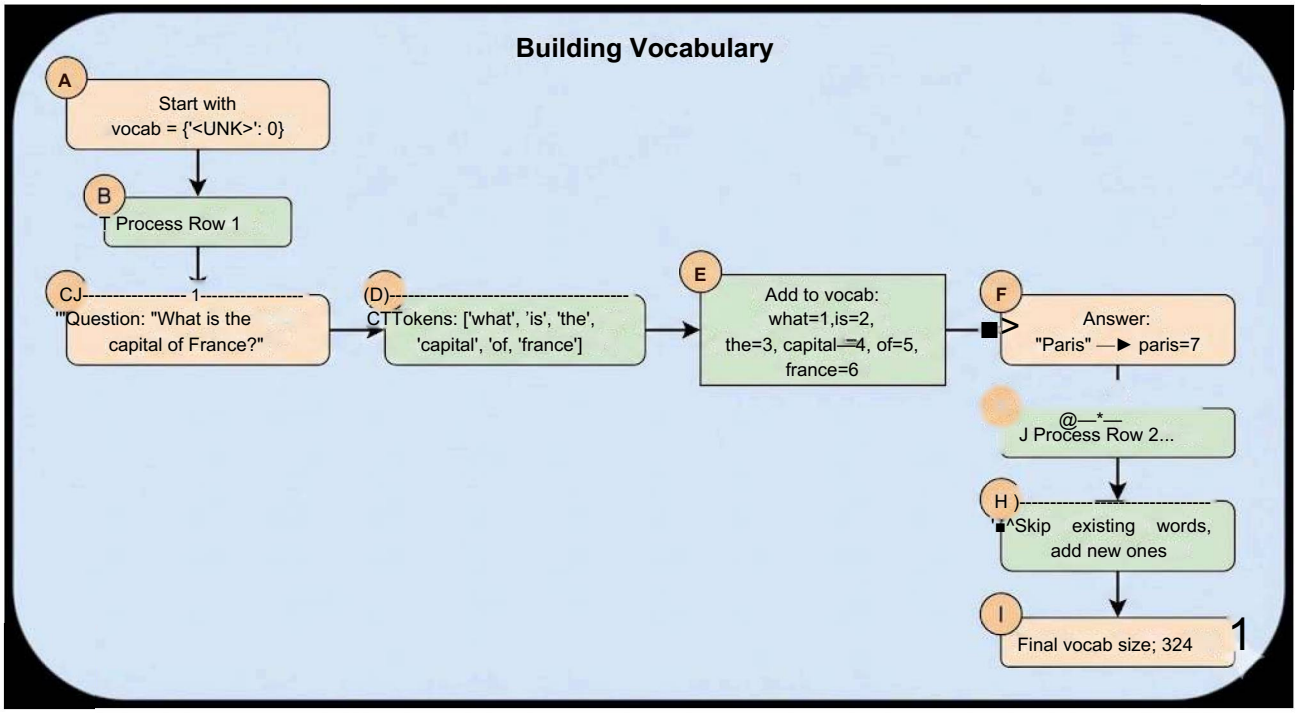
[P

Step 3: Building Vocabulary

Purpose: Create a mapping from unique words to numerical indices.

```
# Initialize vocabulary with unknown token
vocab = {'<UNK>': 0}

def build_vocab(row):
    """
    Build vocabulary from dataset rows.
    Adds unique words from both questions and answers.

    Args:
        row: DataFrame row with 'question' and 'answer' columns
    """
    tokenized_question = tokenize(row['question'])
    tokenized_answer = tokenize(row['answer'])
    merged_tokens = tokenized_question + tokenized_answer

    for token in merged_tokens:
        if token not in vocab:
            vocabftoken] = len(vocab)

# Apply to entire dataset
df.apply(build_vocab, axis=l)

print(f"Vocabulary size: {len(vocab)}") # Output: 324 unique words
```

Vocabulary Building Process:



Example Vocabulary Entries:

Word	Index
<UNK>	0
what	1
is	2
the	3
capital	4
of	5
france	6
paris	7
...	...

Step 4: Converting Text to Indices

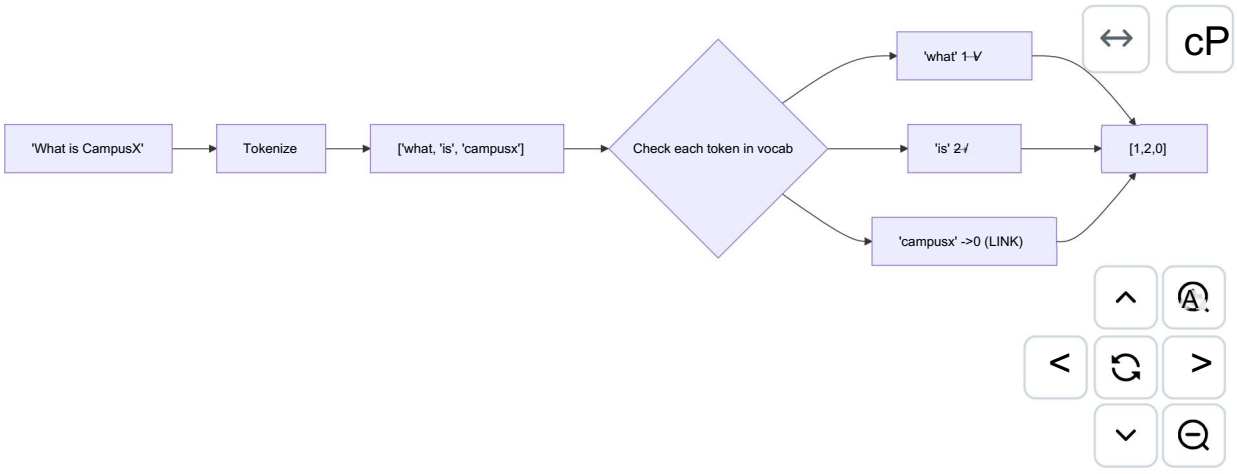
Purpose: Transform text sentences into sequences of numerical indices.

```
def text_to_indices(textj vocab):
    """
    Convert text to list of vocabulary indices.
```

```
    Args:
        text: Input text string
        vocab: Vocabulary dictionary

    Returns:
        List of indices corresponding to each word
    """
    indexed_text = []
    for token in tokenize(text):
        if token in vocab:
            indexed_text.append(vocab[token])
        else:
            indexed_text.append(vocab['<UNK>']) # Handle unknown words
    return indexed_text
```

Example:

```
>>> text_to_indices("What is CampusX", vocab)
[1, 2, 0] # 'what'=1, 'is'=2, 'campusx'=0 (unknown)
```



Step 5: Creating Dataset and DataLoader

PyTorch Dataset Class:

```
import torch
from torch.utils.data import Dataset, DataLoader

class QADataset(Dataset):
    """
    Custom Dataset for Question-Answer pairs,
    """
```

```
    def ___init____(self, df, vocab):
        self.df = df
        self.vocab = vocab

    def ___len__ (self):
        return self.df.shape[0]

    def ___getitem___ (self, index):
        """
        Returns question and answer as tensors of indices.
        """
        numerical_question = text_to_indices(
            self.df.iloc[index]['question'],
            self.vocab
        )
        numerical_answer = text_to_indices(
            self.df.Hoc[index]['answer'],
            self.vocab
        )

        return (
            torch.tenson(numerical_question),
            torch.tensor(numerical_answer)
        )

# Create dataset and dataloader
dataset = QADataset(df, vocab)
dataloader = DataLoader(dataset, batch_size=l, shuffle=True)
```

Why batch_size=1?

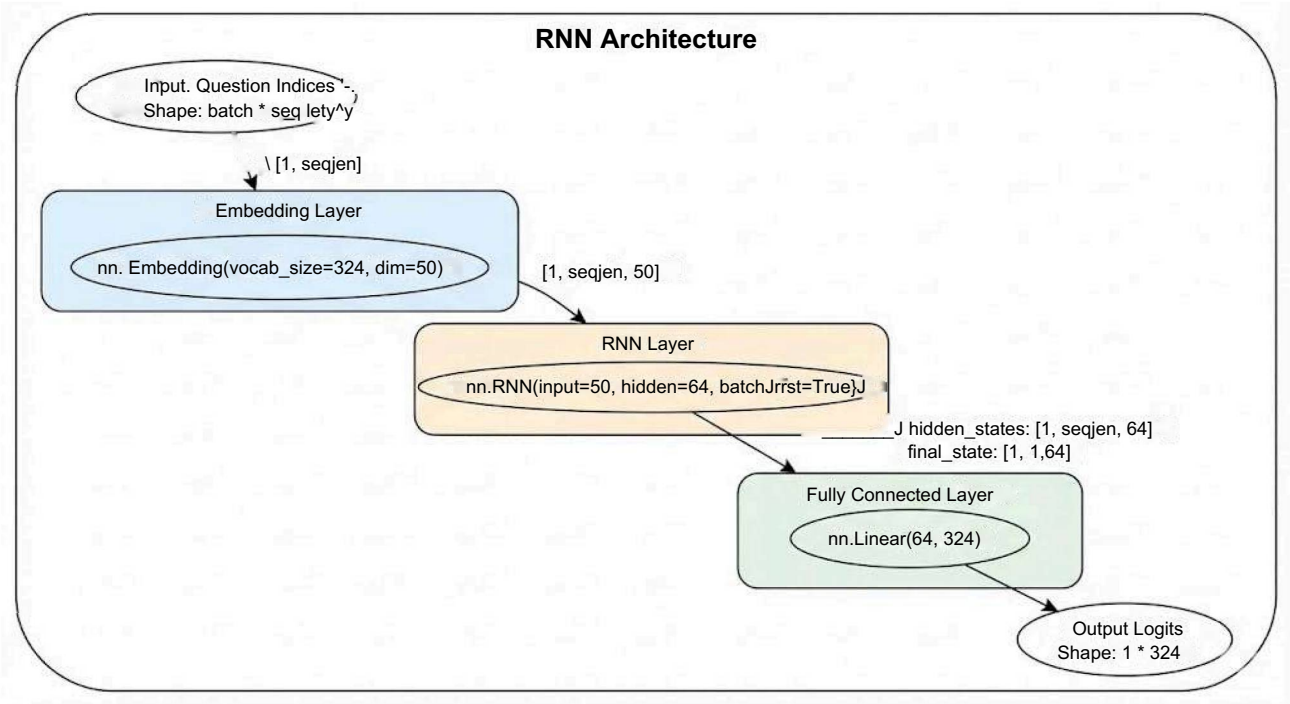
Reason	Explanation
No Padding Needed	Different questions have different lengths; batch_size = 1 avoids padding complexity
Small Dataset	With only ~90 samples, larger batches aren't necessary
Simplicity	Easier to debug and understand

DataLoader Output Example:

```
>>> dataset[0]
(tensor([l, 2, 3, 4, 5, 6]), tensor([7]))
# Question: "what is the capital of france" -» [1, 2, 3, 4, 5, 6]
# Answer: "paris" -> [7]
```

Step 6: RNN Architecture Design

Architecture Diagram



Layer Details

Layer	Input Size	Output Size	Description
Embedding	vocab_size (324)	50	Converts word indices to dense vectors
RNN	50	64	Processes sequence with memory
Fully Connected	64	vocab_size (324)	Maps to vocabulary for classification

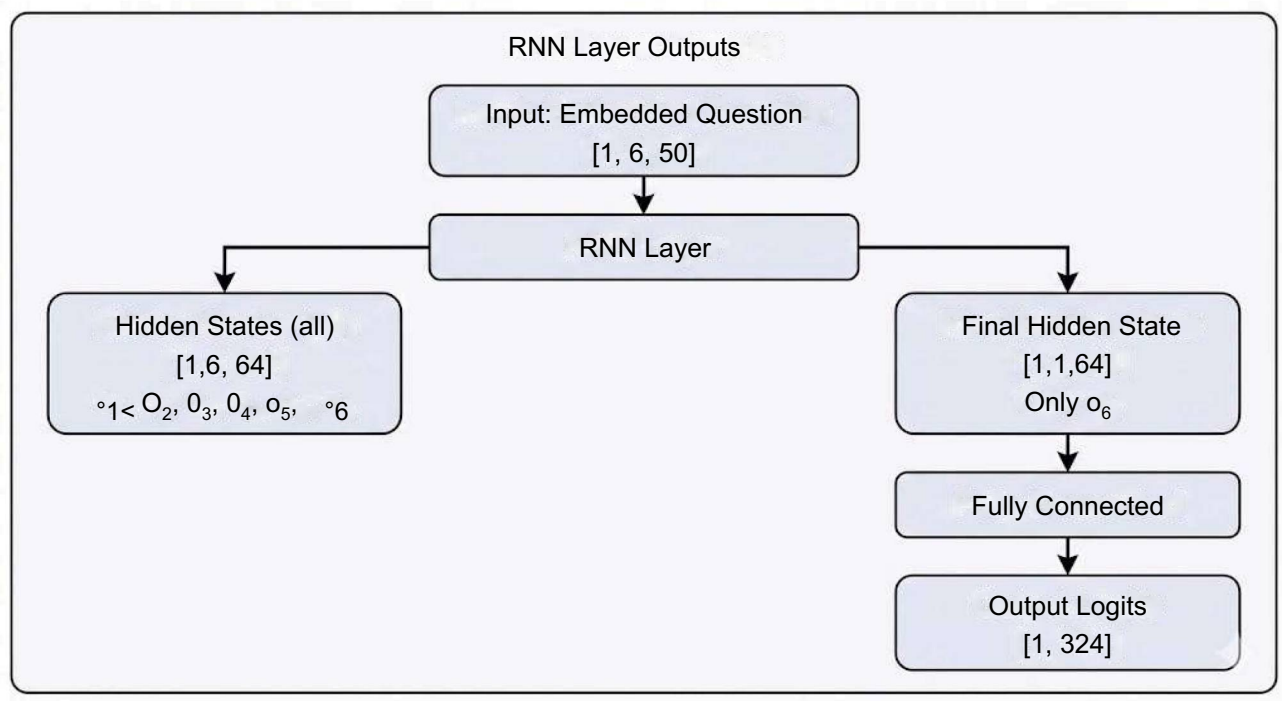
PyTorch Implementation

cP

```
import torch.nn as nn

class SimpleRNN(nn.Module):
    def _____ init____ (self, vocab_size):
        super().____ init____ ()

        # Embedding layer: converts indices to dense vectors
        self .embedding = nn. Embedding(vocab_size., embedding_dim=50)
```

```
        # RNN layer: processes sequence with memory
        # batch_first=True: input shape is (batch, seq, features)
        self.rnn = nn.RNN(50, 64, batch_first=True)

        # Output layer: maps to vocabulary size
        self.fc = nn.Linear(64, vocab_size)

    def forward(self, question):
        # Step 1: Convert indices to embeddings
        # Shape: (batch, seq_len) -> (batch, seq_len, 50)
        embedded_question = self.embedding(question)

        #  Step 2: Process through RNN
        #  hidden: all hidden states (batch, seq_len, 64)
        #  final: last hidden state (1, batch, 64)
        hidden, final = self.rnn(embedded_question)

        #  Step 3: Use final hidden state for classification
        # squeeze(0): remove first dimension (batch, 64) →
        output = self,fc(final.squeeze(0))

        return output
```

Understanding RNN Outputs



Why batch_first=True ?

Without batch_first=True :

- Input shape expected: (seq_len, batch, features)

- Hidden states swap batch and sequence dimensions

With batch_first=True :

- Input shape: (batch, seq_len, features) *V*
- More intuitive and matches our data format

Why .squeeze(0) ?

The final hidden state has shape [1, batch, 64] . The extra dimension at index 0 causes shape mismatch with the FC layer. squeeze(0) removesit: [1, batch, 64] -> [batch, 64]

Step 7: Training Loop

Training Configuration

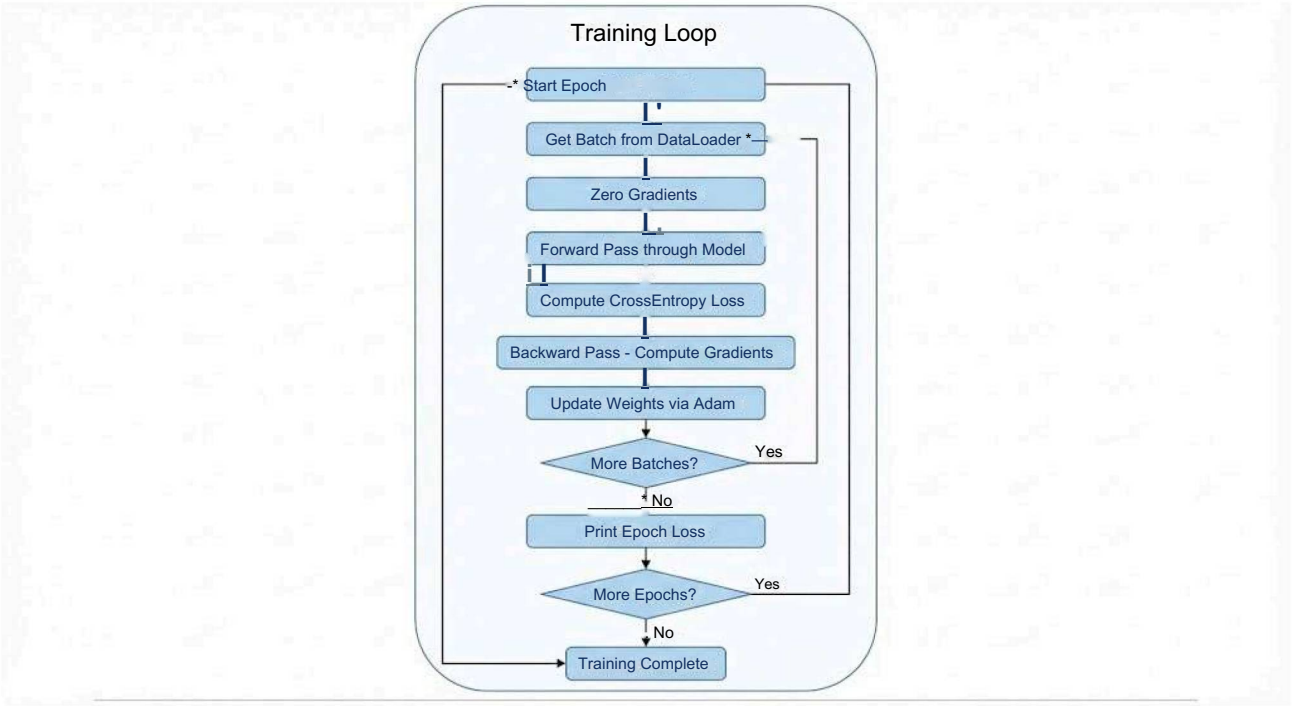
```
# Hyperparameters
learning_rate = 0.001
epochs = 20

# Model initialization
model = SimpleRNN(len(vocab))

# Loss function: Cross Entropy for classification
criterion = nn.CrossEntropyLoss()

# Optimizer: Adam
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

Training Loop Implementation

```
for epoch in range(epochs):
    total_loss = 0

    for question, answer in dataloader:
        # 1. Clear gradients
        optimizer.zero_grad()

        # 2. Forward pass
        output = model(question)

        # 3. Compute loss
        # Note: answer[0] to get 0D tensor for CrossEntropyLoss
        loss = criterion(output, answer[0])
```

```
        # 4. Backward pass (compute gradients)
        loss.backward()

        # 5. Update weights
        optimizer.step()

        # Accumulate loss
        total_loss += loss.item()

    print(f"Epoch: {epoch+1}, Loss: {total_loss:.4f}")
```

Training Flow Diagram



Expected Training Output

```
Epoch: 1, Loss: 527.3421
Epoch: 2, Loss: 421.8765
Epoch: 3, Loss: 298.4532
...
Epoch: 19, Loss: 12.3456
Epoch: 20, Loss: 8.7654
```

cP

Step 8: Prediction Function

Implementation

```
def predict(model, question, threshold=0.5):
    """
    Predict answer for a given question.

    Args:
        model: Trained RNN model
        question: Input question string
        threshold: Confidence threshold (default 0.5)

    Returns:
        Prints the predicted answer or "I don't know"
    """
    # Step 1: Convert question to numerical indices
    numerical_question = text_to_indices(question, vocab)

    # Step 2: Convert to tensor and add batch dimension
    question_tensor = torch.tensor(numerical_question).unsqueeze(0)
    # Shape: [seq_len] -> [1, seq_len]

    # Step 3: Forward pass (no gradient needed for inference)
    output = model(question_tensor)
    # Shape: [1, vocab_size]

    # Step 4: Convert logits to probabilities
    probs = torch.nn.functional.softmax(output, dim=1)

    # Step 5: Find max probability and its index
    max_prob, index = torch.max(probs., dim=1)

    # Step 6: Apply threshold logic
    if max_prob < threshold:
        print("I don't know")
    else:
        # Get the word at the predicted index
        answer = list(vocab.keys())[index]
        print(answer)
```

Usage Examples

```
>>> predict(model, "What is the capital of France?")
paris

>>> predict(model, "What is the largest planet in our solar system?")
jupiter
```

```
>>> predict(model, "What is CampusX?")
I don't know
```

Why Use Threshold?

Scenario	Max Probability	Action
Known question	> 0.5 (e.g., 0.85)	Return answer
Unknown question	< 0.5 (e.g., 0.03)	Return "I don't know"
Ambiguous	~0.5	Depends on threshold

7. Mathematical Foundations

7.1 Embedding Layer

The embedding layer converts discrete word indices to continuous vector representations.

Given:

- Vocabulary size: $V = 324$
- Embedding dimension: $d = 50$

The embedding matrix $E \; E \; R^{Vxd}$ maps each word index i to a vector:

$$et = £[i] \; e \; R^{d}$$

For a sequence of words with indices $[ii, i2, \ldots, i_n]$:

$$\text{Output} = [f[ii], £'[i2]> \blacksquare \blacksquare \text{-}\hat{}Un]] \; R^{\mathbb{C}xd}$$

Example:

- Input: [i, 2, 3, 4j 5, 6] (6 word indices)
- Output: [6, 50] tensor (6 vectors of 50 dimensions each)

7.2 RNN Forward Pass

At each time step t, the RNN computes:

$$h_t = \tanh(fV_{xh} \blacksquare x_t + W_{hh}\text{-}h_{t_1} + \quad b_h)$$

Where:

- x_t = input at time t (embedding vector, dimension 50)
- $b_{t_}x$ = previous hidden state (dimension 64)
- W_{xh} = input-to-hidden weights $(\mathrm{R}^{64\mathrm{X}\ 50})$
- W_{hh} = hidden-to-hidden weights $(R^{64 \times 64})$
- b_h = bias term (/?^{64})
- h_t = current hidden state (7?^{64})

Unrolled computation for "I am Nitish":

Time Step	Input	Hidden State
t = 0	-	h_0 (random initialization)
t = 1	%i = embedding("I")	bi = tanhC^Xi + ^bo + b)
t = 2	x_2 = embeddingC'am")	h_2 = tanh(I¥$_{xh}x_2$ + $W_{hh}h_1$ + b)
t = 3	x_3 = embedding("Nitish")	b_3 = tanh($W_{x/l}x_3$ + $W_{hh}h_2$ + b)

Final output: h_3 (used for classification)

7.3 Output Layer (Fully Connected)

The final hidden state is mapped to vocabulary logits:

$$^{2}_=\text{^}fc\ '\ \text{br} + bf_c$$

Where:

- h_T = final hidden state (/?^{64})
- ™fc = fully connected weights (/?^{324x64})
- ^{b}fc : = bias (/?^{324})
- z = logits (/?^{324})

7.4 Softmax Function

Convert logits to probabilities:

$$P(y = k) = \frac{g^{z}k}{\sum_{j=1}^{V} e^{z_j}}$$

Where:

- z_k = logit for class k
- V = vocabulary size (324)

Properties:

- All probabilities sum to 1: $Y_k P\{y = k) = 1$
- Each probability is between 0 and 1

7.5 Cross-Entropy Loss

For training, we use cross-entropy loss:

$$L = -\sum_{k=1}^{V} y_{fc} \log(P(y = k y>$$

For single-label classification (one correct answer):

$$L = - \log(P(y = y_{true}\text{^}$$

Where y_{true} is the index of the correct answer.

Intuition:

- If model predicts correct answer with high probability -» low loss
- If model predicts wrong answer -» high loss

7.6 Gradient Update (Adam Optimizer)

The Adam optimizer updates weights as:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = /?_2 v_{t_} i + (1 - £_2)\text{^}$$

$$\hat{m}t = \frac{m t}{1\text{-}\text{^}} F', \; \hat{v}t = \frac{V t}{1\text{-}\text{^}} r$$

$$6_t = Ot\text{-}x - a \quad \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + e}$$

Where:

- g_t = gradient at time t
- a = learning rate (0.001)
- $P_1 P_2$ = decay rates (default: 0.9, 0.999)
- e = small constant for numerical stability

8. Debugging Common Issues

Issue 1: Shape Mismatch in Loss Function

Error:

```
RuntimeError: Expected target size (I, 324), got (1,)
```

tP

Cause: The answer tensor has an extra dimension from the DataLoader.

Solution:

```
# Instead of:
loss = criterion(output, answer)

# Use:
loss = criterion(output, answer[0]) # Get 0th element to make it ID
```

LP

Issue 2: RNN Output Shape Mismatch

Error:

```
Expected target size (1, 324), got torch.Size([1, 9, 324])
```

[P

Cause: Without batch_first=True , RNN swaps batch and sequence dimensions, leading to incorrect shapes.

Solution:

```
# Add batch_first=True to RNN layer
self.rnn = nn.RNN(50, 64, batch_first=True)
```

[P

Issue 3: Final Hidden State Shape

Problem: Final hidden state shape is [1, 1, 64] but FC layer expects [1, 64],

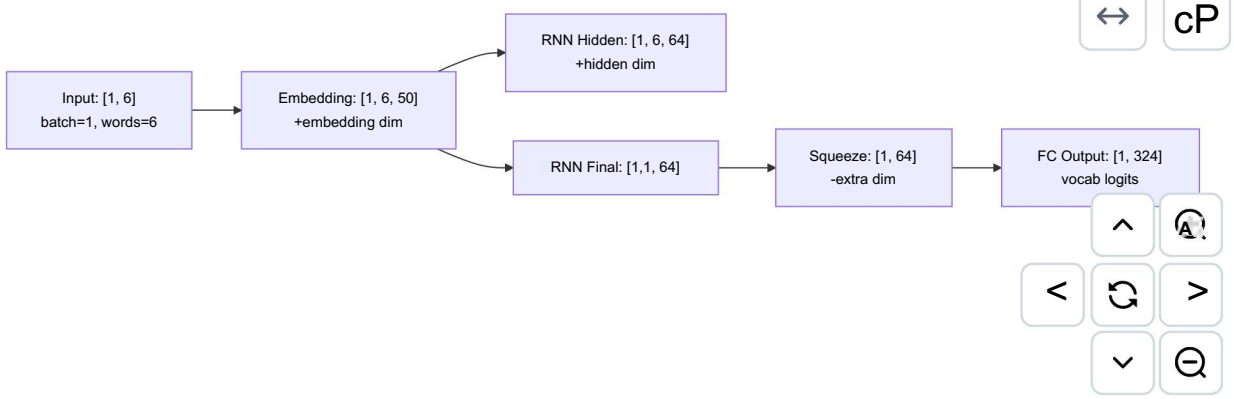
Solution:

```
# In forward method:
output = self.fc(final.squeeze(0)) # Remove first dimension
```

Debug Verification Code

```
# Verify shapes at each layer
x = nn.Embedding(324, embedding_dim=50)
y = nn.RNN(50, 64, batch_first=True)
z = nn.Linear(64, 324)

a = dataset[0][0].reshape(l, 6) # [1, 6] - batch=l, seq=6
print("Input shape:", a.shape)

b = x(a) # Embedding output
print("After embedding:", b.shape) # [1, 6, 50]

c, d = y(b) # RNN outputs
print("Hidden states shape:", c.shape) # [1, 6, 64]
print("Final state shape:", d.shape) # [1, 1, 64]

e = z(d.squeeze(0)) # FC output
print("Output shape:", e.shape) # [1, 324]
```

Shape Transformation Summary



9. Complete Code Implementation

```
# ================================================== ======================
#  SIMPLE QUESTION-ANSWERING SYSTEM USING RNN IN PYTORCH
# ================================================== ======================

#---------------------------------------------------------------------------- ----------------------------
#  STEP 1: IMPORTS AND DATA LOADING
#---------------------------------------------------------------------------- ----------------------------
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

#  Load dataset
df = pd.read_csv('/content/100_Unique_QA_Dataset.csv')

#---------------------------------------------------------------------------------------------------------------------- ------
#  STEP 2: TOKENIZATION FUNCTION
#---------------------------------------------------------------------------------------------------------------------- ------
def tokenize(text):
    """Tokenize text: lowercase, remove punctuation, split by spaces.....
    text = text.lower()
    text = text.replace('?', '')
    text = text.replace(""")          " ,
    return text.split()

#---------------------------------------------------------------------------------------------- ------------------------
#  STEP 3: BUILD VOCABULARY
#---------------------------------------------------------------------------------------------- ------------------------
vocab = {'<UNK>': 0} # Initialize with unknown token

def build_vocab(row):
    """Add unique words from question and answer to vocabulary..... .
    tokenized_question = tokenize(row['question'])
    tokenized_answer = tokenize(row['answer'])
    merged_tokens = tokenized_question + tokenized_answer

    for token in merged_tokens:
        if token not in vocab:
            vocabftoken] = len(vocab)

df.apply(build_vocab, axis=l)
print(f"Vocabulary size: {len(vocab)}")

#---------------------------------------------------------- ----------------------------------------
# STEP 4: TEXT TO INDICES CONVERSION
#---------------------------------------------------------- ----------------------------------------
```

```
def text_to_indices(text, vocab):
    """Convert text to list of vocabulary indices."""
    indexed_text = []
    for token in tokenize(text):
        if token in vocab:
            indexed_text.append(vocab[token])
        else:
            indexed_text.append(vocab['<UNK>'])
    return indexed_text

#---------------------------------------------------- ----------------------------------------
#  STEP 5: CUSTOM DATASET CLASS
#---------------------------------------------------- ----------------------------------------
class QADataset(Dataset):
    def ___init____(self, df, vocab):
        self.df = df
        self.vocab = vocab

    def ___len__  (self):
        return self.df.shape[0]

    def ___getitem___ (self, index):
        numerical_question = text_to_indices(
            self.df.iloc[index]['question'],
            self.vocab
        )
        numerical_answer = text_to_indices(
            self.df.iloc[index]['answer'],
            self.vocab
        )
        return (
            torch.tensor(numerical_question),
            torch.tensor(numerical_answer)
        )

#  Create dataset and dataloader
dataset = QADataset(df, vocab)
dataloader = DataLoader(dataset, batch_size=l, shuffle=True)

#---------------------------------------------------------------------------------------------------------------- ----------
#  STEP 6: RNN MODEL ARCHITECTURE
#---------------------------------------------------------------------------------------------------------------- ----------
class SimpleRNN(nn.Module):
    def___ init____(self, vocab_size):
        super().___init_()
        self.embedding = nn.Embedding(vocab_size, embedding_dim=50)
        self.rnn = nn.RNN(50, 64, batch_first=True)
        self.fc = nn.Linear(64, vocab_size)
```

```
    def forward(self, question):
        embedded_question = self.embedding(question)
        hidden, final = self.rnn(embedded_question)
        output = self.fc(final.squeeze(0))
        return output

#---------------------------------------------------- ----------------------------------------------
#  STEP 7: TRAINING CONFIGURATION
#---------------------------------------------------- ----------------------------------------------
learning_rate = 0.001
epochs = 20

model = SimpleRNN(len(vocab))
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

#---------------------------------------------- ------------------------------------------------------
#  STEP 8: TRAINING LOOP
#---------------------------------------------- ------------------------------------------------------
for epoch in range(epochs):
    total_loss = 0

    for question, answer in dataloader:
        optimizer.zero_grad()
        output = model(question)
        loss = criterion(output, answer[0])
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    print(f"Epoch: {epoch+1}, Loss: {total_loss:.4f}")

#---------------------------------------------------------------------------------------------------------- ----------
#  STEP 9: PREDICTION FUNCTION
#---------------------------------------------------------------------------------------------------------- ----------
def predict(model, question, threshold=0.5):
    """Predict answer for a given question."""
    numerical_question = text_to_indices(question, vocab)
    question_tensor = torch.tensor(numerical_question).unsqueeze(0)

    output = model(question_tensor)
    probs = torch.nn.functional.softmax(output, dim=1)
    max_prob, index = torch.max(probs, dim=1)

    if max_prob < threshold:
        print("I don't know")
    else:
        print(list(vocab.keys())[index])
```

```
#-------------------------------------------------------------------------------------------------------------
# STEP 10: TEST PREDICTIONS
#-------------------------------------------------------------------------------------------------------------
print("\n— Testing Predictions —")
predict(model, "What is the capital of France?")
predict(modelj "What is the largest planet in our solar system?")
predict(model, "What is CampusX?")
```

10. Key Takeaways and Best Practices

Best Practices

Category	Practice	Reason
Data	Use <UNK> token	Handle out-of-vocabulary words
Data	Lowercase all text	Reduce vocabulary size, improve consistency
Model	Set batch_first=True	Maintain intuitive tensor shapes
Model	Use .squeeze(0) on final state	Match FC layer input requirements
Training	Start with small learning rate	Prevent divergence
Inference	Use confidence threshold	Filter unreliable predictions

Limitations and Future Improvements

Limitation	Potential Solution
Small vocabulary (324 words)	Train on larger dataset
Single-word answers only	Extend to sequence-to-sequence
No attention mechanism	Implement attention/transformer
Basic RNN (vanishing gradients)	Use LSTM or GRU
No pre-trained embeddings	Use Word2Vec, GloVe, or BERT

Architecture Comparison: RNN vs LSTM vs Transformer

Feature	RNN	LSTM	Transformer
Memory	Short-term	Long-term via gates	Attention-based
Vanishing Gradients	Severe	Mitigated	None
Parallelization	Sequential only	Sequential only	Fully parallel
Complexity	Low	Medium	High
Performance on Long Sequences	Poor	Good	Excellent

Next Steps in Learning

1. **LSTM/GRU:** Learn gated architectures to handle long sequences
2. Encoder-Decoder: Build sequence-to-sequence models
3. **Attention Mechanism:** Understand how attention improves sequence models
4. **Transformers:** Modern architecture for NLP tasks
5. **Pre-trained Models:** Fine-tune BERT, GPT for specific tasks

Building a Question-Answering System

Using RNN in PyTorch

What is RNN?

A Recurrent Neural Network processes sequential data by maintaining memory of previous inputs

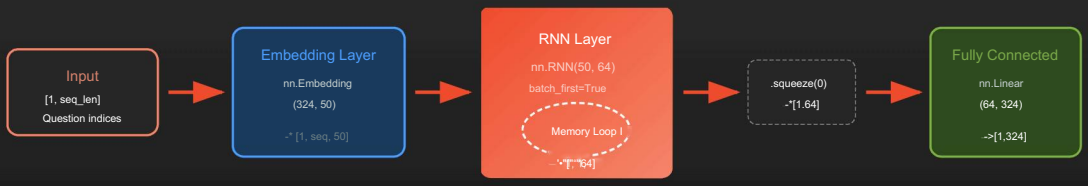
- •/ Sequential Processing
- •/ Memory Mechanism
- *V* Parameter Sharing
- / Variable Length Input
- *V* Context Awareness

Data Pipeline

1. CSV Dataset — Q&A pairs
2. Tokenize — Split words
3. Build Vocab — 324 words
4. To Indices — [1.2, 3...]
5. DataLoader — batch_size=1

"What is the capital of France?" → ['what', 'is', 'the', 'capital', 'of, 'france^ [1, 2, 3, 4, 5, 6]

RNN Model Architecture



Embedding	RNN	FC Layer	Output
vocab —• dense vectors	sequence with memory	maps to vocab size	324 logits (per word)

RNN Mathematics

Hidden State Update:

$$h_t = \tanh(W_{xh} \cdot x_t + W_h h \cdot h_{t-1} + b_h)$$

Where:

- x< = input embedding (dim: 50)
- hi-i = previous hidden state (dim: 64)
- Wxx = input weights (64x50)
- Whh = hidden weights (64*64)

Training Loop

```
for epoch in range(20) :
    # 1. Zero gradients                    optimizer.zero_grad()
    # 2. Forward pass -» output            output = model(question)
    # 3. Compute CrossEntropyLoss          loss = criterion(output, answer)
    # 4. Backward pass (gradients)         loss.backward()
    # 5. Adam optimizer step               optimizer.step()
```

Prediction / Inference

predict("What is the capital of France?") → "paris" | predict("What is CampusX?") → "I don't know"

Key Takeaways & Best Practices

Data Processing

- / Use <UNK> token for unknown words
- 7 Lowercase all text for consistency
- V Remove punctuation before tokenizing
- 4 batch_size=1 for variable lengths

Model Architecture

- / Set batch_first=True in RNN
- / Use .squeeze(O) on final state
- / Embedding dim: 50, Hidden: 64
- / FC output = vocab_size (324)

Training & Inference

- J Adam optimizer, lr=0.001
- *V* CrossEntropyLoss for classification
- / Use confidence threshold (0.5)
- / 20 epochs for small dataset

CampusX

Next Word Prediction using LSTM in PyTorch

1. Introduction

1.1 What is Language Modeling?

Language Modeling is a fundamental task in Natural Language Processing (NLP) that involves **predicting the next word in a sequence based on the context of previous words.**

> **Definition:** Given a sequence of words (w_1,w_2,.. $., w_{n-1}$), language modeling aims to predict the probability distribution over the next word w_n.

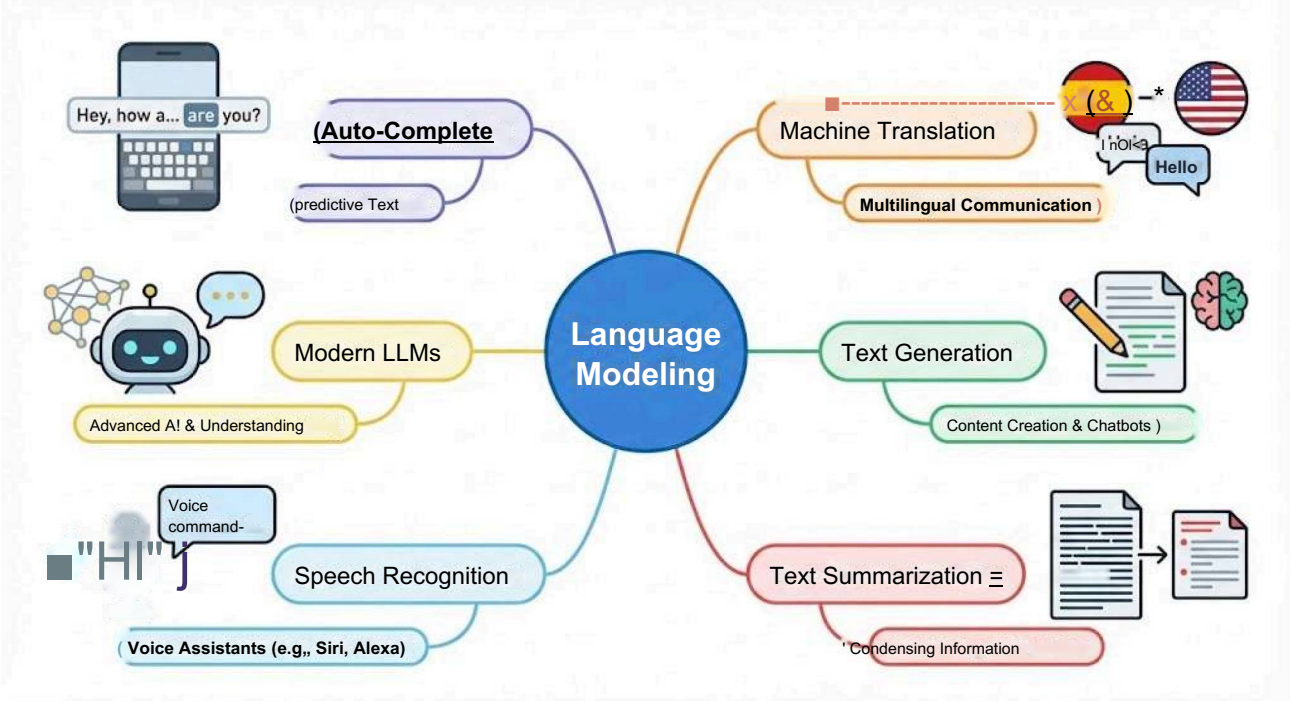
Mathematically, a language model estimates:

$$P(™n \mid w_1, w_2, \blacksquare \blacksquare ., w_{n-1})$$

The goal is to learn patterns in text data such that given any partial sentence, the model can predict the most probable continuation.

1.2 Applications

Language modeling serves as the foundation for numerous NLP applications:



Application	Description	Example
Auto-Complete	Predicting next words while typing	Smartphone keyboard suggestions
Machine Translation	Generating target language words sequentially	Google Translate
Text Summarization	Generating concise summaries	News article summaries
Chatbots	Generating contextual responses	Customer service bots
Speech Recognition	Predicting likely word sequences	Voice assistants

1.3 Project Overview

In this project, we build a **Next Word Predictor** using **LSTM (Long Short-Term Memory)** networks in PyTorch.

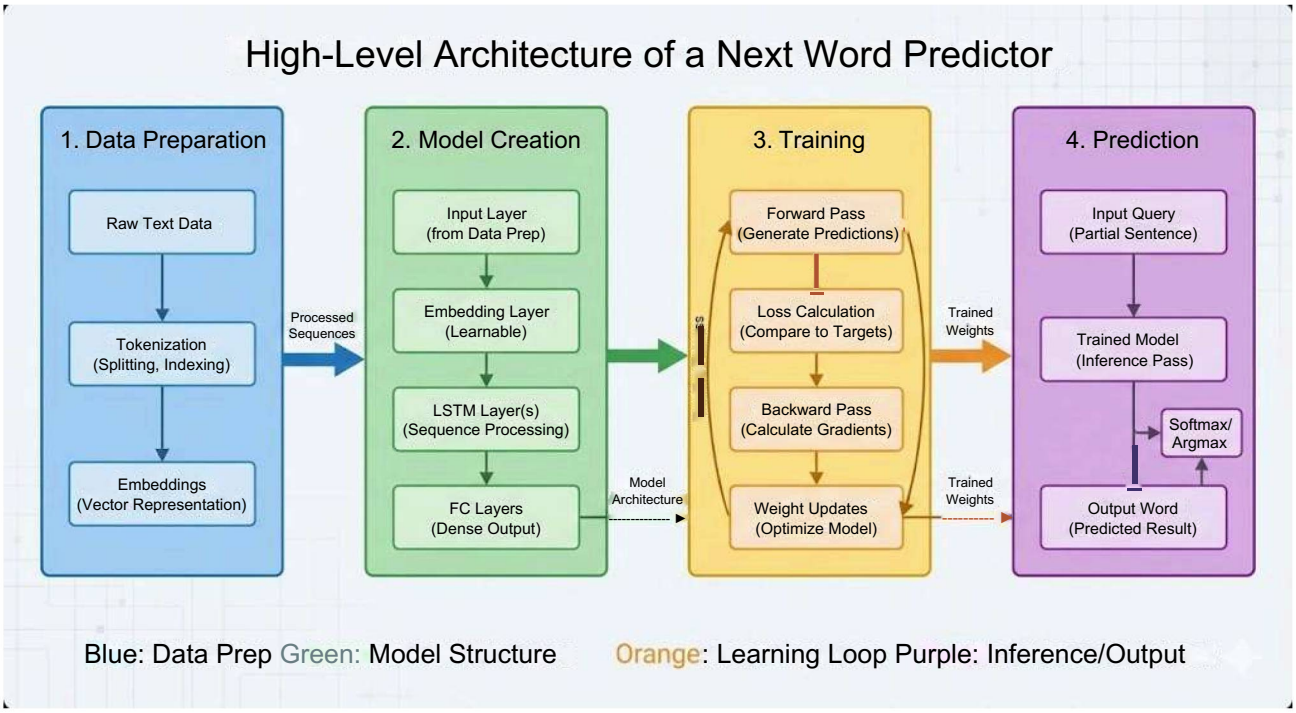
Input: A sentence or phrase (e.g., "How are") **Output:** The predicted next word (e.g., "you")

Dataset: FAQ text from a data science mentorship program containing questions and answers about the course.

2. Strategy and Project Pipeline

2.1 High-Level Architecture

The complete pipeline transforms raw text into a trained model capable of predicting next words:



2.2 Four Stages of Implementation

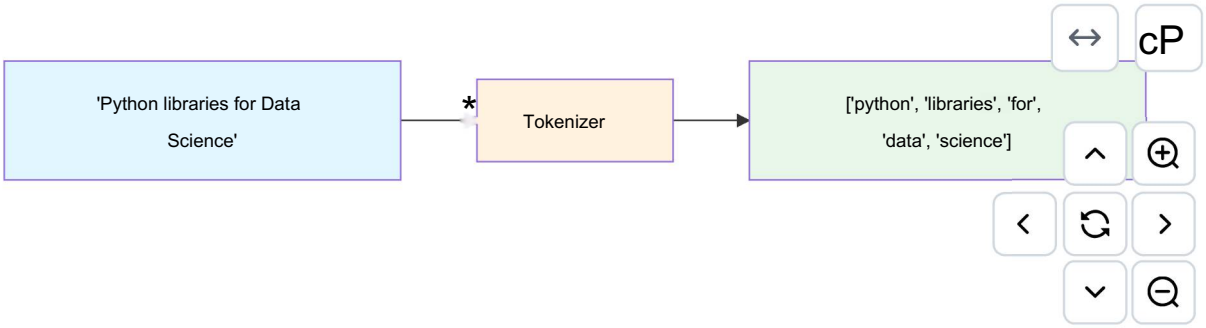
Stage	Description	Key Tasks
Stage 1	Data Preparation	Tokenization, Vocabulary Building, Sequence Creation, Padding
Stage 2	Model Architecture	Define Embedding, LSTM, and FC layers
Stage 3	Training	Implement training loop with loss calculation and backpropagation
Stage 4	Prediction	Build inference function for next word prediction

3. Data Preparation

3.1 Tokenization

Tokenization is the process of breaking text into individual units (tokens), typically words.

```
from nltk.tokenize import word_tokenize
import nltk

nltk.download('punkt')
nltk.download('punkt_tab')

#  Example
document = "Python libraries for Data Science"
tokens = word_tokenize(document.lower())
#  Output: ['python', 'libraries', 'for', 'data', 'science']
```



3.2 Vocabulary Building

A **vocabulary** is a dictionary mapping each unique word to a unique index.

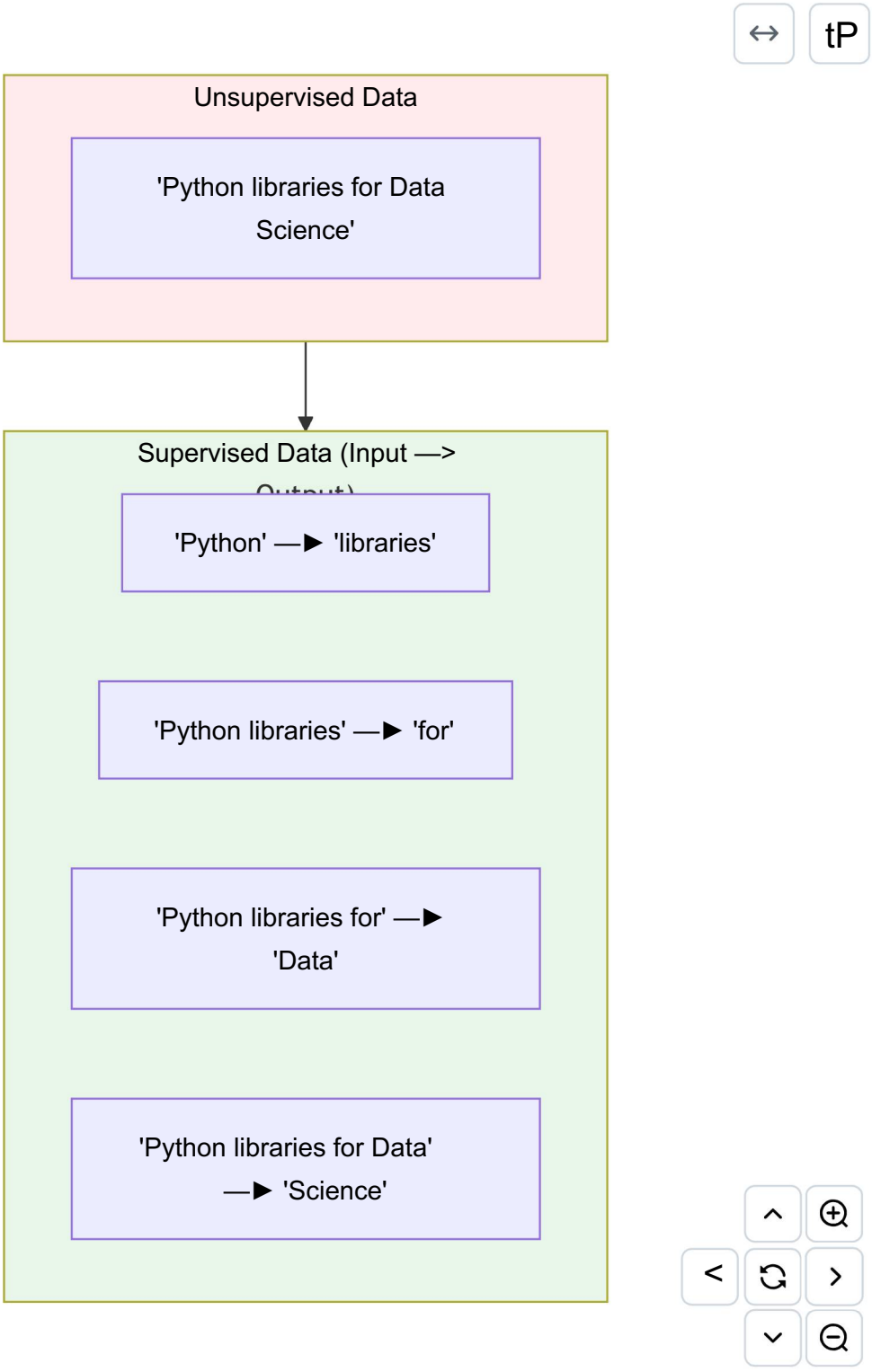
Implementation:

```
from collections import Counter

vocab = {'<unk>': 0} # Unknown token for out-of-vocabulary words

for token in Counter(tokens).keys():
    if token not in vocab:
        vocab[token] = len(vocab)

# vocab = {'<unk>': 0, 'python': 1, 'libraries': 2, 'for': 3, 'data': 4, 'scie
```

Purpose of <unk> Token: Handles words not seen during training by mapping them to a special unknown token.

3.3 Converting Unsupervised to Supervised Learning

Key Insight: The raw text dataset is **unsupervised** (no explicit labels). We convert it to a **supervised learning** task by creating input-output pairs.



Transformation Table:

Input (X)	Output (y)
Python	libraries
Python libraries	for
Python libraries for	Data
Python libraries for Data	Science

Mathematical Representation:

For a sentence S = (w_1, w_2,..., w_n), we create *n* - 1 training examples:

$$(W1,W2),(W_{1:2},W_3),(W_{1:}3,W4), \ldots \quad \ldots,(w_{1:n-1},w_n)$$

Where $w_{1:k}$ denotes the subsequence (w_1,w_2, . . . w_k).

3.4 Creating Training Sequences

Process: For each sentence, create multiple training sequences of increasing length.

```
def text_to_indices(sentence, vocab):
    """Convert tokenized sentence to numerical indices"""
    numerical_sentence = []
    for token in sentence:
        if token in vocab:
            numerical_sentence.append(vocab[token])
        else:
            numerical_sentence.append(vocab[' <unk>'])
    return numerical_sentence

# Create training sequences
training_sequences = []
for sentence in input_numerical_sentences:
    for i in range(l, len(sentence)):
        training_sequences.append(sentence[:i+1])
```

cP

Visual Example:

3.5 Padding Sequences

Problem: Training sequences have variable lengths, but neural networks require fixed-size inputs for batch processing.

Solution: Padding - Add zeros to the beginning of shorter sequences to make all sequences the same length.

Why Pre-padding (zeros at the beginning)?

- The **last token** is used as the **target label** (output)
- Pre-padding ensures the target token is always at the same position

Implementation:

```
# Find maximum sequence length
max_len = max(len(seq) for seq in training_sequences)

# Apply padding
padded_sequences = []
for sequence in training_sequences:
    padded = [0] * (max_len - len(sequence)) + sequence
    padded_sequences.append(padded)

# Convert to tensor
padded_sequences = torch.tensor(padded_sequences, dtype=torch.long)

# Split into X and y
X = padded_sequences[:, :-1] # All except last token
y = padded_sequences[:, -1]  # Last token only
```

tP

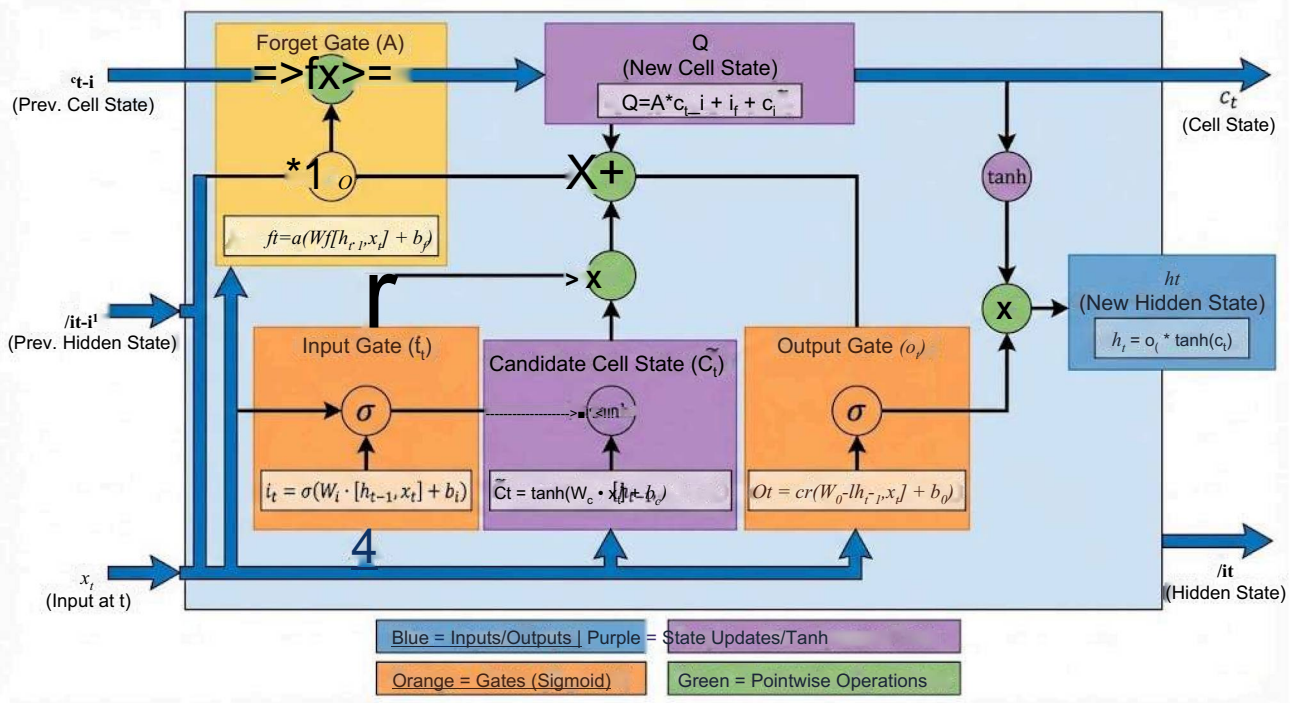
Final Data Format:

Component	Shape	Description
x (Input)	(942, 61)	942 sequences, each with 61 tokens
y (Output)	(942,)	942 target tokens

4. LSTM Architecture Deep Dive

4.1 Understanding LSTM Cells

LSTM (Long Short-Term Memory) networks are a type of Recurrent Neural Network (RNN) designed to learn long-term dependencies.



Key Components:

Component	Symbol	Purpose
Forget Gate	*ft*	Decides what information to discard from cell state
Input Gate	*k*	Decides what new information to add to cell state
Output Gate	Of	Decides what to output based on cell state
Cell State	c_t	Long-term memory carrier
Hidden State	*ht*	Short-term memory and output

4.2 Mathematical Formulation

The LSTM cell computes the following at each time step t:

Step 1: Forget Gate - Decides what to forget from the previous cell state

$$f_t = a\text{^}Wflht\text{-}\text{^}Xt] + b_f)$$

Step 2: Input Gate - Decides what new information to store

$$it = \sigma(W_i \cdot \quad [ht\text{-}\text{^}Xt] + bt)$$

Step 3: Candidate Cell State - Creates candidate values to add

$$\tilde{} \; c_t = \tanh(V/_c \bullet [h_t _ _n x_t] + bf)$$

CampusX

Step 4: Update Cell State - Combines old and new information

$$Ct = ft \text{ O } C_t _ \text{ i } + i_t \odot \tilde{c}_t$$

Step 5: Output Gate - Decides what to output

$$Ot = o(W_o \blacksquare [h_{t-1}, x_t] + b_0)$$

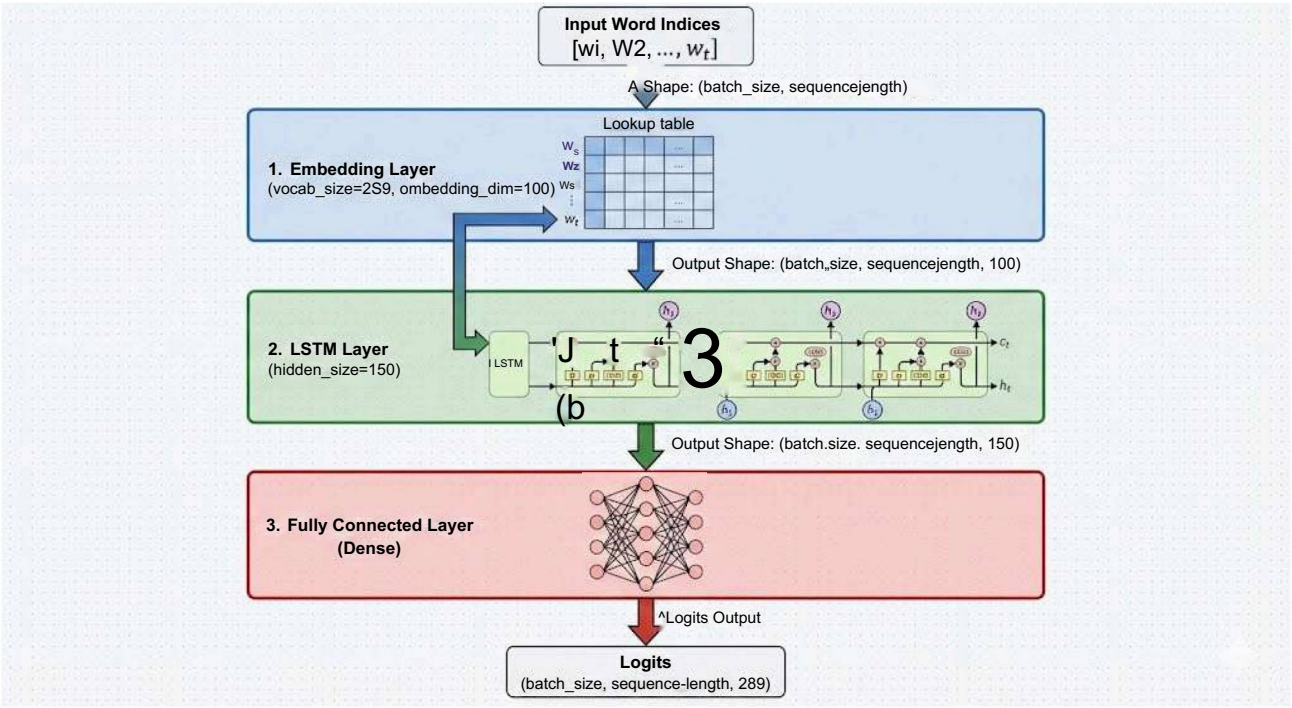
Step 6: Hidden State - Computes the output

$$h_t = o_t \text{ O } \tanh(c_t)$$

Where:

- *a* = Sigmoid activation function: c(x) = $\frac{1}{1 + e\text{—}x}$
- tanh = Hyperbolic tangent activation
- O = Element-wise multiplication (Hadamard product)
- W_f, H/j, V/$_c$, W_o = Weight matrices
- b_f, *bt*, b_c, b_0 = Bias vectors

4.3 Tensor Flow Through LSTM



LSTM Output Structure:

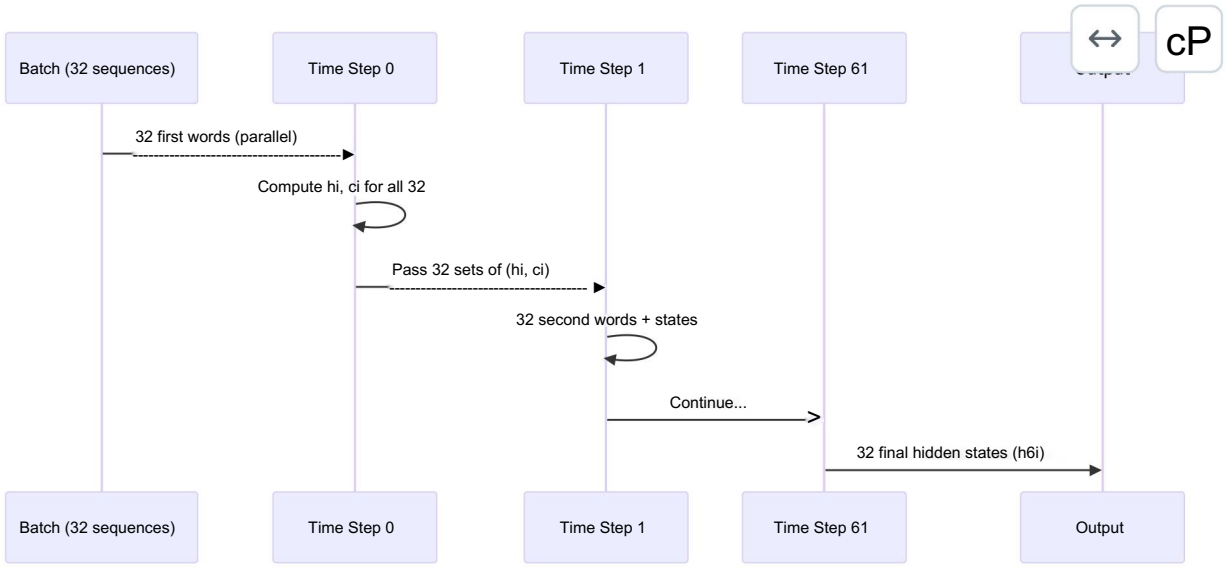
```
# LSTM returns 3 things:
```

```
intermediate_hidden_states, (final_hidden_state, final_cell_state) = lstm(embe
```

Output	Shape	Description
intermediate_hidden_states	(batch, seq_len, hidden_size)	Hidden state at each time step
final_hidden_state	(num_layers, batch, hidden_size)	Last time step's hidden state
final_cell_state	(num_layers, batch, hidden_size)	Last time step's cell state

4.4 Batch Processing in LSTM

How LSTM processes batches:

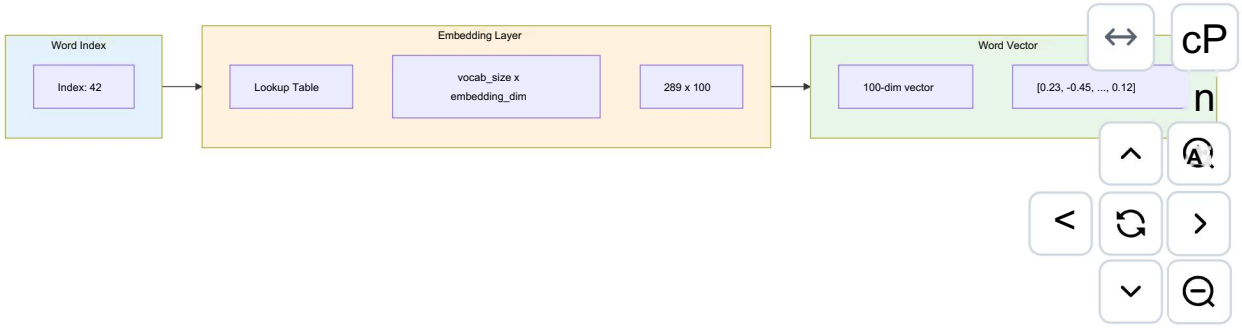


Parallel Processing: At each time step, all 32 sequences in the batch are processed simultaneously through vectorized operations.

5. Model Architecture

5.1 Embedding Layer

Purpose: Convert discrete word indices into dense continuous vectors.



Mathematical Representation:

$$\text{embedding}(W_i) = £[i] \text{ G } R^d$$

Where:

- E G R^{Vxd} is the embedding matrix
- / = vocabulary size (289)
- d = embedding dimension (100)

Implementation:

```
self.embedding = nn.Embedding(vocab_size, 100)
# Input: (batch_size, seq_len) -» Output: (batch_size, seq_len, 100)
```

5.2 LSTM Layer

Configuration:

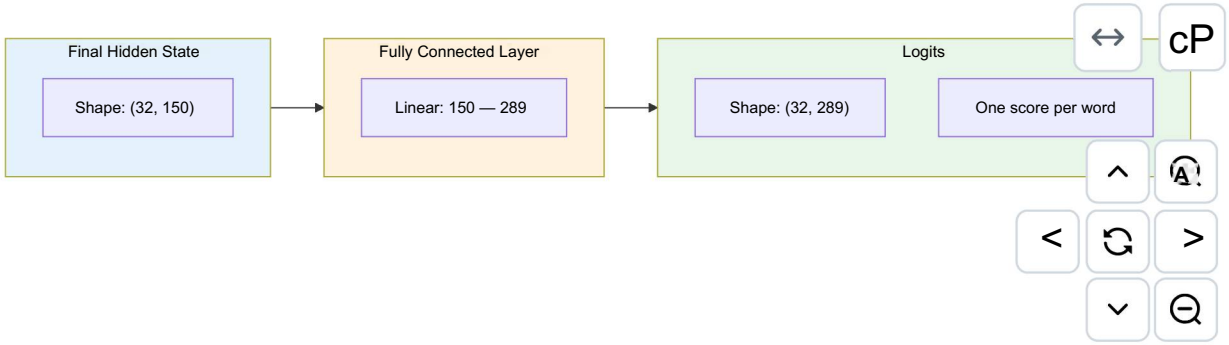
Parameter	Value	Description
input_size	100	Embedding dimension
hidden_size	150	Number of LSTM units
batch_first	True	Input shape: (batch, seq, features)

```
self.lstm = nn.LSTM(input_size=100, hidden_size=150, batch_first=True)
```

CampusX

5.3 Fully Connected Layer

Purpose: Map the final hidden state to vocabulary-sized output for classification.



Mathematical Operation:

$$\text{logits} = \text{VIZ} \bullet h_{final} + b$$

Where:

- VIZ G R^{VxH} (289 x 150)

** hfinal R^^0)*

- *bERv* (289)

```
self.fc = nn.Linear(150, vocab_size)
```

6. Implementation

6.1 Complete Code

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from collections import Counter
from torch.utils.data import Dataset, DataLoader
from nltk.tokenize import word_tokenize
import nltk

# Download required NLTK data
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
# ======================== ====================
# STEP 1: DATA PREPARATION
# ======================== ====================

document  =  """About the Program
What is the course fee for Data Science Mentorship Program (DSMP 2023)
The course follows a monthly subscription model where you have to make monthly
...
"""   # Your full text data here

#  Tokenize entire document
tokens = word_tokenize(document.lower())

#  Build vocabulary
vocab = {'<unk>': 0}
for token in Counter(tokens).keys():
    if token not in vocab:
        vocab[token] = len(vocab)

print(f"Vocabulary size: {len(vocab)}")

#  Split document into sentences
input_sentences = document.split('\n')

#  Function to convert text to numerical indices
def text_to_indices(sentencej vocab):
    """Convert tokenized sentence to numerical indices"""
    numerical_sentence = []
    for token in sentence:
        if token in vocab:
            numerical_sentence.append(vocab[token])
        else:
            numerical_sentence.append(vocab['<unk>'])
    return numerical_sentence

#  Convert all sentences to numerical form
input_numerical_sentences = []
for sentence in input_sentences:
    input_numerical_sentences.append(
        text_to_indices(word_tokenize(sentence.lower()), vocab)
    )

#  Create training sequences
training_sequences = []
for sentence in input_numerical_sentences:
    for i in range(l, len(sentence)):
        trainingsequences.append(sentence[:i+1])
```

```
print(f"Number of training sequences: {len(training_sequences)}")

#  Find maximum sequence length
max_len = max(len(seq) for seq in training_sequences)
print(f"Maximum sequence length: {max_len}")

#  Apply padding (pre-padding with zeros)
padded_sequences = []
for sequence in training_sequences:
    padded = [0] * (max_len - len(sequence)) + sequence
    padded_sequences.append(padded)

# Convert to PyTorch tensor
padded_sequences = torch.tensor(padded_sequences, dtype=torch.long)

# Split into features (X) and labels (y)
X = padded_sequences[:^ :-l]          # All except last token
y = padded_sequences[:, -1]           # Last token only

print(f"X shape: {X.shape}")          # (num_sequences, seq_len-l)
print(f"y shape: {y.shape}")          # (num_sequences,)

# ==============================
# STEP 2: DATASET AND DATALOADER
# ==============================

class CustomDataset(Dataset):
    def _____ init____ (self, X, y):
        self.X = X
        self.y = y

    def ___len__  (self):
        return self.X.shape[0]

    def ___getitem___ (self, idx):
        return self.X[idx], self.y[idx]

dataset = CustomDataset(X, y)
dataloader = DataLoader(dataset, batch_size=32, shuffle=True)

# ==========================
# STEP 3: MODEL ARCHITECTURE
# ==========================

class LSTMModel(nn.Module):
    def _____ init____ (self, vocab_size):
        super()._____ init____ ()

        # Embedding Layer: Convert word indices to dense vectors
```

```
        self.embedding = nn.Embedding(vocab_size, 100)

        # LSTM Layer: Process sequences
        self.lstm = nn.LSTM(
            input_size=100,          #  Embedding dimension
            hidden_size=150,         #  Number of LSTM units
            batch_first=True         #  Input shape: (batch, seq, features)
        )

        # Fully Connected Layer: Map to vocabulary size
        self.fc = nn.Linear(150, vocab_size)

    def forward(self, x):
        #  x shape: (batch_size, seq_len)

        #  Embedding
        embedded = self.embedding(x)
        #  embedded shape: (batch_size, seq_len, 100)

        #  LSTM
        intermediate_hidden_states, (final_hidden_state, final_cell_state) = s
        # final_hidden_state shape: (1, batch_size, 150)

        #  Fully Connected
        output = self.fc(final_hidden_state.squeeze(0))
        #  output shape: (batch_size, vocab_size)

        return output

# ====================== ======================
# STEP 4: TRAINING SETUP
# ====================== ======================

#  Initialize model
model = LSTMModel(len(vocab))

#  Device configuration
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

#  Hyperparameters
epochs = 50
learning_rate = 0.001

#  Loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)

#  ==========================================
```

```
# STEP 5: TRAINING LOOP
# ====================  ======================

for epoch in range(epochs):
    total_loss = 0

    for batch_x., batch_y in dataloader:
        # Move to device
        batch_x., batch_y = batch_x.to(device), batch_y.to(device)

        # Zero gradients
        optimizer.zero_grad()

        # Forward pass
        output = model(batch_x)

        # Calculate loss
        loss = criterion(output, batch_y)

        # Backward pass
        loss.backward()

        # Update weights
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch: {epoch + 1}, Loss: {total_loss:.4f}")
```

6.3 Prediction Function

```
def prediction(model, vocab^ text):                                    cP
    """
    Predict the next word given input text

    Args:
        model: Trained LSTM model
        vocab: Vocabulary dictionary
        text: Input text string

    Returns:
        Complete text with predicted next word
    """
    model.eval() # Set to evaluation mode

    # Step 1: Tokenize input text
```

```
    tokenized_text = word_tokenize(text.lower())

    #  Step 2: Convert to numerical indices
    numerical_text = text_to_indices(tokenized_text, vocab)

    #  Step 3: Apply padding (max_len - 1 because we removed last token for tra
    seq_len = 61 # Same as training
    padded_text = [0] * (seq_len - len(numerical_text)) + numerical_text

    #  Step 4: Convert to tensor and add batch dimension
    padded_text = torch.tensor(padded_text, dtype=torch.long).unsqueeze(0)
    padded_text = padded_text.to(device)

    #  Step 5: Get prediction
    with torch.no_grad():
        output = model(padded_text)

    #  Step 6: Get predicted word index
    _, predicted_index = torch.max(output, dim=l)

    #  Step 7: Convert index to word
    predicted_word = list(vocab.keys())[predicted_index.item()]

    #  Step 8: Return complete text
    return text + " " + predicted_word

#  Generate multiple words
def generate_text(model, vocab, seed_text, num_words=10):
    """Generate multiple words sequentially"""
    import time

    current_text = seed_text

    for i in range(num_words):
        current_text = prediction(model, vocab, current_text)
        print(current_text)
        time.sleep(0.5) # Add delay for visual effect

    return current_text

#  Example usage
generate_text(model, vocab, "The course follows a monthly", num_words=10)
```

7. Results and Extensions

Model Accuracy Calculation

```python
def calculate_accuracy(model, dataloader, device):
    """Calculate prediction accuracy on the dataset"""
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for batch_x, batch_y in dataloader:
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)

            outputs = model(batch_x)
            _, predicted = torch.max(outputs, dim=1)

            correct += (predicted == batch_y).sum().item()
            total += batch_y.size(0)

    accuracy = correct / total * 100
    return accuracy

accuracy = calculate_accuracy(model, dataloader, device)
print(f"Model Accuracy: {accuracy:.2f}%")
```

Potential Extensions

Extension	Description	Benefit
Larger Dataset	Train on conversational data (e.g, Reddit)	Better generalization
Top-K Predictions	Return top 3 most probable words	Similar to keyboard suggestions
Bi-directional LSTM	Process sequences in both directions	Better context understanding
Attention Mechanism	Add attention over hidden states	Focus on relevant words
Temperature Sampling	Control randomness in predictions	More creative outputs
Beam Search	Consider multiple prediction paths	Higher quality generations

Top-K Prediction Implementation

```
def prediction_top_k(model, vocab, text, k=3):
    """Return top-k most probable next words"""
    model.eval()

    #  Process input (same as before)
    tokenized_text = word_tokenize(text.lower())
    numerical_text = text_to_indices(tokenized_text, vocab)
    padded_text = [0] * (61 - len(numerical_text)) + numerical_text
    padded_text = torch.tensor(padded_text, dtype=torch.long).unsqueeze(0).to(

    with torch.no_grad():
        output = model(padded_text)

    #  Get top-k predictions
    probabilities = torch.softmax(output, dim=l)
    top_k_probs, top_k_indices = torch.topk(probabilities, k, dim=l)

    #  Convert to words
    vocab_list = list(vocab.keys())
    predictions = []

    for i in range(k):
        word = vocab_list[top_k_indices[0][i].item()]
        prob = top_k_probs[0][i].item()
        predictions.append((word, prob))

    return predictions

# Example: Get top 3 suggestions
suggestions = prediction_top_k(model, vocab, "The course follows", k=3)
for word, prob in suggestions:
    print(f"{word}: {prob:.4f}")
```

8. Summary

Quick Reference Table

Concept	Description	Code/Formula
Tokenization	Split text into words	word_tokenize(text)
Vocabulary	Map words to indices	vocab = {'<unk>': 0, ...}
Padding	Fixed-length sequences	[0]*pad_len + sequence

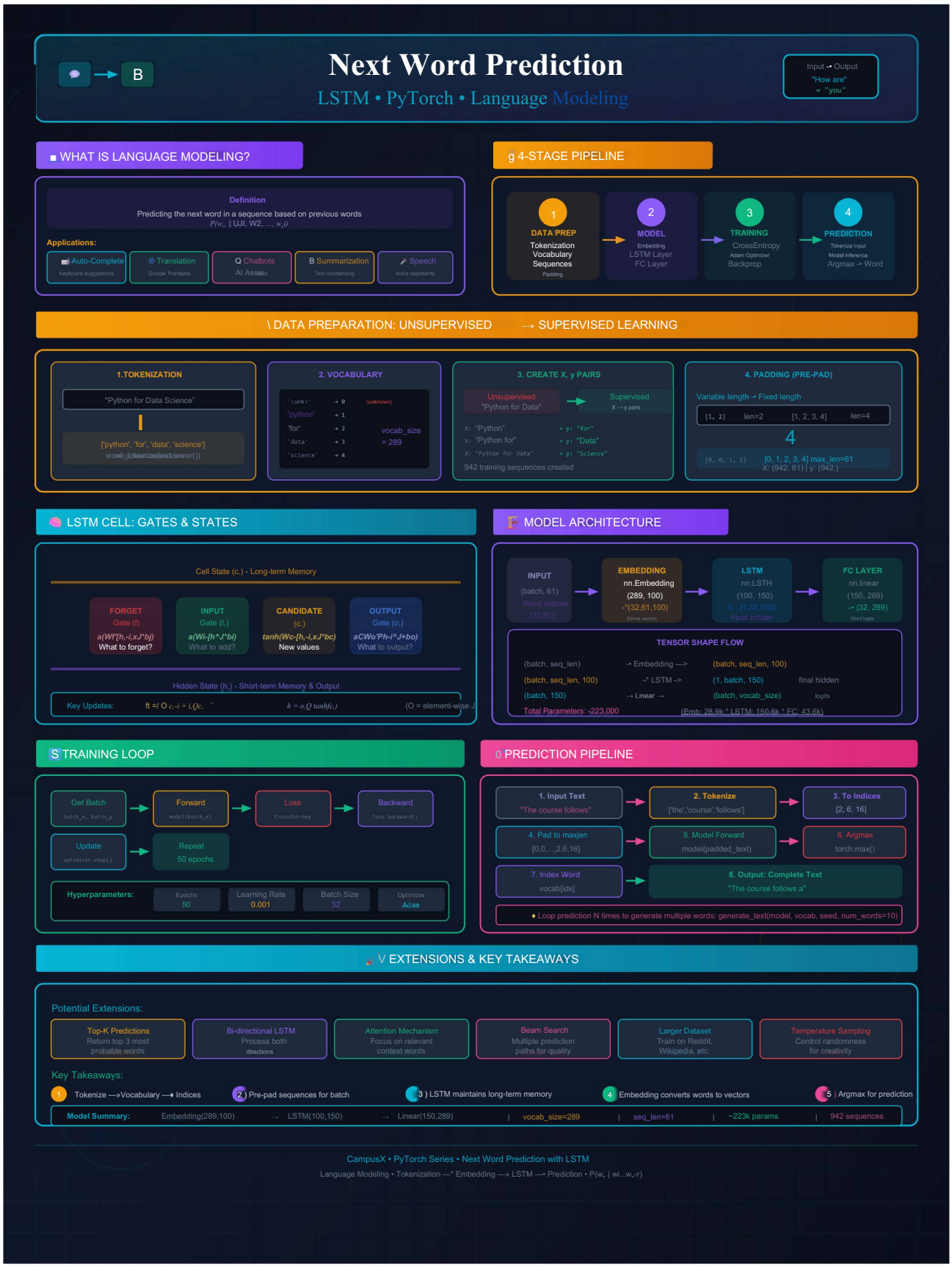
Concept	Description	Code/Formula
Embedding	Dense word vectors	nn. Embedding^, d)
LSTM	Sequence processing	nn.LSTM(input, hidden)
Output	Word probabilities	nn.Linear(hidden, V)
Loss	Cross-entropy	nn.CrossEntropyLoss()
Prediction	Argmax over logits	torch.max(output$_J$ dim=l)

Model Summary

Layer	Input Shape	Output Shape	Parameters
Embedding	(batch, 61)	(batch, 61, 100)	289 x 100 = 28,900
LSTM	(batch, 61, 100)	(batch, 61, 150)	4 x ((100 + 150) x 150 + 150) = 150,600
Linear	(batch, 150)	(batch, 289)	150 x 289 + 289 = 43,639
Total	-	-	**~223,000**

References

1. **LSTM Paper:** Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780.
2. **PyTorch Documentation:** torch.nn.LSTM
3. **Word Embeddings:** Mikolov, T., et al. (2013). Efficient Estimation of Word Representations in Vector Space.


Next Word Prediction
LSTM • PyTorch • Language Modeling
Input → Output
"How are" → "you"
WHAT IS LANGUAGE MODELING?
Definition
Predicting the next word in a sequence based on previous words
P(w_t | w1, w2, ..., w_t-1)
Applications:
Auto-Complete
Keyboard suggestions
Translation
Google Translate
Chatbots
AI Assistants
Summarization
Text condensing
Speech
Voice assistants
4-STAGE PIPELINE
1
DATA PREP
Tokenization
Vocabulary
Sequences
Padding
2
MODEL
Embedding
LSTM Layer
FC Layer
3
TRAINING
CrossEntropy
Adam Optimizer
Backprop
4
PREDICTION
Tokenize input
Model inference
Argmax → Word
DATA PREPARATION: UNSUPERVISED → SUPERVISED LEARNING
1.TOKENIZATION
"Python for Data Science"
['python', 'for', 'data', 'science']
word_tokenize(text.lower())
2. VOCABULARY
'<unk>' → 0 (unknown)
'python' → 1
'for' → 2
'data' → 3
'science' → 4
vocab_size = 289
3. CREATE X, y PAIRS
Unsupervised
"Python for Data"
Supervised
X → y pairs
X: "Python" → y: "for"
X: "Python for" → y: "Data"
X: "Python for Data" → y: "Science"
942 training sequences created
4. PADDING (PRE-PAD)
Variable length → Fixed length
[1, 2] len=2
[1, 2, 3, 4] len=4
4
[0, 0, 1, 2]
[0, 1, 2, 3, 4] max_len=61
X: (942, 61) | y: (942,)
LSTM CELL: GATES & STATES
Cell State (c_t) - Long-term Memory
FORGET Gate (f_t)
σ(Wf·[h_t-1, x_t]+b_f)
What to forget?
INPUT Gate (i_t)
σ(Wi·[h_t-1, x_t]+b_i)
What to add?
CANDIDATE (c̃_t)
tanh(Wc·[h_t-1, x_t]+b_c)
New values
OUTPUT Gate (o_t)
σ(Wo·[h_t-1, x_t]+b_o)
What to output?
Hidden State (h_t) - Short-term Memory & Output
Key Updates: f_t = σ(...) c_t = f_t ⊙ c_t-1 + i_t ⊙ c̃_t h_t = o_t ⊙ tanh(c_t)
(⊙ = element-wise)
MODEL ARCHITECTURE
INPUT
(batch, 61)
Word indices
EMBEDDING
nn.Embedding
(289, 100)
Dense vectors
LSTM
nn.LSTM
(100, 150)
h_n: (1, 32, 150)
Final hidden
FC LAYER
nn.Linear
(150, 289)
→ (32, 289)
Word logits
TENSOR SHAPE FLOW
(batch, seq_len) → Embedding → (batch, seq_len, 100)
(batch, seq_len, 100) → LSTM → (1, batch, 150) final hidden
(batch, 150) → Linear → (batch, vocab_size) logits
Total Parameters: 223,000
(Emb: 28.9k + LSTM: 150.6k + FC: 43.6k)
TRAINING LOOP
Get Batch
batch_x, batch_y
Forward
model(batch_x)
Loss
CrossEntropy
Backward
loss.backward()
Update
optimizer.step()
Repeat
50 epochs
Hyperparameters:
Epochs 50
Learning Rate 0.001
Batch Size 32
Optimizer Adam
PREDICTION PIPELINE
1. Input Text
"The course follows"
2. Tokenize
['the','course','follows']
3. To Indices
[2, 6, 16]
4. Pad to maxlen
[0,0,...,2,6,16]
5. Model Forward
model(padded_text)
6. Argmax
torch.max()
7. Index Word
vocab[idx]
8. Output: Complete Text
"The course follows a"
Loop prediction N times to generate multiple words: generate_text(model, vocab, seed, num_words=10)
EXTENSIONS & KEY TAKEAWAYS
Potential Extensions:
Top-K Predictions
Return top 3 most probable words
Bi-directional LSTM
Process both directions
Attention Mechanism
Focus on relevant context words
Beam Search
Multiple prediction paths for quality
Larger Dataset
Train on Reddit, Wikipedia, etc.
Temperature Sampling
Control randomness for creativity
Key Takeaways:
1 Tokenize → Vocabulary → Indices
2 Pre-pad sequences for batch
3 LSTM maintains long-term memory
4 Embedding converts words to vectors
5 Argmax for prediction
Model Summary: Embedding(289,100) → LSTM(100,150) → Linear(150,289) | vocab_size=289 | seq_len=61 | ~223k params | 942 sequences
CampusX • PyTorch Series • Next Word Prediction with LSTM
Language Modeling • Tokenization → Embedding → LSTM → Prediction • P(w_t | w1...w_t-1)

CampusX